

---

# **Modern GPU Programming For MLSys**

***Release 0.0.1***

**MLC Community**

**Jun 28, 2026**



## 第一部分, 理解 GPU

|          |   |
|----------|---|
| 1 本书组织结构 | 3 |
|----------|---|



机器学习系统是现代 AI 工作负载的核心。在这些系统中,性能往往取决于少数几个 GPU 内核的质量。注意力内核、LLM 预填充 (prefill) 与解码 (decode) 内核、低精度块缩放 GEMM、融合的 MoE 层以及其他大型融合内核,都直接决定了训练与服务两端的端到端速度。

然而,要让这些内核足够快,仅有优化技巧清单是远远不够的。现代 GPU 早已不再是旧设计的简单变体。近年的架构引入了更丰富的存储空间、新的访问模式以及日益特化的执行单元。要把它们编程好,我们既需要对硬件有清晰的心智模型,也需要具备构建高性能内核的实践理解。本书的目标正是同时培养这两者。

本书遵循一条简单的递进脉络:先理解 GPU 硬件,再学习我们将要使用的编程模型,最后一步步构建 SOTA(state-of-the-art) 内核。我们的主要目标硬件是 Blackwell 这一代,主要贯穿全书的示例是快速矩阵乘 (GEMM) 与 FlashAttention。在此过程中,我们还会研究 GPU 优化背后的核心要素:数据布局 (data layout)、异步数据搬运以及异步协调 (asynchronous coordination)。

这些材料源自卡内基梅隆大学 (Carnegie Mellon University) 的 [Machine Learning Systems](#) 课程系列。为了让这些思想更易于学习与运行,本书使用 **TIRx** 这一 Python DSL(领域特定语言) 一步步构建真实可运行的 GPU 内核示例。TIRx 贴近硬件,让我们既能推理底层控制,又能通过可运行代码进行学习。

本书是开源的。欢迎通过 [GitHub](#) 仓库贡献内容、提交勘误与补充示例。



## 本书组织结构

- **第一部分, 理解 GPU**。这一部分介绍 GPU 的整体组织结构、编写快速内核的通用套路, 以及数据布局、异步内存操作与协调等关键概念。它为全书后续内容奠定了硬件直觉基础。
- **第二部分,TIRx 概览**。这一部分介绍 TIRx 的关键要素, 它们构成了全书代码示例的基础。
- **第三部分,GEMM: 从分块到 SOTA**。一份完整的分块 GEMM 优化指南, 通过 TMA 流水线、持久化调度、warp specialization(线程束特化) 与 2-CTA 集群逐步搭建而成。
- **第四部分,FlashAttention 4**。一个基于第三部分技术构建的完整注意力内核: 两个 MMA 之间夹着 softmax、在线 softmax 重缩放、因果掩码 (causal masking) 以及 GQA。
- **参考**。TIRx 语言参考与编译器内部实现。

### 1.1 GPU 执行模型

#### Overview

- 一个 kernel(内核) 在一个线程层次结构 (thread → warp → warpgroup → CTA → cluster → grid) 上运行, 跨越多个不同的存储空间 (寄存器、SMEM、GMEM、TMEM)。
- 计算分为 CUDA cores 与 Tensor Cores(张量核);TMA(张量内存加速器) 等专用引擎负责搬运喂给它们的数据。
- 一个 kernel 是一条流水线, 把数据逐级送过这些存储空间, 并在相互独立的计算引擎与数据搬运引擎之间交接工作; 反复出现的目标, 就是让这些引擎同时保持忙碌。

要写出快速的 GPU 程序, 重要的是先理解硬件本身, 以及代码如何在该硬件上运行。本章概述 GPU 的执行模型: 执行工作的线程层次结构、保存与搬运数据的存储空间, 以及承担主要工作的计算引擎与数据搬运引擎。我们先把这些部件逐一介绍, 然后把它们在一个 GEMM(通用矩阵乘) 流水线中组合起来, 以便看清数据和执行是如何在硬件中流动的。本书后续几乎每一项优化, 都是用某种方式在这些相同部件之间安排工作。

现代 GPU 还包含许多专用硬件单元。为了先给一个直观印象, 在深入每个部件之前, 下面的交互演示展示了 Blackwell 流式多处理器 (streaming multiprocessor;SM) 内部的主要元素。你可以点击每个部件查看其细节。

交互:*Blackwell SM*, 展示其 *warp/warpgroup*、共享内存、*Tensor Memory*, 以及 *Tensor Core* 与 *TMA* 引擎。

### 1.1.1 执行层次结构

我们从执行工作的线程讲起。GPU 并不是把它的数千个线程呈现为一个扁平的线程池。相反, 它把它们组织成一个嵌套的层次结构, 之所以如此, 是因为协作会在多个不同规模上同时发生。每一层的存在, 都是为了在某一种规模上让协作变得廉价。下图展示了 Blackwell 上的层次结构; 你可以点击每个层级来高亮它。

交互: 点击一个层级: *thread* → *warp* → *warpgroup* → *CTA* → *cluster* → *grid*。

- **Thread(线程):** 执行的标量单元。每个线程有自己的程序计数器和自己的寄存器, 并在其所在的 warp 内由一个 lane ID 标识。
- **Warp:** 32 个线程, 以 SIMT(单指令多线程) 方式执行。一个 warp 的各 lane 一起发射同一条指令, 但每个 lane 各自保留自己的寄存器, 并且可以被单独掩码掉, 这正是让单个 warp 的各 lane 能够走上不同分支的机制。
- **Warpgroup(线程束组):** 四个连续的 warp, 即 128 个线程。Hopper 引入了 warpgroup, 把它作为发射 warpgroup 级 MMA(wgmma) 的单元, 而在 Blackwell 上它又承担了第二个角色: 它是 Tensor Memory 访问的协作单元, 128 个线程一起把一个 TMEM 分块搬进或搬出寄存器。
- **CTA(Cooperative Thread Array, 即 CUDA 中所称的 thread block):** 硬件调度的基本单元。一个 CTA 运行在单个 SM 上, 并在其中拥有一份私有的共享内存分配。多个 CTA 可以同时驻留在同一个 SM 上, 此时它们共同瓜分该 SM 的共享内存容量。
- **Cluster(集群):** 一组协作的 CTA, 可能分布在不同 SM 上。一个 cluster 内的 CTA 可以相互同步, 并读写彼此的共享内存, 这一能力被称为分布式共享内存 (distributed shared memory)。

这些层级值得反复体会, 因为与早期架构不同, Blackwell 的关键操作并非全由同一组线程发射。一次 TMA 拷贝由单个线程发起, 再由硬件执行。一次 TMEM 到寄存器的加载是 warpgroup 分布式的: 四个 warp 协作, 各自搬动它那一部分 TMEM 分块。一次 tcgen05 MMA 由一个被选出的线程提交, 而一次集群级 MMA 会一次跨越两个 CTA。因此, 每个操作都有它自己的天然粒度, 执行该操作的线程集合, 就是我们所说的该操作的作用域 (scope), 这是本书反复回到的三个反复出现的设计要素 (作用域、布局、派发) 中的第一个。

### 1.1.2 存储空间

该层次结构中的线程, 其速度只取决于送达它们的数据有多快, 因此接下来我们看数据存放在哪里。不存在一种存储既大又快; 物理规律迫使容量与速度之间做出权衡。因此 GPU 提供的是多种存储而非一种, 每种都在该权衡的不同点上取得平衡, 而 kernel 的工作正是通过这些存储来搬移数据。每个空间都有自己的容量、自己的延迟, 以及自己关于谁可以访问它的规则。

| 存储空间                        | 归属           | 角色         | 说明                          |
|-----------------------------|--------------|------------|-----------------------------|
| <b>Global (GMEM)</b>        | 设备级          | 持久张量存储     | 大容量 HBM, 被所有 SM 共享          |
| <b>Shared (SMEM)</b>        | 每 CTA(单个 SM) | 分块中转       | 低延迟暂存; B200 上每 SM 最多 228 KB |
| <b>Tensor Memory (TMEM)</b> | 每 CTA        | MMA 累加器存储  | Blackwell 新增; 由 tcgen05 使用  |
| <b>Register File (RF)</b>   | 每线程          | 标量与每线程分块片段 | 速度快; 保存收尾/临时值               |

按顺序读下来, 这些空间描绘出一条路径。本书中几乎每个 kernel 的数据通路都是 **GMEM** → **SMEM** → (计算) → 寄存器 → **SMEM** → **GMEM**, 而对于 Tensor Core kernel, TMEM 位于这



条路径的中间, 在数学运算进行时保存累加器。

在这四种里, **Tensor Memory (TMEM)** 是唯一在 Blackwell 之前硬件上没有对应物的, 它的完整细节留到 *Tensor Core: tcgen05*。不过, 理解它的动机是值得的。早期 GPU 把大型 MMA 累加器保存在寄存器里, 在那里它们要竞争一种稀缺资源。Blackwell 则把 tcgen05 累加器输出写到 TMEM——一个 CTA 作用域、规模为 128 lane 乘最多 512 个 32 位列的二维暂存区 (该数组物理上驻留在 SM 上)。然后 kernel 必须在收尾之前, 显式地把 TMEM 读回到寄存器。这一额外步骤并非没有代价, 它的两个后果会在全书反复出现。其一是 TMEM 读取是**显式且 warpgroup 分布式的**, 由 warpgroup 的四个 warp 协作完成。其二是 TMEM 与寄存器不同, 必须被**显式分配和释放**。

## 集群内的分布式共享内存

集群是层次结构中唯一其成员可以跨越多个 SM 的层级, 而这种跨度换来了其他层级所欠缺的一种存储能力。一个 CTA 运行在一个 SM 上, 从该 SM 的共享内存中工作, 但单个 CTA 的 SMEM 预算是有限的, 而大的分块往往需要比单个 block 所能提供的更多的操作数存储或更多的复用。Hopper 的对策是 **thread block cluster**: 一组协作比相互独立的 block 更紧密的 CTA, 它们能够一起同步、读写彼此的共享内存, 这一能力被称为**分布式共享内存 (DSMEM)**。Blackwell 保留了集群并加以增强, 加入了动态调度 (进阶: *Cluster Launch Control*) 与 2-CTA 协作 MMA。

DSMEM 让一个 CTA 能够直接寻址并访问对端 CTA 的共享内存。一个线程可以命名对端 SMEM 中的一个位置, 并把自己的 SMEM 中的一个分块整块拷贝过去, 一旦字节落位, 就触发一个完成屏障 (异步协调: *mbarriers*)。第三部分中的 2-CTA 集群 GEMM 正是建立在这一机制上, 利用它在两个 CTA 之间共享操作数分块, 而不必把它们绕回全局内存。

下图展示了一个 CTA 集群所带来的额外 DSMEM 跳转; 点击某一块可以看到每个 CTA 各自拥有什么, 以及跨 CTA 读取发生在何处。

交互: 一个 2-CTA 集群, 其中每个 CTA 各拥有 *A* 的一半和 *B* 的一半, 通过集群 (DSMEM) 读取对方的 *B*, 这对 CTA 一起产生出一个  $256 \times 256$  的输出分块。

### 1.1.3 计算: CUDA Cores 与 Tensor Cores

线程以及它们搬动的数据, 最终必须在一个算术单元上汇合, 而一个 SM 提供的是两种截然不同的数学引擎, 而非一种。两者之间的分工, 几乎决定了每一个 kernel 的写法, 它们扮演着互补的角色。

- **CUDA cores** 是通用的 SIMT ALU。它们运行标量与向量指令, 处理索引运算、逐元素数学、归约以及控制流, 即环绕繁重矩阵运算之外的粘合逻辑。
- **Tensor Cores** 是固定功能单元, 以分块粒度执行稠密矩阵乘加, 在一条指令中计算  $D = AB + C$ 。

这种划分之所以重要, 是因为 Tensor Cores 提供的算术吞吐量远高于 CUDA cores, 在 FLOP/s 上大约有一个数量级 ( $10 \times$  或更多), 因此稠密线性代数 (GEMM、卷积、attention) 只有在 Tensor Cores 上运行时才能达到峰值性能。于是, 获得性能在很大程度上就是让那些 Tensor Cores 喂饱。从一个 GPU 代际到下一个代际发生变化的是 Tensor Cores 如何被编程, 以及它们的结果落在何处。Hopper 引入了异步 warpgroup MMA (`wgmma.mma_async`); Blackwell 的第五代 Tensor Core tcgen05 把它的累加器放在 Tensor Memory 而非寄存器中, 我们在 *Tensor Core: tcgen05* 中专门讨论它。

集群以两种方式扩展了这些引擎, 这两种方式在 GEMM 各章中反复出现。**2-CTA 协作 MMA** 让两个 CTA 各自把它们 SMEM 操作数贡献给一个更大的单一 Tensor Core MMA 分块。**\*\*TMA multicast(多播)\*\*** 让数据搬运引擎的一次加载把同一个 GMEM 分块同时送达多个 CTA, 消除了各自单独加载本会带来的冗余全局流量。两者都建立在前面介绍的分布式共享内存之上。

### 1.1.4 GEMM 数据流水线

到目前为止, 我们已经分别介绍了各个硬件单元。要看看它们如何协同, 可以用一个典型的 GEMM(通用矩阵乘) 流水线作为例子。下面的交互演示展示了一个三段式 GEMM 分块流水线所涉及的单元; 点击某个动作 (例如 `tma load`) 可以高亮它在各硬件单元之间走过的数据通路。

交互: *Blackwell* 上的 `load`  $\rightarrow$  `MMA`  $\rightarrow$  `epilogue` 流水线; 点击某个动作, 追踪它穿越各硬件单元的数据通路。

单个 GEMM 分块流经三个阶段。

1. **加载。** 一次 TMA 拷贝 ([异步数据搬运:TMA](#)) 把一个 A 或 B 操作数分块从 GMEM 流式加载到 SMEM。单个线程发起拷贝, 预先记录预期到达多少字节。随着字节落位, TMA 引擎汇报进度, 而一个完成屏障只在所有预期字节都送达时才翻转。
2. **计算。** 一次 `tcgen05 MMA` ([Tensor Core:tcgen05](#)) 从 SMEM 读出操作数分块, 并把乘积累加进一个 TMEM 分块。由一个被选出的线程发起, 并在数学运算完成时向一个屏障发信号。
3. **收尾。** `warpgroup` 把 TMEM 累加器读回到寄存器, 把结果转换到输出 dtype, 并存到 GMEM, 常常通过经 SMEM 中转并发出一次 TMA store 来完成。

这样写出来, 三个阶段看起来是严格顺序的, 但慢 kernel 与快 kernel 之间的全部差别就在于**重叠 (overlap)**。一个朴素的 kernel 确实按顺序执行各步 (加载、等待、计算、等待、存储), 于是在等待前一步时让每个引擎都闲置着。快的 kernel 则把它们流水化: 当 Tensor Core 正在计算分块 `k` 时, TMA 引擎已经在取分块 `k+1`, 而收尾正在忙于排空分块 `k-1`, 于是三个引擎同时都保持着占用。让三个异步引擎安全地把工作交接给彼此, 正是屏障与相位模型 ([异步协调:mbarriers](#)) 的工作, 而第三部分的 GEMM 阶梯正是建立在它之上。

### 1.1.5 接下来读什么

既然我们已看清了高层图景, 就可以进入那些深入主要机制的章节:

- [Tensor Core:tcgen05](#) 详细讲解 `tcgen05` 计算与 Tensor Memory。
- [异步数据搬运:TMA](#) 涵盖基于 TMA 的异步数据搬运。
- [异步协调:mbarriers](#) 介绍协调这些引擎的 `mbarrier` 与相位模型。

## 1.2 是什么让内核变快

### Overview

- **roofline(屋顶线)** 模型给出一个内核的性能上限。该上限由显存带宽或计算吞吐量决定。
- **算术强度 (arithmetic intensity)** 决定哪一种上限适用, 即每搬运一个字节所完成的有用算术运算量。
- 算术强度低意味着内核受限于内存 (**memory-bound**)。主要的出路是搬运更少的字节、更多地复用数据、融合操作, 或使用更小的 dtype。
- 算术强度高意味着内核可能受限于计算 (**compute-bound**)。此时的主要任务就是让 Tensor Core(张量核) 保持忙碌。

- 在现代 GPU 内核中, 主要的杠杆是重叠 (overlap)。只要依赖图允许, TMA、Tensor Core、收尾 (epilogue) 和存储就应该同时运行。

一个内核只有相对于某个上限才谈得上快。一个像 330 TFLOP/s 这样的数字单看可能很大, 但放到一块能在稠密 fp16 或 bf16 的 Tensor Core 工作上持续约 2 PFLOP/s 的 GPU 上, 含义就完全不同了。如果没有一个上限, 很难判断一个内核到底是接近了硬件极限, 还是仍让芯片大部分处于空闲。

roofline 模型给出了这个上限。它把内核分解为两类基本活动: 搬运字节和执行算术。如果内核无法足够快地搬运数据, 显存带宽就成了限制; 如果内核有足够的复用的数据和足够的算术运算量, 计算吞吐量就成了限制。

本章的数字以 NVIDIA B200 作为贯穿示例。沿用 GPU 执行模型的约定, 我们用取整后的上限来推理: 稠密 fp16 或 bf16 的 Tensor Core 吞吐量约 2 PFLOP/s, HBM3e 带宽约 8 TB/s。具体数值取决于特定器件、时钟、功耗限制和测量设置, 因此应将其视为数量级意义上的上限, 而非数据手册上的常数。

### 1.2.1 Roofline 模型

每个内核都在搬运数据并执行算术。roofline 模型用这两条路径中较慢的那条来约束内核。

计算上限是硬件的最大算术吞吐量。对于 B200 上的 Tensor Core GEMM, 相关上限就是 Tensor Core 吞吐量。对于标量或逐元素 (elementwise) 内核, 相关上限则可能是 CUDA core 吞吐量或其他功能单元。

内存上限是带宽乘以算术强度。如果一个内核每搬运一个字节只做很少的算术, 显存带宽就会限制性能; 如果每个字节上做很多运算, 内存就不那么可能成为瓶颈。

基本的 roofline 约束为:

```
attainable FLOP/s <= min(peak FLOP/s, memory bandwidth * arithmetic_
    intensity)
```

算术强度为:

```
arithmetic intensity = useful FLOPs / bytes moved
```

必须指明内存层级。对于 HBM roofline, 字节数指 HBM 字节; 对于 L2 roofline, 指 L2 字节; 对于 SMEM(共享内存) roofline, 指共享内存字节。本章默认使用 HBM roofline。

在 roofline 图上, x 轴是算术强度, 单位为每字节 FLOP 数; y 轴是可达性能。内存屋顶是一条斜线:

```
performance = bandwidth * arithmetic intensity
```

计算屋顶是一条水平线:

```
performance = peak FLOP/s
```

两者在屋脊点 (ridge point) 相交:

```
ridge point = peak FLOP/s / bandwidth
```

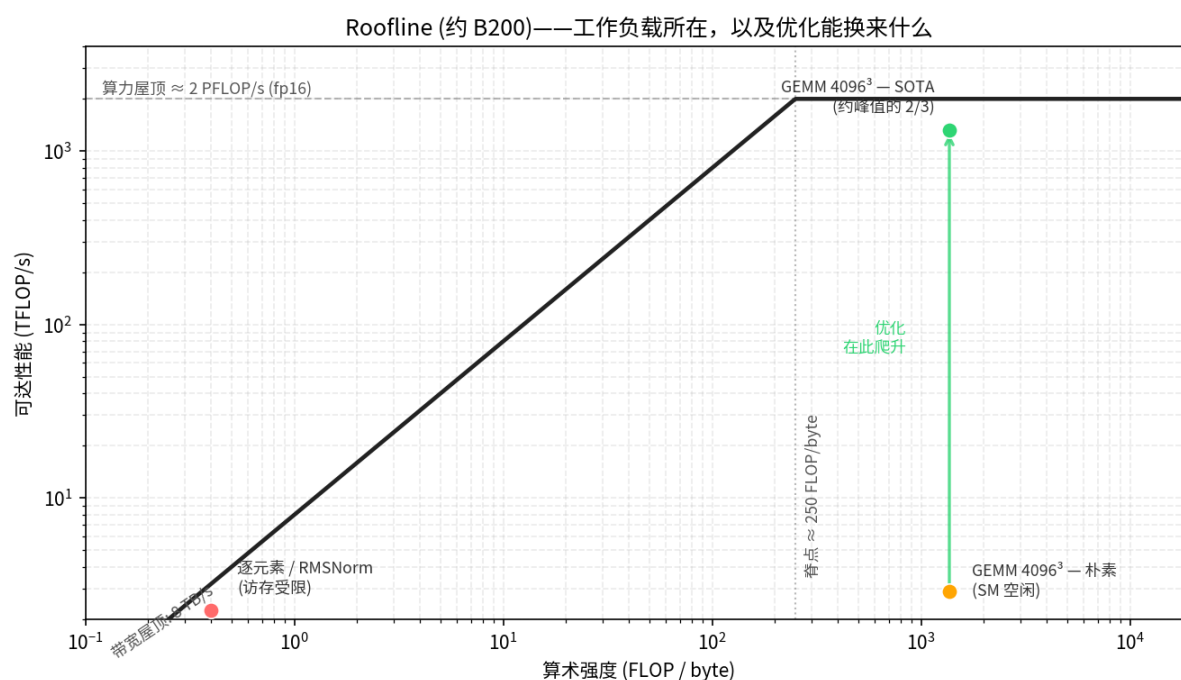
用本章采用的 B200 取整数值:

ridge point  $\approx 2000$  TFLOP/s / 8 TB/s  
 $\approx 250$  FLOP/byte

在 HBM roofline 下, 算术强度低于该值的内核是 **memory-bound** 的。它无法达到峰值 Tensor Core 吞吐量, 因为它无法每秒交付足够多的字节来喂饱那么多的算术运算。

算术强度高于该值的内核则可能是 **compute-bound** 的。此时显存流量不再是第一阶的限制, 剩下的工作是把计算单元驱动得足够好, 以接近那条水平屋顶。

roofline 模型真正有用的不是那张图本身, 而在于它告诉程序员: 哪一种资源是绑定 (binding) 的。一个 **memory-bound** 的内核不会因为其数学指令略微改善就变快; 一个 **compute-bound** 的内核也不会因为节省了几个无关的字节就变快。第一步是搞清楚内核位于屋脊的哪一侧。



## 1.2.2 常见工作负载的算术强度

算术强度往往首先是一种算法性质, 其次才是实现细节。在编写内核之前, 通常就能做出一个粗略估计。

### 逐元素与归约

逐元素内核 (如 GELU) 和归约式内核 (如 RMSNorm) 会读写大量张量, 但每个元素只做很少的 FLOP。

它们的算术强度很低, 位于屋脊点左侧很远处。这类内核的最佳版本通常试图逼近内存带宽屋顶, 而非 Tensor Core 计算屋顶。

对这些内核而言, 重要的问题都很机械:

Are the loads and stores coalesced?  
 Are bytes moved only once?  
 Can the operation be fused with a producer or consumer?  
 Can the dtype be smaller?  
 Can TMA or vectorized accesses help?

如果没有复用、也没有融合机会, 内存屋顶就是真正的上限。

## GEMM

GEMM 是相反的情形。它的算术强度随问题规模增长, 因为每个被加载的分块 (tile) 都可以被复用于许多乘加运算。

对于  $M = N = K$  的方阵 fp16 matmul, 理想算术强度约为:

$$\begin{aligned} AI &\approx 2N^3 / (3 * 2N^2) \\ &= N / 3 \text{ FLOP/byte} \end{aligned}$$

该估计假设 A 和 B 各读一次、C 写一次、beta 为零、片上复用完美, 且没有额外的元数据、padding 或冗余流量。真实内核搬运的数据会多于这一理想模型, 但该估计仍然有用。

在  $N = 4096$  时:

$$\begin{aligned} AI &\approx 4096 / 3 \\ &\approx 1365 \text{ FLOP/byte} \end{aligned}$$

这远在 B200 大约 250 FLOP/byte 的屋脊点右侧。因此在 HBM roofline 下, 大型 GEMM 是 compute-bound 的。目标不再仅仅是减少 HBM 流量, 而是要使用 Tensor Core、让其保持被喂饱, 并把数据搬运与计算重叠起来, 从而使计算屋顶变得可达。

这就是为什么即便 GEMM 算术强度很高, 一个朴素 (naive) 的 GEMM 仍可能很慢: 算法允许高性能, 但实现可能让 Tensor Core 空闲。

## Attention

Attention 介于这两个极端之间。它的算术强度取决于序列长度、头维度、分块化 (tiling)、掩码, 以及中间张量是否被实体化 (materialize)。

标准 attention 中的关键问题是得分矩阵 (score matrix)。如果内核把得分矩阵写入 HBM、之后又读回来, 它就把一个很大的中间量往返搬过了显存。Flash Attention ([Flash Attention 4](#)) 通过把相关分块留在片上、避免这次 HBM 往返, 提高了算术强度。

所以 attention 的优化一部分是 roofline 问题, 一部分是调度问题。先改变算法以减少进入 HBM 的字节, 再调度内核使剩余的搬运与计算相互重叠。

### 1.2.3 当算术强度低时

如果一个内核位于屋脊左侧, 它就是 memory-bound 的。Tensor Core 或 CUDA core 可能处于空闲, 因为瓶颈在于字节, 而不在于算术指令。

有两种应对方式。

第一种应对是提高算术强度。这是杠杆更大的路径, 因为它可能把内核推向 compute-bound 区域。

最重要的技术是融合 (fusion)。算术强度低的一个常见来源是: 把一个中间张量写入 HBM、又在下一个操作里立即读回。把生产者 (producer) 和消费者 (consumer) 融合, 就能把这个中间量留在寄存器、SMEM 或 TMEM (张量内存) 里, HBM 往返随之消失。

例子包括:



```
GEMM plus elementwise epilogue
normalization folded into a neighboring op
attention computed without materializing the full score matrix
```

第二种技术是为复用而分块 (blocking for reuse)。如果一个分块被加载一次、并在被淘汰前被使用多次, 每个字节就能支撑更多的算术运算。GEMM 的高算术强度正是源于这种复用。其他工作负载只要对某个分块有重复使用, 就可以套用同样的思路。

第三种技术是减少每个值所占的字节数。从 fp32 转向 fp16、fp8 或 fp4, 会减少流量并提高每字节的 FLOP 数。当格式需要元数据、缩放因子或额外的转换工作时, 实际增益会小于 dtype 本身的比例。块缩放 (block-scaled) 的 fp8 与 fp4 就是这样的例子。即便如此, 更小的 dtype 往往仍是在 roofline 上把内核右移的最直接手段之一。

第二种应对是接受内存屋顶并试图达到它。有些内核没有足够的工作可融合, 也没有足够的复用可利用。纯粹的拷贝、简单的逐元素操作, 或对大张量的单遍归约, 可能本质上就是 memory-bound 的。

在这种情况下, 目标不是击穿屋顶, 而是打满它。

这意味着:

```
move each byte once
avoid redundant reads
use coalesced or vectorized accesses
use TMA for regular bulk tiles
keep enough memory requests in flight
use smaller storage dtypes when the algorithm allows it
```

一旦一个 memory-bound 的内核达到了内存屋顶, 进一步的计算优化就不再有帮助。要变得更快, 唯一的办法是改变算法以搬运更少的字节。

## 1.2.4 优化阶梯

roofline 说的是什么是可能的, 而不说是达到该极限有多容易。

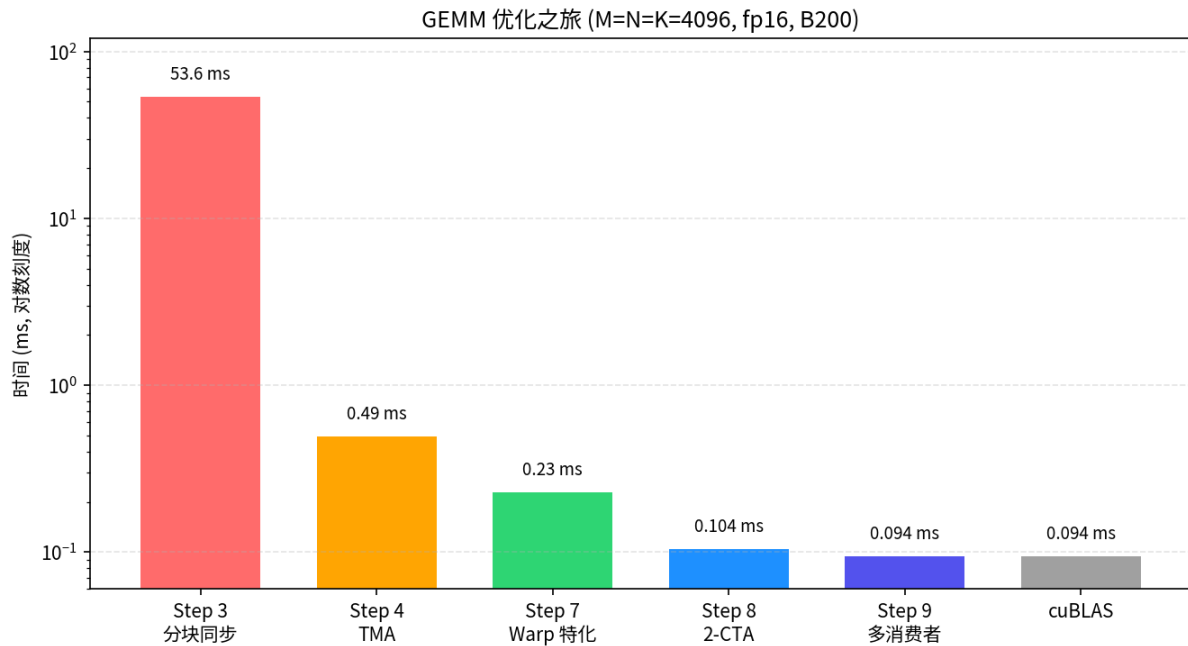
一个大型 fp16 GEMM 在理论上可能是 compute-bound 的。这并不意味着 HBM 屋顶不是主要限制, 并不意味着任何实现都能达到 Tensor Core 屋顶。要弥合这个差距, 需要正确的指令、布局 (layout)、暂存 (staging)、同步与调度。

第三部分中的 GEMM 内核在 B200 上把这一点呈现为一系列步骤 (用线程束特化与集群扩展 GEMM)。每一步都保持相同的基本算法, 只改变分块的计算方式或调度方式。

GEMM 阶梯中第一个被测量到的显著跳变, 是从线程拷贝的分块路径切换到由 TMA 支撑的路径。TMA 把规则的 GMEM -> SMEM 分块搬运从 CTA(协作线程阵列) 线程上卸载下来, 让内核通过硬件托管的大块拷贝来喂饱 Tensor Core。

在那第一次跳变之后, 主要的改进来自重叠与调度。TMA 把未来的分块搬入共享内存, tcgen05.mma 异步运行, 收尾把之前的结果排空。软件流水线 (software pipelining) 和线程束特化 (warp specialization) 把这些片段编排起来, 使硬件引擎同时保持活跃。

也并没有规定每个中间步骤本身都必须更快。像线程束特化这样的步骤, 可能会暂时把资源花在一个不能立即改善数字的结构上。但如果它使得更简单的结构无法表达的后续重叠成为可能, 它仍然可能是正确的一步。



### 1.2.5 重叠是主要的杠杆

一旦一个 GEMM 已经 compute-bound 并用上了 Tensor Core, 剩余的差距通常来自空闲时间。一个简单的内核可能是这样的:

```
load tile k
compute tile k
store tile k
load tile k + 1
compute tile k + 1
store tile k + 1
```

这种调度会让硬件空闲。加载运行时, Tensor Core 在等待; Tensor Core 运行时, 拷贝引擎可能空闲; 存储排空时, 两者可能都在等待。

流水线化的内核则试图把相互独立的阶段放在一起运行:

```
load tile k + 1
compute tile k
store tile k - 1
```

这正是本书后续所用 Blackwell 内核结构背后的核心思想。TMA 负责异步数据搬运, tcgen05.mma 负责异步的 Tensor Core 工作, 收尾与存储负责输出侧, mbarrier(内存屏障) 对象把各个阶段连接起来, 使得每个消费者只有在真正需要其数据时才等待。

关键不在于消除依赖, 而在于围绕依赖来调度。分块 k 的 MMA 在分块 k 被加载之前无法开始; 分块 k 的收尾在分块 k 的 MMA 完成之前无法读取累加器 (accumulator)。但分块 k + 1 的加载通常可以在分块 k 的 MMA 进行中并行运行, 而分块 k - 1 的存储通常也可以同时排空。

这就是为什么后续那么多章节聚焦于异步机制:

```
TMA for global memory to shared memory movement
mbarriers for load completion and resource handoff
```

(continues on next page)

(continued from previous page)

```
tcgen05 for asynchronous Tensor Core compute
TMEM for long-lived accumulators
warp specialization to separate producer and consumer roles
clusters for larger cooperative tiles and multicast
```

它们是不同的机制,但服务于同一个调度目标:让有用的工作同时在多条硬件路径上运行。

### 1.2.6 占用率与资源压力

重叠并不是唯一的延迟隐藏机制。更古老、也更通用的机制是占用率 (occupancy)。

占用率是驻留在单个 SM(流式多处理器)上的工作量。如果某个 warp(线程束)停顿,调度器可以运行另一个就绪的 warp。这通过维持一个可用独立 warp 池来隐藏延迟。

占用率受每 SM 资源的限制。主要限制是寄存器、共享内存、warp 槽位和 CTA 槽位。一个每线程使用大量寄存器、或每 CTA 占用大量共享内存的内核,占用率可能很低,因为只有少量 CTA 或 warp 能装得下 SM。

许多现代 Tensor Core 内核会有意以降低占用率的方式消耗资源。多级共享内存流水线消耗 SMEM, 大寄存器分片消耗寄存器, TMEM 分配消耗张量内存容量, 线程束特化可能为生产者或消费者角色预留整个 warp。

这种权衡是刻意为之。这些内核不靠驻留大量无关 warp 来隐藏延迟,而是通过在较少数量的驻留 CTA 内部进行显式重叠来隐藏延迟。一个低占用率的内核,只要其流水线能让 TMA、Tensor Core 和存储都保持忙碌,就仍然可以很快。

两种方法都不是普遍更优。有些内核需要高占用率,因为它们的内存访问不规则,或显式重叠有限;另一些则需要深度暂存与特化,因为那是高效喂饱 Tensor Core 的唯一途径。正确的问题不是占用率是否高,而是处于活跃的硬件单元是否被保持忙碌。

### 1.2.7 这为后续带来了什么

本书余下部分会不断回到同一个诊断:

```
Which roof is this kernel under?
What resource is binding?
What change moves the kernel closer to that roof?
```

对 memory-bound 的内核,答案通常是更少的字节和更好的带宽利用。这意味着融合、合并访问 (coalescing)、向量化访问、在适用处使用 TMA, 以及更小的 dtype。

对 compute-bound 的 GEMM, 答案是先上 Tensor Core, 再谈重叠。内核必须暂存操作数、派发异步 MMA 工作、让流水线保持填满,并在不阻塞计算路径的前提下排空结果。

对 Flash Attention, 第一步是通过把得分与概率分块留在片上来提高算术强度。之后,它使用与 GEMM 相同的重叠工具:分块化的数据搬运、共享内存暂存、异步计算,以及谨慎的资源交接。

这就给出了一个实用的优化 workflow: 估算算术强度, 定位屋顶, 判断内核是 memory-bound 还是 compute-bound, 然后优化真正设定上限的那项资源。

没有这一步,内核优化就变成了瞎猜。有了它,每一次改动就都有了理由:要么它提高了算术强度,要么它把内存路径推向带宽峰值,要么它减少了计算屋顶下的空闲时间。



## 1.3 数据布局及其记法

### Overview

- 数据布局将一个张量的逻辑索引映射到物理位置, 它决定了访存的合并性、bank 冲突, 以及某个引擎能否读取一个分块。
- 本书用一种记法来书写布局:  $S[(\text{shape}) : (\text{strides})]$ , 并带有具名轴 ( $@\text{laneid}$ 、 $@\text{TLane}$ , ...) 以及一个用于广播或复制数据的复制项  $R[\dots]$ 。
- Swizzle 是一种对地址的 XOR(异或) 重映射, 用于消除共享内存的 bank 冲突。

同样的一组数字, 以不同的物理排布写入内存, 在同一块 GPU 上的运行速度可能相差一个数量级。

原因在于, 一个张量的逻辑索引丝毫不能说明它的字节实际存放在哪里。硬件对这种存放位置极为敏感: 它决定了 32 个 lane 的访存是合并成一次事务, 还是分散成 32 次; 这些地址是落到不同的内存 bank 上, 还是会发生冲突并被串行化; 它甚至决定了一个分块的字节排布是否与 Tensor Core(张量核) 能读取的格式相符。

机器学习程序通常用张量的逻辑形状来描述它们。一个数据布局补上了缺失的物理部分: 它说明具有逻辑索引  $(i, j, \dots)$  的某个元素到底存放在哪里——是在内存中、在寄存器中, 还是在某种其他硬件存储里。

本章介绍现代 GPU 编程中出现的几种主要布局。为了让讨论易于展开, 我们发展出一套紧凑的记法, 用以统一描述机器学习系统在各种情形下遇到的布局。最后我们以 **swizzling** 收尾, 这种机制能同时让对一个分块的行向访问和列向访问都变得高效。

### 1.3.1 形状-步长模型

在进入 GPU 专有的布局之前, 值得从最简单的布局讲起, 因为本章其余所有内容都建立在他之上。布局的核心其实只有两样东西: 一个形状和一组与之匹配的步长。我们把这对值写成  $S[(\text{shape}) : (\text{strides})]$ , 要找到某个逻辑索引对应的位置, 我们取该索引与步长的点积。例如, 一个行主序的  $4 \times 4$  矩阵看起来是这样的:

```
S[(4, 4) : (4, 1)]      addr(i, j) = i·4 + j·1
```

这无非就是经典的形状/步长模型, 只是写得紧凑了一些 (是 CuTe 记法的行主序简化版), 后续所有内容都由它构建而成。

事实上, 你几乎肯定已经用过这个模型了。凡是写过 PyTorch 或 NumPy 的人都用过, 因为在这些库里, 一个张量本身就是一个形状加上在一个平坦存储缓冲区上的一组步长:

```
import torch
t = torch.arange(12).reshape(3, 4)
t.shape          # torch.Size([3, 4])
t.stride()       # (4, 1)           ← exactly S[(3, 4) : (4, 1)]
```

一旦你以这种方式看待张量, 就会明白为什么有那么多“重塑”操作根本不碰数据。它们只是改写步长, 然后返回一个指向同一份存储的视图, 最清晰的例子就是转置, 或者说 permute:

```
tt = t.permute(1, 0)          # or t.T
tt.shape                      # torch.Size([4, 3])
```

(continues on next page)

(continued from previous page)

```
tt.stride()                # (1, 4)          ← strides
↪swapped, no data moved
tt.data_ptr() == t.data_ptr() # True, same bytes
```

这里 `t.permute(1, 0)` 是在同一块内存上的 `S[(4, 3) : (1, 4)]`: 转置纯粹是一次步长的改变, 没有搬动一个字节。对连续张量做 `reshape` 或 `view` 的情形也一样: 在旧存储上换一组新的形状和新的步长。(NumPy 的行为完全一致; 唯一的区别是它的 `.strides` 以字节而非元素来计数。)

GPU 上的布局也正是这样工作的, 本章余下部分其实都是围绕同一个想法展开的一系列变体: 一个分块的映射 (无论映射到内存, 还是通过我们马上引入的具名轴映射到 lane 和寄存器) 都是一条作用在固定缓冲区上的步长规则, 所以重排一个分块通常只是改变布局, 而不是一次拷贝。不过, 我们要小心这种推理的边界。零拷贝的故事只对一个线性地址空间之上的逻辑视图才干净成立; 在 GPU 上, 它只有在新的视图与既有的字节排布和归属关系相容时才适用。一旦你改变了哪个线程或哪个寄存器拥有某个元素, 或者改动了 SMEM(共享内存) 的 `swizzle`, 通常就需要真正的数据搬动: `load`、`store`、`shuffle`、`ldmatrix`、转置。

### 1.3.2 分块布局

到目前为止, 我们描述的都是整张张量的布局。然而 GPU 的内核很少一次性处理整张矩阵; 它们处理的是更小的分块, 这些分块被硬件的不同部分加载、变换、计算。好消息是, 分块化并不需要任何新东西。它仍然只是一个布局, 只不过现在写出来多了几个维度。把一个  $8 \times 8$  矩阵切成  $2 \times 4$  的分块, 我们就得到一个 4 维布局, 其坐标为 `(tile_row, row_in_tile, tile_col, col_in_tile)`, 步长则被选成让每个分块保持连续:

```
S[(4, 2, 2, 4) : (16, 4, 8, 1)]
```

一个逻辑坐标  $(i, j)$  先变成  $(i//2, i\%2, j//4, j\%4)$ , 再经过这些步长计算。值得注意的是, 这套记法在表达分块化时根本用不到任何特殊的“分块”概念: 它就是和之前一样的形状-步长模型, 只是索引被拆成了外层和内层坐标。

下方的交互式可视化展示了逻辑矩阵索引是如何被分解成分块坐标, 再映射到物理地址的。

交互: 点击某个单元格, 查看它的分块化索引和地址。

### 1.3.3 具名轴

到目前为止, `S[...]` 中的每一条步长都指向线性内存里的某个偏移, 我们也把地址当作内存里的某个位置来看待。但在 GPU 上, 数据可以存放在不止一个地方: 除了内存, 一个分块还可以分布在 `warp`(线程束) 的 lane 之间、线程的寄存器之间, 或者 TMEM(张量内存) 的 lane 和列之间。为了统一描述所有这些情形, 我们用具名轴来扩展这套记法。其想法是让每个步长系数带上一枚轴标签, 说明它是在哪个空间里移动的: `@m` 表示普通内存, `@laneid` 表示 `warp` 的 lane, `@reg` 表示寄存器, `@warpid` 表示 `warp`, `@TLane / @TCol` 表示 TMEM 坐标。有了这些标签, 一个布局就不仅能描述数据在内存中的位置, 还能描述数据是如何分布在操作它的那些硬件资源之上的。

一旦内存标签被明确写出来, 内存中一个行主序的  $8 \times 16$  分块就简单地写成

```
S[(8, 16) : (16@m, 1@m)]
```

当一个布局描述的是分散在线程之间而非铺在内存里的数据时, 这些标签才真正派上用场。以 `S[(8, 4, 2) : (4@laneid, 1@laneid, 1@reg)]` 为例: 它不再指向线性内存, 而是把行和列映射到 lane ID 和每个 lane 的一个寄存器上。这里的 `laneid` 指 `warp` 内的 lane 索引,

即 `thread_index % warp_size`。这正是你将在跨 GPU 世代的 *Tensor Core* 操作数布局中遇到的 Tensor Core 寄存器片段。

下方的交互式可视化展示了布局如何把张量元素分散到 warp 的 lane 和每个 lane 的寄存器上, 而不是放进线性内存。

交互: 建立在 `@laneid` 和 `@reg` 之上的布局; 点击某个单元格, 查看哪个 lane / 寄存器持有它。

### 1.3.4 分布式布局

具名轴之所以如此有用, 在于它让我们能统一地描述系统多个层级上的数据放置, 包括跨整台设备的放置。我们刚刚把它用在单个 GPU 内部的 lane 和寄存器上, 但同样的想法也能向外延伸: `@gpuuid_x`、`@gpuuid_y` 这样的轴可以说明数据位于 GPU 阵列中的何处, 有了它们, 这套记法就能刻画分布式训练和推理中出现的分片模式。轴尚未刻画的一件事是复制, 即被拷贝到不止一个地方的数据, 为此我们加入记法 `R[n : stride]`, 其中 `R` 标记被复制的维度。例如, `R[2 : 1@gpuuid_x]` 描述沿 `@gpuuid_x` 轴的复制。把两者合在一起, 一个表达式就能既把张量分片到一个  $2 \times 2$  的 GPU 阵列上, 又沿其中一根轴复制它:

```
S[(2, 4, 8) : (1@gpuuid_y, 8@m, 1@m)] + R[2 : 1@gpuuid_x]
```

下方的演示在一个小的 GPU 阵列上展示了这种“分片加复制”的组合模式。点击任意单元格, 查看它由哪个设备持有, 并观察 `@gpuuid_x` 复制是如何在配对设备上放置一份完全相同的副本的; 按钮可以在“全分片”、“分片 + 副本”和“分片 + 偏移”三种布局之间切换。

交互: 分布在  $2 \times 2$  GPU 阵列上的布局; 点击某个单元格, 查看哪些设备持有它。

### 内核内的复制模式: TMEM 中的缩放因子

我们刚刚为 GPU 阵列引入的复制维度 `R[...]` 并不只是关于多台设备。同样的构造恰好也能描述完全发生在单个内核内部的一件事: 跨 lane 广播的数据。Blackwell(架构代号)的块缩放 MMA(矩阵乘加, Matrix Multiply-Accumulate)(跨 GPU 世代的 *Tensor Core* 操作数布局)就是一个好例子。它的缩放因子存放在 TMEM 里, 一个 128 行的缩放向量只存在 **32 个 TMEM lane** 中: 逻辑行 `r` 落到 TMEM lane `r % 32` 上, 而 `r // 32` 则沿列方向延伸。这 32 个被存储的 TMEM lane 随后沿 **TMEM 的 T Lane 轴** 被复制, 从 32 扩展到 128 个 TMEM lane, 这样发起读取的 `warpgroup`(线程束组, 4 个 warp) 中的四个 warp 就都能在各自那 32-lane 的 TMEM 窗口里找到一份副本。这是一次 `warpx4` 广播, 我们用复制维度来书写它。这些读取本身则由这些 warp 的线程来完成:

```
S[(32, ...) : (1@TLane, ...)] + R[4 : 32@TLane]
```

于是得到四份副本, 间隔为 32 个 TMEM lane: TMEM lane `l`、`l+32`、`l+64`、`l+96` 都持有同一个缩放值。如前所述, 复制维度并不携带新数据; 它只是说明“同一个值, 出现在四个 TMEM-lane 位置上”, 其方式与刚才 `@gpuuid_x` 在 GPU 阵列上广播一行完全相同。

下方的交互式演示把两步合在一起展示: 先紧凑地打包进 32 个 TMEM lane, 再 `warpx4` 广播到 128 个读取 lane。

交互: 点击某个缩放因子 `SFA[m, sf]`; 它会打包进 TMEM 中 lane 为 `m mod 32`、列号为 `(m // 32) * 4 + sf` 的位置, 然后沿 T Lane 轴 `warpx4` 广播到四个 lane 副本 (`l`、`l+32`、`l+64`、`l+96`), 每个 warp 的 32-lane 窗口各得一份。

每一列内部的字节打包 (`scale_vec` 的 1X/2X/4X 模式) 以及 `cta_group:::2` 的拆分, 在跨 GPU 世代的 *Tensor Core* 操作数布局中介绍。

已经熟悉 CuTe 的读者, 可以把本章这套记法看作 CuTe 的一个行主序变体, 只是额外加上了显式的硬件具名轴和一套专门的复制结构。

### 1.3.5 Swizzle 布局

本章的最后一种布局是为了解决一个具体的硬件问题而存在的。GPU 的共享内存被组织成多个内存 bank, 当不同的 lane 落到不同的 bank 上时, 访存速度最快。当若干 lane 反而落到同一个 bank 内的不同地址时, 硬件别无选择, 只能把它们串行化, 于是我们就要为一次 bank 冲突付出代价。

在张量程序中这一点很难避免, 因为内存并非按纯线性的顺序访问。在处理矩阵时, 我们常常需要同时读取同一分块的行切片和列切片, 这造成了一种实实在在的张力: 一种对行向访问高效的布局往往会在列向访问时引发 bank 冲突, 而偏向列的布局又会损害行向访问。Swizzling 正是用来打破这种张力的技术。

swizzle 背后的想法是对地址映射做置换, 通常是把列索引与行索引做 XOR(异或), 使得行访问和列访问同时都被分散到各个 bank 上。它所提供的无冲突保证是具体的: 它只对匹配的元素位宽、swizzle 模式和访问模式 (也就是某个引擎的描述符所期望的那种) 成立, 而对任意的元素位宽或对齐方式并不成立。

下方的第一个交互式演示把它讲得很具体。点击某个列索引, 观察每个元素落到哪个 bank: 左侧纯行主序的分块里, 一列会把全部八个元素都塞进同一个 bank, 于是这次读取被串行化成八个周期; 而右侧经过 XOR swizzle 的布局里, 同一列被分散到八个不同的 bank, 一个周期就能读完。

交互: 一个  $8 \times 8$  分块, 纯行主序下列方向存在 bank 冲突, 经 XOR swizzle 后变为无冲突。

这个小小的  $8 \times 8$  例子抓住了核心想法, 但真实 GPU 的内存 bank 数远比这个示意要多。要让 swizzle 在全规模下生效, 我们并不把整个分块当作一个整体来对待, 而是把内存切成一个个小段, 并在每一段内部套用 swizzle 模式。实践中最常见的情况是 SWIZZLE\_128B, 它围绕 128 字节的段来组织, 这样同一套行/列重映射技巧就能很自然地嵌入到 32-bank 的内存系统中。

下方的交互式演示展示了那个具体的硬件 swizzle——SWIZZLE\_128B, 在我们推广到各种格式之前, 先把这种逐段重复的模式看清楚。

交互: 128 字节段内的 SWIZZLE\_128B 模式; 逐步走过各个读取周期, 可以看到  $physical\_sector = logical\_sector \oplus row$  把每一列都分散到不同的 bank 上。

同样的想法可以推广到这种 128 字节情形之外。为了简化可视化, 接下来我们用一个色块来表示一个段, 而不再逐个画出 bank。一般而言, 硬件会定义一个小的、可重复的原子单元, 置换就其上加, 不同的 swizzle 模式选择不同的原子大小。SWIZZLE\_128B 使用  $8 \times 128$  B 的原子, SWIZZLE\_64B 使用  $8 \times 64$  B 的原子, SWIZZLE\_32B 使用  $8 \times 32$  B 的原子; 然后整个分块就由当时所用原子来分块化铺满。

最后一个交互式演示让你在这些格式 (包括一种 16 B 交错模式) 之间切换、选择数据类型, 并把鼠标悬停在任意单元格上来直接查看一个原子内部的元素排布——这正是讨论某条 load/store 指令期望哪种 swizzle 时所需要的细节粒度。

交互: 选择一种 swizzle 格式 (和数据类型), 查看它的原子形状 ( $8 \times N$  B); 悬停某个单元格, 查看其内部元素是如何被置换的。

该选哪种模式呢? 经验法则是优先选用分块所能填满的最大原子。一个 N 字节的原子要求分块的连续维度至少有 N 字节、并且是它的整数倍, 所以 SWIZZLE\_128B 仅在一行至少跨 128 字节、即 64 个 float16 元素时才适用。一旦能用, 它就是默认选择, 因为它的  $8 \times 128$  B 原子正好覆盖完整的一条 128 字节 bank 线, 从而能一次把一列分散到全部 32 个 bank 上, 在 fp16 下同时对 8 行和 8 列提供无冲突访问。而当问题的形状迫使连续维度变小时, 分块就填不满 128 B 原子, 这时你就退到 SWIZZLE\_64B 或 SWIZZLE\_32B, 即这一行仍能覆盖的最大原子。



你绝不需要手工算出这些置换后的地址, 这里值得把 `swizzle` 与 `S[...]` 记法的关系说精确: 它不是那条仿射映射的一部分。它是叠加在其上的一层独立的、非仿射的层。`S[...]` 布局把一个元素放到一个线性内存 (`@m`) 地址, `swizzle` 则置换这个地址, 在 `TIRx`(`TIRx`, Python DSL) 的布局 API 中写作 `ComposeLayout(swizzle, tile)`([TIRx 布局 API](#))。你要做的只是为涉及该分块的每一个 `op` 都选定一个一致的模式, 剩下的交给这个复合布局即可。

这个复合布局也正是硬件要填充的东西, `swizzling` 和分块化也正是在这里走到一起。一个 `TMA`(张量内存加速器, `Tensor Memory Accelerator`) 描述符是多维的, 所以一个单一的三维方框就能同时描述分块的原子分块化, 以及每个原子内部的 `swizzle`; 一次 `TMA` 加载就会逐个原子地把分块铺开, 并在写共享内存的同时完成 `swizzle`([异步数据搬运: TMA](#)), 不需要单独的 `swizzle` 步骤。至于每种引擎需要哪种 `swizzle`, 则因架构代际而异, 这正是下一章的主题。

## 1.4 跨 GPU 世代的 Tensor Core 操作数布局

### Overview

- 在 Ampere、Hopper 和 Blackwell 三代中, `Tensor Core`(张量核) 仍然执行同样的高层运算:  $D = A B + C$ 。
- 各世代之间变化的是: 操作数如何抵达 `Tensor Core`、支持哪些分块形状与 `dtype`(数据类型), 以及累加器存放在何处。
- Ampere 使用 `warp`(线程束) 级的寄存器片段。共享内存 (`SMEM`) 分块通过 `ldmatrix` 加载进该片段, 累加器保留在寄存器中。
- Hopper 允许 `wgmma` 通过矩阵描述符直接从共享内存读取操作数。描述符指定了 `Tensor Core` 所期望的共享内存 `swizzle`(混淆) 格式。
- Blackwell 保留了共享内存操作数通路, 但把累加器搬进了 `TMEM`(张量内存)。块缩放 `MMA` 也会通过 `TMEM` 暂存其缩放因子。
- 在所有世代中都一直存在两条内存约束: 全局内存合并访问与共享内存 `bank`(存储体) 冲突。

从远处看, `Tensor Core` 的运算显得很稳定。它把 `A` 和 `B` 的分块相乘, 加上累加器 `C`, 得到 `D`。这种形式自 `Volta` 以来就没变过。

围绕该运算的细节却并非固定不变。在某一代上跑得很快的内核, 到了下一代可能就很慢。使用了错误布局的内核还可能算出错误结果, 哪怕逻辑上的数学式仍写作  $D = A B + C$ 。原因是 `Tensor Core` 并不消费抽象矩阵。它消费的是处于非常具体的硬件布局中的操作数。

本章追随这一布局契约走过三代。Ampere 通过 `warp` 级寄存器片段暴露 `Tensor Core`。Hopper 把输入操作数移到共享内存描述符。Blackwell 保留共享内存操作数, 但把累加器搬进 `TMEM`。运算始终是矩阵乘加, 但进出 `Tensor Core` 的路径每一次都在变。

[数据布局](#) 一章中的布局记法, 正是我们用来描述这些契约的语言。Blackwell `TMEM` 的细节另行在 [特殊内存: TMEM](#) 中讨论。

### 1.4.1 始终未曾消失的两条约束

在 `Tensor Core` 介入之前, 两条普通的内存约束就已经在塑造 GPU 内核的布局了。

第一条是全局内存合并访问。当 `warp` 的 32 条 `lane`(通道) 发起一次全局内存加载时, 内存系统希望这些地址落在少数几段连续、对齐的内存段内。如果地址是分散的, `warp` 加载就会变成多

次内存事务。同样的逻辑数据搬运会消耗更多带宽、花费更多时间。

第二条是共享内存 bank 冲突。共享内存被划分为 32 个 bank。如果 warp 中的若干 lane 访问映射到同一个 bank 的不同地址, 这些访问就无法一次性全部服务。硬件会把它们串行化。于是一个在平坦共享内存数组看来无害的布局, 可能因为其 bank 模式而变得很慢。

Swizzle 是修复共享内存一侧的常用手段。逻辑分块保持不变, 但物理地址映射被置换, 使访问模式在各个 bank 之间铺开, 而不是堆叠到同一个 bank 上。

这两条约束即便对从不使用 Tensor Core 的内核也成立。Tensor Core 内核还加上第三条约束: 操作数必须按 Tensor Core 指令自身所期望的布局来排列。本章余下部分就讲这第三条约束如何在 Ampere、Hopper 和 Blackwell 之间变迁。

### 1.4.2 Ampere: 分布在 warp lane 上的寄存器片段

在 Ampere 类 GPU 上, 主要的 Tensor Core 指令是 warp 级的 `mma.sync.aligned.m16n8k*` 家族。关键事实是指令在哪里读写数据: 寄存器。

A、B 以及 C 或 D 累加器, 都是分布在 warp 的 32 条 lane 上的逐线程寄存器片段。共享内存只是中转区。在 MMA 能运行之前, 操作数分块必须从共享内存搬到指令所期望的精确寄存器片段布局中。

数据通路如下:

```
SMEM to registers with ldmatrix
registers to registers with mma.sync
registers back to SMEM with ordinary stores
```

Ampere 的布局故事大多源自这条通路。内核必须把分块以可高效加载的形式存放在共享内存中, 然后用 `ldmatrix` 产生 `mma.sync` 所需的寄存器片段。

### 1.4.3 Ampere Tensor Core 的期望

Ampere 的 Tensor Core 读取由  $8 \times 8$  子分块单元构建的寄存器片段。这些单元正是 `ldmatrix` 加载、MMA 消费的对象。

以 `mma.m16n8k16`、fp16 或 bf16 输入、fp32 累加为例。累加器分块的形状是  $16 \times 8$ 。它以固定模式分布在 32 条 lane 上。

对于 C 或 D 累加器, lane  $l$  持有的行是:

```
l / 4
l / 4 + 8
```

列是:

```
2 * (l % 4)
2 * (l % 4) + 1
```

因此每条 lane 拥有四个 fp32 累加值: 来自两个 8 行半区的各两行, 与两个相邻列交叉。四条连续 lane 覆盖一行的八列。

A 操作数使用相同的 M 侧行切分。K 维度散布在  $l \% 4$  与该 lane 所持寄存器之间。对 fp16 或 bf16, 每个 32 位寄存器打包两个 K 值。

B 操作数使用相匹配的 K 摆放, 并把 N 侧散布到 lane 组与寄存器之间。

确切细节随指令形状与 `dtype` 而异, 但原则固定不变。Tensor Core 期望某种特定的逐 lane 寄存器片段。如果数值不在这些寄存器、不在此模式中, 指令就会乘错元素。

用布局记法写, `m8n8` 片段正是那种以命名 lane 轴书写的模式, 例如:

```
S[(8, 4, 2) : (4@laneid, 1@laneid, 1@m)]
```

两个 `laneid` 迭代量合起来描述行块和列块如何在 lane 间散开, 而最后的 `m` 分量描述逐 lane 的寄存器槽位。

#### 1.4.4 ldmatrix: 从共享内存到寄存器片段

`ldmatrix` 是 Ampere 中连接共享内存与 Tensor Core 寄存器片段的指令。它是一种 warp 协作加载。一条指令把一个或多个  $8 \times 8$  的 16 位矩阵从共享内存搬入 `mma.sync` 所期望的分布式寄存器布局。

指令形式为:

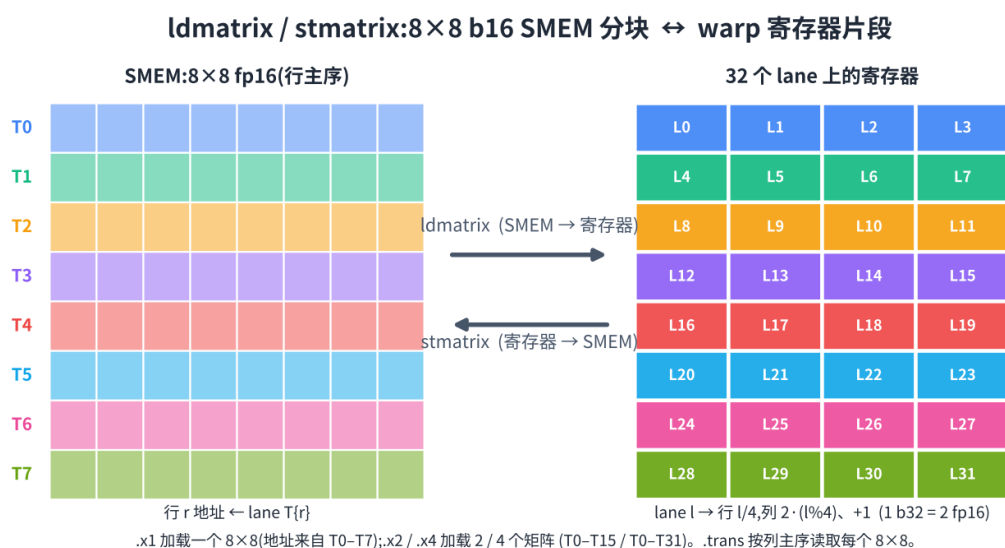
```
ldmatrix.sync.aligned.m8n8.x1.shared.b16
ldmatrix.sync.aligned.m8n8.x2.shared.b16
ldmatrix.sync.aligned.m8n8.x4.shared.b16
```

可附带可选的 `.trans` 限定符。

`.x1`、`.x2`、`.x4` 形式分别加载一、二、四个  $8 \times 8$  矩阵。行基地址由 lane 提供。对矩阵 `m` 与行 `r`, 基地址来自 `lane m * 8 + r`。也就是说 `.x1` 用 lane 0 到 7 提供行地址, `.x2` 用 lane 0 到 15, `.x4` 用 lane 0 到 31。

结果直接落入 MMA 片段。对于基本的  $8 \times 8$  情形, lane `l` 收到 Tensor Core 所期望的那一对行与列。一段朴素的逐 lane `ld.shared` 指令循环本要手动复现那种散布。`ldmatrix` 则以一条 warp 协作指令完成共享内存到片段的重组。

`.trans` 形式在加载时对每个  $8 \times 8$  矩阵做转置。当操作数的存放朝向与 MMA 指令所期望的相反时, 就用得上它。





### 1.4.5 把 Ampere 片段写回

`mma.sync` 完成后,累加器仍是寄存器片段。收尾必须把这个片段搬出去。

在 Ampere 上没有 `ldmatrix` 的专用逆操作。内核使用普通的逐线程存储,有时在存储前配以 `warp shuffle` 或局部重排,把累加器以有用的布局写入共享内存或全局内存。

这让 Ampere 的模型保持简单,但也把大量布局工作暴露给内核。输入侧用 `ldmatrix` 生成片段。计算指令读写寄存器片段。输出侧则由从这些片段出发的普通存储来处理。

### 1.4.6 Ampere 上的 Swizzle

Ampere 内核已经需要共享内存 `swizzle`。原因是共享内存分块通常以一种访问模式写入、又以另一种模式读出。

假设某个分块沿行从全局内存填入。行主序布局使那次写入既合并又 `bank` 友好。但 `ldmatrix` 之后可能以一种实际上沿列下行或横跨  $8 \times 8$  子分块的模式读取该分块。在朴素的行主序布局下,这些读取可能堆叠到同一个共享内存 `bank` 上。

对一个简单的  $(8, 64)$  float16 分块,一行是:

```
64 * 2 bytes = 128 bytes
```

恰好是一整条共享内存 `bank` 线。沿固定列每往下一行就前进 128 字节,于是 `bank` 索引不断重复。八行可能塌缩到同一个 `bank` 上,造成 8 路冲突。

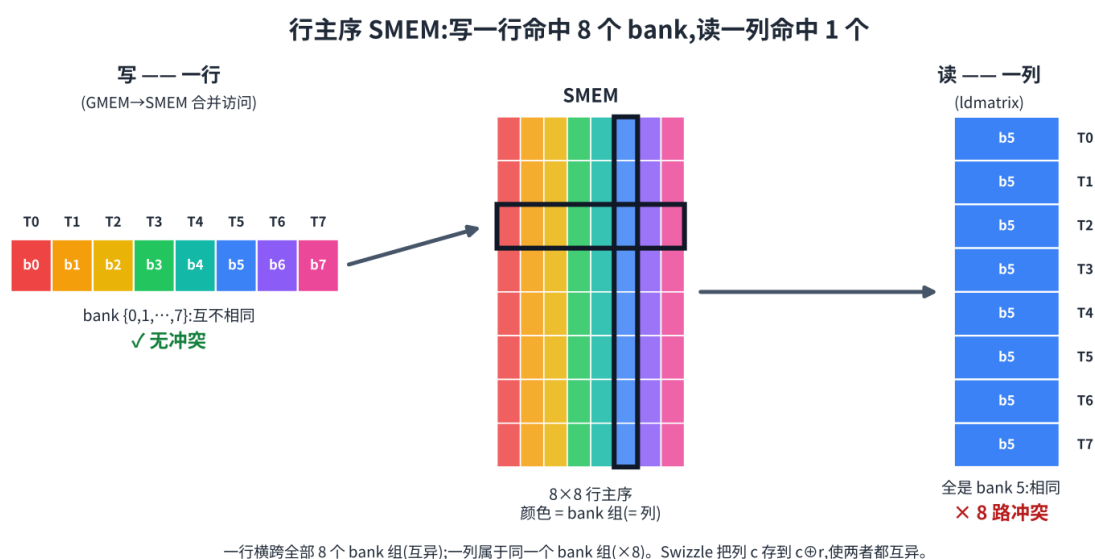
改成朴素的列主序布局并不能解决全部问题。它通常只是把冲突挪到另一次访问。行写入变差,而列式读取变好。

XOR swizzle 通过让物理列依赖于行来修复这一点。一个简单版本是:

```
physical_col = logical_col xor row
```

逻辑分块不变。共享内存中的物理摆放被置换,使行式写入与 Tensor Core 读取模式都能避开 `bank` 冲突。

在 Ampere 上,这种 `swizzle` 通常通过手写的共享内存索引算术来表达。后续世代把它纳入了硬件引擎所用的描述符格式。



### 1.4.7 Hopper:wgmma、共享内存描述符与 swizzle 格式

Hopper 改变了 Tensor Core 通路的输入侧。Hopper 的 wgmma 不再要求每个操作数都通过 ldmatrix 加载进寄存器, 而能直接从共享内存读取操作数。

B 操作数从共享内存矩阵描述符读取。A 操作数既可从共享内存描述符读取, 也可从寄存器读取, 这便有了 .ss 与 .rs 两种形式。

这去掉了源在 SMEM 的操作数那段显式 ldmatrix 步骤, 但并未去掉布局要求。Tensor Core 仍期望操作数以精确的共享内存格式存放。区别在于, 该格式现在通过矩阵描述符告知硬件。

### 1.4.8 Hopper Tensor Core 的期望

Hopper 的共享内存矩阵描述符是对共享内存中一个矩阵分块的紧凑描述。它告诉 wgmma 如何把逻辑操作数坐标转换成共享内存地址。

描述符包含如下字段:

```
start address
leading dimension offset
stride dimension offset
swizzle mode
base offset
```

确切含义取决于操作数的主维模式。对一个 K 主序分块, 一条步长沿 K 推进, 另一条沿 M 推进。对一个 MN 主序分块, 两者角色互换。

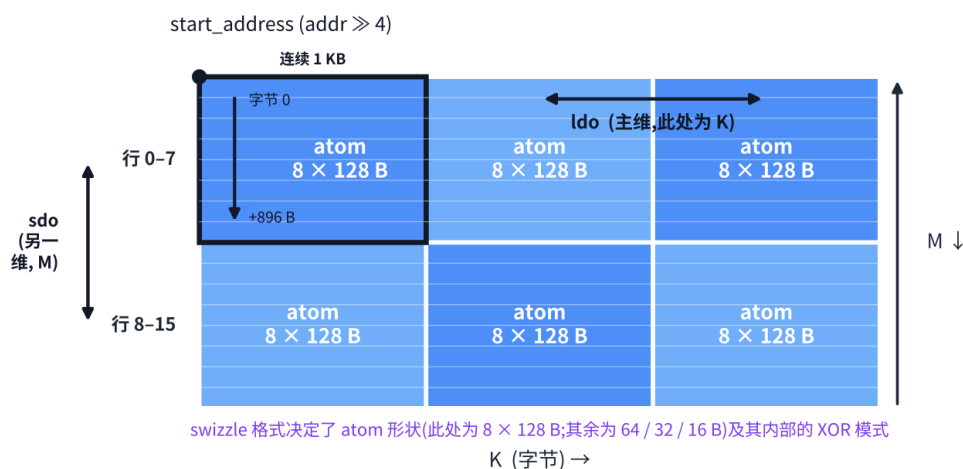
swizzle 模式是若干共享内存描述符格式之一, 例如:

```
SWIZZLE_NONE
SWIZZLE_32B
SWIZZLE_64B
SWIZZLE_128B
```

swizzle 模式决定两件事。它决定描述符所用的原子形状, 也决定在该原子内部施加的 XOR 置换。例如, 128 字节 swizzle 模式把操作数视为 8 行 × 128 字节原子的网格, swizzle 在每个原子内部施加。

内核仍须把字节摆对。TMA(张量内存加速器) 通常负责填充共享内存分块, 而 TMA 描述符必须使用与 wgmma 描述符后续所指定的相同的 swizzle 格式。如果 TMA 写出的是 128 字节 swizzled 分块, wgmma 描述符就必须按 128 字节 swizzled 分块来读。描述符与数据若不一致, Tensor Core 就会读到乱序的操作数。

这是相对 Ampere 的主要转变。swizzle 不再仅隐藏在手写的共享内存索引里。Hopper 把它升格为一等描述符格式。写分块的 TMA 加载与读分块的 wgmma 指令, 都能指名同一种格式。

SMEM 矩阵描述符  $\rightarrow A(M \times K)$  在共享内存中的摆放方式

每个 atom 是一个连续的  $8 \times 128$  B 块——其 8 行在内存中背靠背排列; ldo / sdo 在 atom 之间跳步。  
ldo = 沿主维的步幅, sdo = 沿另一维。图中为 K-major (主维 = K); MN-major 操作数则交换 ldo 与 sdo。

## 1.4.9 Hopper 的输出仍用寄存器

Hopper 改变了输入通路, 但累加器仍住在寄存器里。

一条 wgmma 指令把累加器写入逐线程寄存器片段。片段的确切大小与寄存器数量取决于指令形状, 例如 m64nNk16, 其中 N 改变累加器寄存器的数量。但基本思路与 Ampere 相同: 收尾消费的是一个寄存器片段。

因此 Hopper 拥有混合的布局模型。输入操作数可直接来自共享内存描述符, swizzle 由硬件描述。输出累加器则仍是一个寄存器布局问题。

Blackwell 改变了这一输出侧。

## 1.4.10 Blackwell:tcgen05 与 TMEM

Blackwell 为数据操作数保留了共享内存描述符的思路。A 和 B 仍以 Tensor Core 所期望的布局在共享内存中备好。某些模式还能从 TMEM 读取 A 操作数。

主要变化在累加器。tcgen05.mma 把累加器写入 Tensor Memory (张量内存, TMEM), 而不是让它作为一个长寿命的寄存器片段留存。在计算阶段, 累加器留在 TMEM 中。收尾随后用 tcgen05.ld 把它装回寄存器。

这把输出布局问题从寄存器挪到了 TMEM。内核必须分配 TMEM、选择正确的 TMEM 布局、等待 MMA 完成, 然后用匹配的 tcgen05.ld 通路为收尾恢复累加器片段。

cta\_group::1 与 cta\_group::2 如何把累加器在一个或两个 CTA (协作线程阵列) 之间切分, 其细节在 *Tensor Core:tcgen05* 中讨论。与早期世代差异最大的布局, 是块缩放的缩放因子布局。

### 1.4.11 TMEM 中的缩放因子布局

块缩放 MMA 模式, 例如 mxfp8 与 nvfp4, 增加了缩放因子操作数。除 A 和 B 之外, MMA 还读取:

```
SFA(M, SFK)
SFB(N, SFK)
```

其中 SFK 是 K 缩放块的数量。

数据操作数 A 和 B 住在共享内存中。缩放因子住在 TMEM 中。于是它们拥有不同的搬运通路。TMA 从全局内存加载到共享内存。它不直接加载进 TMEM。因此缩放因子通常分两步搬运:

```
global memory to shared memory with TMA
shared memory to TMEM with tcgen05.cp
```

只有在这次拷贝之后, 缩放因子才进入 tcgen05.mma 所期望读取的内存空间。

TMEM 缩放因子布局使用 TMEM 硬件坐标 Lane 与 Col。在 TIRx 布局记法中, 这两根轴写作 TLane 与 TCol。

一个 128 行的缩放向量被压缩进一个 32 lane 组, 然后在 TMEM 的四个 32 lane 窗口之间复制。在布局记法中, 核心模式是:

```
S[(32, sf_per_mma) : (1@TLane, 1@TCol)] + R[4 : 32@TLane]
```

分片摆放基础的 32 行组:

```
TLane = r
TCol  = s
```

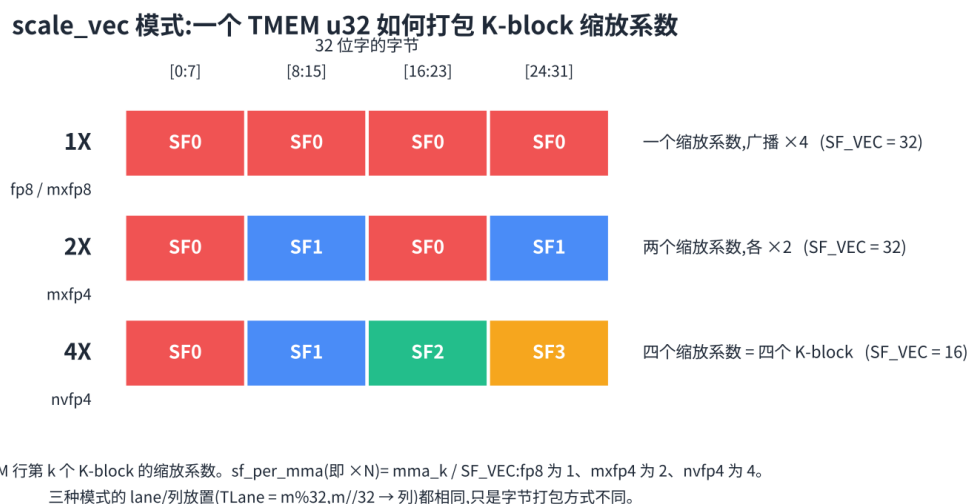
复制项在 lane 偏移 0、32、64、96 处添加副本:

```
TLane = r + 32 * q, where q in {0, 1, 2, 3}
TCol  = s
```

这就是 warpx4 广播模式。同一个压缩缩放因子组在整个 128 lane 的 TMEM 空间中都变得可见。

在 32 位 TCol 单元内部还有字节打包。打包取决于 scale\_vec 模式:

```
1X: one scale value is broadcast across the 32-bit cell
2X: two scale values are packed, each duplicated
4X: four K-block scale values are packed
```



这种打包在 Ampere 或 Hopper 上没有直接的对应物, 因为那两代没有针对 tcgen05 块缩放 MMA 的 TMEM 缩放因子操作数。

在 `cta_group::2` 中, 缩放因子跟随它们所缩放的数据。SFA 缩放 A, 因此它沿 M 在两个 CTA 之间切分, 与每个 CTA 所拥有的 A 行相匹配。SFB 缩放 B, 而 B 为计算的两个 CTA 半体所共享, 因此 SFB 被多播到两个 CTA (*Tensor Core: tcgen05*)。

### 1.4.12 一个反复出现的片段

尽管周围的内存通路在变, 有一种结构不断归来: m8n8 式的寄存器片段。

在 Ampere 上, `ldmatrix` 构建该片段供 `mma.sync` 读取。

在 Hopper 上, `wgmma` 把累加器作为寄存器片段写出来供收尾使用。

在 Blackwell 上, 累加器在计算期间住在 TMEM 中, 但 `tcgen05.ld` 会在收尾处理并存储它之前把它装回成寄存器片段 (*特殊内存: TMEM*)。

因此片段并未消失。它的角色在变。早期世代在整个计算阶段都把累加器放在那里。Blackwell 主要在 TMEM 与收尾的边界处使用它。

### 1.4.13 主线

在 Ampere 上, 内核显式构建 Tensor Core 寄存器片段。共享内存 `swizzle` 主要是内核通过索引算术来承担的责任。

在 Hopper 上, Tensor Core 可通过矩阵描述符直接从共享内存读取操作数。`swizzle` 变成了 TMA 与 `wgmma` 共享的具名描述符格式。

在 Blackwell 上, 输入侧仍使用共享内存操作数, 但累加器搬到了 TMEM。块缩放 MMA 还增加了必须暂存进 TMEM 的缩放因子操作数。

描述符并不消除布局工作。它把契约显式化。内核仍须确保数据搬运通路、内存布局与 Tensor Core 指令三者一致。写 `swizzled SMEM` 分块的 TMA 描述符、读该分块的 MMA 描述符, 以及附着于缓冲区的布局, 三者必须描述同一种物理摆放。

这些部件中只要有任何一处不一致, 硬件照常运行。它只是会读错字节, 或者读得慢。正因如此, 布局不是 Tensor Core 内核之外的装饰。它是指令接口的一部分。

## 1.5 异步数据搬运:TMA

### Overview

- TMA(张量内存加速器) 是一种硬件引擎, 用于在全局显存 (GMEM) 与共享内存 (SMEM) 之间进行异步分块拷贝。由一个线程发起拷贝, 引擎负责搬运字节。
- 一次 TMA 拷贝由一个 `tensor-map` 描述符 (descriptor) 描述。该描述符告知引擎全局张量的形状、步长、分块坐标, 以及共享内存的 `swizzle`(混洗) 模式。
- 在加载路径上,TMA 可以在写入共享内存时对分块进行 `swizzle`, 使分块直接落到 Tensor Core(张量核) 所期望的布局上。
- TMA 加载通过带字节计数追踪的 `mbarrier`(内存屏障) 完成;TMA 存储则使用提交组 (commit group) 与等待组 (wait group)。

只有当数据准备就绪可供消费时,Tensor Core 才能发挥作用。在 GEMM(通用矩阵乘) 或注意力 kernel(内核) 中, 一旦流水线填满, 计算可能成为计算受限的部分 (是什么让内核变快), 但流水线只有当下一个操作数分块及时到达时才能保持填满。

搬运一个分块的旧做法是让线程自行拷贝。每个线程计算地址、从全局显存发起加载, 并把数值写入共享内存。这可行, 但它把 `warp` 指令花在了地址计算与拷贝簿记上, 而不是用在计算上。它还使得拷贝路径暴露在那些本应喂给 Tensor Core 的同一批 `warp` 的指令流中。

Tensor Memory Accelerator, 即 TMA, 把这部分工作搬进了硬件拷贝引擎。一个线程发起一次分块拷贝, 拷贝引擎随后在全局显存与共享内存之间异步搬运一个矩形分块。当引擎在搬运字节时,CTA(协作线程阵列) 的其余部分可以继续做其他工作。

TMA 还处理了布局问题的一部分。Tensor Core 不仅需要共享内存中放正确的数值, 还需要它们处于正确的共享内存布局上。在加载路径上,TMA 可以在写入分块时应用共享内存 `swizzle`。这使得分块可以直接落到后续 MMA(矩阵乘加) 所期望的布局上。

交互演示:TMA 将一个分块从全局显存拷贝到共享内存。切换 `swizzle` 模式, 并将鼠标悬停在某个源单元上, 即可看到它落在共享内存中的位置。

### 1.5.1 一个线程发起, 硬件搬运分块

一次 TMA 拷贝从一个发起线程开始。该线程并不会遍历分块中的所有元素。它向硬件提供一份拷贝描述, 然后由 TMA 引擎执行传输。

主要输入是一个 `tensor-map` 描述符。该描述符描述了全局张量以及应当如何从中读取一个分块。它记录了诸如张量形状、步长、元素大小、分块形状与 `swizzle` 模式等信息。发起线程还需提供分块应当落入的共享内存地址。

指令发起后, 拷贝异步执行。发起线程可以继续执行。CTA 中的其他线程也可以继续执行。此时传输由 TMA 引擎负责, 而不再由一组普通的加载/存储指令循环承担。

这给 kernel 提供了两种不同的方式来表达同一个逻辑操作——「拷贝这个分块」。

一种路径是线程拷贝。线程协作从全局显存加载并写入共享内存。这让 kernel 能对每一次访问进行直接控制, 但会消耗线程指令与寄存器来计算地址。

另一种路径是 TMA 拷贝。由一个线程发起传输, 硬件拷贝引擎执行矩形拷贝。这是大型规整分块 (尤其是 Tensor Core kernel 所使用的操作数分块) 的自然路径。



这两条路径有不同的同步规则和不同的性能表现。在二者之间作出选择是一个派发 (dispatch) 决策。布局告诉 kernel 它想要何种内存排布; 作用域 (scope) 告诉它有哪些线程或 CTA 参与; 派发则决定这次拷贝是由普通线程代码实现, 还是由 TMA 实现。

### 1.5.2 Swizzled 布局

仅搬运分块是不够的。分块还必须以一种 Tensor Core 能够高效读取的布局放入共享内存。

这正是 TMA swizzling 派上用场的地方。当 TMA 把分块写入共享内存时, 它可以对共享内存的地址模式进行置换。全局内存中的分块仍是一个逻辑矩形, 但共享内存中的目标布局可以被 swizzle。

swizzle 模式是 TMA 描述符的一部分。一旦描述符设置好, 发起线程就无需手动应用 swizzle。引擎会在字节落入共享内存时应用它。

关键的要求是一致性。TMA 描述符、共享内存分块布局以及后续的 MMA 指令, 三者必须描述同一个布局 (数据布局及其记法)。如果 TMA 用一种 swizzle 写入分块, 而 MMA 却按另一种来读取它, 硬件仍会一丝不苟地执行被要求做的事——只不过对计算而言, 字节的排布将是错误的。

正是在这一点上, 布局记号不再只是簿记工具。DSL(领域特定语言) 所使用的布局必须与 TMA 描述符和 Tensor Core 指令所使用的硬件布局相匹配。例如, 如果 kernel 声明某个操作数分块以 128 字节 swizzled 布局存储, 那么 TMA 描述符就必须使用匹配的 swizzle 模式, MMA 派发也必须期待同样的共享内存排布。上面的演示允许你在「无 swizzle」与「128 字节 swizzle」之间切换; 将鼠标悬停在某个源元素上, 即可看到应用 swizzle 后它落在哪里。

理解 swizzle 的一个有用方式是: TMA 并没有改变逻辑分块, 它改变的是逻辑元素在物理上落在共享内存的何处。后续 MMA 仍消费同一个逻辑 A 或 B 分块。swizzle 只决定该分块如何分布在共享内存的各 bank 之间。

### 1.5.3 用于分块化与 Swizzling 的 3D TMA

一次普通的 TMA 拷贝搬运的是扁平的 2D 分块, 但 Tensor Core 所期望的共享内存布局通常已被 \* 分块化 (tiled) \* 成 swizzle 原子 (即数据布局及其记法中  $8 \times 128$  字节的原子)。TMA 通过一个额外的描述符维度来处理这一点。一个 3D TMA 把共享内存盒子描述为 (group, row, col), 其中 group 维度跨原子推进, 内部两个维度在单个原子内寻址。于是, 单次 3D 拷贝既逐原子地铺排分块 (分块化), 又在每个原子内部应用 swizzle, 使数据抵达时便已处于 MMA 所期望的布局, 无需单独的分块化或 swizzling 步骤。

交互演示: 一次 3D TMA 拷贝, 以 (group, row, col) 寻址, 分块化地写入 swizzled 共享内存。

选择 swizzle 格式与这种分块化紧密相关。更宽的 swizzle 会把一列打散到更多 bank 上, 因此在能够容纳时 128 字节 swizzle 是默认选择; 但一个 N 字节原子要求分块的连续维度能够填满它。因此, 因形状约束而偏小的分块无法使用 128 字节 swizzle, 必须退而使用 64 字节或 32 字节: 经验法则是选取分块所能填满的最大 swizzle (数据布局及其记法)。下面的演示直接展示了这一约束: 对  $16 \times 16$  分块应用 128 字节 swizzle, 只有在把分块拆分成与原子匹配的  $16 \times 8$  组之后, 才能做到无冲突。

交互演示: 对  $16 \times 16$  分块应用 128 字节 swizzle, 只有分块化为  $16 \times 8$  组后才能做到无冲突。

### 1.5.4 完成: 加载

拷贝是异步的, 因此仅仅发起它是不够的。消费者不能仅仅因为 TMA 指令已发起就去读共享内存中的分块。只有当引擎写完字节之后, 分块才是可安全读取的。

对 TMA 加载而言, 完成信号是一个 mbarrier (异步协调: mbarriers)。

通常的流程是:

1. 为该流水级初始化或复用一个 mbarrier;
2. 告知屏障 TMA 传输预期将写入多少字节;
3. 发起 TMA 加载;
4. 让 TMA 引擎随着字节到达而更新屏障;
5. 让消费者在读共享内存分块之前, 先在屏障相位 (phase) 上等待。

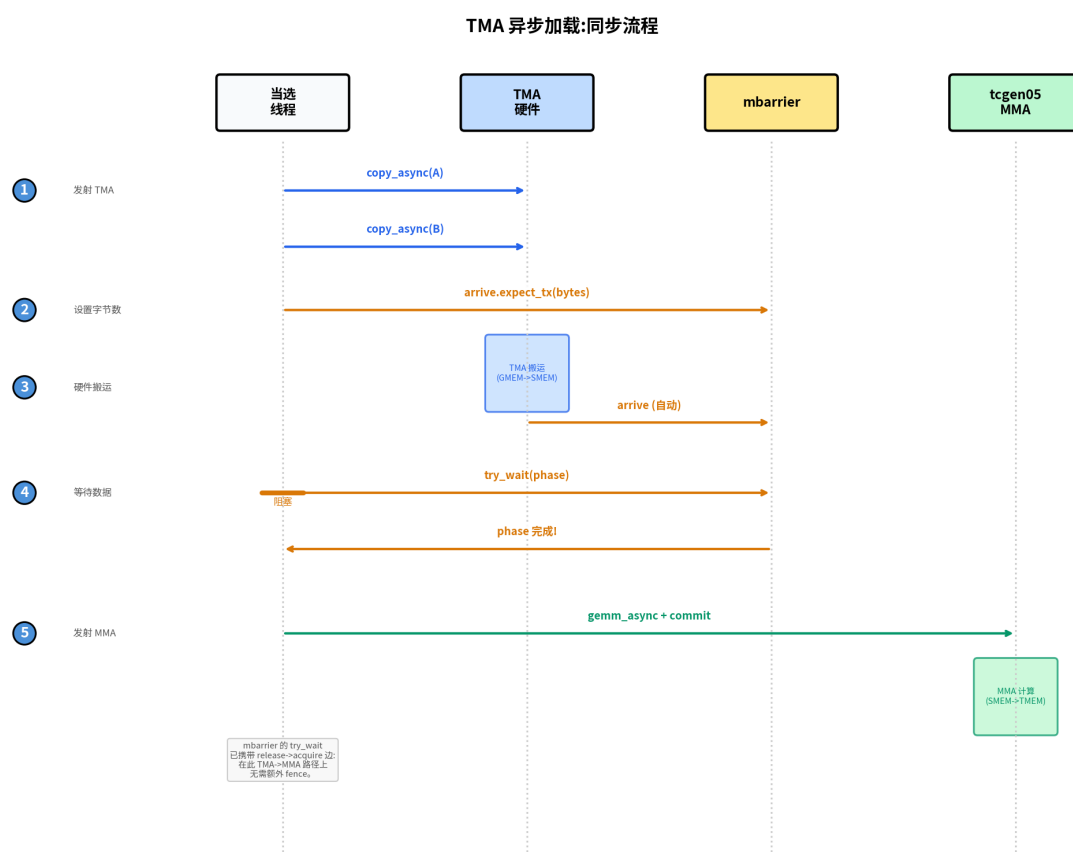
字节数通过如下操作来设置:

```
mbarrier.arrive.expect_tx(bytes)
```

它做两件事: 记录预期传输大小, 同时执行发起线程对屏障的到达 (arrival)。屏障并不会仅仅因为这个调用发生过就完成。它仍要等待 TMA 引擎报告预期的字节已经到达。

随着传输推进, 引擎对屏障执行 complete-tx 更新。屏障相位只有在两个条件都满足时才会翻转: 到达计数满足, 且待处理字节数归零。

随后消费者在该屏障上等待。一旦对预期相位的等待完成, 共享内存分块就准备就绪。此时 MMA 路径就可以安全地读取它了。



这与其他异步生产者-消费者交接所使用的屏障模型是同一个。生产者是 TMA 引擎, 消费者是 MMA 路径或任何读取该共享内存分块的代码。屏障则是它们之间显式的交接点。

### 1.5.5 完成: 存储

TMA 存储按相反方向搬运数据, 从共享内存到全局显存。它们同样是异步的, 但完成机制不同。

TMA 加载通常喂给同一个 kernel 内部的消费者。MMA 路径需要知道共享内存分块何时就绪, 这正是加载路径使用 `mbarrier` 的原因。

TMA 存储通常把最终数据写出至全局显存。往往没有即时的、kernel 内部的消费者在等待被存储的结果。kernel 主要需要知道的是: 何时可以安全地复用共享内存缓冲区, 或者何时可以结束这串存储。

为此, TMA 存储使用提交组与等待组。kernel 发起一个或多个存储, 提交该组, 随后等待该组排空。等待完成后, 从 kernel 视角看该组内的存储已经完成, 被存储所占用的共享内存区域即可安全复用。

所以规则很简单:

```
TMA load: wait through an mbarrier with byte-count tracking
TMA store: wait through a commit group and wait group
```

这两种机制在不同的交接点上服务于同一目的。加载需要让一个共享内存分块对后续消费者可见; 存储则需要在 kernel 复用源存储或依赖该存储已排空之前, 确保这次外出传输已经完成。

### 1.5.6 TMA 对流水化的意义

TMA 在作为流水线一部分时最为有用。一个 kernel 可以在 Tensor Core 计算当前分块的同时, 发起对未来分块的加载。加载在后台运行, 计算在前台运行。当未来的分块变成当前分块时, 屏障把二者衔接起来。

一个典型的 GEMM 循环会反复使用这种结构。共享内存的一段持有当前由 MMA 消费的分块, 另一段正由 TMA 填充。随着循环推进, 这些角色轮换。在 MMA 读取某一段之前, 它先在该段的加载屏障上等待。在 TMA 覆盖某一段之前, kernel 要确保之前的消费者已经用完它。

这就是为什么 TMA 与 `mbarrier` 通常一起出现在 Blackwell 和 Hopper 风格的 kernel 中。TMA 给 kernel 提供了一个异步拷贝引擎, 屏障则给 kernel 提供了一种精确获知被拷贝字节何时就绪的方式。

## 1.6 Tensor Core: tcgen05

### Overview

- `tcgen05` 是 Blackwell 的 Tensor Core(张量核) 指令族。其 MMA(矩阵乘加) 指令以协作方式完成分块矩阵乘加, 指令由一个被选中的 `thread`(线程) 提交。
- 累加器存放在 TMEM(张量内存) 而非寄存器中。收尾阶段随后通过 `tcgen05.ld` 把它读回寄存器。
- `cta_group::1` 与 `cta_group::2` 控制一次 MMA 由一个 CTA(协作线程阵列) 还是两个 CTA 协作完成。该选择还会改变 M 维度到 TMEM 的映射方式。
- 块缩放 (block-scaled) MMA 模式 (例如 `mxfp8` 和 `nvfp4`) 会增加缩放因子操作数。数据操作数位于 SMEM(共享显存), 而缩放因子则经 TMEM 暂存。

稠密线性代数是现代 GPU 完成大部分有效计算的地方。普通的 CUDA core(核) 矩阵乘无法逼近芯片标称的峰值 ([GPU 执行模型](#))。快速的 GEMM(通用矩阵乘) 与 attention(注意力) 内核通过以正确的分块形状、布局 and 同步方式喂给 Tensor Core 来达到该峰值。

其基本思想自 Volta 起并未改变。Tensor Core 消费矩阵分块, 把它们相乘并累加结果。代与代之间变化的是: 指令如何派发、操作数如何布局、以及累加器存放在何处。

Blackwell 对最后这一点做了大幅改动。tcgen05 的累加器不再作为长寿命的寄存器片段保留, 而是写入 Tensor Memory, 即 TMEM([特殊内存:TMEM](#))。这一改动影响整个内核。MMA 写入 TMEM。完成情况以异步方式跟踪。收尾阶段随后把累加器从 TMEM 加载出来, 并转换回它进行类型转换与存储所需的寄存器片段。

本章聚焦于计算指令本身。TMA([异步数据搬运:TMA](#)) 负责把操作数搬入 SMEM。TMEM 负责保存累加器以及部分缩放因子操作数。tcgen05.mma 则是介于这两次访存动作之间的 Tensor Core 操作。

交互演示:tcgen05 累加器行为。切换 A 或 B 的转置, 选择输出宽度 N, 并逐步推进 K 次迭代, 观察部分和在 TMEM 中累积的过程。

## 1.6.1 tcgen05 MMA

tcgen05 MMA 是 Blackwell 的 Tensor Core 矩阵乘加指令。它是一条协作指令。其工作针对一个 warpgroup(线程束组) 完成, 在某些模式下还可涉及同一 cluster(集群) 中的两个 CTA。该指令并非由每个 thread 独立派发, 而是由一个被选中的 thread 代表参与组提交该操作。

把 MMA 拆成三个问题来看会更容易理解。

第一个问题是谁在协作。普通模式使用一个 CTA, 写作 `cta_group::1`。更大的模式使用 cluster 中的两个 CTA, 写作 `cta_group::2`。两种情形下, 该指令都代表针对一个分块的一次 Tensor Core 操作, 而不是某个 thread 的一次标量操作。

第二个问题是操作数与结果存放在哪里。数据操作数通常位于 SMEM。某些变体也可以从 TMEM 读入 A 操作数。累加器写入 TMEM。操作数布局必须与 Tensor Core 期望的布局一致, 包括数据操作数所使用的经过 swizzle(混排) 的共享显存布局 ([数据布局及其记法](#))。

第三个问题是如何观察到完成。tcgen05.mma 是异步的。派发 MMA 并不意味着乘加已经完成。该指令在操作提交后即返回, 而 Tensor Core 继续运行。内核通过一个 commit group(提交组) 和一道 mbarrier(内存屏障) 来获知结果何时就绪 ([异步协调:mbarriers](#))。

正是这种异步行为使重叠成为可能。一个快速的内核不会派发一条 MMA 后立即停顿直到它完成。它可以派发 MMA、开始准备后续的分块, 并只在真正需要结果时才等待。代价是每一次交接都必须显式完成。如果收尾阶段在 MMA 完成屏障触发之前读取 TMEM, 那就是读得太早了。

## 1.6.2 累加器位于 TMEM

在 Ampere 和 Hopper 上, 累加器以寄存器形式暴露给程序。MMA 产生一个按 lane(通道) 划分的寄存器片段, 收尾阶段直接消费该片段。这种方式简单, 但累加器的大小被绑定到每个 thread 的寄存器预算上。

Blackwell 打破了这一绑定。tcgen05.mma 把其累加器写入 TMEM——一种 Blackwell 上作用域为 CTA 的存储空间。累加器可以在计算阶段一直留在 TMEM 中, 收尾阶段随后用 tcgen05.ld 把它加载回寄存器。

这改变了内核的形态。寄存器片段在边界处仍然重要。收尾阶段仍然需要寄存器, 以便进行类型转换、逐元素运算以及存储结果。但长寿命的累加器状态不再是寄存器分配问题, 而是一个 TMEM 分配与布局问题 ([特殊内存:TMEM](#))。

这就是为什么必须把 `tcgen05` 与 `TMEM` 放在一起理解。`MMA` 指令决定计算哪个分块。`TMEM` 决定累加器落在何处。收尾阶段必须使用与之匹配的加载路径, 才能以它期望的寄存器布局恢复出累加器。

### 1.6.3 `cta_group::1` 与 `cta_group::2`

`tcgen05 MMA` 既可在 `cta_group::1` 模式下运行, 也可在 `cta_group::2` 模式下运行。

在 `cta_group::1` 中, 一个 CTA 独占该 `MMA`。其操作数位于该 CTA 的 `SMEM`, 其累加器写入该 CTA 的 `TMEM`。

在 `cta_group::2` 中, `cluster` 内的两个 CTA 协作完成一个 `MMA` 分块。每个 CTA 有自己的 `SMEM` 和自己的 `TMEM`。累加器并不存储在一个横跨两个 CTA 的物理 `TMEM` 区域里, 而是被拆分到两个 CTA 之间, 每个 CTA 各持有一部分。偶数 CTA 负责派发指令并为这对 CTA 提交完成屏障。

这一选择很关键, 因为它改变了逻辑累加器分块  $C(M, N)$  到 `TMEM` 的映射方式。`TMEM` 有 128 条硬件 Lane 行以及最多 512 条硬件 Col 列。在 `TIRx` 的布局记法中, 这些轴写作 `TLane` 与 `TCol`。`MMA` 模式决定  $C$  的行与列如何被放置到这些 `TMEM` 轴上。

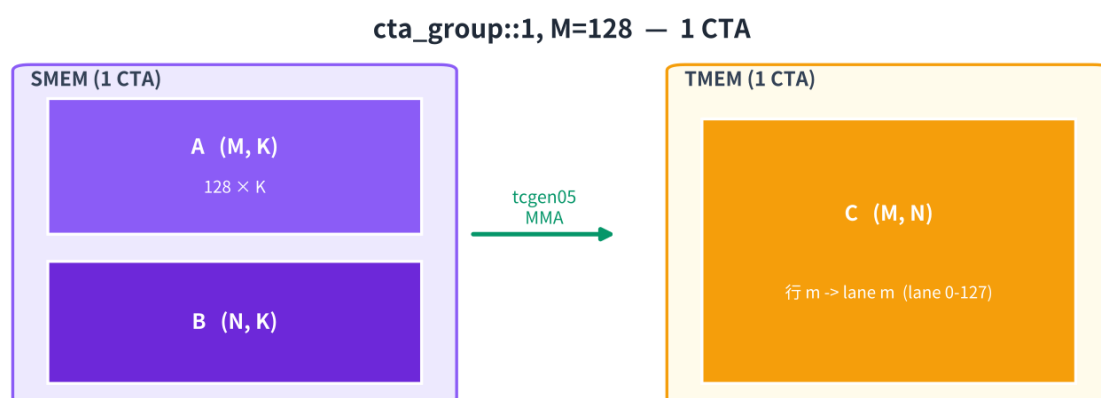
有四种值得记住的有用情形。

下面的图遵循演示的配色约定: 紫色标记 `SMEM` 操作数, 橙色标记 `TMEM` 累加器状态, 绿色标记 `Tensor Core MMA` 路径。CTA 的身份通过标签和位置来区分, 而不是改变这些硬件颜色。

#### `cta_group::1, M = 128`

这是最简单的情形。一个 CTA 计算一个 128 行的分块。`TMEM` 也有 128 条 Lane 行。因此映射是直接的: 累加器的第  $m$  行映射到 Lane  $m$ , 而  $N$  维度映射到 `TMEM` 列。

结果填满 128 条 Lane 行乘  $N$  条 Col 列。这是基准图景。该 CTA 在 `SMEM` 中持有  $A$  和  $B$ , 并在其 `TMEM` 中持有完整的累加器分块。



#### `cta_group::1, M = 64`

当  $M = 64$  时, 累加器只有 64 行, 但 `TMEM` 仍有 128 条 Lane 行。硬件并不会简单地把第 0 到 63 行装进 Lane 0 到 63, 而是把它们分散到 128 条 Lane 上, 排成四段、每段 16 行。

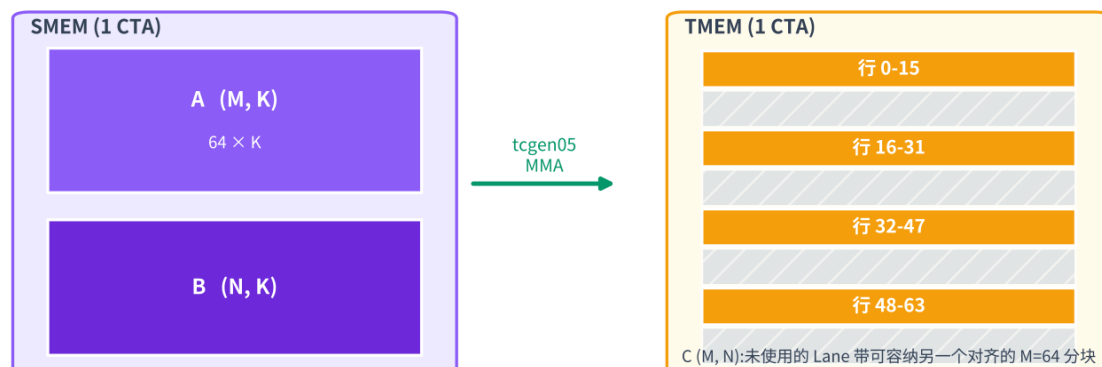
第 0 到 15 行进入 Lane 0 到 15。第 16 到 31 行进入 Lane 32 到 47。第 32 到 47 行进入 Lane 64 到 79。第 48 到 63 行进入 Lane 96 到 111。



这样在 Lane 16 到 31、48 到 63、80 到 95 以及 112 到 127 处留下了空隙。这些空隙是有意为之。在另一种 Lane 对齐方式下, 另一个独立的  $M = 64$  MMA 可以占据互补的 Lane。这使得两个较小的  $M$  分块可以共享 128 条 Lane 的 TMEM 结构而互不踩踏。

$N$  维度仍然映射到 TMEM 列。不寻常之处仅在于  $M$  行在 Lane 上的排布方式。

cta\_group::1,  $M=64$  — 1 CTA



cta\_group::2,  $M = 256$

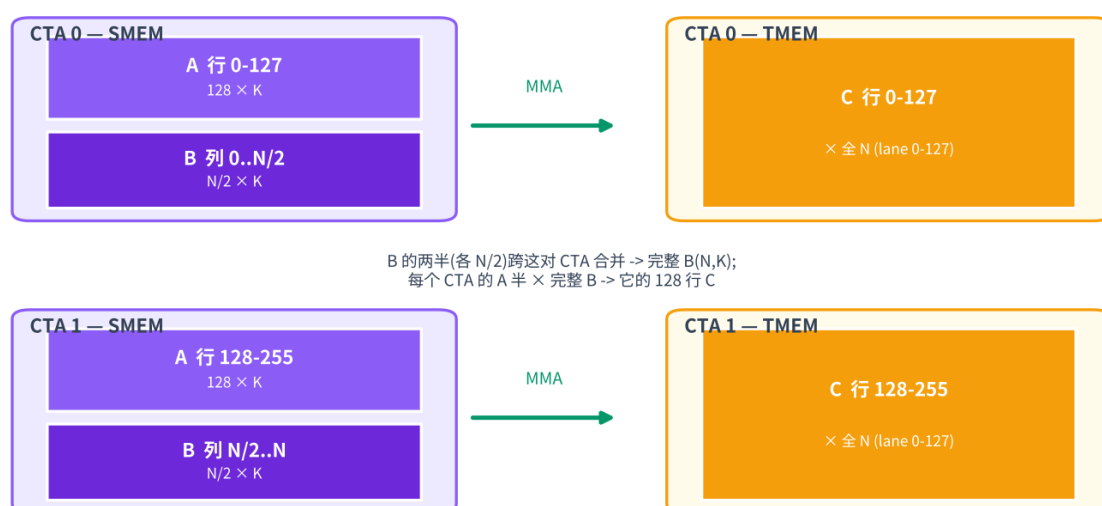
当  $M$  维度大于一个 CTA 所能自然容纳的大小时, MMA 可以使用 `cta_group::2`。对于  $M = 256$ , 拆分是直接的。CTA 0 持有第 0 到 127 行。CTA 1 持有第 128 到 255 行。

每个 CTA 使用自己的 TMEM Lane 第 0 到 127 行以及全部  $N$  列。在物理上, 这是两个独立的 128 行 TMEM 区域, 分别位于每个 CTA 中。在逻辑上, 它们组成一个 256 乘  $N$  的累加器分块。

每个 CTA 还提供与其  $M$  行对应的那部分  $A$ 。  $B$  按模式要求对两个 CTA 都可见。偶数 CTA 负责派发 MMA 并为这对 CTA 提交完成屏障。

这就是用线程束特化与集群扩展 *GEMM* 中双 CTA cluster GEMM 所使用的模式。

cta\_group::2,  $M=256$  — 2 个 CTA ( $M$  划分)



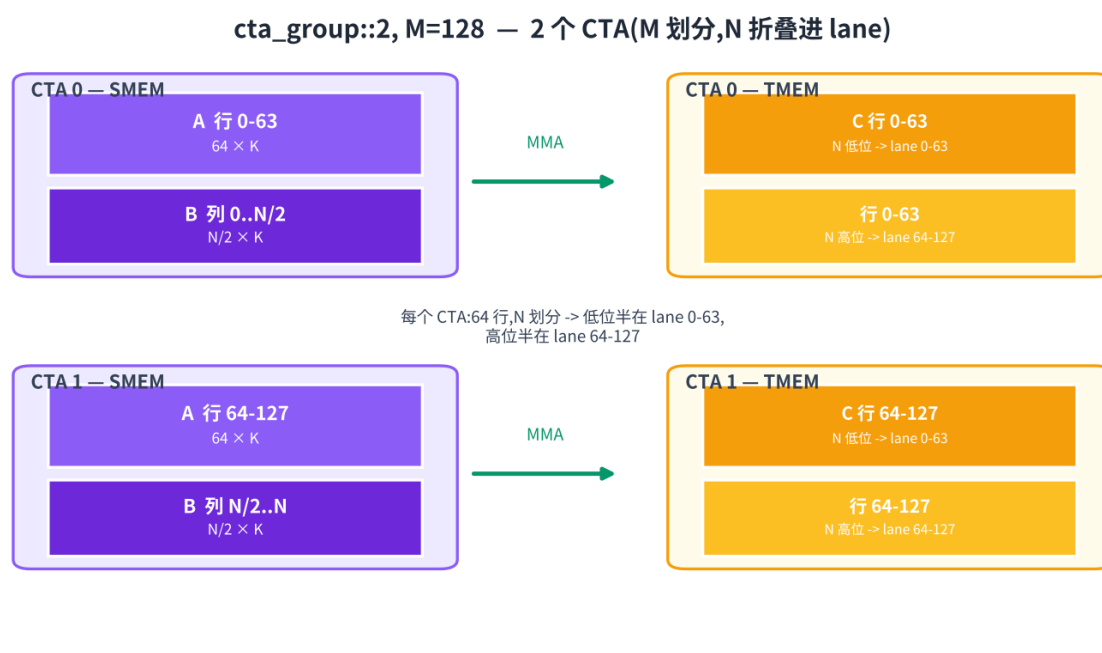


**cta\_group::2, M = 128**

cta\_group::2, M = 128 模式仍然使用两个 CTA, 但 M 维度更短。由于总共只有 128 行, 每个 CTA 分得 64 行 M。

剩余的 lane 容量用于打包 N 维度。在每个 CTA 内部, N 的一半占据 Lane 0 到 63, 另一半占据 Lane 64 到 127。这样每个 CTA 即便只拥有 64 行 M, 也能用满全部 128 条 Lane 行。

因此该拆分包含两部分。M 跨这对 CTA 拆分, 每个 CTA 64 行。N 则在每个 CTA 内部, 沿 TMEM Lane 行的下半部分和上半部分再各拆一次。



在这些模式中, 原理是相同的。tcgen05.mma 计算的是一个逻辑累加器分块, 但该分块必须被放置到物理上的 128 Lane 乘最多 512 Col 的 TMEM 空间中。模式与 M 形状决定了该放置方式。内核其余部分在随后把累加器读回时, 必须使用同样的映射。

对于本书中的内核, TMEM 中的累加器通常是 f32。这是常见的高精度路径。它并非唯一的累加器类型。kind::f16 路径可以在 f16 下累加。

**1.6.4 操作数放置**

对于稠密 MMA 模式, A 和 B 在 MMA 运行之前被准备在 SMEM 中。TMA 负责把全局显存分块搬入 SMEM。内核按照 Tensor Core 期望的布局 (包括任何必需的 swizzle) 来安排这些 SMEM 分块。

累加器 C 写入 TMEM。这是与早前几代的主要区别。收尾阶段不会把累加器直接当作 MMA 指令的输出接收, 而是必须显式地用 tcgen05.ld 从 TMEM 加载。

在 cta\_group::1 中, 一个 CTA 提供操作数并独占累加器。在 cta\_group::2 中, 每个 CTA 从自己的 SMEM 提供自己一侧的操作数, 并且每个 CTA 拥有自己那份 TMEM 中的累加器。当 A 按 M 拆分时, 每个 CTA 保留与自身 M 切片对应的 A 行。B 按模式共享, 因为两个 M 切片都要与同一个 N 乘 K 的分块相乘。

阅读内核时这种区分很重要。SMEM 放置回答的是 Tensor Core 如何读 A 与 B。TMEM 放置回答的是累加器落在何处。这两种布局通过 MMA 模式相互关联, 但它们不是同一个存储空间, 不能视为可互换。

### 1.6.5 块缩放 MMA

稠密模式直接从 SMEM 读入其数据操作数, 并累加到 TMEM 中。块缩放 MMA 额外增加了两个操作数: A 和 B 的缩放因子张量。

这用于 mxfp8 和 nvfp4 这类极低精度格式。低精度格式效率高, 但其动态范围小。单一的 global scale(全局缩放) 通常过于粗糙。如果按最大值选取 scale(缩放), 较小值会损失精度; 如果按小值选取 scale, 较大值可能会被截断。

块缩放通过为小的 K 块分配缩放因子来解决这个问题。一组连续的 K 元素共享一个 scale。MMA 在概念上先用各自的 scale 对每个块反量化, 再在累加器类型中累加乘积。

对 A 和 B 而言, 这引入了两个缩放因子张量:

```
SFA(M, SFK)
SFB(N, SFK)
```

其中  $SFK = K / B$ , B 是沿 K 的块大小。

确切的块大小取决于格式。要点在于缩放轴以更粗的粒度跟随 K。每个缩放因子描述的是一整块 K 值, 而不是单个元素, 也不是整张矩阵。

其数学形式为:

```
acc += (Aq * scale_a) * (Bq * scale_b)
```

其中 Aq 与 Bq 是量化后的低精度值, scale 在累加之前恢复它们的近似量级。

缩放因子的 dtype(数据类型) 也很重要。使用 e8m0 scale 时, 每个 scale 实际上是 2 的幂。使用 e4m3 scale 时 (nvfp4 即如此), scale 是一个小的浮点值, 可以表示介于相邻 2 的幂之间的值。

### 1.6.6 缩放因子存放在何处

块缩放 tcgen05.mma 与稠密 MMA 有一处重要的放置规则差异: 缩放因子从 TMEM 读取。

数据操作数 A 和 B 仍然暂存在 SMEM 中。缩放因子 SFA 和 SFB 则经 TMEM 暂存。由于 TMA 加载的目标是 SMEM, 缩放因子通常需要额外一步。内核先把它们加载到 SMEM, 再用 tcgen05.cp 从 SMEM 拷贝到 TMEM。只有当缩放因子进入 TMEM 之后, 块缩放 MMA 才能读取它们。

这就给缩放因子赋予了与数据操作数不同的移动路径:

```
A, B:      全局显存到 SMEM, 然后 MMA 读 SMEM
SFA, SFB:  全局显存到 SMEM, 然后 tcgen05.cp 把 SMEM 拷到 TMEM, 然后 MMA
            ↪ 读 TMEM
```

缩放因子的 TMEM 布局很紧凑。一个 128 行的 scale 向量可以打包进 32 条 Lane 行, 使用基于  $r \% 32$  决定 lane 位置、 $r / 32$  沿列排布的映射。随后数据可以广播到读取完整 128 Lane 空间的四个 warp 上 (跨 GPU 世代的 Tensor Core 操作数布局)。

这是一个说明为何 TMEM 布局必须显式的恰当例子。累加器布局与缩放因子布局都在 TMEM 中, 但它们并不是同一种布局。累加器使用 MMA 输出映射; 缩放因子使用块缩放 MMA 所期望的紧凑布局。

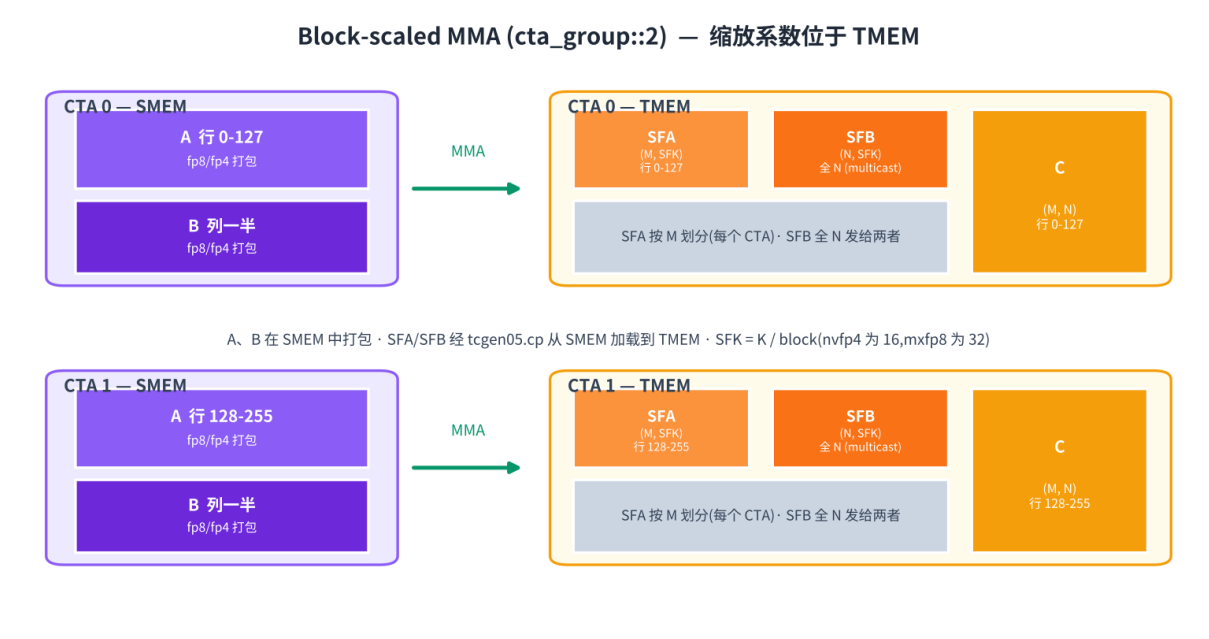
### 1.6.7 cta\_group::2 中的缩放因子

在双 CTA 情形下, 缩放因子跟随它们所缩放的数据。

SFA 缩放 A。由于 A 按 M 拆分到这对 CTA 之间, SFA 也按 M 拆分。每个 CTA 持有与自身 A 行对应的 SFA 行。

SFB 缩放 B。由于两个 CTA 都要与同一个 B 分块相乘, SFB 必须对两个 CTA 都可见。在实践中, 这意味着 SFB 在这对 CTA 之间以 multicast(组播) 方式传送。

这正是块缩放 cluster GEMM 中常见加载模式的来源。SFA 按 CTA 各自加载, 使用该 CTA 自身 M 切片的掩码。SFB 则广播给这对 CTA, 因为两个 CTA 都需要相同的 N 侧缩放因子。



### 1.6.8 保持各 MMA 契约相互匹配

一个 Blackwell GEMM 分块要穿过若干条专用路径。

TMA 把 A 和 B 从全局显存搬入 SMEM。对于块缩放模式, 它还把缩放因子搬入 SMEM。tcgen05.cp 在需要时把这些缩放因子搬入 TMEM。tcgen05.mma 读入其操作数, 在 Tensor Core 上异步运行, 并累加到 TMEM。完成屏障告诉内核累加器何时就绪。收尾阶段随后用 tcgen05.ld 把累加器从 TMEM 加载回寄存器, 并存储最终输出。

跨越这些路径, 内核必须保持三处契约相互匹配: SMEM 操作数布局、TMEM 累加器或缩放因子布局、以及让下一个消费者得以安全运行的异步完成信号。

## 1.7 特殊内存: TMEM

### Overview

- TMEM(张量内存) 是一种仅 Blackwell 才有的内存空间, 由 tcgen05 使用。它是每个 SM(流式多处理器) 上的二维便笺存储, 有 128 个 Lane 行、最多 512 个 Col 列。
- tcgen05.mma 将其累加器写入 TMEM。块缩放 MMA 还使用 TMEM 存放缩放因子。

- TMEM 通过 Lane 和 Col 来寻址。在 TIRx 的布局记法中, 这两条硬件坐标轴写作 TLane 和 TCol。
- TMEM 不像寄存器那样被分配。内核必须自行分配并显式释放, 分配单位为 32 列。
- 普通的共享内存加载与存储无法访问 TMEM。数据在 TMEM、寄存器与共享内存之间通过专用的异步 `tcgen05` 指令流动。

在 Hopper 及更早的 GPU 上, Tensor Core(张量核, 见 *Tensor Core: tcgen05*) 的累加器位于寄存器中。这种模型很容易推理: MMA(矩阵乘加) 指令产生一个寄存器片段, 内核在计算阶段保持该片段有效, 收尾阶段随后读取它、做类型转换并存储结果。

问题在于寄存器压力。寄存器是按线程固定的资源。随着 MMA 分块变大, 累加器片段也随之变大。到一定程度后, 累加器会挤占该线程需要持有的其它值。更大的分块有利于 Tensor Core 的吞吐量, 但把整个累加器都留在寄存器中, 会让这些更大的分块更难使用。

Blackwell 改变了数据通路上的这一环节。tcgen05 的累加器不必在整个计算阶段都留在寄存器中。相反, tcgen05.mma 将累加器写入张量内存, 即 TMEM。TMEM 是一种早期 NVIDIA GPU 不具备的内存空间。它是 SM 上的一个二维便笺存储, 形状为 128 个 Lane 行乘最多 512 个 Col 列, 作用域限定在使用它的 CTA(协作线程阵列) 内。

这一额外的内存空间让 Blackwell 能够支持更大的 Tensor Core 分块, 而不必把整个累加器塞进每线程的寄存器中。但 TMEM 并不像寄存器那样是自动的。编译器不会简单地把它当作普通寄存器存储来发放。内核必须分配 TMEM、用正确的布局对其寻址、用正确的指令搬入搬出数据, 并在 CTA 完成时释放它。

### 1.7.1 二维地址空间

TMEM 不是一维的字节数组。它是一个二维地址空间。硬件将这两个坐标命名为 Lane 和 Col。共有 128 个 Lane 行、最多 512 个 Col 列。每个 Col 是一个 32 位的列。

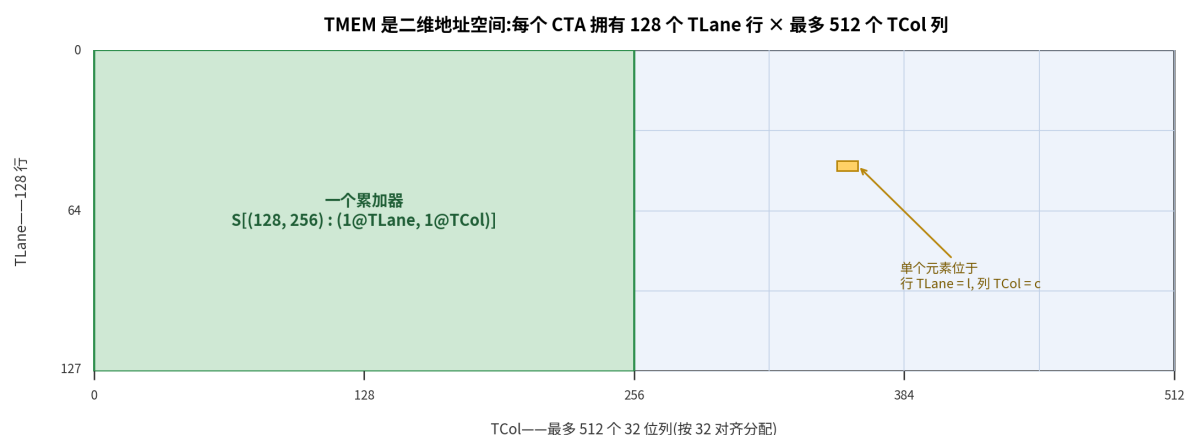
这个形状很重要, 因为 tcgen05.mma 正是按照这种二维结构把累加器写入 TMEM 的。一个 TMEM 位置由一个 Lane 坐标和一个 Col 坐标共同描述, 而不是像共享内存那样用一个字节偏移量描述。

当内核在 TIRx 中声明一个 TMEM 缓冲区时, 它会将该缓冲区在这两条硬件坐标上指定一个布局。在布局记法(见 *数据布局及其记法*)中, 我们把 TMEM 的 Lane 轴写作 TLane, 把 TMEM 的 Col 轴写作 TCol。这些命名并不旨在取代官方硬件术语。它们只是布局轴的名称, 用以在 DSL(领域特定语言) 内部把 TMEM 的各维度显式化。

例如, 一个累加器分块可以写作:

```
S[(128, N) : (1@TLane, 1@TCol)]
```

这表示该分块沿硬件 Lane 维度有 128 行, 沿硬件 Col 维度有 N 列。在布局记法中, 这两个维度表现为 TLane 和 TCol。该布局是直接的: 相邻行沿 TLane 推进, 相邻列沿 TCol 推进。下图展示了这一网格, 硬件 Lane 沿 128 行向下排列, 硬件 Col 沿各列横向排列。



要点在于:TMEM 是分块布局叙事的一部分。它不仅仅是 Tensor Core 的一个隐藏后备存储。内核必须为这块内存命名、从中分配列,并使用一种与 `tcgen05` 指令读写该内存方式相匹配的布局。

## 1.7.2 分配

内核在使用 TMEM 之前,必须先在其中预留空间。这与寄存器不同。寄存器由编译器分配,而 TMEM 由内核显式分配。

分配以 CTA 为单位进行。CTA 中的某一个 warp 申请一段 TMEM 列。申请以 32 列为单位,且请求的列数会按硬件分配规则向上取整。分配之后,该 CTA 会获得一个 TMEM 基地址。随后的 `tcgen05` 指令用该基地址访问已预留的区域。

把 TMEM 看作一种按预算分配的 CTA 资源是很有益的,这与共享内存很相似。CTA 拥有它所分配到的 TMEM 列。内核自行决定它需要多少列用于累加器、缩放因子或临时中转。当该 CTA 完成时,必须释放所分配的区域。

这使得 TMEM 成为内核资源规划的一部分。更大的累加器分块或许能提升 Tensor Core 吞吐量,但会消耗更多 TMEM 列。块缩放 MMA 可能还需要额外的 TMEM 空间存放缩放因子。内核必须把这些用途安置在可用的 TMEM 预算之内,正如它必须把共享内存缓冲区安置在 SMEM 预算之内一样。

## 1.7.3 读写 TMEM

普通的 `ld.shared` 和 `st.shared` 指令无法访问 TMEM。TMEM 是一个独立的地址空间,因此数据通过专用的 `tcgen05` 指令流动。

共有三条主要通路。

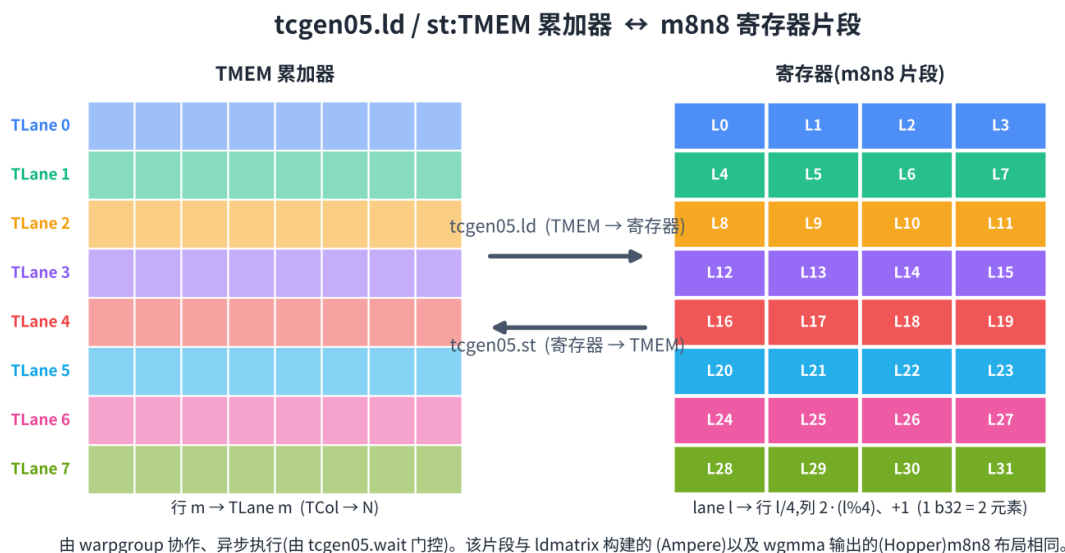
第一条通路是 `tcgen05.ld`,它把数据从 TMEM 加载到寄存器。这是收尾阶段在 MMA 阶段之后使用的通路。累加器已经在 TMEM 中产出,但收尾阶段通常需要一个寄存器片段,以便做类型转换、施加逐元素运算并存储最终结果。

在 DSL 层面,一次 TMEM 加载会分布到一个 `warpgroup` 上,并降级为四个 warp 级别的 `tcgen05.ld` 操作,每个 warp 一个。每个 warp 处理 128 个 TMEM Lane 行中的 32 行,因此四个 warp 合起来覆盖完整的 Lane 维度。在布局记法中,这一完整维度就是 T lane 轴。

该指令本身来自一组加载形状族,例如 `.16x64b`、`.16x128b`、`.16x256b`、`.32x32b` 和 `.16x32bx2`,重复因子从 `.x1` 到 `.x128`。所选形状决定了读取多少 TMEM 列,以及每个线程收到多少寄存器。



重要的结果是寄存器片段布局。对于常见的收尾通路, lane  $l$  接收来自 TMEM 第  $l / 4$  行以及两列的值。这会产生与早期几代从 MMA 直接暴露出来的那种每 lane 累加器片段相同的结构 (见跨 GPU 世代的 *Tensor Core* 操作数布局)。这种延续性很重要。它意味着 Blackwell 的收尾阶段可以复用此前已用于 Ampere *mma* 或 Hopper *wgmma* 的同一种寄存器级类型转换与存储结构, 即便累加器在计算阶段是存放在 TMEM 中的。



第二条通路是 `tcgen05.st`, 它把数据从寄存器存回 TMEM。这是 `tcgen05.ld` 的反方向。它用于某个线程已经持有一个寄存器片段、并需要将其放入 TMEM 的情形。例如, 某些操作数或中间值可能先经过寄存器中转, 再写入 TMEM 以供后续的 `tcgen05` 操作使用。

第三条通路是 `tcgen05.cp`, 它把数据从共享内存拷贝进 TMEM。这是一条批量拷贝通路, 常用于块缩放 MMA 中的缩放因子。在此情形下, TMA(张量内存加速器) 或普通线程代码先把缩放数据准备到共享内存中, 然后 `tcgen05.cp` 将其搬入 Tensor Core 所期望的 TMEM 布局。

这三条通路都是异步的。一条 `tcgen05.ld`、`tcgen05.st` 或 `tcgen05.cp` 指令可以在数据搬运尚未完成时就返回。因此, 内核在消费结果或复用存储之前, 必须使用正确的完成机制 (见异步协调: *mbarriers*)。

等待通路依指令而定。`tcgen05.ld` 通过 `tcgen05.wait::ld` 完成。`tcgen05.st` 通过 `tcgen05.wait::st` 完成。`tcgen05.cp` 与 `tcgen05.mma` 一样, 通过一个 `commit group` 和一个 `mbarrier`(内存屏障) 完成。如果数据从一组线程传递给另一组线程, 内核可能还需要加入栅栏 (fence), 以便接收方线程按预期顺序看到已完成的写入。

TMEM 位于 Blackwell Tensor Core 数据通路的中间。TMA 把操作数搬入共享内存。`tcgen05.mma` 读取其操作数并累加进 TMEM。对于块缩放 MMA, 缩放因子也可以被搬入 TMEM。计算阶段结束后, `tcgen05.ld` 把累加器带回寄存器, 收尾阶段再做类型转换并存储最终输出。

## 1.8 异步协调:mbarriers

### Overview

- TMA 与 Tensor Core 都是异步的, 因此派发工作并不等于完成工作, 消费者需要一个显式的完成信号。



- `mbarrier`(内存屏障)就是这个信号:生产者到达,消费者等待,它跟踪到达计数以及(对 TMA 而言的)字节计数。
- 每个屏障都带有一个 *phase*(相位),每一轮都会翻转;在正确的 *phase* 上等待,正是安全放行消费者的关键。

TMA(异步数据搬运:TMA)与 Tensor Core(*Tensor Core:tcgen05*)操作是异步的。当内核派发一个 TMA 加载或一个 `tcgen05 MMA` 时,发起线程并不会等待操作完成。指令只是被提交给硬件引擎;真正的数据搬运或矩阵运算会与程序的其余部分并行进行。

这很有用,因为它让数据搬运与计算得以重叠。这也意味着仅靠程序顺序不足以证明数据已经就绪。一条靠后的指令可能会在更早的异步操作完成之前就执行。如果 TMA 仍在写入一个共享内存分块时 MMA 就开始读取它,MMA 会读到不完整的数据。如果收尾(*epilogue*)在 Tensor Core 写完累加器之前就去读 TMEM,它会读到错误的值。如果内核在错误的条件上等待,它可能永远无法推进。

因此,内核在每一次异步交接处都需要一个显式的完成信号。`mbarrier`就是这个信号。生产者在工作完成时到达屏障,消费者在使用生产出的数据之前在该屏障上等待。同一套机制既用于 TMA 到 MMA 的交接,也用于 MMA 到收尾的交接,以及流水级之间的缓冲复用。

屏障不只是一个一次性标志。它带有一个相位位(*phase bit*),而每当屏障完成一轮到达时这个相位位就会改变。正是 *phase* 让同一个屏障能够在多次循环迭代中复用,而不会把一次迭代的完成与另一次迭代的完成混淆。

### 1.8.1 mbarrier

`mbarrier`,即 `memory barrier`(内存屏障)的简写,是存储在共享内存中的硬件同步对象。在概念上,它包含两部分状态:一个到达计数器和一个相位位。计数器告诉屏障当前这一轮还缺少多少次到达。相位位告诉内核屏障当前处于哪一轮。

互动:一个 `mbarrier` 状态视图,展示到达计数器、相位位以及 *init*、*arrive*、*wait* 操作;点击某个字段即可聚焦它。

屏障以初始化开始。在 *init* 期间,内核设置这个屏障应当期望多少次到达。屏障以 *phase 0* 起始,其计数器装载为那个期望的到达计数。从那一刻起,屏障就在等待某个资源的全部所需生产者或使用者报告它们已完成。

一次到达会减少屏障仍在等待的工作量。内核的不同部分可以以不同方式在屏障上到达,而这种区分很重要。

对于 TMA 加载,通常的到达路径是 *tx-count*(按传输计数)到达。诸如 `mbarrier.arrive.expect_tx(bytes)` 这样的操作做了两件事。第一,它算作发起线程在该屏障上的一次到达。第二,它记录下 TMA 引擎预期要传输的字节数。屏障并不会仅仅因为发起线程已经到达就完成。它还要等待 TMA 引擎在传输完成时把那个字节计数耗尽。只有在两个条件都满足时 *phase* 才会翻转:常规到达计数已经归零,且待处理的 *tx* 字节计数已经归零。

这正是为什么 `expect_tx` 不应被理解为「多一次普通到达」。它为这次异步拷贝设定了一个字节预算。硬件随后通过 `complete_tx` 更新来核对实际拷贝的完成情况。只有当到达与字节传输两者都已完成时,屏障才完成。

对于 Tensor Core 的工作,到达路径有所不同。一个 `tcgen05 MMA` 并不会仅仅因为 MMA 被派发了就自动推进屏障。内核必须显式地把一次屏障到达挂接到提交路径上,例如通过 `tcgen05.commit.mbarrier::arrive` 操作。当那个被提交的组完成时,Tensor Core 一侧会执行这次屏障到达。如果内核忘记那次提交到达,在该屏障上等待的消费者将永远等待下去。

普通线程也可以直接在一个屏障上到达。这用在普通线程代码充当生产者的时候, 或者用在一组线程宣告它已经用完某个资源的时候。例如, 在一个消费者读完一个共享内存缓冲后, 它可以在屏障上到达, 以此告诉生产者该缓冲已可复用。

等待是同一协议中消费者一侧行为。消费者会等待, 直到屏障完成了当前迭代所期望的 `phase`。只有到那时, 读取数据或复用该屏障所保护的资源才是安全的。

关键在于: 异步硬件不仅会跑在程序前面; 它也会通过屏障把完成情况回报回来。TMA 可以发出信号, 表示一个共享内存分块已就绪。Tensor Core 的工作可以发出信号, 表示 TMEM 的结果已就绪。普通线程可以发出信号, 表示一个缓冲已不再被使用。屏障赋予所有这些情形以同一种生产者-消费者形态: 生产者到达, 消费者等待。

### 1.8.2 Phase 跟踪

一个屏障通常不会只为单次使用而分配。一个流水化的 `K` 循环可能会执行同一次交接数百次, 为每次迭代都分配一个新的共享内存屏障并不现实。相反, 内核保留一小批固定的屏障, 随着循环推进而复用它们。

相位正是让这种复用得以安全的关键。

互动: 一个在多次流水线迭代间复用的屏障, 展示相位位在每完成一轮后翻转。

每当屏障完成了当前这一轮的全部到达, 它就翻转 `phase`: `phase 0` 变为 `phase 1`, `phase 1` 变为 `phase 0`, 以此类推。一次 `wait` 操作会检查消费者所期望的 `phase`。那个期望 `phase` 由内核保存在一个寄存器中。在一个 `stage` 成功等待了一轮之后, 内核会在下一次复用该屏障之前切换它本地的 `phase` 值。

这防止了内核把一次旧的完成错当成一次新的完成。假设一个屏障被用于一次 TMA 加载, 并且已经完成。如果下一次循环迭代在不跟踪 `phase` 的情况下复用同一个屏障, 一个消费者可能观察到上一次完成, 从而错误地假定新的加载已就绪。相位位把这两轮区分开来。迭代 0 等待某个 `phase`, 迭代 1 等待相反的 `phase`, 迭代 2 再次等待第一个 `phase`, 该模式如此延续。

在真实的流水线里, 这套簿记通常是按 `stage` 进行的。内核有固定数量的共享内存 `stage`、数量匹配且固定的屏障, 以及一小撮位于寄存器中的 `phase` 值。随着循环推进, 每个逻辑迭代都映射到某个物理 `stage` 上, 而 `phase` 值告诉 `wait` 操作它正在等待的是那个物理屏障的哪一轮。

这就是为什么后面的 GEMM 代码并不需要每个 `K` 分块配一个屏障 (用 TMA 为 GEMM 建立流水线)。它需要的是每个可复用 `stage` 配一个屏障, 再加上 `phase` 跟踪。stage 索引选择共享内存缓冲与屏障。phase 值则把该 `stage` 的当前使用与上一次使用区分开来。

与你的 agent 一試: 给它一个两级流水线, 让它跟踪四次迭代。对每次迭代, 列出 `stage` 索引、本地 `phase` 值、屏障何时翻转, 以及如果在 `stage` 复用之前不切换 `phase` 会出什么问题。

### 1.8.3 同步规则

一旦屏障与 `phase` 机制清晰了, tensor-core 内核中的同步模式就相当机械化了。每当一条路径生产出数据, 或释放出另一条路径将要消费的资源时, 这次交接就必须被显式化。

有三种常见情形。

第一种情形是线程代码为某个异步引擎生产数据。如果线程写共享内存, 而稍后某个 TMA store 或 MMA 指令读取那块共享内存, 内核必须在线程写入对引擎可见之后才能让引擎读取它们。这要求相应的线程级同步或 `fence`。确切的指令取决于交接的作用域, 但原因总是一样的: 在生产线程写完共享内存缓冲之前, 引擎绝不能观察到它。

第二种情形是 TMA 为 MMA 生产数据。一次 TMA 加载异步地填充一个共享内存分块。MMA 路径不能仅仅因为 TMA 指令已被派发就推断该分块已就绪。TMA 操作必须关联到一个 `mbarrier`, 而 MMA 路径必须在读取该分块之前等待那个屏障。

第三种情形是 MMA 为收尾生产数据。一个 `tcgen05` MMA 把它的结果异步写入 TMEM。在 Tensor Core 完成相关工作之前, 收尾无法安全地读取累加器。因此 MMA 的提交路径会在一个完成屏障上到达, 而收尾在读取 TMEM 之前会等待那个屏障。

互动: 一次 TMA 加载通过 `mbarrier` 发出完成信号。MMA 路径在读取共享内存分块之前等待该屏障。Tensor Core 到收尾的交接遵循同样的形态, 只是到达由 Tensor Core 的提交路径而非 TMA 执行。

同样的思路也适用于资源复用。屏障不仅是一个数据就绪信号。它也可以是一个「资源已空闲」信号。一个共享内存 stage 不能被覆写, 直到旧分块的所有消费者都已用完它。一片 TMEM 区域不能被复用, 直到上一个使用者已读完或写完它。在这些情形下, 到达意味着「我已经用完这个资源」, 而等待意味着「现在为下一个 stage 复用这个资源是安全的」。

这才是阅读一个流水化 GEMM 内核中同步的正确方式。那些 `wait` 与 `arrive` 并不是作为防御式编程散落各处的。每一次都标记着一次具体的所有权转移: 一个分块变得就绪, 一个累加器变得可读, 或一个缓冲变得可复用。一旦这些交接被识别出来, 控制流就变得易于跟读得多。

## 1.9 进阶:Cluster Launch Control

### Overview

- 持久化内核会让一组固定数量的 CTA(协作线程阵列) 或 CTA 集群常驻 (规模通常按每个 SM 大致对应一个活跃的工作属主来设置, 但并不依赖严格的 1:1 映射保证), 让它们循环处理多个输出分块, 而不是为每个分块派发一个 CTA。
- Cluster Launch Control 是 Blackwell(架构代号) 的硬件机制, 允许常驻集群在运行时再申请一个分块。它是一条硬件 `work-stealing` 路径, 围绕两条 PTX 指令构建: 一条指令申请工作, 另一条指令读回申请是否成功。
- 主要收益是更好的尾部行为。当各分块开销不均, 或分块数无法在可用 SM 之间均分时, 提前完成的 CTA 可以拉取更多工作, 而不是空等。

持久化 GEMM(通用矩阵乘) 并不把 CUDA grid 当作「每个输出分块对应一个 CTA」的固定派发。相反, 它派发一组数量更少、生命周期更长的 CTA 或 CTA 集群。每个 CTA 或集群计算一个分块、推进到下一个分块、再计算, 如此反复直到整个输出空间完成。这就是用线程束特化与集群扩展 GEMM 中逐步构建起来的执行模式。

一旦内核成为持久化内核, 主要的调度问题就变得简单: 在一个 CTA 或集群完成当前分块之后, 下一个分块从哪里来?

最简单的答案是静态公式。例如, 内核可以根据 CTA id 算出分块坐标, 然后按 grid 步长推进。这种做法实现简单, 在所有分块开销大致相同且分块数能在 GPU 上均匀分布时效果良好。但调度是在工作真正运行之前就决定好的。如果少数分块耗时更长, 或最后几个分块分配不均, 某些 SM 会提前完成自己的份额, 而其他 SM 仍在处理尾部。

Cluster Launch Control, 即 CLC, 改变了这种调度模型。它不再预先决定整个分配, 而是允许持久化集群向硬件 grid 调度器申请另一个尚未派发的集群的工作。如果申请成功, 当前集群就接管那个集群坐标并计算对应的分块。如果申请失败, 说明已经没有可窃取的工作, 循环退出。

这与 thread block cluster 本身并不是一回事。Thread block cluster(一起派发的 CTA, 具备



集群级别的同步和对分布式共享内存的访问)是在 Hopper 上引入的 (*GPU 执行模型*)。CLC 是 Blackwell 上的新增机制, 它让这些集群坐标之上的调度变得动态化。集群本身就是派发单元; CLC 让一个已经在运行的集群可以取消一个待派发的集群, 并继承其坐标。

### 1.9.1 两条指令

Cluster Launch Control 通过两条 PTX 指令暴露出来。第一条指令向 grid 调度器发送一个异步请求。第二条指令读取响应。

请求指令是 `clusterlaunchcontrol.try_cancel.async`。

`try_cancel` 请求调度器取消一个待派发集群的启动, 并把该集群的坐标返回给调用者。响应以一条 16 字节的记录写入共享内存。由于请求是异步的, 指令不会等待响应到达。相反, 完成事件通过一个 `mbarrier`(内存屏障) 上报, 使用的是与 TMA(张量内存加速器) 相同的屏障与相位模型。

这是一个重要细节, 因为它意味着 CLC 并不引入新的等待模型。内核发出请求, 把它与一个屏障关联, 之后再在该屏障上等待, 然后才读取响应。响应到达通过屏障以字节计数完成的方式发出信号, 总体风格与其他异步硬件操作一致 (见 *异步协调: mbarriers*)。

一旦屏障触发, 内核就使用查询指令。

第一条查询是 `clusterlaunchcontrol.query_cancel.is_canceled`。它返回一个谓词, 告诉内核取消是否成功。谓词为真表示调度器找到了一个待派发的集群启动, 取消了它, 并返回了其坐标。谓词为假表示已经没有剩余的待处理工作可取。

只有当 `is_canceled` 为真时, 内核才应读取坐标。这通过 `clusterlaunchcontrol.query_cancel.get_first_ctaid` 完成, 它提取被取消集群的第一个 CTA id。该 CTA id 是一个坐标向量, 通常按 (x, y, z) 读取, 内核把它解码为接下来应当计算的输出分块。

这个协议里没有数值型的哨兵分块 id。内核根据谓词分支。如果谓词为真, 坐标有效。如果谓词为假, `work-stealing` 循环结束。

在底层, 这种形态直接来自 CLC 实际在做的事情。硬件并不是从软件队列中分配一个抽象任务。它是在取消一个尚未发生的集群启动。因此, 成功的响应包含一个真实的集群坐标。失败的响应仅仅表示派发队列已经被耗尽。

### 1.9.2 Work-Stealing 循环

有了这两条指令, 持久化调度器就变成一个简短的循环。

在循环中的任何时刻, 集群都有一个负责计算的分块。在开始该分块之前, 它先为下一个分块发送一个 `try_cancel` 请求。请求异步执行。当调度器在处理该请求时, 集群计算自己的当前分块。

当前分块完成后, 集群在与 `try_cancel` 响应关联的 `mbarrier` 上等待。一旦响应就绪, 它调用 `query_cancel.is_canceled`。如果谓词为真, 它调用 `query_cancel.get_first_ctaid`, 解码返回的坐标, 并把它用作下一个分块。如果谓词为假, 说明已经没有剩余工作, 集群退出。

在代码形态上, 该循环是:

1. 为可能的下一个分块发出 `try_cancel`;
2. 在请求在途时计算当前分块;
3. 等待响应屏障;
4. 查询取消是否成功;

5. 要么带着返回的坐标继续, 要么退出。

请求的放置位置正是让这个循环有用的关键。集群不会等到当前分块完成之后才去申请更多工作。它先申请, 再计算。这样就把调度器请求与有用的工作重叠起来。到当前分块完成时, 下一个分块的答案往往已经就绪。

这与持久化内核在其他地方使用异步拷贝和 `tensor-core` 屏障是出于同样的基本原因。内核避免把一个长延迟操作直接放在关键路径上。CLC 把同样的思路应用到分块调度上: 尽早申请下一个工作单元, 计算当前单元, 然后在需要时再消费调度结果。

### 1.9.3 与持久化 GEMM 的关系

用线程束特化与集群扩展 `GEMM` 中的持久化 `GEMM` 在主线讲解中使用静态调度器。静态调度器更容易讲解, 因为下一个分块可以直接从循环状态计算出来。例如, `ClusterPersistentScheduler2D` 这样的调度器可以在输出分块空间上按 `grid` 步长模式分配分块。

CLC 是对那种静态分配的动态替代。外层循环保持不变: 每个常驻集群反复计算一个输出分块, 然后推进到另一个。变化的是下一个分块从哪里来。使用静态调度器时, 下一个分块由公式计算。使用 CLC 时, 下一个分块由硬件 `work stealing` 返回。

这种差异在派发的尾部附近最为重要。在静态调度中, 剩余工作可能分布不均。某些 `SM` 可能已经用完了分配到的分块, 而其他 `SM` 还剩好几个。有了 CLC, 提前完成的集群会申请另一个待派发的集群坐标。只要派发队列里还有工作, 提前完成者就会持续拉取更多分块。

当分块开销不均匀时, 这同样重要。某些 `GEMM` 分块可能因为边界、掩码、稀疏性、分组调度或围绕主矩阵乘的融合工作而走不同的路径。静态调度假设分块分配在任何这些开销被观察到之前就已经足够好。CLC 不需要这个假设。它只在一个集群变为可用之后才分配更多工作。

因此在 `TIRx` 中, CLC 可以作为一个动态分块调度器暴露出来。编程模型不需要改变一个分块的计算。分块体与静态调度器使用的持久化 `GEMM` 体相同。调度器从「用公式计算我的下一个分块坐标」变为「向硬件询问下一个可用的集群坐标」。结果是同样的持久化循环, 但采用硬件驱动的工作分配, 而非固定的派发时调度。

## 1.10 TIRx 简介

### Overview

- `TIRx` 是一个用于在 `IR` 层编写 GPU 内核的 Python DSL(领域特定语言): 你直接命名硬件, 但通过结构化的 `IR` 来命名。
- 每一个分块操作都由三个设计要素控制: `scope`(作用域, 哪些线程)、`layout`(布局, 分块位于何处) 和 `dispatch`(派发, 走哪条硬件路径)。
- 一个可运行的单 `MMA GEMM`(通用矩阵乘) 就展示了全部三者; 本书余下部分就是这些设计要素在更大尺度上的展开。

### Running the examples

这些示例需要一块 Blackwell GPU(sm\_100a, 例如 B200)。`TIRx` 编译器以 Apache TVM wheel 包的 `tvm.tirx` 模块形式发布; 请与 PyTorch 的 CUDA 构建一起安装:

```
pip install apache-tvm
```

用 `python -c "import tvm, tvm.tirx; print(tvm.__version__)"` 确认能正常导入。同样的这套环境可以运行书中每一个可运行的示例。

第一部分讲解了硬件是什么。要让硬件真正算出东西, 我们需要一种编程方式。

我们可以直接写裸 CUDA 或 PTX, 许多快速的内核正是这样写出来的。问题在于, 真正决定内核行为的那些决策在那里很难看清: 哪些线程执行一个操作、每一块数据放在哪里、由哪条硬件路径执行。这些选择被埋没在内联函数参数、地址算术和约定之中。

TIRx(Tensor IR neXt) 是一个 Python DSL(领域特定语言), 它把上述三个决策提升到明面上: **scope**(作用域, 哪些线程执行一个操作)、**layout**(布局, 操作数分块放在哪里)和 **dispatch**(派发, 由哪条硬件路径执行)。它仍然直接命名硬件概念, 包括线程、共享内存和张量内存、屏障, 以及 tcgen05 MMA。区别在于, 这些选择现在变成了编译器可以降级、检查和调度的结构化 IR。

我们不抽象地引入这些概念, 而是从一个完整的内核出发: 一个最小的单 MMA GEMM。我们先让它跑起来, 然后再逐行回读, 看 `scope`、`layout` 和 `dispatch` 各自如何塑造它, 以及内核如何被编译。该内核所依赖的张量布局模型将在 [TIRx 布局 API](#) 中独立展开, 完整的语言特性集则在 [TIRx 语言参考](#) 中给出; 这里我们只聚焦于这一个内核和三个设计要素。

### 1.10.1 第一个内核: 单 MMA GEMM

我们承诺要给出的内核是一个最小化的 GEMM, 精简到仍能用到 Tensor Core 的最小版本。它计算  $D = A \cdot B^T$  的单个  $128 \times 128$  输出分块,  $K = 64$ 。整个计算从头到尾被表达为一个 `Tx.gemm_async` 分块操作。(那一个分块操作并不对应单条硬件指令: 因为硬件 MMA 的 K-atom 是 16,  $K=64$  的分块会降级成一小段沿  $K$  步进的 `tcgen05.mma` 指令序列。这个 DSL 的要点恰恰在于我们写的是分块, 而不是序列。)围绕这个操作, 内核还做些常规杂活: 分配共享内存 (SMEM, Shared Memory) 和张量内存 (TMEM, Tensor Memory), 把  $A$  和  $B$  从全局内存拷贝到共享内存, 把分块 MMA 派发进一个 TMEM 累加器, 通过寄存器把该累加器读回, 再存储结果。这个内核虽小, 却是我们在构建分块 [GEMM](#) 中攀登的 GEMM 阶梯的第 1 步, 在那里它会有一次完整的走读。

每个 TIRx 内核都从同样的若干导入开始, 所以值得先一次性把它们看清楚:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_
    shared_layout, SwizzleMode
from tvm.tirx.layout import TileLayout, S, Tlane, TCol, tid_in_wg
```

我们把内核包在一个小型构造器 `hgemm_v1(M, N, K)` 里, 它接受问题形状并返回一个 `PrimFunc`。对于我们选定的形状  $M=N=128$ ,  $K=64$ , 启动恰好只包含一个输出分块, 这正是让这个第一个版本简单到能一口气读完的原因:

```
def hgemm_v1(M, N, K):
    a_type = tvm.DataType("float16")
    b_type = tvm.DataType("float16")
    d_type = tvm.DataType("float16")
    acc_type = tvm.DataType("float32")
```

(continues on next page)



(continued from previous page)

```

BLK_M, BLK_N, BLK_K = 128, 128, 64
# MMA_M/MMA_N/MMA_K 记录底层硬件 MMA 分块的大小; 它们不会
# 传给 gemm_async(gemm_async 会从操作数和累加器分块推导出 MMA 形状),
# 因此后续步骤中略去了它们。
MMA_M, MMA_N, MMA_K = 128, 128, 16

A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_
→ATOM, (BLK_M, BLK_K))
B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_
→ATOM, (BLK_N, BLK_K))

@T.prim_func
def kernel(
    A: T.Buffer((M, K), a_type),
    B: T.Buffer((N, K), b_type),
    D: T.Buffer((M, N), d_type),
):
    T.device_entry()
    # Step 1 是一个单分块内核: M = BLK_M 且 N = BLK_N, 所以 grid
    # 是 1x1。从 1x1 grid 出发能让每个 CTA 的分块偏移
    # (m_st, n_st) 平凡地为零; Step 3 及以后会把它推广到更大的 M / N。
    bx, by = T.cta_id([M // BLK_M, N // BLK_N])
    wg_id = T.warpgroup_id([1]) # 单个 warpgroup, 所以 wg_id_
→恒为 0(下面未使用)
    warp_id = T.warp_id_in_wg([4])
    lane_id = T.lane_id([32])

    # --- SMEM 分配 ---
    pool = T.SMEMPool()
    tmem_addr = pool.alloc((1,), "uint32")
    mma_bar = pool.alloc((1,), "uint64", align=8)
    pool.move_base_to(1024)
    Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout)
    Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout)
    pool.commit()

    # --- 屏障 + TMEM 初始化 (仅 warp 0) ---
    if warp_id == 0:
        if lane_id == 0:
            T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1)
            T.ptx.tcgen05.alloc(T.address_of(tmem_addr), n_cols=512,
→ cta_group=1)

    T.ptx.fence.proxy_async("shared::cta")
    T.ptx.fence.mbarrier_init()
    T.cuda.cta_sync()

    tmem = T.decl_buffer(

```

(continues on next page)

(continued from previous page)

```

        (128, 512), "float32", scope="tmem", allocated_
→addr=tmem_addr[0],
        layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)])
    )

    m_st = T.meta_var(bx * BLK_M)
    n_st = T.meta_var(by * BLK_N)
    phase_mma: T.int32 = 0

    # --- 加载: 所有线程把 global -> shared 拷贝 (同步)。
    # M=BLK_M 且 N=BLK_N 时, 下面的切片覆盖了完整矩阵;
    # 保留切片形式是为了让与 Step 3(多分块) 的 diff 最小。
    Tx.cta.copy(Asmem[:, :], A[m_st:m_st + BLK_M, :])
    Tx.cta.copy(Bsmem[:, :], B[n_st:n_st + BLK_N, :])
    T.cuda.cta_sync()

    # --- 计算: 单个被选中的线程派发 MMA ---
    if warp_id == 0:
        if T.ptx.select_sync():
            Tx.gemm_async(
                tmem[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
                accum=False, dispatch="tcgen05", cta_group=1
            )
            T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_
→group=1)

            T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma)

    # --- 回写: TMEM -> RF -> GMEM ---
    Dreg = T.alloc_local((BLK_N,), acc_type)
    Dreg_f16 = T.alloc_local((BLK_N,), d_type)
    Dreg_wg = Dreg.view(128, BLK_N,
        layout=TileLayout(S[(128, BLK_N) :
→(1@tid_in_wg, 1)]))
    Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N])
    T.ptx.tcgen05.wait_ld()
    Tx.cast(Dreg_f16[:, :], Dreg[:, :])
    m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)
    Tx.copy(D[m_thr, n_st : n_st + BLK_N], Dreg_f16[:, :])

    # --- 释放 TMEM ---
    T.cuda.cta_sync()
    if warp_id == 0:
        T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
        T.ptx.tcgen05.dealloc(tmem_addr[0], n_cols=512, cta_
→group=1)

    return kernel

```

在读这个内核之前, 我们先确保它能用。我们编译它, 并把输出与一个 torch 参考实现对照

检查。我们不必写出确切的架构: 架构 (例如 `sm_100a`) 会从设备自动探测, 所以 `target` 写成 `"cuda"` 就够了, 而 `tir_pipeline="tirx"` 正是用来选择 TIRx 降级流水线的。编译完成后, `ex.mod(...)` 直接接受 torch 张量, 中间无需任何手动转换。

```
import torch

target = tvm.target.Target("cuda")
device = torch.device('cuda') # gpu(0)

M, N, K = 128, 128, 64
kernel = hgemm_v1(M, N, K)
with target:
    ex = tvm.compile(tvm.IRModule({"main": kernel}), target=target,
    →tir_pipeline="tirx")

torch.cuda.empty_cache()
torch.cuda.synchronize()
A_tensor = torch.randn(M, K, dtype=torch.float16, device=device)
B_tensor = torch.randn(N, K, dtype=torch.float16, device=device)
D_tensor = torch.zeros(M, N, dtype=torch.float16, device=device)

# ex.mod(...) 直接接受 torch 张量, 每一章用的都是这种调用形式。
ex.mod(A_tensor, B_tensor, D_tensor)

D_ref = (A_tensor.float() @ B_tensor.float().T).half()
max_err = float((D_tensor - D_ref).abs().max())
print(f"Max error vs torch reference: {max_err:.6f}")
torch.testing.assert_close(D_tensor, D_ref, rtol=2e-2, atol=1e-2)
print("PASS")
```

### 1.10.2 Scope、Layout、Dispatch

既然内核能跑了, 我们就可以回过头读它, 追问它的每一行到底决定了什么。这样看, 整个内核就是沿三个设计要素做出的一组选择。其中的每一个操作都回答同样三个问题: 谁运行它、它的数据在哪、它怎么执行, 而这三个回答正是 `scope`、`layout` 和 `dispatch`。本节余下部分逐个讨论这些设计要素; 下方的交互演示可以让你看到每个设计要素分别控制了哪些行。

交互演示: 点击 *Scope / Layout / Dispatch*, 高亮显示每个设计要素所控制的内核行。

在使用这个演示时, 留意三个问题:

- **Scope:** 谁运行这个操作? `Tx.cta.copy(...)` 是 CTA(协作线程阵列) 作用域的, 所以全部 128 个线程都参与 GMEM -> SMEM 的拷贝。`Tx.gemm_async(...)` 由一个被选中的线程派发一次, 因为每条降级后的 `tcgen05.mma` 指令本身就已经是一次协作式 MMA 启动。`Tx.wg.copy_async(...)` 是 warpgroup(线程束组) 作用域的, 所以该 warpgroup 的 128 个线程逐行瓜分 TMEM 的回读。
- **Layout:** 每个分块放在哪里? A 和 B 使用 `tcgen05.mma` 所期望的 `swizzle` SMEM 布局。累加器位于 TMEM 中, 采用 `TLane/TCol` 布局。寄存器回读视图把行映射到 `tid_in_wg`, 于是每个 warpgroup 线程拥有一段行片段。
- **Dispatch:** 由哪条硬件路径执行? `Tx.gemm_async(..., dispatch="tcgen05", ...)` 选择 Blackwell 的 Tensor Core 路径。拷贝操作也有 `dispatch` 选择: 这第一个内核

使用普通的线程拷贝, 后续的 GEMM 步骤会在不改变周围 scope 或 layout 的前提下, 把这些拷贝换成 TMA(张量内存加速器)。

**和你的 agent 一起试试:** 从第一个内核中挑三行——一个拷贝、一个 MMA、一个 TMem 回读。让它按 scope、layout 和 dispatch 给每行打标签, 然后核对答案是否与代码中的守卫、buffer 布局以及 dispatch= 参数一致。

### 1.10.3 编译是如何工作的

我们在上面为了测试已经编译过这个内核; 现在我们更仔细地看看这一步做了什么。套路很短: 把 PrimFunc 包进一个 IRModule, 交给 `tvm.compile(mod, target=..., tir_pipeline="tirx")`。这会运行 TIRx 降级流水线, 并返回一个你可以直接调用的 Executable。

```
target = tvm.target.Target("cuda")
ex = tvm.compile(tvm.IRModule({"main": kernel}), target=target, tir_pipeline="tirx")
```

至少在轮廓层面, 值得了解一下 `tir_pipeline="tirx"` 启动了什么。流水线的核心 pass `LowerTIRx` 依据每个分块原语的 scope / layout / dispatch 契约来解析它: 我们刚刚讨论的三个设计要素正是在这里被实际兑现为指令。之后, 常规的 host/device 切分和一个收尾步骤会产生可启动的 module。如果你愿意, 也可以在一个 `with target:` 块内编译, 这样内核能拾取周围的 target 上下文。

这套流程的一个好处是对你毫无隐瞒: 结果在两个层级上都可检视。你可以用 `.show()` 或 `.script()` 读 IR 本身, 也可以直接从编译后的 module 上读到编译器最终吐出的 CUDA C。

```
kernel.show()                # 美化打印 TIRx(TVMScript)
print(kernel.script())        # .....同样的内容, 以字符串形式

# 编译后的 Executable 生成的 CUDA C 源码:
print(ex.mod.imports[0].inspect_source())
```

这只是一个轮廓。完整的降级故事——涵盖所有 pass、分块原语派发如何被解析、host/device 切分如何完成——请见编译器内部实现。

### 1.10.4 接下来去哪里

一个内核就足以让我们认识 scope、layout 和 dispatch, 并看到它们被编译和运行。三个设计要素中的每一个, 以及这个内核本身, 都通向一个把它们进一步展开的章节:

- **TIRx 布局 API:** 张量布局模型 (TileLayout、命名轴、swizzle), 上面的操作数与累加器放置正是建立在它之上。如果三个设计要素里 layout 感觉最神秘, 从这里开始。
- **TIRx 语言参考:** 完整的语言特性集, 涵盖解析器工具、数据类型、buffer 与内存、控制流以及线程同步, 适合你想获得完整词汇表而非一次导览时使用。
- **构建分块 GEMM:** 把这个内核作为 GEMM 优化路径的第 1 步, 经由 K 循环累加、空间分块、TMA 和 warp specialization(线程束特化) 逐步构建起来。如果你想看这三个设计要素如何扩展到一个真实内核, 这里是自然的下一站。

## 1.11 TIRx 布局 API

### Overview

- TIRx 布局 API 把数据布局及其记法 中的布局记法转换为编译器对象。主要对象有 `TileLayout`、`SwizzleLayout` 和 `ComposeLayout`。
- `TileLayout` 描述在命名硬件轴上的仿射放置。它由分片规格 `S[...]`、副本规格 `R[...]` 以及可选偏移构成。
- 一个布局将一个逻辑坐标映射到一个或多个物理坐标。`layout.apply()` 对该映射求值。
- `SwizzleLayout` 描述基于 XOR 的共享内存 swizzle, 用于避免 bank 冲突。`ComposeLayout` 在 tile 布局之上叠加一层 swizzle。
- 诸如 `tmem_datapath_layout`、`tcgen05_atom_layout` 和 `wg_local_layout` 这类现成构造器, 覆盖了内核中反复出现的硬件布局。

数据布局及其记法 引入了贯穿全书的记法: 一个 tile 形状、一组在命名轴上的步幅 (stride), 以及一个用于被复制而非被划分的值的可选复制项。本章把这种记法转换为编译器使用的 API。

目标是让页面上的记法与内核中的代码看起来几乎一致。当你写出一个诸如以下的布局时:

```
S[(128, 256) : (1@TLane, 1@TCol)]
```

你写的不仅仅是一段说明, 而是在构造一个可附加到 `buffer` 的 `TileLayout` 对象。此后, 每一个触及该 `buffer` 的 tile 操作都能从布局中读取自己的放置位置。放置只写一次, 只检查一次, 然后由编译器复用。

布局既可以在从池中分配时附加, 也可以在声明 `buffer` 时附加:

```
pool.alloc(shape, dtype, layout=layout)
```

```
T.decl_buffer(shape, dtype, scope=scope, layout=layout)
```

从此刻起, 该 `buffer` 就携带了自己的物理放置。各 tile 操作无需重复说明每个元素位于何处。

这些布局对象都位于同一个模块中:

```
from tvm.tirx.layout import (
    TileLayout,
    SwizzleLayout,
    ComposeLayout,
    S,
    R,
    laneid,
    warpid,
    tid_in_wg,
    TLane,
    TCol,
    m,
    tcgen05_atom_layout,
```

(continues on next page)



(continued from previous page)

```
tmem_datapath_layout,
)
```

该 API 背后有一个核心思想: 布局不必把一个逻辑索引映射到单一物理地址, 而是把一个逻辑索引映射到一组位于命名轴上的物理坐标。在通常情况下这组坐标只有一个元素; 当存在复制时, 同一个逻辑元素会有多个物理放置。

这正是布局模型由三部分组成的原因: 分片 (shard)、副本 (replica) 和偏移 (offset)。分片负责放置元素; 副本把它复制到额外的坐标; 偏移则平移整个放置。

### 1.11.1 通过示例看布局

下面的示例展示了 API 的基本形态。

TMEM(张量内存) 中的累加器可以直接写成一个在 TMEM 轴上的放置:

```
acc = TileLayout(S[(128, 256) : (1@TLane, 1@TCol)])
```

这里逻辑行映射到 TLane, 逻辑列映射到 TCol。在特殊内存:TMEM 中, 硬件坐标称作 Lane 和 Col。在 TIRx 布局记法里, 这些硬件轴被写作 TLane 和 TCol。

一个块缩放 (block-scaled)MMA 的缩放因子布局用到了复制:

```
scale_factor_layout = TileLayout(
    S[(32, sf_per_mma) : (1@TLane, 1@TCol)] + R[4 : 32@TLane]
)
```

分片把一个 32 行的组放置在 TMEM 中。副本以 32 条 lane 的步幅将该组重复四次, 使这个 32 行的组在整个 128 lane 的 TMEM 空间中都可见。

一个 Tensor Core(张量核) 寄存器片段可以分布在 lane 和 warp 之间:

```
frag = TileLayout(
    S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)]
)
```

同一个物理轴可以出现不止一次。在本例中, 两个不同的 iter 都贡献到 laneid。没有显式轴的步幅使用默认内存轴 m。

在真实内核中, 常见的硬件布局通常来自构造器:

```
acc = tmem_datapath_layout("D", 128, 256)

ld = tcgen05_atom_layout("32x32b", (128, 64), "float32")
```

这些构造器返回的是普通的 TileLayout 对象。它们只是便利封装, 并非另一套机制。你可以检查返回的布局, 把它与其他布局组合, 或者在形状特殊时手写出底层的 S[...] 和 R[...] 形式。

### 1.11.2 交互式演示

在讲解机制之前,先有一个具体可玩的东西会有帮助。下面的演示允许你选择一个预设布局、编辑逻辑形状和 S 或 R 项、选择 dtype 与 swizzle 模式,然后点击某个元素,查看它被哪个或哪些物理坐标所拥有。

这个演示之所以有用,是因为 API 的大部分内容只不过是该演示所展示内容的精确版本。一个逻辑元素进入布局,布局把它展平、按 iter 切分、在命名轴上累加坐标,最后在需要时应用复制。

### 1.11.3 TileLayout

TileLayout 是主要的仿射布局对象。它通常用与正文相同的记法书写:

```
TileLayout(S[shape : strides])
```

S 项是分片规格。可以这样读它: 取一个具有该形状的逻辑 tile, 并按这些步幅在命名轴上放置它。

当一个值需要出现在多个位置时, 分片规格会扩展出一个副本规格:

```
TileLayout(S[shape : strides] + R[replica_shape : replica_stride])
```

还可以加上一个可选的偏移:

```
TileLayout(S[shape : strides] + R[replica_shape : replica_stride] +  
↪offset)
```

在底层, 这些组成部分由 iter 表示。一个 iter 是一个三元组:

```
(extent, stride, axis)
```

它描述了沿某一个命名轴的一次步进式行走。extent 表示该 iter 拥有多少个位置; stride 表示每一步移动多远; axis 表示正在改变哪一个硬件坐标。

一个布局有三部分。

### 分片

分片, 即 D, 是由 S[...] 构造的部分。它把逻辑索引划分到一个或多个 iter 上, 并产生基础物理坐标。

例如:

```
S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)]
```

有四个分片 iter。它们的 extent 分别是 8、2、4 和 2。它们的步幅分别把数据放置到 laneid、warpid、再次到 laneid, 以及默认内存轴 m 上。

这是对普通「形状-步幅」规则的推广。区别在于, 步幅是绑定到命名硬件轴上的, 而不是绑定到单一扁平地址。

## 副本

副本, 即  $R$ , 描述同一个逻辑元素的额外物理拷贝。副本  $iter$  与逻辑索引相互独立。它们枚举的是硬件空间中的额外偏移。

例如:

```
R[2 : 4@warpid]
```

在 `warpid` 轴上创建两个相隔四个 `warp` 的拷贝。

复制并不是为图方便而要的小把戏, 它描述的是真实的硬件行为。某些数据会被广播到多个 `warp`、`lane` 或内存区域。逻辑到物理的映射天然就能支持这一点, 因为一个逻辑元素可以映射到一组物理坐标。

## 偏移

偏移, 即  $O$ , 是加到每个结果上的一个固定坐标。

例如:

```
5@warpid
```

把整个放置在 `warpid` 轴上平移五。

偏移可用于把一个 `tile` 放置到选定的基础坐标、为独占使用预留一段区域, 或描述一个在同一资源中紧接另一个 `tile` 之后开始的 `tile`。

## 把各部分组合起来

一个布局按顺序应用这三部分。

首先, 分片计算出基础坐标; 然后, 副本把该坐标扇出到零个或多个额外拷贝; 最后, 偏移平移每一个坐标。

对于逻辑坐标  $x$ , 结果是:

$$L(x) = \{ D(x) + r + O \mid r \in R \}$$

如果没有副本,  $R$  只包含零偏移, 所以结果是单元素集合。如果有副本, 结果中每个副本位置对应一个坐标。

在 TIRx 语法里, 一个完整布局看起来可以是这样的:

```
layout = TileLayout(
    S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)]
    + R[2 : 4@warpid]
    + 5@warpid
)
```

从左向右读: 分片放置该逻辑 `tile`, 副本在相隔四个 `warp ID` 处创建第二份拷贝, 偏移则把整个放置平移到从 `warpid = 5` 开始。

如果各 `iter` 已经作为对象构造好, 同一个布局也可以直接构造:

```
TileLayout.from_iters(shard, replica, offset)
```

大多数用户代码使用  $S[\dots]$  和  $R[\dots]$  记法, 因为它更接近数学形式。

### 1.11.4 命名轴

布局中的轴并非匿名维度。每个轴都命名一个真实的硬件坐标或一个编译器层面的放置坐标。

例如:

```
bx, by, bz
cbx, cby, cbz
tx
warpid
laneid
wgid
tid_in_wg
wid_in_wg
m
P, F
Bank
TLane, TCol
```

grid 轴 (如  $bx$ 、 $by$ 、 $bz$ ) 把工作分配到各 CTA 之间。cluster 轴 (如  $cbx$ 、 $cby$ 、 $cbz$ ) 在一个 CTA 集群 (CTA cluster) 内部分配工作。线程轴 (如  $tx$ 、 $warpid$ 、 $laneid$ 、 $tid\_in\_wg$ 、 $wid\_in\_wg$ ) 描述一个 CTA 或 warpgroup 内部的归属。轴  $m$  是默认的线性内存轴。 $P$  和  $F$  用于二维 scratchpad 式放置。 $Bank$  命名共享内存的 bank。 $TLane$  和  $TCol$  是 TIRx 布局中对 TMEM 的 Lane 与 Col 坐标的命名。

轴名是布局的一部分。这很重要, 因为两个取相同整数值的坐标可能意味着不同的硬件含义。 $1@tx$  与  $1@tid\_in\_wg$  不同;  $1@laneid$  与  $1@TLane$  也不同。布局把这些含义保持显式。

### 1.11.5 正向映射

对布局求值, 意味着取一个逻辑坐标并计算它落在哪个物理位置。API 方法是:

```
layout.apply(*coord)
```

对于没有复制的布局, 结果是一个坐标字典。对于带复制的布局, 结果是一组坐标字典。坐标字典把轴名映射到整数位置, 例如:

```
{"laneid": 7, "warpid": 2, "m": 1}
```

求值规则有四步。

第一步, 按行优先顺序展平逻辑坐标。对于逻辑坐标:

```
x = (x0, x1, ..., xr-1)
```

在逻辑形状:

```
(s0, s1, ..., sr-1)
```

内, 扁平索引是:

```
flat = x0 * S1 * S2 * ... * Sr-1
      + x1 * S2 * ... * Sr-1
      + ...
      + xr-2 * Sr-1
      + xr-1
```

第二步, 按分片 extent 切分该扁平索引。如果分片 extent 是:

```
(e0, e1, ..., en-1)
```

那么切分得到的分量是:

```
c0, c1, ..., cn-1
```

对分片 extent 使用同样的行优先顺序。

第三步, 用每个分量的步幅把它累加到对应轴上。如果分片 iter k 的 extent 为  $e_k$ 、步幅为  $s_k$ 、轴为  $a_k$ , 那么分量  $c_k$  贡献:

```
ck * sk @ ak
```

所有对同一轴的贡献都相加在一起, 随后再加上偏移。

第四步, 应用副本 iter。每个副本 iter 贡献一个与逻辑索引无关的额外偏移。如果有多个副本 iter, 布局会枚举所有组合。

这条规则的一个有用推论是: 布局不需要硬编码输入形状。它需要的只是逻辑 tile 的元素总数等于各分片 extent 的乘积。只要这一点成立, 展平和切分就定义了该映射。

### 1.11.6 案例:Tensor Core 寄存器 tile

考虑一个分布在两个各含 32 条 lane 的 warp 之间的逻辑 (8, 16) tile。每条 lane 拥有一个小的寄存器片段。寄存器槽位由默认内存轴 m 表示。

```
layout = TileLayout(
    S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)]
    + R[2 : 4@warpid]
    + 5@warpid
)
```

从 (8, 16) tile 中取一个逻辑元素 (i, j)。

行优先扁平索引是:

```
flat = 16 * i + j
```

按分片 extent (8, 2, 4, 2) 切分得到:

```
c0 = i
c1 = floor(j / 8)
c2 = floor(j / 2) mod 4
c3 = j mod 2
```

分片贡献是:



```
laneid = 4 * c0 + c2
warpid = c1
m      = c3
```

加上偏移 5@warpid 后, 变为:

```
laneid = 4 * i + floor(j / 2) mod 4
warpid = floor(j / 8) + 5
m      = j mod 2
```

副本项:

```
R[2 : 4@warpid]
```

要么给 warpid 加 0, 要么加 4。所以完整映射是:

```
laneid = 4 * i + floor(j / 2) mod 4
warpid = floor(j / 8) + 5 + 4 * r, where r in {0, 1}
m      = j mod 2
```

分片把 tile 放置在 warp 5 和 6 上。副本再把它复制到 warp 9 和 10。因此同一个逻辑元素出现在两个 warp 位置。

本例说明了为什么模型要使用一组物理坐标。复制无法用「从物理坐标到逻辑坐标的函数」自然地表示, 但可以用「从一个逻辑坐标到多个物理坐标的函数」自然地表示。

### 1.11.7 案例:Blackwell 张量内存

同一个布局模型也适用于内存放置。轴不必是线程轴, 也可以是内存轴。

TMEM 由硬件的 Lane 和 Col 坐标寻址。在 TIRx 布局记法中, 这两个轴写作 TLane 和 TCol。

考虑如下布局:

```
layout = TileLayout(
    S[(2, 128, 112) : (112@TCol, 1@TLane, 1@TCol)]
)
```

如果逻辑 tile 形状是 (2, 128, 112), 那么切分分量就是逻辑坐标本身。对于元素 (a, l, c), 映射是:

```
TLane = l
TCol  = 112 * a + c
```

extent 为 128、步幅为 1@TLane 的 iter 填满 128 行 TMEM Lane。extent 为 2、步幅为 112@TCol 的 iter 与 extent 为 112、步幅为 1@TCol 的 iter 合起来覆盖 224 列:

```
TCol in [0, 224)
```

这个 224 列的跨度是有意为之的。TMEM 布局不必是 2 的幂。一个块缩放的 FP8 GEMM 可能会选择 224 列的累加器, 因为一个完整的 256 列 tile 不会为两个累加器流水级加上缩放因子留下足够的 TMEM 容量。布局 API 能够直接表达这种形状。

### 1.11.8 缩放因子布局

上面的累加器布局是纯放置: 每个逻辑累加器元素映射到一个 TMEM 坐标。块缩放 MMA 的缩放因子则不同, 因为同一个物理组可能需要在多个 warp 窗口中都可见。这正是复制派上用场的地方。

一个紧凑的缩放因子布局可以写成:

```
scale = TileLayout(
    S[(32, sf_per_mma) : (1@TLane, 1@TCol)]
    + R[4 : 32@TLane]
)
```

分片把一个 32 行的缩放因子组放置在 TMEM 中:

```
TLane = r
TCol  = s
```

这是对逻辑缩放坐标 (r, s) 而言。

副本项创建四个相隔 32 条 lane 的拷贝:

```
TLane = r + 32 * q, where q in {0, 1, 2, 3}
TCol  = s
```

于是这个 32 行的组在 TMEM lane 0 到 31、32 到 63、64 到 95、96 到 127 处都可见。这就是 warpx4 广播模式 (跨 GPU 世代的 *Tensor Core* 操作数布局)。四个 warp 大小的 TMEM lane 窗口各自都能看到同一个缩放因子组。

在完整的块缩放 MMA 布局中, 这个 atom 会与 M 行和 K 个缩放因子组上的外层 iter 组合在一起。依据缩放因子的 dtype, 多个缩放因子也可能被打包进同一个 32 位 TCol 单元。例如, fp8 缩放因子可以把四个值打包进一个 32 位列单元。可选的步幅为零复用和流水级深度的 iter 随后可以描述跨多个 MMA 的缩放因子复用以及双缓冲 (double buffering)。

关键之处在于, 同一个 TileLayout 模型描述了这两种情形: 累加器是 TMEM 中的单一放置, 而缩放因子是同一 TMEM 地址空间中的复制放置。

### 1.11.9 现成布局

大多数内核并不逐一手写每一个硬件布局。TIRx 为常见布局提供了构造器。

```
tmem_datapath_layout(datapath, rows, cols)
```

返回由 tcgen05.mma 写入的 TMEM 累加器布局。datapath 参数选择行放置模式。例如, "D" 对应 M = 128 的恒等式风格放置, 而 "F" 对应 M = 64 的分散式放置。

```
tcgen05_atom_layout(instr_shape, tensor_shape, dtype)
```

返回由一个 tcgen05.ld 或 tcgen05.st atom 搬运的寄存器 tile 布局。指令形状的例子包括 .32x32b、.16x64b、.16x128b 及相关形式。在 DSL 层面这是一个 warpgroup 分布式 tile。在降级 (lowering) 过程中它会变成四条 warp 协作的 tcgen05.ld 或 tcgen05.st 指令, 每条 warp 一条, 各自处理自己的 32 条 TMEM lane。

```
wg_local_layout(cols, rows=128)
```

返回一个 warpgroup 本地寄存器 tile, 通常在 tid\_in\_wg 上每个线程一行。

这些辅助函数是为了避免手写重复的常见硬件映射。它们并不隐藏模型: 每个辅助函数返回的都是一个由上述同样的 S 和 R 构造的普通 TileLayout。

### 1.11.10 SwizzleLayout 与 ComposeLayout

TileLayout 是仿射的, 它能表达命名轴上的步幅、复制和偏移。这对许多放置已足够, 包括线程片段、TMEM tile 以及紧凑的缩放因子布局。

共享内存的 swizzle 却需要别的东西。用于避免 bank 冲突的 swizzle 并不是仿射步幅模式, 而是对线性共享内存地址的一次基于 XOR 的置换。

因此 TIRx 把 swizzle 保留为一个独立的布局对象:

```
SwizzleLayout(...)
```

并把它与 tile 布局组合:

```
ComposeLayout(swizzle, tile)
```

tile 布局先产生一个线性内存地址, swizzle 再对该地址做置换。把这两层保持分离, 比把 XOR 置换硬塞进仿射布局模型更干净。

### 1.11.11 为什么要 swizzle

共享内存被划分为 32 个 bank, 每个 bank 字 (word) 占 4 字节。当一次访问的各 lane 触及同一 bank 内的不同地址时, 这次访问会因 bank 冲突而被串行化。

一个普通的行优先 tile 在结构上就可能制造这种冲突。考虑一个采用行优先布局的 (8, 64) float16 tile:

```
TileLayout(S[(8, 64) : (64@m, 1@m)])
```

逻辑元素 (i, j) 的线性元素地址是:

```
m = 64 * i + j
```

每一行是 64 个 float16 值, 即 128 字节, 正好是一整条共享内存 bank 线。如果某个 warp 固定 j 沿一列向下读, 每前进一行就跨过整整一条 128 字节的线。bank 索引随之重复, 于是该列读会跨行塌缩到同一个 bank 上。

swizzle 通过让低位地址比特依赖于更高的行比特来改变这一点。原本会反复落在同一个 bank 上的一列, 会被分散到不同 bank 上。

### 1.11.12 Swizzle 变换

一个 SwizzleLayout 由三个整数参数控制:

```
per_element = M
swizzle_len = B
atom_len    = S
```

输入是一个线性元素地址 m。

m 的低 M 位保持不变, 以此保留一小段连续的元素组。较高位则被右移进一个临时值:

```
x = m >> M
```

随后,  $x$  中位于  $[S, S + B)$  的比特组会被异或进  $x$  的  $[0, B)$  比特组。把保持不变的低  $M$  位放回去, 就得到 `swizzle` 后的地址。

等价地:

```
mask = (1 << B) - 1

low  = m & ((1 << M) - 1)
x    = m >> M
x2   = x ^ ((x >> S) & mask)

addr = (x2 << M) | low
```

为使布局良构,  $S$  至少要等于  $B$ 。

这个变换的要点不在于改变 `tile` 中包含哪些逻辑元素, 而在于改变这些元素落在共享内存中的何处。MMA 仍然读同一个逻辑 `tile`, `swizzle` 只是让物理 `bank` 模式更优。

### 1.11.13 选择 `swizzle` 参数

在常规使用中, `swizzle` 参数依据 `dtype` 和共享内存 `swizzle` 模式选定。常见模式有 32 字节、64 字节和 128 字节 `swizzle`。

`per_element` 参数的选择要让一小段向量大小的组保持连续。对 `float16` 而言, 一个 16 字节向量含 8 个元素, 故:

```
M = log2(8) = 3
```

采用 128 字节 `swizzle` 时, 布局使用:

```
SwizzleLayout(per_element=3, swizzle_len=3, atom_len=3)
```

这既保持了 16 字节向量组完整, 又足以置换较大的共享内存地址模式, 从而打破列上的 `bank` 冲突。

大多数代码不应手工推导这些参数。`dtype` 和描述符模式通常会决定它们。对程序员而言重要的是: TIRx 布局里的 `swizzle`、TMA 描述符和 MMA 的期望三者必须匹配。

因此, 一个 `swizzle` 过的共享内存分配看起来是这样的:

```
tile = TileLayout(S[(8, 64) : (64@m, 1@m)])
swizzle = SwizzleLayout(per_element=3, swizzle_len=3, atom_len=3)

layout = ComposeLayout(swizzle, tile)
```

组合后的布局才是附加到共享内存 `buffer` 上的那个。

### 1.11.14 元素的 bank 与 line

要判断一个 swizzle 是否有帮助, 可把 swizzle 后的元素地址换算回共享内存的 bank。

设  $addr$  为 swizzle 后的元素地址,  $b$  为元素大小 (字节)。字节地址是:

```
byte = addr * b
```

bank 是:

```
bank = floor(byte / 4) mod 32
```

128 字节的 bank 线是:

```
line = floor(byte / 128)
```

对 float16,  $b = 2$ , 所以 bank 公式变为:

```
bank = floor(addr / 2) mod 32
```

这就是下面计算示例所用的公式。

### 1.11.15 计算示例: 在 (8, 64) float16 tile 上的 128B swizzle

回到行优先的 float16 tile:

```
m = 64 * i + j
```

使用:

```
SwizzleLayout(per_element=3, swizzle_len=3, atom_len=3)
```

变换变为:

```
x    = m >> 3
addr = ((x ^ ((x >> 3) & 7)) << 3) | (m & 7)
```

由于:

```
m = 64 * i + j
```

我们可以写:

```
q = floor(j / 8)
r = j mod 8
```

而 swizzle 后的地址是:

```
addr = 64 * i + 8 * (q xor i) + r
```

现在看列  $j = 0$ 。此时  $q = 0$  且  $r = 0$ , 故:

```
addr = 72 * i
```

对 float16, bank 是:



```
bank = floor(addr / 2) mod 32
```

所以这八行映射到:

```
i = 0: bank 0
i = 1: bank 4
i = 2: bank 8
i = 3: bank 12
i = 4: bank 16
i = 5: bank 20
i = 6: bank 24
i = 7: bank 28
```

该列现在触及八个不同的 **bank**, 冲突消失了。

若不 **swizzle**, 同一列的地址是:

```
m = 64 * i
```

因此:

```
bank = floor(64 * i / 2) mod 32 = 0
```

每一行都落在 **bank 0** 上, 于是该访问被串行化。**swizzle** 只改变了物理放置, 但这已经足够把列访问变成无冲突访问。

这一保证依赖于按其设计意图使用 **swizzle**。**dtype**、**swizzle** 宽度和访问形状必须与 **TMA** 和 **MMA** 描述符模式匹配。128 字节 **float16 swizzle** 是围绕相关的 16 字节行块与 **Tensor Core** 访问模式设计的, 它并不承诺任意共享内存访问都能无冲突。本章顶部的演示把这一点可视化: 选择一个 **dtype** 和 **swizzle** 模式, 观察一列在不加 **swizzle** 时塌缩到同一个 **bank** 上, 再在施加匹配的 **swizzle** 后于 **bank** 视图中散开。

### 1.11.16 设计依据

布局 **API** 遵循三个设计取舍。

第一, 它支持一般形状。硬件 **tile** 并不总是 2 的幂。全局张量、共享内存的各流水级、**TMEM** 累加器和缩放因子 **buffer**, 常常具有源自容量限制或算法选择的形状。布局模型把这些形状当作正常情况对待。

第二, 映射方向是从逻辑坐标到物理坐标。这一方向很重要, 因为复制很常见: 一个逻辑元素可能存在于多个物理位置。逻辑到物理的映射能直接用一组坐标表示这一点。

第三, 硬件轴是显式的。布局不使用匿名维度、再依赖上下文事后解释。**tx**、**tid\_in\_wg**、**laneid**、**warpid**、**TLane** 与 **TCol** 之间的区别, 被直接写进布局本身。

合法性与可行性检查并不只由布局对象负责。布局能说明数据放在何处, 更高层的 **tile** 原语决定某次操作能否合法且高效地使用该放置。这种分离使布局 **API** 保持精简, 同时仍给编译器足够信息来派发真实的硬件操作。

## 1.12 构建分块 GEMM

### Overview

- 从 TIRx 的分块原语出发, 以单个输出分块为起点, 构建一个正确的分块 GEMM。
- 第 1 步是单分块 GEMM, 第 2 步加入 K 维循环累加, 第 3 步跨 CTA 做空间分块以覆盖完整矩阵。
- 先求正确; 性能是后续两章的任务。

GEMM(通用矩阵乘, General Matrix Multiply) 是贯穿整本书的工作负载。它位于线性层、注意力投影和卷积这些占据 GPU 主要计算时间的算子之下, 因此一个正确的 GEMM 与一个足够快的 GEMM 之间的差距, 就等同于让芯片大部分时间空闲与将其打满之间的差距。

这个差距太大, 无法一步跨过。一个能打满硬件的内核会迫使你同时调试数据搬运、累加、分块和 Tensor Core(张量核) 调度, 而且没有任何可信赖的参照物。更稳妥的做法是: 从一个能得到正确结果的最小内核起步, 然后每次只做一个决策地逐步扩展。

本章就是要写出这第一个正确的分块 GEMM。前几章抽象地介绍了 TIRx(Python DSL) 的作用域 / 布局 / 派发模型, 本章则把它应用到一个真实的内核上。我们从一个  $128 \times 128$  的输出分块开始, 把它扩展成能够处理全尺寸矩阵的内核, 先后加入 K 维累加和跨多个 CTA 的空间分块。

本章是从头到尾走通一条完整 GEMM 优化路径的三章中的第一章。本章我们只构建一个正确的分块内核, 就此打住。下一章(用 TMA 为 GEMM 建立流水线)用 TMA(张量内存加速器, Tensor Memory Accelerator) 替换线程拷贝, 并通过流水线让数据搬运与计算重叠; 再下一章用线程束特化与集群扩展 GEMM 则更进一步, 引入线程束特化和 CTA 集群。每一章都建立在前一章之上, 因此各个内核逐步累积功能, 而不是从头再来。

把每一步理解成对同一份契约的编辑, 会有所帮助: 这份契约有三项条款——哪一个作用域执行操作、操作数分块用哪一种布局、哪一条派发路径执行它。多数步骤只有一项主要变化, 所以我们用一张小卡片开头, 点明这一变化, 并指出为安全复用所需的任何同步细节。第 1 步确立了基线, 之后的整条优化路径都在编辑它。

### 1.12.1 GEMM

GEMM 是位于线性层、注意力投影和许多卷积实现之下的稠密矩阵乘, 这也是为什么一个快的 GEMM 内核几乎在所有地方都能带来收益。本教程的示例使用  $D = AB^T$ :

- $A$  的形状为  $M \times K$ 。
- $B$  的形状为  $N \times K$ 。
- $D$  的形状为  $M \times N$ 。
- $D[m, n] = \sum_k A[m, k] \cdot B[n, k]$ 。

这里的转置并非我们额外选择执行的操作, 它是由数据的存储方式自然产生的。示例中  $B$  保持为  $N$  行、每行长度为  $K$ , 这正是线性层权重通常采用的布局, 因此沿  $K$  做内积时自然读到的就是  $B^T$ , 无需任何重排。

贯穿整篇教程, 我们以吞吐量 (单位 TFLOPS) 衡量一个内核, 用墙钟时间去除每次乘加里的两次浮点运算:

$$\text{TFLOPS} = \frac{2 \times M \times N \times K}{t_{\text{seconds}} \times 10^{12}}$$

## GEMM 数据通路

本教程中的每一项优化最终都归结为数据存放在哪里、如何移动,因此在动手写代码之前把它们梳理清楚是值得的。本质上,一个 Blackwell GEMM 内核只围绕两类活动组织:在各级存储之间搬运分块,以及在这些分块上做计算。下图追踪了一个分块从输入到输出经过的每一级存储:



上图展示的是基线路径,后续每一次优化都在编辑它,但从不替换它。从左到右读:操作数分块先从 GMEM 搬到 SMEM;随后 `tcgen05.mma` 消费 SMEM 中的操作数并把累加器写到 TMEM;最后收尾部分把 TMEM 读回到寄存器,再把结果写回 GMEM。请把这条链路记在心里,因为下面每一步改变的只是其中一跳「怎么跳」;它从不改变跳本身。

### 1.12.2 优化路径

上面这条朴素数据通路足以得到正确结果,但它把硬件的大部分都闲置着。教程余下部分通过逐一加入 Blackwell 的特性来弥合这一差距,每个特性都通过一个 TIRx 分块原语表达。我们将要走的路径依次经过这些特性:

- **TMA 异步搬运**通过 Blackwell 的硬件拷贝路径搬运 GMEM <-> SMEM 分块,并用屏障跟踪完成情况。
- **软件流水线**使用多个 SMEM 流水级,使下一个 K 分块的数据搬运能够与当前分块上的 Tensor Core 计算重叠。
- **持久化调度**保持一个固定规模的 CTA 池,每个 CTA 通过分块调度器处理多个输出分块,而不是每个分块启动一个 CTA。
- **线程束特化**把生产者、MMA 消费者和回写三种角色分到不同的 warpgroup 上。
- **CTA 集群**让两个 CTA 合作完成一个更大的 Blackwell MMA 分块。
- **多消费者执行**使用多个消费者 warpgroup 同时计算分块的不同部分,以提高计算密度。

### 1.12.3 第 1 步: 串行单分块 GEMM

仍然能走通完整硬件路径的最简 GEMM,就是只计算单个输出分块的那个。所以我们从这里开始。第 1 步计算一个  $128 \times 128$  的输出分块,  $K = 64$ , 小到无需任何循环,数据通路里的每个部分都恰好出现一次。没有任何东西重复,我们就能在不得不去推敲一个循环之前,孤立地看清每一跳。

#### 这一步确立的内容: 基线

- **作用域:** 一个由 128 个线程组成的单一 warpgroup 依次走完整条路径,一级接一级。
- **布局:** A 和 B 分块放在 SMEM 里,累加器放在 TMEM 里,结果经由寄存器暂存后写出。
- **派发:** 同步 `Tx.copy` 承担加载, `tcgen05` 执行 MMA。

## 单分块数据流

把基线契约定下来之后,接下来要确定的就是一个分块以何种顺序穿过这条契约。这第一个内核恰好走一遍核心的 GEMM 数据通路,也就是数据流图里那条 GMEM → SMEM → TMEM → 寄存器 → GMEM 链,外面没有套任何循环。它分配工作内存、加载操作数、计算乘积、写回结果,最后自我清理:

1. 分配: SMEM(池分配器)、TMEM(tcgen05.alloc)、mbarrier
2. 加载: 全部 128 个线程协作地把 A 和 B 分块从 GMEM 拷到 SMEM(同步 Tx.copy)
3. 计算: 单个被选中的线程发起 Tx.gemm\_async + tcgen05.commit; 所有线程在 mbarrier 上等待
4. 回写: warpgroup 读 TMEM → 寄存器; 每个线程把 fp32→fp16 转换后写回 GMEM
5. 释放: 释放 TMEM

## 第一个内核的四部分

完整内核只有几十行,但分块来看更易消化。我们分四段来读(内存分配、同步加载、MMA 派发、回写),最后再拼装成一个内核。沿途出现的 API 名字,正是第二部分([TIRx 简介](#)、[TIRx 布局 API](#))介绍过的 TIRx 分块原语词汇。

**内存分配。**内核开头先为操作数切出共享内存,再留一个存放 TMEM 地址的槽和一个 mbarrier:

```
pool = T.SMEMPool()
tmem_addr = pool.alloc((1,), "uint32")           # TMEM 地址 (4 字节)
mma_bar = pool.alloc((1,), "uint64", align=8)     # mbarrier(8 字节)
pool.move_base_to(1024)                          # 跳到偏移 1024
Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout) # ↪
↪128×64 fp16
Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout) # ↪
↪128×64 fp16
pool.commit()
```

这里有两处细节值得停下来想一想。pool.move\_base\_to(1024) 把 Asmem 和 Bsmem 推到偏移 1024 处,从而把它们上方那些小块元数据的低地址保留下来,让大块的操作数分块落在一个干净的边界上。而 layout=A\_layout 向 tma\_shared\_layout 请求一种经过 swizzle 的 SMEM 摆放方式,使得 TMA 和 tcgen05.mma 都能直接读取——这正是第二部分描述的那种「以布局为契约」的义务。

**同步加载。**缓冲区就位后,操作数还得真正进入 SMEM。在第一个版本里,我们让 CTA 自己的线程来做这次拷贝:

```
Tx.cta.copy(Asmem[:, :], A[:, :])
Tx.cta.copy(Bsmem[:, :], B[:, :])
T.cuda.cta_sync()
```

因为这里只有一个分块 (M=N=128,K=64), 把整个 A 和 B 拷过去就是全部的加载工作。Tx.cta.copy(...) 让 CTA 在这次拷贝上协作,每个线程负责自己那一份数据切片。随后的 T.cuda.cta\_sync() 一身二用: 它既等待每个线程完成,又发布它们的共享内存写操作,从而保证 MMA 之后读 Asmem 和 Bsmem 时看到的是完整分块,而不是只填了一半的缓冲区。这种由线程驱动的拷贝也正是我们最先要替换的东西;下一章(用 [TMA](#) 为 GEMM 建立流水线)会把它换成 TMA。

**MMA 派发。**操作数现在已落在 SMEM 里, 我们可以发起 MMA, 并从一个被选中的线程来发起:

```
if warp_id == 0:
    if T.ptx.select_sync():
        Tx.gemm_async(tmemb[:, :BLK_N], Asmem[:, :], Bsmemb[:, :],
                      accum=False, dispatch="tcgen05", cta_group=1)
        T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1)
```

两层嵌套的守卫用两步把发起者收窄下来。外层的 `if warp_id == 0` 只保留 warpgroup 中的 warp 0, 内层的 `if T.ptx.select_sync()`: 再从该 warp 中选出一个活跃 lane。两者合起来恰好只剩一个线程来运行 `Tx.gemm_async` 和 `tcgen05.commit`。

关于这一个线程到底意味着什么、不意味着什么, 有必要讲清楚, 因为最自然的解读是误导性的。单个发起线程并不意味着一次单线程的乘法。计算本身仍然是一次完整的分块级 MMA: 硬件对由 SMEM 操作数布局 and TMEM 累加器布局共同描述的那个分块执行协作式乘法。关键在于, `Tx.gemm_async` 是一个分块操作, 而不是一条硬件指令。K = 64 的分块比硬件 MMA 的 K 原子 (MMA\_K = 16) 更宽, 因此这一次分块操作会降级为沿 K 步进的一小串原始 `tcgen05.mma` 指令, 并由 warpgroup 协作地驱动每一条。只让一个线程发起这个分块操作的原因在于, 每一条底层 `tcgen05.mma` 本身就是一次单指令的协作操作: 一次发起就驱动分块 MMA 中那个 K 原子的计算。如果 128 个线程都发起这一串指令, 同样的工作只不过被发起 128 次而已。最后, `accum=False` 标志告诉 MMA 去覆写 TMEM 目标, 而不是累加进去——这正是我们在此想要的, 因为还没有任何先前的部分和可供延续。

**回写。**乘积现在落在 TMEM 里, 但调用方希望它以 fp16 形式回到 GMEM。因此收尾部分必须把结果经由寄存器搬下来, 并在途中做类型转换:

```
Dreg = T.alloc_local((BLK_N,), acc_type) # 每线程一行的 fp32 寄存器
Dreg_f16 = T.alloc_local((BLK_N,), d_type) # 同一行, 转换成 fp16
Dreg_wg = Dreg.view(128, BLK_N, layout=TileLayout(S[(128, BLK_N) : ↵
↵(1@tid_in_wg, 1)]))
Tx.wg.copy_async(Dreg_wg[:, :], tmemb[:, :BLK_N])
T.ptx.tcgen05.wait_ld()
Tx.cast(Dreg_f16[:, :], Dreg[:, :])
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)
Tx.copy(D[m_thr, n_st : n_st + BLK_N], Dreg_f16[:, :])
```

MMA 在 TMEM 中留下一个 128 x 128 的 fp32 累加器分块。这里用 fp32 是刻意为之: GEMM 沿 K 对许多乘积求和, 以更高精度保存这个累加和可以压住本会累积起来的舍入误差。但 D 是 fp16, 所以这些值不能直接送出去。它们先落到寄存器, 在那里收窄成 fp16, 然后才进入 GMEM。

两个寄存器缓冲区各司其职。Dreg 是一个每线程 BLK\_N 个元素的缓冲区, 而 Dreg\_wg 则是在某种选定布局下, 对同一批寄存器的一个 warpgroup 级别视图:

```
TileLayout(S[(128, BLK_N) : (1@tid_in_wg, 1)])
```

这一布局把分块的第一维映射到 warpgroup 的线程上: 线程 0 拥有第 0 行, 线程 1 拥有第 1 行, 依此类推到第 127 行。第二维则留在每个线程自己的寄存器缓冲区内, 因此单个线程持有自己那一行的所有列。warpgroup 有 128 个线程, 分块有 128 行, 这个 128 x 128 的输出就整整齐齐地划分为每线程一行。

在这个视图下读出累加器, 正是 `Tx.wg.copy_async(Dreg_wg, tmemb)` 所做的事, 它会降级为 Blackwell 的 TMEM 加载路径 `tcgen05.ld`。由于该加载是异步的, 任何线程在触碰 Dreg



之前,都必须先让 `T.ptx.tcgen05.wait.ld()` 完成; 否则线程会读到加载尚未填满的寄存器。

等待返回之后, 每个线程私有的 `Dreg[:]` 里就持有了它所负责的那一逻辑输出行的 `fp32` 值。线程把这些值在 `Dreg_f16` 里收窄成 `fp16`, 算出自己负责的全局行号,

```
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)
```

再写 `D[m_thr, n_st:n_st + BLK_N]`。这些行在四个 warp 之间干净地划分:warp 0 写第 0–31 行,warp 1 写第 32–63 行,warp 2 写第 64–95 行,warp 3 写第 96–127 行。

## 完整内核

现在我们把四部分缝合回一个可运行的内核 ( $M=N=128, K=64$ )。导入语句排在最前面:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_
    shared_layout, SwizzleMode
from tvm.tirx.layout import TileLayout, S, TLane, TCol, tid_in_wg
```

内核包在后续步骤同样使用的 `hgemm_vX(M, N, K)` 风格里。第 1 步以  $M=N=128, K=64$  运行, 所以启动配置里恰好只有一个输出分块:

```
def hgemm_v1(M, N, K):
    a_type = tvm.DataType("float16")
    b_type = tvm.DataType("float16")
    d_type = tvm.DataType("float16")
    acc_type = tvm.DataType("float32")

    BLK_M, BLK_N, BLK_K = 128, 128, 64
    # MMA_M/MMA_N/MMA_K 记录底层硬件 MMA 分块的大小; 它们不会被
    # 传给 gemm_async(后者会从操作数和累加器分块推导出 MMA 形状),
    # 因此后续步骤省略了它们。
    MMA_M, MMA_N, MMA_K = 128, 128, 16

    A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_
    →ATOM, (BLK_M, BLK_K))
    B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_
    →ATOM, (BLK_N, BLK_K))

    @T.prim_func
    def kernel(
        A: T.Buffer((M, K), a_type),
        B: T.Buffer((N, K), b_type),
        D: T.Buffer((M, N), d_type),
    ):
        T.device_entry()
        # 第 1 步是一个单分块内核:  $M = BLK_M$  且  $N = BLK_N$ , 所以 grid
        # 是  $1 \times 1$ 。从  $1 \times 1$  的 grid 出发, 可使每个 CTA 的分块偏移
```

(continues on next page)



(continued from previous page)

```

# (m_st, n_st) 平凡地为零; 第 3 步及以后会把它推广到更大的 M / N。
bx, by = T.cta_id([M // BLK_M, N // BLK_N])
wg_id = T.warpgroup_id([1]) # 单个 warpgroup, 所以 wg_id_
→ 恒为 0 (下方未使用)
warp_id = T.warp_id_in_wg([4])
lane_id = T.lane_id([32])

# --- SMEM 分配 ---
pool = T.SMEMPool()
tmem_addr = pool.alloc((1,), "uint32")
mma_bar = pool.alloc((1,), "uint64", align=8)
pool.move_base_to(1024)
Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout)
Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout)
pool.commit()

# --- 屏障 + TMEM 初始化 (仅 warp 0) ---
if warp_id == 0:
    if lane_id == 0:
        T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1)
        T.ptx.tcgen05.alloc(T.address_of(tmem_addr), n_cols=512,
→ cta_group=1)

    T.ptx.fence.proxy_async("shared::cta")
    T.ptx.fence.mbarrier_init()
    T.cuda.cta_sync()

    tmem = T.decl_buffer(
        (128, 512), "float32", scope="tmem", allocated_
→ addr=tmem_addr[0],
        layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)])
    )

    m_st = T.meta_var(bx * BLK_M)
    n_st = T.meta_var(by * BLK_N)
    phase_mma: T.int32 = 0

# --- 加载: 所有线程协作把 global -> shared(同步)。
# 由于 M=BLK_M 且 N=BLK_N, 下面的切片覆盖了整个矩阵;
# 保留切片形式, 使与第 3 步 (多分块) 的 diff 最小。
Tx.cta.copy(Asmem[:, :], A[m_st:m_st + BLK_M, :])
Tx.cta.copy(Bsmem[:, :], B[n_st:n_st + BLK_N, :])
T.cuda.cta_sync()

# --- 计算: 单个被选中的线程发起 MMA ---
if warp_id == 0:
    if T.ptx.select_sync():
        Tx.gemm_async(
            tmem[:, :BLK_N], Asmem[:, :], Bsmem[:, :],

```

(continues on next page)

(continued from previous page)

```

        accum=False, dispatch="tcgen05", cta_group=1
    )
    T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_
→group=1)

    T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma)

    # --- 回写:TMEM -> RF -> GMEM ---
    Dreg = T.alloc_local((BLK_N,), acc_type)
    Dreg_f16 = T.alloc_local((BLK_N,), d_type)
    Dreg_wg = Dreg.view(128, BLK_N,
                        layout=TileLayout(S[(128, BLK_N) :
→(1@tid_in_wg, 1)]))
    Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N])
    T.ptx.tcgen05.wait.ld()
    Tx.cast(Dreg_f16[:, :], Dreg[:, :])
    m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)
    Tx.copy(D[m_thr, n_st : n_st + BLK_N], Dreg_f16[:, :])

    # --- 释放 TMEM ---
    T.cuda.cta_sync()
    if warp_id == 0:
        T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
        T.ptx.tcgen05.dealloc(tmем_addr[0], n_cols=512, cta_
→group=1)

    return kernel

```

之后每一个 GEMM 步骤都以同样的方式编译、运行并自检, 所以我们把这套脚手架在这里完整地讲一遍, 从此之后就只展示内核。要运行后面的某个步骤, 只需把下面的 `hgemm_vX` 换成对应的版本, 再配上匹配的问题规模即可。有一点值得记住: 每个新步骤请在一个全新的 Python 会话里单独编译, 在尝试下一个之前重启, 因为示例复用了内部名字, 而编译器持有按会话维度的状态。

```

import torch

target = tvm.target.Target("cuda")
device = torch.device('cuda') # gpu(0)

M, N, K = 128, 128, 64
kernel = hgemm_v1(M, N, K)
with target:
    ex = tvm.compile(tvm.IRModule({"main": kernel}), target=target,
→tir_pipeline="tirx")

torch.cuda.empty_cache()
torch.cuda.synchronize()
A_tensor = torch.randn(M, K, dtype=torch.float16, device=device)
B_tensor = torch.randn(N, K, dtype=torch.float16, device=device)

```

(continues on next page)

(continued from previous page)

```

D_tensor = torch.zeros(M, N, dtype=torch.float16, device=device)

# ex.mod(...) 直接接收 torch 张量, 与每一章使用的调用形式一致。
ex.mod(A_tensor, B_tensor, D_tensor)

D_ref = (A_tensor.float() @ B_tensor.float().T).half()
max_err = float((D_tensor - D_ref).abs().max())
print(f"Max error vs torch reference: {max_err:.6f}")
# 相对容差, 与线程束特化和 Flash Attention 的单元一致:
# 输出幅值随 K 增长, 所以一个固定的绝对上界会在更大的 K 下失效。
torch.testing.assert_close(D_tensor, D_ref, rtol=2e-2, atol=1e-2)
print("PASS")

# 可选: 对更大的内核做计时。
ITERS = 10
for _ in range(3):
    ex.mod(A_tensor, B_tensor, D_tensor)
    torch.cuda.synchronize()
    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    start.record()
    for _ in range(ITERS):
        ex.mod(A_tensor, B_tensor, D_tensor)
    end.record()
    torch.cuda.synchronize()
    ms = start.elapsed_time(end) / ITERS
    tflops = 2 * M * N * K / ms / 1e9
    print(f"Performance: {ms:.3f} ms, {tflops:.1f} TFLOPS")

```

第 1 到第 3 步刻意采用很小的规模 (这里是  $128 \times 128$ , 第 3 步是  $256^3$ ), 以让这几段最初的走读保持简单易跟。用线程束特化与集群扩展 GEMM 末尾那张跨步骤的端到端结果表则采取相反的做法: 它在单一的  $M=N=K=4096$  规模下测量每一个步骤 (包括这个第 1 步算法), 从而使其加速比能够直接比较。

### 单分块内核的局限

这个内核是正确的, 这正是第 1 步的全部意义所在, 但它只在一种非常狭窄的设定下正确。有四点局限是刻意为之的, 优化路径的其余部分会逐一消除它们:

- 它只处理单个 K 分块, 因此无法在一个大的 K 上做内积。
- 它只处理单个输出分块, 所以 M 和 N 都被钉死在 128。
- 它使用同步的 GMEM  $\rightarrow$  SMEM 拷贝, 而不是 TMA。
- 它不让数据搬运与计算重叠, 所以两者从不同时运行。

### 1.12.4 第 2 步:K 维循环累加

第一个要去除的局限是最小的那个。第 1 步只处理一个 64 宽的 K 分块, 而真实矩阵上要做的内积远不止于此。第 2 步里我们仍保留单个输出分块, 但让 K 跨越许多个 64 宽的块。

思路很直接: 对每个块重复一次加载 -> MMA -> 等待序列, 并让每次 MMA 累加到同一个 TMEM 槽里。真正的工作量其实在于同步。在多次迭代间复用同一个 mbarrier, 引出了本章第一个真正意义上的正确性陷阱。如果代码跟踪错了相位, 一次等待可能在其 MMA 实际完成之前就返回, 悄无声息地破坏结果。下面的机制说明了这究竟是怎么出错的, 以及如何避免。

#### 这一步改变的是: 布局复用

- 作用域: 不变, 仍是一个单一的 warpgroup。
- 布局/复用: 同一对 SMEM 分块和同一个 TMEM 累加器槽在 K 循环间被复用。不分配新的存储; 操作数分块流过一对固定的缓冲区, 累加器状态停留在一个 TMEM 槽里。
- 同步: 被复用的 MMA 屏障必须在每一个 K 块上推进到正确的相位, 否则后续的某次等待可能观测到的是更早一次的完成。
- 派发: 不变。

#### K 循环机制

第 1 步在单个 64 宽的 K 分块上做内积; 这里我们保留它的单个输出分块, 但让 K 想跑多长就跑多长。为了覆盖大于 64 的 K, 我们以 `BLK_K=64` 为步长逐块地走完 K。每次迭代把下一个 A 和 B 的 K 切片加载进 SMEM, 并发起 `Tx.gemm_async`。`accum` 标志正是把这些块缝合进同一点积的关键: 第一个块上 `accum=False` 初始化 TMEM 累加器, 其后每个块上 `accum=True` 把该块的乘积加到已经在 TMEM 里的累加和上。

同步是这里需要小心的地方。我们为每一次 MMA 完成复用同一个 mbarrier, 而安全复用它归结为跟踪我们正在等待的是哪一个屏障相位。mbarrier 持有一个 1 比特的相位, 取 0 或 1, 每当预期的到达落下来时就翻转到另一个值。微妙之处在于等待条件本身: `try_wait(bar, phase)` 会一直阻塞, 直到屏障内部相位与传入的 `phase` 不同。所以我们传入的实参, 必须命名我们期望「离开」的那个相位, 而不是我们正「等待到达」的那个相位:

| K 迭代 | 等待前本地 <code>phase_mma</code> | <code>try_wait</code> 等待什么 | 等待后本地更新                    |
|------|------------------------------|----------------------------|----------------------------|
| 0    | 0                            | 屏障翻转到 1                    | <code>phase_mma = 1</code> |
| 1    | 1                            | 屏障翻转到 0                    | <code>phase_mma = 0</code> |
| 2    | 0                            | 屏障翻转到 1                    | <code>phase_mma = 1</code> |

`phase_mma ^= 1` 这一行正是让上表成立的关键。把它去掉, 第二次迭代仍会调用 `try_wait(bar, 0)`, 但屏障在第一次 MMA 之后已经翻到相位 1, 于是这次等待看到不匹配就立即返回, 赶在第二次 MMA 完成之前。内核随之读到一个算了一半的累加器, 在没有任何报错的情况下给出错误答案。这种 bug 编译和运行起来都完美无瑕, 这也正是相位翻转值得如此重视的原因。

#### 完整内核

下面的完整内核, 无非就是把第 1 步折入 K 循环和相位翻转。导入语句与之前相同:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_
    ↪ shared_layout, SwizzleMode
from tvm.tirx.layout import TileLayout, S, TLane, TCol, tid_in_wg
```

它包在 `hgemm_v2(M, N, K)` 里。grid 仍是 `[1, 1]`, 因为我们仍在计算单个输出分块; 增长的只是它的 `K` 范围:

```
def hgemm_v2(M, N, K):
    a_type = tvm.DataType("float16")
    b_type = tvm.DataType("float16")
    d_type = tvm.DataType("float16")
    acc_type = tvm.DataType("float32")

    BLK_M, BLK_N, BLK_K = 128, 128, 64
    K_TILES = K // BLK_K

    A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_
    ↪ ATOM, (BLK_M, BLK_K))
    B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_
    ↪ ATOM, (BLK_N, BLK_K))

    @T.prim_func
    def kernel(
        A: T.Buffer((M, K), a_type),
        B: T.Buffer((N, K), b_type),
        D: T.Buffer((M, N), d_type),
    ):
        T.device_entry()
        bx, by = T.cta_id([M // BLK_M, N // BLK_N]) # 仍是单个输出分块
        (M=N=128)
        wg_id = T.warpgroup_id([1])
        warp_id = T.warp_id_in_wg([4])
        lane_id = T.lane_id([32])

        pool = T.SMEMPool()
        tmem_addr = pool.alloc((1,), "uint32")
        mma_bar = pool.alloc((1,), "uint64", align=8)
        pool.move_base_to(1024)
        Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout)
        Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout)
        pool.commit()

        if warp_id == 0:
            if lane_id == 0:
                T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1)
                T.ptx.tcgen05.alloc(T.address_of(tmem_addr), n_cols=512,
```

(continues on next page)



(continued from previous page)

```

→ cta_group=1)

    T.ptx.fence.proxy_async("shared::cta")
    T.ptx.fence.mbarrier_init()
    T.cuda.cta_sync()

    tmem = T.decl_buffer(
        (128, 512), "float32", scope="tmem", allocated_addr=tmem_
→ addr[0],
        layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)]))

    phase_mma: T.int32 = 0
    m_st = T.meta_var(bx * BLK_M)
    n_st = T.meta_var(by * BLK_N)

    # === K 循环: 以 BLK_K 为步长在 K 上迭代 ===
    for i in T.serial(K_TILES): # 串行 device 循环 (保持全 K 的
→ A/B 参数形状正确)
        # 加载第 i 个 K 块
        Tx.cta.copy(Asmem[:, :], A[:, i*BLK_K:(i+1)*BLK_K])
        Tx.cta.copy(Bsmem[:, :], B[:, i*BLK_K:(i+1)*BLK_K])

        T.cuda.cta_sync()

        # MMA: 首个分块 accum=False, 其余为 True
        if warp_id == 0:
            if T.ptx.elect_sync():
                Tx.gemm_async(tmem[:, :BLK_N], Asmem[:, :],
→ Bsmem[:, :],
                                accum=(i != 0), dispatch="tcgen05
→ ", cta_group=1)
                T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_
→ group=1)

            # 等待 MMA, 然后翻转相位
            T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma)
            phase_mma ^= 1

        # === 回写 (与第 1 步相同)===
        Dreg = T.alloc_local((BLK_N,), acc_type)
        Dreg_f16 = T.alloc_local((BLK_N,), d_type)
        Dreg_wg = Dreg.view(128, BLK_N,
→ layout=TileLayout(S[(128, BLK_N) :
→ (1@tid_in_wg, 1)]))

        Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N])
        T.ptx.tcgen05.wait.ld()

        Tx.cast(Dreg_f16[:, :], Dreg[:, :])

```

(continues on next page)

(continued from previous page)

```

m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)
Tx.copy(D[m_thr, n_st : n_st + BLK_N], Dreg_f16[:])

T.cuda.cta_sync()
if warp_id == 0:
    T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
    T.ptx.tcgen05.dealloc(tmем_addr[0], n_cols=512, cta_
→group=1)

return kernel

```

### 1.12.5 第 3 步: 空间分块 (多 CTA)

K 循环处理了内积维度, 但 M 和 N 仍被钉死在单个 128 x 128 分块上。真实的输出远大于一个分块, 所以基本内核的最后一块拼图, 是用许多分块同时覆盖 M 和 N。第 3 步启动一个二维的 CTA grid, 每个输出分块对应一个 CTA, 让 GPU 并行地计算所有分块。示例采用 M=N=K=256, 得到一个 2x2 的分块网格, 刚好足以让下标变得不再平凡, 又不至于把它埋没。

这一步改变的是: 作用域

- 作用域: 一个二维的 CTA grid, 每个 CTA 拥有一个 128 x 128 的输出分块。
- 布局: 不变; 在一个 CTA 内部, 与第 2 步一样是同一条 SMEM/TMEM/寄存器路径。
- 派发: 不变。

#### Grid 映射

grid 的形状直接由分块方式决定: 每个 128 x 128 输出分块对应一个 CTA, 总共需要  $\lceil M / \text{BLK\_M} \rceil, \lceil N / \text{BLK\_N} \rceil$  个 CTA。与第 2 步相比, 唯一真正全新的工作, 就是教会每个 CTA 矩阵里的哪一片是它自己要计算的那一片。

CTA (bx, by) 拥有这个输出区域:

```

D[bx * BLK_M : (bx + 1) * BLK_M,
  by * BLK_N : (by + 1) * BLK_N]

```

为产出它, 该 CTA 的 K 循环会反复加载它自己的 A 行带和 B 列带所对应的那些 K 切片:

```

A[bx * BLK_M : (bx + 1) * BLK_M, k : k + BLK_K]
B[by * BLK_N : (by + 1) * BLK_N, k : k + BLK_K]

```

这里的下标直接源自  $D = A @ B.T$  的约定: bx 选出 A 和 D 的行, 而 by 选出 B 的行, 这些行在施加转置后就成了 D 的列。

每个 CTA 一个分块是最简单可行的映射, 但也很浪费。同一行里的每个 CTA 都会从 GMEM 重新加载相同的 A 分块, 同一列里的每个 CTA 都会重新加载相同的 B 分块, 所以完全谈不上复用相邻 CTA 已经拉进来的数据。我们暂且把这份浪费搁置在原地; 持久化调度 (用 TMA 为 GEMM 建立流水线的第 6 步) 会回过头来处理它, 把这些共享的操作数保持在 L2 里热着。

交给你的 **agent** 试一试: 在  $M=N=K=256$ 、 $BLK\_M=BLK\_N=128$ 、 $BLK\_K=64$  下, 让它追踪 CTA (1, 0) 和 CTA (0, 1)。对每个 CTA, 列出  $m\_st$ 、 $n\_st$ 、每次  $K$  迭代加载的  $A$  和  $B$  切片, 以及所写的  $D$  区域。因为内核计算的是  $D = A @ B$ , 哪些  $B$  行会变成  $D$  的列?

## 完整内核

这个内核再一次是第 2 步, 这次只有两处改动:  $grid$  形状和每个 CTA 的偏移。内部的  $K$  循环和回写原封不动。导入语句相同:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_
    shared_layout, SwizzleMode
from tvm.tirx.layout import TileLayout, S, Tlane, TCol, tid_in_wg
```

$grid$  变成  $[M // BLK\_M, N // BLK\_N]$  而不是  $[1, 1]$ , 加载和存储都按 CTA 自己的  $m\_st$  与  $n\_st$  偏移:

```
def hgemv_v3(M, N, K):
    a_type = tvm.DataType("float16")
    b_type = tvm.DataType("float16")
    d_type = tvm.DataType("float16")
    acc_type = tvm.DataType("float32")

    BLK_M, BLK_N, BLK_K = 128, 128, 64
    K_TILES = K // BLK_K

    A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_
    ATOM, (BLK_M, BLK_K))
    B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_
    ATOM, (BLK_N, BLK_K))

    @T.prim_func
    def kernel(
        A: T.Buffer((M, K), a_type),
        B: T.Buffer((N, K), b_type),
        D: T.Buffer((M, N), d_type),
    ):
        T.device_entry()
        # 二维 grid: 每个 128x128 输出分块对应一个 CTA
        bx, by = T.cta_id([M // BLK_M, N // BLK_N])
        wg_id = T.warpgroup_id([1])
        warp_id = T.warp_id_in_wg([4])
        lane_id = T.lane_id([32])

        pool = T.SMEMPool()
        tmem_addr = pool.alloc((1,), "uint32")
        mma_bar = pool.alloc((1,), "uint64", align=8)
        pool.move_base_to(1024)
```

(continues on next page)

(continued from previous page)

```

Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout)
Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout)
pool.commit()

if warp_id == 0:
    if lane_id == 0:
        T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1)
        T.ptx.tcgen05.alloc(T.address_of(tmем_addr), n_cols=512,
→ cta_group=1)

        T.ptx.fence.proxy_async("shared::cta")
        T.ptx.fence.mbarrier_init()
        T.cuda.cta_sync()

        tmем = T.decl_buffer(
            (128, 512), "float32", scope="tmем", allocated_addr=tmем_
→addr[0],
            layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)]))

        phase_mma: T.int32 = 0

        # 每个 CTA 的分块偏移
        m_st = T.meta_var(bx * BLK_M)
        n_st = T.meta_var(by * BLK_N)

        # 带偏移 A、B 切片的 K 循环
        for i in T.serial(K_TILES): # 串行 device 循环 (保持全 K 的
→A/B 参数形状正确)
            Tx.cta.copy(Asmem[:, :], A[m_st:m_st+BLK_M, i*BLK_
→K:(i+1)*BLK_K])
            Tx.cta.copy(Bsmem[:, :], B[n_st:n_st+BLK_N, i*BLK_
→K:(i+1)*BLK_K])

            T.cuda.cta_sync()

            if warp_id == 0:
                if T.ptx.elect_sync():
                    Tx.gemm_async(tmем[:, :BLK_N], Asmem[:, :],
→Bsmem[:, :],
                                accum=(i != 0), dispatch="tcgen05
→", cta_group=1)
                    T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_
→group=1)

                    T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma)
                    phase_mma ^= 1

            # 回写到正确的输出分块
            Dreg = T.alloc_local((BLK_N,), acc_type)

```

(continues on next page)

(continued from previous page)

```

Dreg_f16 = T.alloc_local((BLK_N,), d_type)
Dreg_wg = Dreg.view(128, BLK_N,
                    layout=TileLayout(S[(128, BLK_N) :
↪(1@tid_in_wg, 1)]))

Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N])
T.ptx.tcgen05.wait.ld()

Tx.cast(Dreg_f16[:, :], Dreg[:, :])
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)
Tx.copy(D[m_thr, n_st:n_st+BLK_N], Dreg_f16[:, :])

T.cuda.cta_sync()
if warp_id == 0:
    T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
    T.ptx.tcgen05.dealloc(tmem_addr[0], n_cols=512, cta_
↪group=1)

return kernel

```

### 1.12.6 练习

1. 在第 1-3 步中, Tx.copy 在 MMA 之前把 A 和 B 分块搬进 SMEM。为什么内核在 Tx.gemm\_async 读取这些 SMEM 分块之前需要 T.cuda.cta\_sync()?
2. 在第 2 步中, 如果把 phase\_mma ^= 1 从 K 循环里去掉会怎样? 内核还会等待每一次 MMA 吗, 还是某次后续的等待会过早通过?
3. 对于 M=N=4096、BLK\_M=BLK\_N=128, 第 3 步会启动多少个 CTA? 哪些操作数分块在相邻 CTA 之间被逻辑上复用, 而第 3 步是否利用了这种复用?

## 1.13 用 TMA 为 GEMM 建立流水线

### Overview

- 基础 GEMM 浪费了时间在轮替上 (拷贝一个分块、计算、再拷贝下一个), 而这两件事本可以同时进行。
- Step 4 切换到 TMA 异步加载, Step 5 对 SMEM 做双缓冲并预取 (PIPE\_DEPTH=2); 完整的加载/计算重叠要到 Step 7 借助 warp specialization 才能到来, Step 6 则通过分块调度器把内核改造成持久化内核。
- 目标是在 Tensor Core 咀嚼当前分块的同时, 加载下一个分块。

Tensor Core 是芯片上最昂贵的单元, 而上一章那版正确的分块化 GEMM 让它们在大部分时钟周期里都处于空闲。内核在轮替: 线程把一个分块拷贝进共享显存, Tensor Core 咀嚼它, 线程再拷贝下一个分块, Tensor Core 继续等。每一个阶段都要卡在前一个阶段上, 哪怕加载下一个分块和在当前分块上做计算用的是完全独立的硬件、本可以同时跑。要弥合这个空隙并不需要新

的数据通路;分块、布局和数学都已经是对的。真正需要改变的是工作发生的时机和由谁来调度。本章保持分块数据通路原封不动,直接攻击空闲问题。

我们分三个递进步骤走到那里,而动手之前先看清终点会有帮助。在 Step 4,我们把 GMEM 与 SMEM 之间的大块传输交给 TMA,由专门的拷贝硬件去搬运分块,而不是让线程来搬。在 Step 5,我们加入一个两级的软件流水线,让下一个 K 分块在当前分块还在被乘的时候有地方可落。在 Step 6,我们把启动方式改造成由分块调度器驱动的持久化内核,从而摊薄每个分块的启动开销,并让我们能挑选一个让操作数保持热的分块顺序。贯穿全程,SMEM、TMEM 和寄存器的布局都保持上一章留下的原样。唯一真正的新想法,是硬件单元之间的异步交接:让一个引擎跑在另一个引擎前面,而不是让它们齐步前进。

### 1.13.1 Step 4:TMA 异步加载

我们的第一招,是把拷贝本身挪出关键路径。回想一下 Step 1-3 里 CTA 在做什么:它的每一个线程都计算地址、派发加载指令,目的不过是为了把分块搬进 SMEM。这些指令带宽都花在了管道铺设上,而不是数学上。Step 4 用 TMA 替换掉同步的 `Tx.copy`,只需一个线程派发一条命令,TMA 引擎就会自己完成整个分块的传输。从这一步起,示例都跑在完整的  $M=N=K=4096$  规模上,而不再是 Step 1-3 的小规模,它们的端到端计时出现在[用线程束特化与集群扩展 GEMM](#)末尾的 *End-to-End Result* 表里。

本步改变的是:Dispatch

- Scope: 不变,仍是一个 warpgroup。
- Layout: 不变,沿用相同的 SMEM/TMEM/寄存器分块。
- Dispatch:GMEM  $\rightarrow$  SMEM 的加载从同步 `Tx.copy` 改走 TMA 引擎。

#### TMA 的派发模式

Step 4 的唯一改动,就是把同步的分块拷贝换成 TMA 加载,所以值得仔细看看这一加载是如何派发的。对源码的修改只有寥寥几行,但这些行背后的执行模型却是本质上的不同。同步的 `Tx.copy` 是 CTA 线程用自己的指令自己干的活;TMA 拷贝则是由一个线程派发的一条命令,之后所有搬运都由 TMA 硬件完成。两者并排放在一起看一看是有价值的。

之前 (Step 3): 全部 128 个线程参与拷贝,随后 `cta_sync` 让共享显存的写入对后续可见:

```
Tx.cta.copy(Asmem[:, :], A[m_st:m_st+BLK_M, i*BLK_K:(i+1)*BLK_K])
↪# 全部 128 个线程
Tx.cta.copy(Bsmem[:, :], B[n_st:n_st+BLK_N, i*BLK_K:(i+1)*BLK_K])
T.cuda.cta_sync()
```

之后 (Step 4): 一个线程派发 TMA 加载,mbarrier 跟踪硬件传输何时完成:

```
tid = warp_id * 32 + lane_id # warpgroup 内的 0..127
if tid == 0: # 恰好由一个线程启动 TMA
    Tx.copy_async(Asmem, A[...], dispatch="tma")
    Tx.copy_async(Bsmem, B[...], dispatch="tma")
    T.ptx.mbarrier.arrive.expect_tx(tma_bar, byte_count) # TMA 期望
收到的字节数
T.ptx.mbarrier.try_wait(tma_bar, phase) # 在 MMA 读
取 SMEM 之前等待
```

注意,加载是以 `tid == 0` 为门控,而不是以 `elect_sync()` 为门控,这个区别比看上去更重要。`elect.sync` 是每个 warp 选出一个活跃 lane,而一个 warpgroup 有四个 warp,所以



`elect_sync()` 实际上会让四个线程进入加载协议。问题在于, 该协议要向 `mbarrier` 宣告期望的字节数, 而且必须恰好宣告一次; 宣告四次会破坏这个计数, 等待也就永远无法正确释放。按照 `warpgroup` 范围内的 `id` 精确挑出一个线程, 才是干净避开这一问题的办法。

关于加速来自何处, 必须诚实。Step 4 在每次 TMA 加载之后仍然要等待, 所以此时我们还没有把加载和计算重叠起来; 那是 Step 5 的工作。此处的收益纯粹来自数据搬移通路的改变:

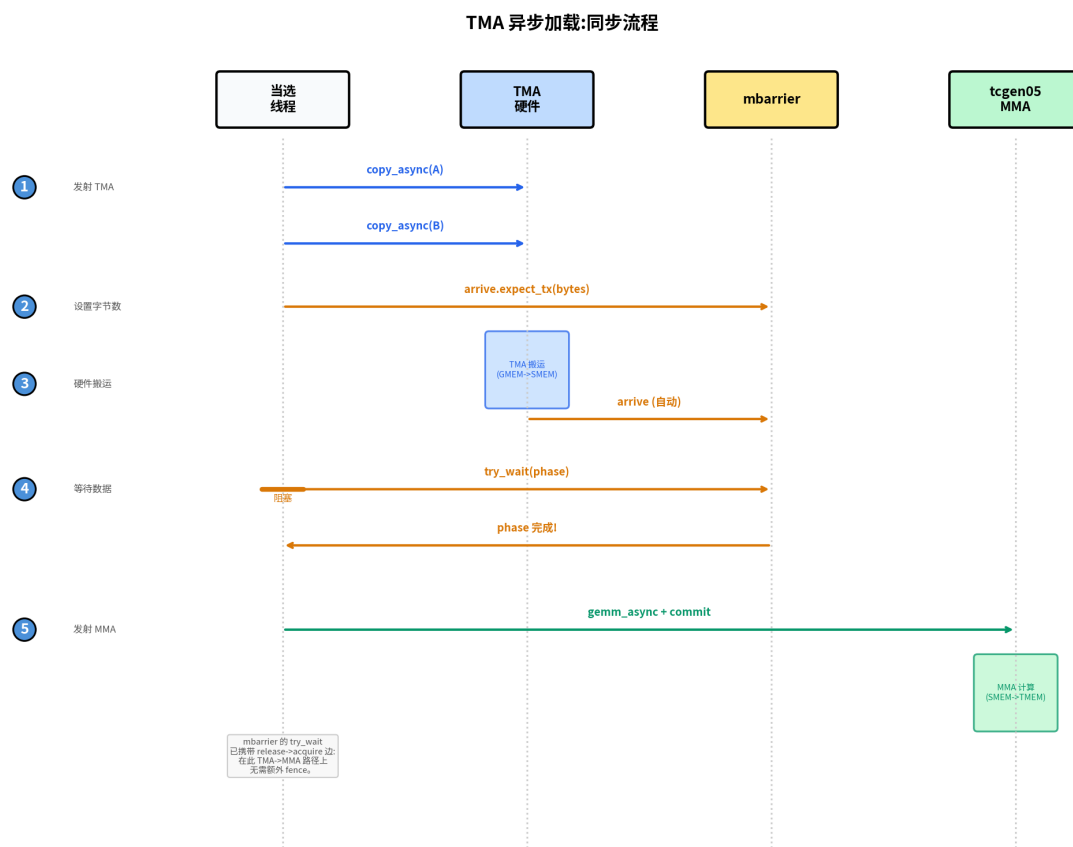
- `Tx.copy` 用 CTA 线程去计算地址、派发加载/存储指令。
- TMA 用一条派发出去的命令启动一次硬件分块传输。地址生成、合并和 `swizzling` 都由 TMA 描述符描述, 并由 TMA 引擎执行。

所以, 即便 Step 4 仍在每次加载上阻塞, 它最终依然更快。TMA 吸收掉了大块传输, 把 CTA 线程从为搬运分块耗费指令带宽中解放出来, 光是这一项节省就足以带来可观的提升。

## TMA 加载与存储的同步

我们已经看到 TMA 拷贝是如何派发的; 故事的另一半, 是知道它何时完成。切换到 TMA 会同时改变两件事: 谁启动一次拷贝, 以及代码如何知道它完成了。第一点从代码里一目了然; 第二点却很容易被忽视, 而且弄错了不会崩溃, 只会给你一个悄无声息的正确性 bug。用 `Tx.cta.copy` 时, CTA 线程一起做拷贝, 后面跟一个 `cta_sync()` 就足以知道它完成了。用 TMA 时, 一个被选中的线程派发 `Tx.copy_async(..., dispatch="tma")`, 引擎按自己的节奏完成传输, 并通过一个 `mbarrier` 信号告知完成。

这正是 `cta_sync()` 不再够用的原因。`cta_sync()` 只等待 CTA 自己的线程, 也只对这些线程的共享显存写入排序; 它对一次在途的 TMA 传输一无所知, 所以会在分块还在抵达时就欣然返回。修正办法是把完成判定显式化: 对于一次 TMA 加载, 被选中的线程先告诉 `mbarrier` 期望多少字节, 然后 CTA 在任何 MMA 触碰这个 SMEM 分块之前, 在那个 `mbarrier` 上等待。下图端到端地描绘了这一握手。



上图把加载侧的握手单独剥离出来: 一个被选中的线程启动 TMA, mbarrier 计数期望的字节, MMA 在读取 SMEM 之前等待其释放。文中凡说“Elected Thread”之处, 都指那个启动 TMA 的被选中的线程, 在我们的代码里就是 `tid == 0` 那个线程, 而不是 `elect_sync()` 选出的某个 lane。

于是把加载通路拼起来: 被选中的线程派发出两个 `copy_async`, 再跟上 `arrive.expect_tx(total_bytes)`, 其中字节数恰好就是 mbarrier 应当等候多少数据。一旦引擎搬完了那么多字节, 与之匹配的 `mbarrier.try_wait(phase)` 就释放, 只有到那时这个 SMEM 分块才安全地可以喂给 MMA。

存储侧走的是同一套硬件, 但等待方式不同, 所以值得在脑子里把这两个协议清楚地区分开: 加载用 mbarrier 和字节数来追踪完成, 而存储用提交组 (commit group) 和等待组 (wait group) 来追踪完成。在线程把它们的 fp16 结果写进 Dsmem 并同步之后, 一个被选中的线程启动 `Tx.copy_async(D[...], Dsmem, dispatch="tma")`, 随后 `cp_async.bulk.commit_group()` 加上 `cp_async.bulk.wait_group(0)` 会阻塞到存储排干为止。这个等待不是可选的: Dsmem 在前一次存储排干之前, 不能被下一个分块复用。

**和你的 agent 一起试:** 针对一个 K 分块, 追踪 Step 4 的加载和存储同步。指出哪个线程启动每条 TMA 命令、哪个 mbarrier 或提交组追踪其完成、哪个等待保护 MMA 对 Asmem 和 Bsmem 的读取、哪个等待保护 Dsmem 的复用。为什么在这里, `elect_sync()` 对 TMA 加载协议而言会是错误的线程选择?

## 完整内核

完整内核把 TMA 加载和存储折叠进 Step 3 的结构里, 其余部分一概不动。导入语句和之前一样:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.layout import TileLayout, S, Tlane, Tcol, tid_in_wg
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_
    ↪ shared_layout, SwizzleMode
```

它被包在 `hgemm_v4(M, N, K)` 里, 这是我们贯穿全书遵循的模式: 这层包装把依赖形状的量 and 布局放在使用它们的内核紧旁边。

```
def hgemm_v4(M, N, K):
    a_type = tvm.DataType("float16")
    b_type = tvm.DataType("float16")
    d_type = tvm.DataType("float16")
    acc_type = tvm.DataType("float32")

    BLK_M, BLK_N, BLK_K = 128, 128, 64
    K_TILES = K // BLK_K
    F16_SIZE = 2

    A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_
    ↪ ATOM, (BLK_M, BLK_K))
    B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_
    ↪ ATOM, (BLK_N, BLK_K))
```

(continues on next page)

(continued from previous page)

```

D_layout = tma_shared_layout(d_type, SwizzleMode.SWIZZLE_128B_
→ATOM, (BLK_M, BLK_N))

@T.prim_func
def kernel(
    A: T.Buffer((M, K), a_type),
    B: T.Buffer((N, K), b_type),
    D: T.Buffer((M, N), d_type),
):
    T.device_entry()
    bx, by = T.cta_id([M // BLK_M, N // BLK_N])
    wg_id = T.warpgroup_id([1])
    warp_id = T.warp_id_in_wg([4])
    lane_id = T.lane_id([32])

    # --- SMEM 分配 (现在为 TMA 存储纳入了 Dsmem) ---
    pool = T.SMEMPool()
    tmem_addr = pool.alloc((1,), "uint32")
    tma_bar = pool.alloc((1,), "uint64", align=8)
    mma_bar = pool.alloc((1,), "uint64", align=8)
    pool.move_base_to(1024)
    Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout)
    Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout)
    Dsmem = pool.alloc((BLK_M, BLK_N), d_type, layout=D_layout)
    pool.commit()

    # --- 屏障与 TMEM 初始化 ---
    if warp_id == 0 and lane_id == 0:
        T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1)
        T.ptx.mbarrier.init(tma_bar.ptr_to([0]), 1)
    if warp_id == 0:
        T.ptx.tcgen05.alloc(T.address_of(tmem_addr), n_cols=512,
→ cta_group=1)

    T.ptx.fence.proxy_async("shared::cta")
    T.ptx.fence.mbarrier_init()
    T.cuda.cta_sync()

    tmem = T.decl_buffer(
        (128, 512), "float32", scope="tmem", allocated_
→addr=tmem_addr[0],
        layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)])
    )

    m_st = T.meta_var(bx * BLK_M)
    n_st = T.meta_var(by * BLK_N)
    phase_tma: T.int32 = 0
    phase_mma: T.int32 = 0

```

(continues on next page)

(continued from previous page)

```

# --- 内联辅助函数 ---
@T.inline
def tma_load(k_st):
    tma_config = T.meta_var({
        "dispatch": "tma", "cta_group": 1,
        "mbar": tma_bar.ptr_to([0])
    })
    Tx.copy_async(Asmem[:, :],
                  A[m_st : m_st + BLK_M, k_st : k_st + BLK_
↪K],
                  **tma_config)
    Tx.copy_async(Bsmem[:, :],
                  B[n_st : n_st + BLK_N, k_st : k_st + BLK_
↪K],
                  **tma_config)
    T.ptx.mbarrier.arrive.expect_tx(
        tma_bar.ptr_to([0]),
        (BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE
    )

@T.inline
def mma(accum):
    Tx.gemm_async(
        tmem[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
        accum=accum, dispatch="tcgen05", cta_group=1
    )
    T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1)

# --- 带 TMA 异步的 K 循环 ---
tid = T.meta_var(warp_id * 32 + lane_id)
for k in range(K_TILES):
    k_st = T.meta_var(k * BLK_K)

    # 单个线程派发 TMA 加载
    if tid == 0:
        tma_load(k_st)

    # 等待 TMA 完成;mbarrier 的释放会把 SMEM 可见性带给
    # 后续的 MMA, 所以不需要额外的 fence。
    T.ptx.mbarrier.try_wait(tma_bar.ptr_to([0]), phase_tma)

    # 单个线程派发 MMA
    if tid == 0:
        mma(accum=k != 0)

    # 等待 MMA 完成
    T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma)
    phase_tma ^= 1
    phase_mma ^= 1

```

(continues on next page)

(continued from previous page)

```

# --- TMA 存储回写 ---
Dreg = T.alloc_local((BLK_N,), acc_type)
Dreg_f16 = T.alloc_local((BLK_N,), d_type)
Dreg_wg = Dreg.view(128, BLK_N,
                    layout=TileLayout(S[(128, BLK_N) :
→(1@tid_in_wg, 1)]))

# 读取 TMEM -> 寄存器 (异步; 先 wait.ld 再 cta_sync 以确保读取完
成)
Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N])
T.ptx.tcgen05.wait.ld()
T.cuda.cta_sync()
# 把 fp32 转为 fp16
Tx.cast(Dreg_f16[:, :], Dreg[:, :])
# 写寄存器 -> Dsmem, 刷出, 然后同步
Tx.copy(Dsmem[warp_id * 32 + lane_id, 0:BLK_N], Dreg_f16[:, :])
T.ptx.fence.proxy_async("shared::cta")
T.cuda.warpgroup_sync(10)
# TMA 存储:Dsmem -> GMEM。一个被选中的线程启动存储,
# 并在 Dsmem 被复用之前把存储组排干。
if tid == 0:
    Tx.copy_async(D[m_st : m_st + BLK_M, n_st : n_st + BLK_
→N],
                  Dsmem[:, :], dispatch="tma")
    T.ptx.cp_async.bulk.commit_group()
    T.ptx.cp_async.bulk.wait_group(0)
    T.cuda.warpgroup_sync(10)

# --- 释放 TMEM ---
T.cuda.cta_sync()
if warp_id == 0:
    T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
    T.ptx.tcgen05.dealloc(tmem_addr[0], n_cols=512, cta_
→group=1)

return kernel

```

## 内核里的 TMA 配置

那个内核里几乎一切都沿袭自 Step 3。真正承载 TMA 语义的只有五个配置点, 每一个都值得按名字认识清楚:

- **TMA 配置**:`{"dispatch": "tma", "cta_group": 1, "mbar": tma_bar.ptr_to([0])}` 告诉 `Tx.copy_async` 使用 TMA, 并通过 `tma_bar` 报告加载完成。
- **字节数**:  $(\text{BLK\_M} * \text{BLK\_K} + \text{BLK\_N} * \text{BLK\_K}) * 2$  是两个 `fp16` 操作数分块加载的字节数。 `arrive.expect_tx(...)` 把这个计数交给 `mbarrier`。
- **mbarrier 初始化**: `init(tma_bar.ptr_to([0]), 1)` 创建出 TMA 加载所用的完成屏障。

- **@T.inline:** `tma_load(...)` 和 `mma(...)` 是辅助函数。它们在编译期被展开进内核体里, 可以使用外层内核的变量。
- **TMA 存储同步:** 收尾先把 `fp16` 行写进 `Dsmem`。 `fence.proxy_async` 和 `warp-group_sync` 让这些由线程写入的 `SMEM` 值为 `TMA` 存储通路做好准备。随后存储用 `commit_group()` 和 `wait_group(0)` 等待 `SMEM` 到 `GMEM` 的传输完成。

到这里, 我们手里有了对的零件, 却还没有对的节奏。Step 4 仍然要在开始对应的 `MMA` 之前先完成每一次加载, 所以加载和乘法从未真正同时跑过; 我们费了那么大力气分开的两个引擎, 还是在轮替。下一步保持 `TMA` 加载和存储通路原封不动, 转而重排调度, 使得加载某个 `K` 分块能在对另一个分块做计算的同时进行。

### 1.13.2 Step 5: 软件流水线 (PIPE\_DEPTH=2)

既然两个引擎明显相互独立, 为什么 Step 4 不能把加载和计算重叠起来? 障碍其实是存储。只有一个 `SMEM` 分块对时, 下一次加载无处可去: 它要等到当前 `MMA` 读完了那一对才能开始, 因为提前开始会覆盖仍在使用的数据。Step 5 通过对共享显存做双缓冲来消除这个存储冲突。单 `warpgroup` 的循环仍然在每次 `MMA` 之后才启动下一次 `TMA` 加载, 但现在它有了不同的流水级可以预取进去、再复用。我们仍然跑在完整的 `M=N=K=4096` 规模上。

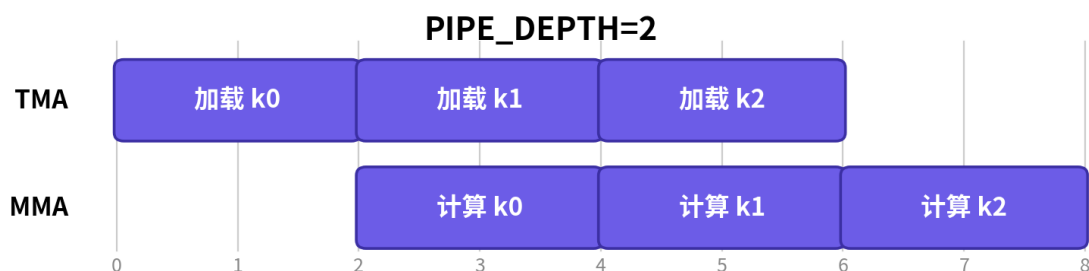
本步改变的是: **Layout**

- **Scope:** 不变, 仍是一个 `warpgroup`。
- **Layout:** 单一 `SMEM` 分块对变成一个 `PIPE_DEPTH` 级的环形缓冲。
- **Dispatch:** 不变, `TMA` 加载和 `tcgen05 MMA`; 本步加入预取和流水级复用, 而完整的加载/计算重叠要到 Step 7 才到来。

#### 流水线走查

设 `PIPE_DEPTH=2`, 内核分配两个 `SMEM` 流水级, 给加载通路和 `MMA` 通路各自独立的工作槽位。

请把下图当作两级缓冲意在支撑的流水线结构来读, 而不是这个单 `warpgroup` 内核的精确执行轨迹。Step 5 建好了环形缓冲并预取更后面的流水级, 但主循环在派发下一次 `TMA` 加载之前仍然要等待当前 `MMA`。完整的加载/计算重叠要到 Step 7 才到来, 届时 `warp specialization` 给 `TMA` 和 `MMA` 赋予了各自独立的角色。



一旦它被灌满, 循环就在两个流水级之间交替。两个 `TMA` 加载一开始把两个流水级都填满; 此后, 循环等待当前流水级、在其上跑 `MMA`、等待该 `MMA` 读完这个流水级, 然后再为 `k + PIPE_DEPTH` 把加载派发进刚刚变得可复用的那个流水级。这还不是并发的 `TMA/MMA` 调度, 但它建立了环形缓冲结构, Step 7 会把这一结构在 `producer` 和 `consumer` 角色之间拆开。

具体而言, 代码与 Step 4 在四处不同:



1. Asmem 和 Bsmem 多出一个前导的 PIPE\_DEPTH 维, 所以每个流水级都有自己独立的 SMEM 存储。
2. tma\_bar 变成一个数组, 每个流水级一个 mbarrier。
3. 在主 K 循环之前, 内核预取头两个流水级。
4. K 循环使用  $\text{stage} = k \% \text{PIPE\_DEPTH}$ : 等待当前流水级、在其上跑 MMA, 然后把这个流水级复用给  $k + \text{PIPE\_DEPTH}$ 。

## 流水线机制

**1. 预取:** 在主循环真正运行之前, 我们加载头 PIPE\_DEPTH 个流水级, 这样循环在第一次迭代时就能找到等在那里的数据:

```
for s in range(min(PIPE_DEPTH, K_TILES)):
    tma_load(s, s * BLK_K)
```

**2. 主循环:** 对每一个 K 分块, 我们等它的流水级就绪、在其上跑 MMA, 然后立刻把这个现已空闲的流水级重新派上用场——为 PIPE\_DEPTH 之后的那个分块派发加载:

```
stage = k % PIPE_DEPTH
wait(tma_bar[stage], phase_tma)
mma(stage, accum)
wait(mma_bar[0], phase_mma)
phase_mma ^= 1
tma_load(stage, next_k * BLK_K)
```

**3. 相位管理:** 这部分最容易绊倒人, 但规则比乍看上去要简单。每个屏障的相位翻转规则, 直接来源于该屏障有多少个槽位, 这也是为什么两个屏障以不同的节奏翻转。MMA 累加器住在一个 TMEM 槽位里, 所以 mma\_bar 是一个每次迭代都要重访的单屏障 (mma\_bar.ptr\_to([0])), 而一个你每次迭代都要重访的屏障, 必须每次迭代都翻转相位。TMA 屏障讲的则是另一个故事: 它们构成一个 PIPE\_DEPTH 元素的数组, 每个流水级一个屏障, 而任一给定流水级的屏障在一圈环形里只回来一次。所以 phase\_tma 只在流水级索引绕回 0 时才翻转:

```
if stage == PIPE_DEPTH - 1:
    phase_tma ^= 1
```

**和你的 agent 一起试:** 设 PIPE\_DEPTH=2 且 K\_TILES=5, 让它追踪主循环。对每个 k, 列出 stage、传给各次等待的 phase\_tma 和 phase\_mma 值, 以及是否派发了新的预取。phase\_tma 到底在哪里翻转, 为什么最后两次迭代没有预取?

## 完整内核

完整内核逐字保留 Step 4 的 TMA 加载和存储通路, 再把它们包进我们刚才描述的分级缓冲和相位逻辑里。导入语句不变:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.layout import TileLayout, S, Tlane, TCol, tid_in_wg
```

(continues on next page)

(continued from previous page)

```
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_
    ↪ shared_layout, SwizzleMode
```

它被包在 `hgemm_v5(M, N, K)` 里。PIPE\_DEPTH=2 常量设定流水级的数量 (这里是两个, 恰好就是双缓冲):

```
PIPE_DEPTH = 2

def hgemm_v5(M, N, K):
    a_type = tvm.DataType("float16")
    b_type = tvm.DataType("float16")
    d_type = tvm.DataType("float16")
    acc_type = tvm.DataType("float32")
    F16_SIZE = 2
    BLK_M, BLK_N, BLK_K = 128, 128, 64
    K_TILES = K // BLK_K

    # 双缓冲布局: 第一维是流水级
    A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_
    ↪ATOM,
                                (PIPE_DEPTH, BLK_M, BLK_K))
    B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_
    ↪ATOM,
                                (PIPE_DEPTH, BLK_N, BLK_K))
    D_layout = tma_shared_layout(d_type, SwizzleMode.SWIZZLE_128B_
    ↪ATOM,
                                (BLK_M, BLK_N))

    @T.prim_func
    def kernel(
        A: T.Buffer((M, K), a_type),
        B: T.Buffer((N, K), b_type),
        D: T.Buffer((M, N), d_type),
    ):
        T.device_entry()
        bx, by = T.cta_id([M // BLK_M, N // BLK_N])
        wg_id = T.warpgroup_id([1])
        warp_id = T.warp_id_in_wg([4])
        lane_id = T.lane_id([32])

        # --- SMEM 分配 ---
        pool = T.SMEMPool()
        tmem_addr = pool.alloc((1,), "uint32")
        # 双缓冲的 TMA 屏障 (每流水级一个), 单个 MMA 屏障
        tma_bar = pool.alloc((PIPE_DEPTH,), "uint64", align=8)
        mma_bar = pool.alloc((1,), "uint64", align=8)
        pool.move_base_to(1024)
        Asmem = pool.alloc((PIPE_DEPTH, BLK_M, BLK_K), a_type,
    ↪layout=A_layout)
```

(continues on next page)

(continued from previous page)

```

        Bsmem = pool.alloc((PIPE_DEPTH, BLK_N, BLK_K), b_type,
→ layout=B_layout)
        Dsmem = pool.alloc((BLK_M, BLK_N), d_type, layout=D_layout)
        pool.commit()

        # 初始化屏障:TMA 用 PIPE_DEPTH 个,MMA 用 1 个
        if warp_id == 0:
            if lane_id == 0:
                T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1)
                for s in range(PIPE_DEPTH):
                    T.ptx.mbarrier.init(tma_bar.ptr_to([s]), 1)
            if warp_id == 0:
                T.ptx.tcgen05.alloc(T.address_of(tmем_addr), n_cols=512,
→ cta_group=1)

        T.ptx.fence.proxy_async("shared::cta")
        T.ptx.fence.mbarrier_init()
        T.cuda.cta_sync()

        tmем = T.decl_buffer(
            (128, 512), acc_type, scope="tmем", allocated_addr=tmем_
→ addr[0],
            layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)])
        )

        m_st = T.meta_var(bx * BLK_M)
        n_st = T.meta_var(by * BLK_N)
        phase_tma: T.int32 = 0
        phase_mma: T.int32 = 0

        @T.inline
        def tma_load(stage, k_offset):
            tma_config = T.meta_var({
                "dispatch": "tma", "cta_group": 1,
                "mbar": tma_bar.ptr_to([stage])
            })
            Tx.copy_async(Asmem[stage, :, :],
                A[m_st:m_st+BLK_M, k_offset:k_offset+BLK_
→ K],
                **tma_config)
            Tx.copy_async(Bsmem[stage, :, :],
                B[n_st:n_st+BLK_N, k_offset:k_offset+BLK_
→ K],
                **tma_config)
            T.ptx.mbarrier.arrive.expect_tx(
                tma_bar.ptr_to([stage]),
                (BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE)

        @T.inline

```

(continues on next page)

(continued from previous page)

```

def mma(stage, accum):
    Tx.gemm_async(tmemb[:, :BLK_N], Asmem[stage, :, :],
    ↪Bsmemb[stage, :, :],
                                accum=accum, dispatch="tcgen05", cta_
    ↪group=1)
    T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1)

    tid = T.meta_var(warp_id * 32 + lane_id)

    # === 预取: 加载头 PIPE_DEPTH 个流水级 ===
    if tid == 0:
        for s in range(min(PIPE_DEPTH, K_TILES)):
            tma_load(s, s * BLK_K)

    # === 主循环 ===
    for k in range(K_TILES):
        stage = k % PIPE_DEPTH

        # 等待 TMA 加载完本流水级
        T.ptx.mbarrier.try_wait(tma_bar.ptr_to([stage]), phase_
    ↪tma)

        # 在本流水级的数据上做 MMA
        if tid == 0:
            mma(stage, accum=(k != 0))

        T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma)
        phase_mma ^= 1

        # 派发下一次预取加载 (k + PIPE_DEPTH)
        next_k = k + PIPE_DEPTH
        if next_k < K_TILES:
            if tid == 0:
                tma_load(stage, next_k * BLK_K)

        # 流水级绕回时 TMA 相位翻转
        if stage == PIPE_DEPTH - 1:
            phase_tma ^= 1

    # === TMA 存储回写: TMEM -> RF -> Dsmem -> TMA -> GMEM ===
    Dreg = T.alloc_local((BLK_N,), acc_type)
    Dreg_f16 = T.alloc_local((BLK_N,), d_type)
    Dreg_wg = Dreg.view(128, BLK_N,
                                layout=TileLayout(S[(128, BLK_N) :
    ↪(1@tid_in_wg, 1)]))
    Tx.wg.copy_async(Dreg_wg[:, :], tmemb[:, :BLK_N])
    T.ptx.tcgen05.wait.ld()
    T.cuda.cta_sync()
    Tx.cast(Dreg_f16[:, :], Dreg[:, :])

```

(continues on next page)

(continued from previous page)

```

Tx.copy(Dsmem[warp_id * 32 + lane_id, 0:BLK_N], Dreg_f16[:])
T.ptx.fence.proxy_async("shared::cta")
T.cuda.warpgroup_sync(10)
if tid == 0:
    Tx.copy_async(D[m_st : m_st + BLK_M, n_st : n_st + BLK_
↪N],
                  Dsmem[:, :], dispatch="tma")
    T.ptx.cp_async.bulk.commit_group()
    T.ptx.cp_async.bulk.wait_group(0)
    T.cuda.warpgroup_sync(10)

# 释放 TMEM
T.cuda.cta_sync()
if warp_id == 0:
    T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
    T.ptx.tcgen05.dealloc(tmem_addr[0], n_cols=512, cta_
↪group=1)

return kernel

```

### 1.13.3 Step 6: 持久化内核 + 分块调度器

到目前为止的一切，都只是在优化单个分块内部的工作。Step 6 改变问题的尺度，改为跨分块进行优化。

Step 5 为每个 128 x 128 的输出分块启动一个 CTA。对一个 4096 x 4096 的输出，这意味着 1024 个独立的 CTA，每个都要付出各自的启动开销，然后在自己的分块一做完就消失。

Step 6 则启动一个固定数量的 CTA 池，再让每个 CTA 依次处理多个分块。这给我们换来两样东西：启动工作被摊薄到多个分块上，而分块指派搬进了内核内部——在这里调度器可以挑选一个能复用操作数的顺序。我们仍然跑在完整的  $M=N=K=4096$  规模上。

#### 本步改变的是:Scope

- Scope: 一个固定数量的持久化 CTA 池，每个 CTA 通过调度器循环处理多个输出分块。
- Layout: 不变，沿用相同的按分块的 SMEM/TMEM/寄存器通路。
- Dispatch: 不变。

#### 持久化调度

持久化内核的定义性想法，是把自己的网格尺寸对齐硬件，而不是对齐问题本身。它启动 `SM_COUNT` 个 CTA，大致每个 SM 一个，而不管恰好有多少个输出分块，目的是让每个 SM 持续被占满。我们特意说“大致”：精确的 1:1 占用并不保证，因为它取决于占用率以及硬件如何选择调度 CTA。

在我们此处作为目标的 B200 上，`SM_COUNT=148`。这 148 个 CTA 中的每一个，都在循环处理由 `ClusterPersistentScheduler2D` 交给它的分块。

第一份回报来自摊薄。TMEM 分配、屏障初始化以及调度器状态，现在每个 CTA 只发生一次，并在该 CTA 处理的那大约 7 个分块上复用，而不是在 1024 个用完即弃的 CTA 上被重复 1024 次。

第二份回报来自调度器挑选的顺序。设 `l2_group_size=8` 把相邻分块成组, 使得共享同一行带的分块复用相同的 A 行分块, 共享同一列带的分块复用相同的 B 分块。把这些分块背靠背地跑, 就让操作数在 L2 里保持热, 而不是从 HBM 里重新取。这恰好就是 Step 3 留在桌上的那份复用。

```
bx = T.cta_id([SM_COUNT]) # 一维网格, 每个 SM 一个 CTA

tile_scheduler = ClusterPersistentScheduler2D(
    "ts",
    num_m_tiles=M // BLK_M,
    num_n_tiles=N // BLK_N,
    l2_group_size=8,          # 把相邻 8 个分块成组
    num_clusters=SM_COUNT
)
tile_scheduler.init(bx)
```

跨分块循环会带来一个容易忽视的正确性后果。每个分块都跑自己全新的 K 循环, 这意味着它的屏障相位必须从一个已知状态开始。在 Step 5 里, 一个 CTA 恰好只处理一个分块, 所以把 `phase_tma` 和 `phase_mma` 初始化一次完全没问题。在 Step 6 里, 这些初始化必须移到 `while tile_scheduler.valid()` 循环内部, 好让每个分块都以匹配自身 TMA 和 MMA 工作的相位状态开始, 而不是继承上一个分块恰好留下来的状态:

```
while tile_scheduler.valid():
    phase_tma: T.int32 = 0
    phase_mma: T.int32 = 0
    ...
```

## 完整内核

在结构上, 这个内核无非就是把 Step 5 的流水线包进一个分块级别的外层循环里。唯一的新依赖是调度器本身, 我们把它和其余导入一起引入:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.layout import TileLayout, S, Tlane, TCol, tid_in_wg
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_
    ↪ shared_layout, SwizzleMode
from tvm.tirx.lang.tile_scheduler import
    ↪ ClusterPersistentScheduler2D
```

网格维度现在直接就是 `SM_COUNT`, 而不再是 `(M//BLK_M, N//BLK_N)`, 并且一个 `ClusterPersistentScheduler2D` 接手了把分块派发给各 CTA 的工作:

```
SM_COUNT = 148 # NVIDIA B200 GPU 上的 SM 数量
PIPE_DEPTH = 2

def hgemm_v6(M, N, K):
    a_type = tvm.DataType("float16")
    b_type = tvm.DataType("float16")
```

(continues on next page)



(continued from previous page)

```

d_type = tvm.DataType("float16")
acc_type = tvm.DataType("float32")
F16_SIZE = 2
BLK_M, BLK_N, BLK_K = 128, 128, 64
K_TILES = K // BLK_K

A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_
→ATOM,
                                (PIPE_DEPTH, BLK_M, BLK_K))
B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_
→ATOM,
                                (PIPE_DEPTH, BLK_N, BLK_K))
D_layout = tma_shared_layout(d_type, SwizzleMode.SWIZZLE_128B_
→ATOM,
                                (BLK_M, BLK_N))

@T.prim_func
def kernel(
    A: T.Buffer((M, K), a_type),
    B: T.Buffer((N, K), b_type),
    D: T.Buffer((M, N), d_type),
):
    T.device_entry()
    # 一维网格: 每个 SM 一个 CTA(不再是二维网格!)
    bx = T.cta_id([SM_COUNT])
    wg_id = T.warpgroup_id([1])
    warp_id = T.warp_id_in_wg([4])
    lane_id = T.lane_id([32])

    # --- SMEM 分配 (同 Step 5)---
    pool = T.SMEMPool()
    tmem_addr = pool.alloc((1,), "uint32")
    tma_bar = pool.alloc((PIPE_DEPTH,), "uint64", align=8)
    mma_bar = pool.alloc((1,), "uint64", align=8)
    pool.move_base_to(1024)
    Asmem = pool.alloc((PIPE_DEPTH, BLK_M, BLK_K), a_type,
→layout=A_layout)
    Bsmem = pool.alloc((PIPE_DEPTH, BLK_N, BLK_K), b_type,
→layout=B_layout)
    Dsmem = pool.alloc((BLK_M, BLK_N), d_type, layout=D_layout)
    pool.commit()

    # --- 屏障与 TMEM 初始化 (同 Step 5)---
    if warp_id == 0 and lane_id == 0:
        T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1)
        for s in range(PIPE_DEPTH):
            T.ptx.mbarrier.init(tma_bar.ptr_to([s]), 1)
    if warp_id == 0:
        T.ptx.tcgen05.alloc(T.address_of(tmem_addr), n_cols=512,

```

(continues on next page)

(continued from previous page)

```

→ cta_group=1)
    T.ptx.fence.proxy_async("shared::cta")
    T.ptx.fence.mbarrier_init()
    T.cuda.cta_sync()

    tmem = T.decl_buffer(
        (128, 512), acc_type, scope="tmem", allocated_addr=tmem_
→ addr[0],
        layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)])
    )

    # 分块调度器: 以对 L2 友好的顺序把分块派给各 CTA
    tile_scheduler = ClusterPersistentScheduler2D(
        "ts",
        num_m_tiles=M // BLK_M,
        num_n_tiles=N // BLK_N,
        l2_group_size=8,
        num_clusters=SM_COUNT
    )
    tile_scheduler.init(bx)

    tid = T.meta_var(warp_id * 32 + lane_id)

    @T.inline
    def tma_load(stage, k_offset, m_st, n_st):
        tma_config = T.meta_var({
            "dispatch": "tma", "cta_group": 1,
            "mbar": tma_bar.ptr_to([stage])
        })
        Tx.copy_async(Asmem[stage, :, :],
            A[m_st:m_st+BLK_M, k_offset:k_offset+BLK_
→ K],
            **tma_config)
        Tx.copy_async(Bsmem[stage, :, :],
            B[n_st:n_st+BLK_N, k_offset:k_offset+BLK_
→ K],
            **tma_config)
        T.ptx.mbarrier.arrive.expect_tx(
            tma_bar.ptr_to([stage]),
            (BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE)

    @T.inline
    def mma(stage, accum):
        Tx.gemm_async(tmem[:, :BLK_N], Asmem[stage, :, :],
→ Bsmem[stage, :, :],
            accum=accum, dispatch="tcgen05", cta_
→ group=1)
        T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1)

```

(continues on next page)

(continued from previous page)

```

# === 外层循环: 遍历分块 ===
while tile_scheduler.valid():
    # 从调度器取当前分块位置
    m_st = T.meta_var(tile_scheduler.m_idx * BLK_M)
    n_st = T.meta_var(tile_scheduler.n_idx * BLK_N)

    # === 内层循环: 同 Step 5 的流水线 ===
    phase_tma: T.int32 = 0
    phase_mma: T.int32 = 0

    # 预取头 PIPE_DEPTH 个流水级
    if tid == 0:
        for s in range(min(PIPE_DEPTH, K_TILES)):
            tma_load(s, s * BLK_K, m_st, n_st)

    # 主 K 循环
    for k in range(K_TILES):
        stage = k % PIPE_DEPTH
        T.ptx.mbarrier.try_wait(tma_bar.ptr_to([stage]),
        ↪ phase_tma)

        if tid == 0:
            mma(stage, accum=(k != 0))
            T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_
            ↪ mma)

            phase_mma ^= 1
            next_k = k + PIPE_DEPTH
            if next_k < K_TILES:
                if tid == 0:
                    tma_load(stage, next_k * BLK_K, m_st, n_st)
            if stage == PIPE_DEPTH - 1:
                phase_tma ^= 1

    # === TMA 存储回写: TMEM -> RF -> Dsmem -> TMA -> GMEM ===
    Dreg = T.alloc_local((BLK_N,), acc_type)
    Dreg_f16 = T.alloc_local((BLK_N,), d_type)
    Dreg_wg = Dreg.view(128, BLK_N,
        layout=TileLayout(S[(128, BLK_N) :
    ↪ (1@tid_in_wg, 1)]))
    Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N])
    T.ptx.tcgen05.wait.ld()
    T.cuda.cta_sync()
    Tx.cast(Dreg_f16[:, :], Dreg[:, :])
    Tx.copy(Dsmem[warp_id * 32 + lane_id, 0:BLK_N], Dreg_
    ↪ f16[:, :])

    T.ptx.fence.proxy_async("shared::cta")
    T.cuda.warpgroup_sync(10)
    if tid == 0:
        Tx.copy_async(D[m_st : m_st + BLK_M, n_st : n_st +
    ↪ BLK_N],

```

(continues on next page)

(continued from previous page)

```

                                Dsmem[:, :], dispatch="tma")
        T.ptx.cp_async.bulk.commit_group()
        T.ptx.cp_async.bulk.wait_group(0)
        T.cuda.warpgroup_sync(10)

        T.cuda.cta_sync()
        tile_scheduler.next_tile() # 移到下一个分块

    # 释放 Tmem
    T.cuda.cta_sync()
    if warp_id == 0:
        T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
        T.ptx.tcgen05.dealloc(tmем_addr[0], n_cols=512, cta_
→group=1)

    return kernel

```

### 1.13.4 练习

1. 在 Step 4 里, `arrive.expect_tx` 用了  $(BLK\_M * BLK\_K + BLK\_N * BLK\_K) * 2$  字节。如果这个字节数太小或太大, `mbarrier` 会等待什么?
2. 在 Step 5 里, 为什么每个 SMEM 流水级需要自己独立的 TMA 屏障, 而不是两个流水级共用一个 `tma_bar`?
3. 在 Step 6 里, 一个  $4096 \times 4096$  的输出, 在  $BLK\_M=BLK\_N=128$  下有多少个输出分块? 在  $SM\_COUNT=148$  时, 每个持久化 CTA 平均处理多少个分块?

## 1.14 用线程束特化与集群扩展 GEMM

### Overview

- 流水线化的 GEMM 仍由一个 `warpgroup` 顺序完成加载、MMA 与回写, 这正是本章要消除的瓶颈。
- 第 7 步把 `warp` 特化成不同角色, 第 8 步加入 2-CTA 集群, 第 9 步加入多个消费者。
- 每一步都移除一个串行瓶颈, 最终逼近当前最优的吞吐量。

上一章 (用 TMA 为 GEMM 建立流水线) 得到的流水线 GEMM 已经很快, 但它仍要求一个 `warpgroup` 包揽一切: 发起加载、运行 MMA、再把结果写回。即便有软件流水线, 这一队线程也仍是三台引擎交汇的唯一通道。

症状很明显。Tensor Cores 运行时 TMA 单元安静下来, 结果排空到显存时 Tensor Cores 又安静下来, 每台引擎都通过同一组线程等待彼此。突破之道就是不再让一支队伍包揽一切。

我们分三步逐步扩大协作范围来实现这一思路。第 7 步 (第 7 步: 线程束特化 + 流水线) 把 `warp` 特化为生产者、消费者与回写三种角色。第 8 步 (第 8 步: 2-CTA 集群) 把两个 CTA 联合成一个集群, 通过彼此的共享内存共享操作数。第 9 步 (第 9 步: 多消费者的线程束特化) 加入第二个 MMA 消费者, 让一份已加载的分块驱动两倍的运算。

把这三步看作同一模式在不同尺度上的展开会很有帮助。第 7 步把整条流水线保留在一个 CTA 内:TMA 与 MMA 共用一个 warpgroup, 回写在另一个 warpgroup 中运行。第 8 步把协作扩展到 CTA 之间, 产生一个横跨两者的  $256 \times 256$  分块。第 9 步进一步推高计算密度: 集群输出增长到  $512 \times 256$ , 每份已加载的 B 分块被两个消费者复用, 由此得到本教程中最密集的变体。

这一切中有一件事始终不变。SMEM、TMEM 与寄存器布局仍然遵循前两章建立的契约; 变化的是谁来协作, 而不是数据如何摆放。第 8 步是协作作用域首次超出单个 CTA, 因此它的操作数分块被拆分到两个 CTA 的共享内存中, 一份布局沿 cbx 集群轴横跨两个 CTA。

### 1.14.1 第 7 步: 线程束特化 + 流水线

单 warpgroup 内核之所以把性能留在桌上, 原因很简单: 每个线程走同一条路径——加载、计算、再写回, 于是加载时 Tensor Cores 无事可做, 计算时 TMA 引擎也无事可做。解决办法是 *warp specialization*(线程束特化)。我们不再让一队线程轮流承担每项工作, 而是把每项工作交给一个专职 warp, 让这些 warp 同时运行, 再由一条软件流水线把它们缝合起来。这是 GEMM 路径上最大的架构变化, 本章余下部分都建立在它之上。这里的基准测试取  $M=N=K=4096$ 。

#### 本步改变了什么: 作用域

- 作用域: 一个 warpgroup 顺序走过加载  $\rightarrow$  MMA  $\rightarrow$  回写, 变成由满/空屏障连接的三个并发角色 (TMA 生产者、MMA 消费者、回写)。
- 布局: 不变, 沿用第 6 步的 SMEM 流水级与 TMEM 累加器。
- 派发: 不变, TMA 加载, tcgen05 MMA。

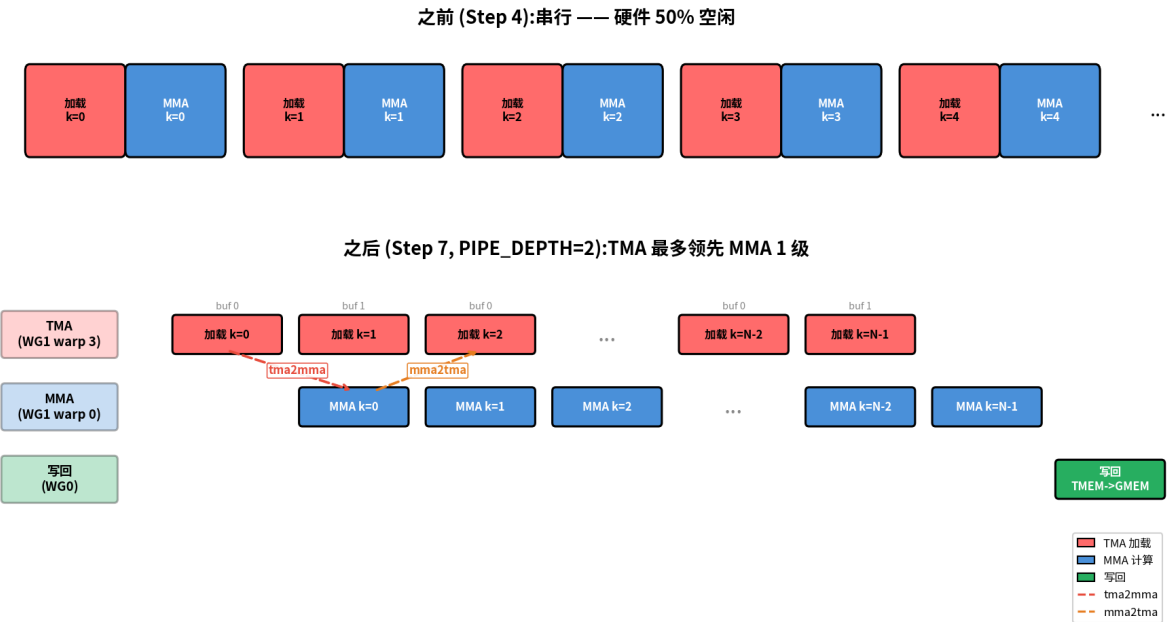
主题。

- warp specialization(线程束特化): 把不同的 warp/warpgroup 专门用于不同任务
- 高层屏障抽象: TMABar、TCGen05Bar、MBarrier
- PipelineState 自动管理流水级/相位
- warpgroup\_sync 的屏障 ID 用于按 warpgroup 同步

(多级 SMEM 流水线与持久化的 ClusterPersistentScheduler2D 直接复用第 5–6 步的实现; 这里只有作用域拆分是新的。)

### 从串行到并发

在介绍角色与屏障之前, 先孤立出 warp specialization 所移除的调度瓶颈会很有帮助。下图用第 4 步风格的串行时间线, 作为第 4–6 步中前置特化内核的紧凑参照, 再把它放在第 7 步 warp 特化调度的上方, 使引擎利用率差异一目了然。



上方是特化前的单 warpgroup 模式: 同一组未特化的线程同时拥有加载路径和 MMA 路径, 因此一台引擎很容易在另一台活跃时空闲。第 5、6 步用双缓冲和持久化调度改进了这一基线, 但它们尚未把加载与计算拆成独立的生产者和消费者角色。下方, 特化打破了这种轮替。TMA 生产者在 MMA 消费者忙于计算时预取下一个分块, 回写则自行推进。生产者 warp 3 在消费者 warp 0 仍在处理当前 MMA 时就发起下一次加载, 因此两台引擎都不必等待对方。加载/MMA 的交接使用两个屏障:

- **tma2mma**(TMA → MMA): 表示已加载的 SMEM 数据已就绪, 可供 MMA 消费。
- **mma2tma**(MMA → TMA): 表示 MMA 已读完某个缓冲, 该缓冲可由 TMA 用于下一次加载。

图中有一个细节乍看像是错的:mma2tma 的箭头会跳过一个流水级。原因在于环形缓冲。在 PIPE\_DEPTH=2 时有两个 SMEM 缓冲, 即 0 号和 1 号流水级;TMA Load k=0 填充缓冲 0,TMA Load k=1 填充缓冲 1。当 MMA Compute k=0 读完缓冲 0 后, 它会发出 mma2tma 信号表示该缓冲已空闲, 但真正想要回缓冲 0 的加载是 TMA Load k=2, 而不是 k=1(k=1 用的是缓冲 1)。这就是为什么从 MMA Compute k=0 发出的 mma2tma 箭头会一直延伸到 TMA Load k=2。释放跳过一个流水级, 纯粹是因为这个环有两个槽位。

Warp 角色

时间线展示了为什么要拆分工作; 接下来的问题是每部分由谁来做。特化把三项工作 (加载、计算、回写) 分配给特定的 warp, 使它们能并发运行。在 WG\_NUMBER=2 下, 内核使用两个 warpgroup(角色表中简写为 WG):

| 参与方     | 位置                   | 职责                   |
|---------|----------------------|----------------------|
| TMA 生产者 | Warpgroup 1,warp 3   | 持续通过 TMA 加载 A 和 B 分块 |
| MMA 消费者 | Warpgroup 1,warp 0   | 数据一就绪就立即运行 MMA       |
| 回写      | Warpgroup 0(全部 warp) | 读 TMEM 结果, 写入 GMEM   |



## 4 个屏障

三个并发参与方需要四个屏障, 这四个屏障可以整齐地分成两个相反的方向。前向路径 (TMA → MMA → 回写) 发出数据就绪信号; 它的消息是”你等的那个分块到了”。反向路径 (回写 → MMA → TMA) 发出缓冲释放信号:”你要的那个槽位又空了。”一旦掌握了命名约定, 这些名字就不言自明: 每个屏障都形如 `source2destination`, 所以 `tma2mma` 不过是 TMA 用来通知 MMA 的那个屏障。

| 屏障             | 类型         | 方向         | 含义                   |
|----------------|------------|------------|----------------------|
| <b>tma2mma</b> | TMABar     | TMA -> MMA | “SMEM 数据已就绪”         |
| <b>mma2tma</b> | TCGen05Bar | MMA -> TMA | “SMEM 缓冲可复用”         |
| <b>mma2ld</b>  | TCGen05Bar | MMA -> 回写  | “TMEM 结果已就绪”         |
| <b>ld2mma</b>  | MBarrier   | 回写 -> MMA  | “TMEM 已空闲, 可用于下一个分块” |

为什么每个屏障的类型各是现在这样? 类型取决于生产者如何宣布自己完成。**TMA Loads** 用 `TMABar`, 一种带字节计数的 `mbarrier`: 传输的字节一落地, TMA 硬件本身就到达该屏障, 于是消费者无需任何线程轮询就能得知数据已就绪。**TMA Stores** 用不了它 (一次存储没人可通知), 所以回退到 `cp_async.bulk.commit_group() + wait_group(0)`, 由发起线程自己等待它的写操作排空。**MMA 操作用** `TCGen05Bar`, 其上 `tcgen05.commit()` 指令在 MMA 完成时通知屏障。

这里有个小细节会在第 8 步派上用场。各 `arrive` 调用传入 `cta_mask=0`, 因为在单 CTA 内核中没有别的 CTA 可通知。等第 8 步组成集群时, 正是这个参数变为非零, 成为唤醒协作 CTA 的机制。

## PipelineState

四个屏障告诉各角色某个缓冲何时就绪; 但当流水线循环时, 还需要有人跟踪每个角色正在哪个缓冲。这种记账正是 `PipelineState` 管理的内容。环形缓冲同时带着两份记账: 当前在哪个槽位, 以及等待的是该槽位屏障的哪个”相位”。在流水化循环里手工同时跟踪这两者, 正是滋生差一错误的地方, 而这里一个差一就会让整个内核死锁。`PipelineState` 的存在就是把两者绑在一起, 省去你来操心:

```
tma_ps = PipelineState(PIPE_DEPTH, phase=1)  # 生产者以就绪状态启动 (phase=1)
# tma_ps.stage = 当前的流水级索引
# tma_ps.phase = 当前的相位 (0 或 1)
tma_ps.advance()                             # 推进到下一个流水级
```

初始的 `phase` 决定了一个角色的第一次 `wait` 是放行还是阻塞, 而流水线两端正确答案正好相反, 这正是容易绊倒人的地方:

- `phase=1`(生产者)-> 第一次 `wait(phase=1)` 时屏障仍处于相位 0, 由于 `0 != 1`, 它会立即放行。这正是我们想要的, 因为缓冲一开始是空的, 生产者应当能自由地立刻开始填充。
- `phase=0`(消费者)-> 第一次 `wait(phase=0)` 时屏障处于相位 0, 由于 `0 == 0`, 它会阻塞。同样是我们想要的, 因为此时还没有数据, 在生产者到达之前消费者无从读取。

给两端设相同的起始相位会得到一个死锁, 更糟的话是悄无声息的数据损坏, 所以这一处选择值得做对。

## warpgroup\_sync 屏障 ID

线程束特化引入了一种容易踩中的同步隐患。一旦每个 warpgroup 跑不同的代码路径, 熟悉的 `cta_sync()` 就会死锁: 它使用硬件屏障 #0 并坚持每个 CTA 线程都到达, 然而在 warpgroup 分支内只有其中一部分线程在场。我们真正需要的是一个作用域限于单个 warpgroup 的屏障。GPU 提供了 16 个具名屏障 (ID 0–15), 因此这些内核改用 `warpgroup_sync(10)`, 它只同步一个 warpgroup 内的线程。当多个 warpgroup 各自需要独立同步时 (如多消费者的第 9 步), 它们通过 `warpgroup_sync(wg_id + 10)` 取用不同的 ID, 从而永远不会撞上同一个硬件屏障。

实现。

这里用 `PIPE_DEPTH=2`, 这是仍能让加载与计算有任何重叠的最小深度。再深一些可以隐藏更多访存延迟, 上限是 `SMEM` 预算; 下文当第 7 步出问题时会详细推演这一权衡。至此所有部件 (角色、四个屏障、`PipelineState` 和 warpgroup 作用域同步) 都已齐备, 我们可以拼出完整内核:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.layout import TileLayout, S, Tlane, Tcol, tid_in_wg
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_
    ↪ shared_layout, SwizzleMode
from tvm.tirx.lang.pipeline import TMABar, TCGen05Bar, MBarrier,
    ↪ PipelineState
from tvm.tirx.lang.tile_scheduler import
    ↪ ClusterPersistentScheduler2D

SM_COUNT = 148 # NVIDIA B200 GPU 上的 SM 数量
F16_SIZE = 2

def hgemm_v7(M, N, K):
    a_type = tvm.DataType("float16")
    b_type = tvm.DataType("float16")
    d_type = tvm.DataType("float16")
    acc_type = tvm.DataType("float32")

    BLK_M, BLK_N, BLK_K = 128, 128, 64
    K_TILES = K // BLK_K
    PIPE_DEPTH = 2
    WG_NUMBER = 2

    A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_
    ↪ ATOM, (PIPE_DEPTH, BLK_M, BLK_K))
    B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_
    ↪ ATOM, (PIPE_DEPTH, BLK_N, BLK_K))
    D_layout = tma_shared_layout(d_type, SwizzleMode.SWIZZLE_128B_
    ↪ ATOM, (BLK_M, BLK_N))

    @T.prim_func
    def kernel(
        A: T.Buffer((M, K), a_type),
        B: T.Buffer((N, K), b_type),
```

(continues on next page)

(continued from previous page)

```

D: T.Buffer((M, N), d_type),
):
    T.device_entry()
    bx = T.cta_id([SM_COUNT])
    wg_id = T.warpgroup_id([WG_NUMBER])
    warp_id = T.warp_id_in_wg([4])
    lane_id = T.lane_id([32])

    # --- 分配 ---
    pool = T.SMEMPool()
    tmem_addr = pool.alloc((1,), "uint32")
    tma2mma = TMABar(pool, PIPE_DEPTH)
    mma2tma = TCGen05Bar(pool, PIPE_DEPTH)
    mma2ld = TCGen05Bar(pool, 1)
    ld2mma = MBarrier(pool, 1)
    pool.move_base_to(1024)
    Asmem = pool.alloc((PIPE_DEPTH, BLK_M, BLK_K), a_type,
→ layout=A_layout)
    Bsmem = pool.alloc((PIPE_DEPTH, BLK_N, BLK_K), b_type,
→ layout=B_layout)
    Dsmem = pool.alloc((BLK_M, BLK_N), d_type, layout=D_layout)

    # --- 屏障初始化 ---
    tma2mma.init(1)
    mma2tma.init(1)
    mma2ld.init(1)
    ld2mma.init(128) # Warpgroup 0 的全部 128 个线程都到达
    pool.commit()

    # --- TMEM 分配 + 栅栏 ---
    if wg_id == 0:
        if warp_id == 0:
            T.ptx.tcgen05.alloc(T.address_of(tmem_addr), n_
→ cols=512, cta_group=1)
            T.ptx.fence.proxy_async("shared::cta")
            T.ptx.fence.mbarrier_init()
            T.cuda.cta_sync()

            tmem = T.decl_buffer(
                (128, 512), acc_type, scope="tmem", allocated_addr=tmem_
→ addr[0],
                layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)]))

    # --- 分块调度器 ---
    tile_scheduler = ClusterPersistentScheduler2D(
        "ts", num_m_tiles=M // BLK_M, num_n_tiles=N // BLK_N,
        l2_group_size=8, num_clusters=SM_COUNT)
    tile_scheduler.init(bx)
    m_st = T.meta_var(tile_scheduler.m_idx * BLK_M)

```

(continues on next page)

(continued from previous page)

```

n_st = T.meta_var(tile_scheduler.n_idx * BLK_N)

# =====
# Warpgroup 1:TMA 生产者 (warp 3)+ MMA 消费者 (warp 0)
# =====
if wg_id == 1:
    if warp_id == 3:
        # === TMA 生产者 ===
        tma_ps = PipelineState(PIPE_DEPTH, phase=1)

        @T.inline
        def tma_load(k_offset):
            Tx.copy_async(Asmem[tma_ps.stage, :, :],
                          A[m_st:m_st+BLK_M, k_offset:k_
→offset+BLK_K],
                          dispatch="tma", cta_group=1,
                          mbar=tma2mma.ptr_to([tma_ps.
→stage]))
            Tx.copy_async(Bsmem[tma_ps.stage, :, :],
                          B[n_st:n_st+BLK_N, k_offset:k_
→offset+BLK_K],
                          dispatch="tma", cta_group=1,
                          mbar=tma2mma.ptr_to([tma_ps.
→stage]))

            if T.filter(lane_id, T.ptx.select_sync()):
                while tile_scheduler.valid():
                    for k in range(K_TILES):
                        mma2tma.wait(tma_ps.stage, tma_ps.phase)
                        tma_load(k * BLK_K)
                        tma2mma.arrive(tma_ps.stage,
                                      (BLK_M * BLK_K + BLK_N *
→BLK_K) * F16_SIZE)
                        tma_ps.advance()
                        tile_scheduler.next_tile()

    elif warp_id == 0:
        # === MMA 消费者 ===
        mma_ps = PipelineState(PIPE_DEPTH, phase=0)
        ld_ps = PipelineState(1, phase=1)

        if T.filter(lane_id, T.ptx.select_sync()):
            while tile_scheduler.valid():
                # 等待 Tmem 被上一分块的回写释放
                ld2mma.wait(ld_ps.stage, ld_ps.phase)
                ld_ps.advance()

                for k in range(K_TILES):
                    tma2mma.wait(mma_ps.stage, mma_ps.phase)

```

(continues on next page)

(continued from previous page)

```

        Tx.gemm_async(
            tmem[:, :BLK_N],
            Asmem[mma_ps.stage, :, :],
            Bsmem[mma_ps.stage, :, :],
            accum=(k != 0), dispatch="tcgen05",
→ cta_group=1)
        mma2tma.arrive(mma_ps.stage, cta_
→ group=1, cta_mask=0)
        mma_ps.advance()

        # 通知回写结果已就绪
        mma2ld.arrive(0, cta_group=1, cta_mask=0)
        tile_scheduler.next_tile()

# =====
# Warpgroup 0: 回写
# =====
elif wg_id == 0:
    wb_ps = PipelineState(1, phase=0)
    reg_f16 = T.alloc_local((BLK_N,), d_type)

    while tile_scheduler.valid():
        # 等待 MMA 结果
        mma2ld.wait(wb_ps.stage, wb_ps.phase)
        wb_ps.advance()

        # 读 TMEM -> 寄存器 (warpgroup 作用域)
        reg = T.alloc_local((BLK_N,), acc_type)
        reg_wg = reg.view(128, BLK_N,
            layout=TileLayout(S[(128, BLK_N) : (1@tid_in_wg,
→ 1)]))
        Tx.wg.copy_async(reg_wg[:, :], tmem[:, :BLK_N])
        T.ptx.tcgen05.wait.ld()

        # 通知 TMEM 已空闲 (全部 128 个线程到达)
        ld2mma.arrive(0, cta_id=0, pred=True)

        # 把 fp32 转换为 fp16
        Tx.cast(reg_f16[:, :], reg[:, :])

        # 写入 Dsmem + TMA 存储
        Tx.copy(Dsmem[warp_id * 32 + lane_id, :], reg_
→ f16[:, :])
        T.ptx.fence.proxy_async("shared::cta")
        T.cuda.warpgroup_sync(10)
        if warp_id == 0:
            if lane_id == 0:
                Tx.copy_async(D[m_st:m_st+BLK_M, n_st:n_
→ st+BLK_N],

```

(continues on next page)

(continued from previous page)

```

                                Dsmem[:, :], dispatch="tma")
        T.ptx.cp_async.bulk.commit_group()
        T.ptx.cp_async.bulk.wait_group(0)
        T.cuda.warpgroup_sync(10)

        tile_scheduler.next_tile()

    # --- 清理 ---
    T.cuda.cta_sync()
    if warp_id == 0:
        T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
        T.ptx.tcgen05.dealloc(tmем_addr[0], n_cols=512, cta_
→group=1)

    return kernel

```

要运行其中任何一个内核, 复用第 1 步 (构建分块 *GEMM*) 中展示过一次的编译/运行/校验脚手架即可: 把 `hgemm_v1` 换成 `hgemm_v7`、`hgemm_v8` 或 `hgemm_v9`, 并选一个问题规模例如  $M=N=K=4096$ 。注意集群化的步骤需要  $M$  和  $N$  是其集群分块的整数倍 (第 8 步为  $256 \times 256$ , 第 9 步为  $512 \times 256$ ), 所以一个极小的  $128 \times 128$  规模根本产生不出任何分块。每一步都要在一个全新的 Python 会话中编译, 切换步骤前重启内核, 因为这些内核复用了内部名字, 而编译器会持有每会话状态。各步的耗时汇总在下面的端到端结果中。

## 收尾 (回写) 细节

第 7 步可以采用一种简单得令人愉快的收尾。由于只有  $BLK\_N=128$  列, 回写 `warpgroup` 只需一轮就把整块 `TMEM` 分块读进寄存器, 然后发起一次 `TMA` 存储。第 8、9 步没有这种奢侈, 这正是它们后来要引入分块读取的原因; 就目前而言, 顺序是:

1. 等待 `MMA:mma2ld.wait(phase)`。本教程第 8、9 步会在此额外加一个 `fence.after_thread_sync()` 作为保守补强; `MMA` 完成的 `mbarrier` 已经覆盖了该顺序, 而大多数内核 (包括 `CUTLASS`) 都省略它, 所以第 7 步同样省略。
2. 读 `TMEM` -> 寄存器 (每线程 128 个 `fp32`, `warpgroup` 作用域, 通过 `Tx.copy_async(reg_wg, tmem[:, :BLK_N])` 再接 `T.ptx.tcgen05.wait.ld()`)。
3. 通知 `MMA:ld2mma.arrive(0, cta_id=0, pred=True)` (全部 128 个线程到达); `TMEM` 此刻对下一个分块空闲。两个 `arrive` 关键字参数在集群化步骤中会再次出现: `cta_id` 指明哪个 `CTA` 的屏障副本被通知 ( $0$  = 本 `CTA`, 即本地屏障; 第 8 步中协作式的 `arrive` 通过 `cta_mask` 指向 `CTA-0`), 而 `pred` 是逐线程的谓词, 门控该线程是否真正参与到达 (此处为 `True`, 因此每个回写线程都计入到达总数)。
4. 在寄存器中把 `fp32` 转换为 `fp16`。
5. 把寄存器 -> `Dsmem` 写回, 然后 `fence.proxy_async("shared::cta") + warpgroup_sync(10)` 刷出。
6. 通过 `cp_async.bulk.commit_group() + wait_group(0)` 把 `Dsmem` 用 `TMA` 存储 -> `GMEM`。

第 8 步 (用  $BLK\_N=256$ ) 和第 9 步 (每个消费者用  $MMA\_N=256$ ) 无法保持这种单轮形态, 原因在于寄存器压力。每线程读 256 个 `fp32` 值意味着  $256 \times 4 = 1024$  字节必须同时存活在每个线程的寄存器中, 这有溢出到 `local memory` 的风险, 此外还迫使 `Dsmem` 缓冲更大。所以这些步骤



把回写拆成 EPI\_N 列的小块 (EPI\_N=64): 每次迭代只让 EPI\_N 个 fp32 寄存器处于活跃, 并发起相应更小的 TMA 存储, 用多一点存储指令换取仍可舒适容纳的寄存器预算。

实现说明。

- 持久化内核: `bx = T.cta_id([SM_COUNT])` — 每个 SM 一个 CTA, 循环遍历各分块
- 对 L2 友好的调度: `ClusterPersistentScheduler2D` 按缓存局部性排序分块
- 这种模式—warp specialization 加软件流水线—在高性能 GEMM 内核中很常见, 包括 CUTLASS 风格的设计。

## 当第 7 步出问题

第 7 步是第一个让 TMA 加载、`tcgen05 MMA` 与回写同时并发的 GEMM 内核。同样的失败模式在第 8、9 步会再次出现: 屏障计数不匹配、角色守卫放错位置、缺失栅栏, 或暂存缓冲在 TMA 存储排空前被复用。这些情况的调试清单汇总在[调试线程束特化内核](#)。

**流水线深度调优。**第 7 步内核以 `PIPE_DEPTH=2`(最小值) 运行。把它推到 4 或 6 能让 TMA 生产者比 MMA 消费者跑得更靠前, 隐藏更多访存延迟, 但代价是消耗更多 SMEM, 而 SMEM 是有限的。B200 每个 SM 提供 228 KB(见[GPU 执行模型](#)的需要记住的数字)。在 `BLK_M=BLK_N=128`, `BLK_K=64`, `fp16` 下, 每个流水级仅 A 和 B 就占  $(128 \times 64 + 128 \times 64) \times 2 = 32 \text{ KB}$ , 再加上 Dsmem 回写暂存缓冲又多 32 KB。这让 `PIPE_DEPTH=4` 大约 160 KB, `PIPE_DEPTH=6` 大约 224 KB, 正好贴着预算。要再深一些, 就得重新考虑回写暂存策略。

warp specialization 让一个 CTA 内的线程协作起来。下一步把这种协作扩大到 CTA 自身的边界之外, 让两个 CTA 共同处理一个更大的分块。

### 1.14.2 第 8 步: 2-CTA 集群

第 7 步让各引擎重叠起来, 但每个 CTA 仍在孤立地计算自己的  $128 \times 128$  分块, 重新加载任何邻居都借不到的操作数。第 8 步打破这种孤立。两个 CTA 联合成一个集群, 并获得访问彼此共享内存的能力, 于是一次协作式的 `tcgen05 MMA` 就能产出一个横跨两者的  $256 \times 256$  分块, 而一次 B 的加载现在能驱动两倍的 MMA 工作。一如既往,  $M=N=K=4096$ 。

**本步改变了什么: 作用域 + 布局 + 派发**

- 作用域: 协作作用域现在横跨集群中的两个 CTA, 而不止一个。
- 布局: 操作数分块被拆分到两个 CTA 的 SMEM 上; CTA 0 拥有共享的完成屏障 (`remote_view`)。
- 派发: MMA 增加 `cta_group / cta_mask`, 使 `tcgen05` 作为 2-CTA 协作操作运行。

主题。

- CTA cluster(CTA 集群): 多个 CTA 协作处理一个更大的分块
- 通过 `map_shared_rank` 跨 CTA 访问 SMEM
- 用 `cta_group=2` 在  $256 \times 256$  集群分块上做协作式 MMA
- 用 `cta_mask` 跨 CTA 发出屏障信号

## 集群分块形状

整个优化建立在一项硬件能力之上: 在 `cta_group=2` 下, MMA 允许读取由两个 CTA 暂存的操作数分块, 而不只是它所在的那个。每个 CTA 加载一行 128 行的存储 B 切片, 经转置后变成 128 个逻辑输出列, 而协作式 MMA 把这两个切片重新拼合成一个操作数。下图追踪两个 CTA 的 A、B 切片如何合并为单一的  $256 \times 256$  集群分块:

可交互: 每个 CTA 拥有一片 A 行切片和一片存储-B 行切片, 然后跨集群 (DSMEM) 读取另一个 CTA 的存储-B 切片。B.T 之后, 两片存储-B 切片覆盖完整的输出列跨度, 于是这对 CTA 产出一个  $256 \times 256$  输出分块。

**为何 A 和 B 跨集群拆分:** 要看清  $256 \times 256$  分块如何划分, 回忆本教程把 GEMM 存储为  $D = A @ B.T$ , 其中存储的 B 形状为  $N \times K$ 。集群中有两个 CTA, 划分就干净地落下来:

- **A 按列拆分:** CTA-0 持有 A0(行 0-127), CTA-1 持有 A1(行 128-255)。堆叠起来: [A0; A1](256 行)。
- **存储 B 按行拆分:** CTA-0 加载 B 的行 0-127, CTA-1 加载行 128-255。由于计算用的是 B.T, 这两片存储的行切片变成逻辑右操作数的两片 128 列切片。
- 在 `cta_group=2` 下, MMA 硬件通过跨 CTA 共享内存访问从两个 CTA 的 SMEM 读 B, 于是它看到完整的逻辑输出列跨度。
- 结果: 两个 CTA 在一个  $256 \times 256$  输出分块上协作。每个 CTA 写出该分块的一条  $128 \times 256$  行带。

值得停下来看清这是真正的收益, 而不仅仅是对工作的重新洗牌。每个 CTA 仍只加载  $128 \times K$  的 A 和  $128 \times K$  的 B, 所以集群整体暂存的操作数大约是单个 CTA 的 2x, 然而它产出  $256 \times 256$  的分块, 携带约 4x 于  $128 \times 128$  分块的输出 FLOPs。于是 MMA 对每字节已暂存操作数做的工作大约翻倍, 因为每个 CTA 的 B 切片会通过协作式 MMA 与另一个 CTA 的 A 切片配合复用。换句话说, 算术强度大约翻倍, 而这正是一个仍偏访存的内核所需要的杠杆: 端到端表中约 2.2x 的加速就来自把同样的字节喂给更多运算。

## 分块地址计算

既然集群成了工作单元, 分块调度器也得以集群分块来计数。它返回的每个 (`m_idx`, `n_idx`) 命名一整块  $256 \times 256$  区域, 而集群内的两个 CTA 在它们之间拆分该区域。把一个集群坐标翻译成每个 CTA 实际加载的逐 CTA 切片, 看起来是这样:

```
m_st = (m_idx * CTA_GROUP + cbx) * BLK_M
n_st = (n_idx * CTA_GROUP + cbx) * BLK_N
```

两个 CTA 处理同一个  $256 \times 256$  集群分块, 而单一坐标 `cbx`(CTA 在集群内的位置, 0 或 1) 正是沿两条轴选出本 CTA 贡献的关键。`m_st` 选出本 CTA 拥有的输出行带, `n_st` 选出它喂给协作式 MMA 的存储-B 切片, 而稍后的回写则发出 256 列输出跨度的两段 128 列。还要注意 `num_m_tiles = M // 256` 和 `num_n_tiles = N // 256` 数的是集群分块, 而不是单个 CTA 分块。

乍看之下 `cbx` 同时出现在 `m_st` 和 `n_st` 中, 仿佛一个行偏移不知怎么漏进了列里, 但两处用法都是对的, 值得理清原因。在回写路径上, `cbx` 只属于 M 轴: 每个 CTA 拥有一条不同的 128 行带 (`m_st = (m_idx * CTA_GROUP + cbx) * BLK_M`, 于是 CTA-0 写行 `m_idx*256 .. +128`, CTA-1 写接下来的 128 行), 然而两个 CTA 都写出集群分块的完整 256 输出列。这正是为什么存储从集群的 `n_idx` 推导列 (`n_st_epi = n_idx * 256 + no * 128`, 看不见 `cbx`), 而不是从逐 CTA 的 `n_st` 推导。`n_st` 之所以带 `cbx`, 是因为每个 CTA 把不同的存储-B 行切片加载进 MMA: 在那里, `cbx` 是一个加载偏移, 而不是该 CTA 的输出列偏移。

## 相对第 7 步的代码改动

相对第 7 步的 diff 有六处编辑, 每一处编码了上面所述集群契约中的一条:

```
# 1. 集群启动
cbx, cby = T.cta_id_in_cluster([CTA_GROUP, 1]) # cbx = CTA 在集群内的索引 (0 或 1)

# 2. 协作式 MMA(原来是 cta_group=1)
Tx.gemm_async(..., cta_group=2)

# 3. 跨 CTA 共享内存访问
B_remote = T.ptx.map_shared_rank(Bsmem, cta_id=1)

# 4. 跨 CTA 屏障
tma2mma_cta0 = T.decl_buffer(
    [CTA_GROUP], "uint64",
    data=T.ptx.map_shared_rank(tma2mma.ptr_to([0]), 0),
    scope="shared"
)

# 5. mma2tma / mma2ld 的 arrive 从 cta_mask=0(单 CTA, 第 7 步)
# 变为 cta_mask=3(通知集群内的两个 CTA)
mma2tma.arrive(mma_ps.stage, cta_group=CTA_GROUP, cta_mask=3)
mma2ld.arrive(0, cta_group=CTA_GROUP, cta_mask=3)

# 6. 结尾用 cluster_sync 取代 cta_sync
T.cuda.cluster_sync()
```

## 集群作用域的改动

这六处编辑都源于同一个转变: 协作作用域现在是集群, 而不是单个 CTA。下面几点具体说明这种扩大在实践中意味着什么: 每个 CTA 如何找到自己的位置、集群在谁的屏障上协调、以及究竟哪个 CTA 发起协作式 MMA。

- **集群 CTA ID:** cbx 告诉每个 CTA 它在集群中的位置 (0 或 1)。CTA-0 处理 A 的行 0-127, CTA-1 处理行 128-255。
- **远程屏障视图:** 在集群中, 每个 CTA 有自己的 SMEM 和自己的屏障, 这引出一个显而易见的问题: 如果 CTA-1 需要等待 CTA-0 产生的某物, 它实际触碰的是谁的屏障? 答案是提名 CTA-0 的屏障作为唯一协调点, 并让集群内任何 CTA 都能访问它。map\_shared\_rank(tma2mma.ptr\_to([0]), 0) 返回一个集群范围内指向 CTA-0 屏障的指针, 经 TIRx 包装为 tma2mma.remote\_view(0), 此后每个 arrive 和 wait 都指向 CTA-0 的副本。
- **仅由 CTA-0 派发 MMA:** 很容易把 cta\_group=2 读成同时触发两台引擎并行, 但实际并非如此。CTA-0 恰好发起一次 tcgen05.mma, 硬件随后驱动一次单一协作式 MMA, 横跨两个 CTA, 从两个 SM 的 SMEM 读操作数, 并把累加器写到两个 SM 的 TMEM 上。CTA-1 完全不发 MMA。(每个 SM 只有一个 tcgen05 引擎, 所以 cta\_group=2 是一次跨 SM 的 MMA, 而不是两台引擎并排运行。)这就是为什么代码用 if cbx == 0: 守护 MMA。
- **多播 arrive:** tcgen05.commit(..., cta\_group=2, cta\_mask=3) 只由 CTA-0 发起, 但通知两个 CTA 的屏障。cta\_mask=3(二进制 11) 意味着同时指向 CTA-0 和 CTA-1。

- **ld2mma** 初始化计数: `init(128 * CTA_GROUP)` — 两个 CTA 的回写 warpgroup(各 128 个线程) 都到达。

实现。

```
def hgemv_v8(M, N, K):
    a_type = tvm.DataType("float16")
    b_type = tvm.DataType("float16")
    d_type = tvm.DataType("float16")
    acc_type = tvm.DataType("float32")

    CTA_GROUP = 2
    BLK_M, BLK_N, BLK_K = 128, 128, 64
    MMA_M, MMA_N = 256, 256
    K_TILES = K // BLK_K
    PIPE_DEPTH = 4
    WG_NUMBER = 2
    F16_SIZE = 2 # fp16

    A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_
    ↪ATOM, (PIPE_DEPTH, BLK_M, BLK_K))
    B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_
    ↪ATOM, (PIPE_DEPTH, BLK_N, BLK_K))
    D_layout = tma_shared_layout(d_type, SwizzleMode.SWIZZLE_128B_
    ↪ATOM, (BLK_M, 128))

    @T.prim_func
    def kernel(
        A: T.Buffer((M, K), a_type),
        B: T.Buffer((N, K), b_type),
        D: T.Buffer((M, N), d_type),
    ):
        T.device_entry()
        bx = T.cta_id([SM_COUNT])
        cbx, cby = T.cta_id_in_cluster([CTA_GROUP, 1])
        wg_id = T.warpgroup_id([WG_NUMBER])
        warp_id = T.warp_id_in_wg([4])
        lane_id = T.lane_id([32])

        # --- 分配 ---
        pool = T.SMEMPool()
        tmem_addr = pool.alloc((1,), "uint32")
        tma2mma = TMABar(pool, PIPE_DEPTH)
        mma2tma = TCGen05Bar(pool, PIPE_DEPTH)
        mma2ld = TCGen05Bar(pool, 1)
        ld2mma = MBarrier(pool, 1)
        pool.move_base_to(1024)
        Asmem = pool.alloc((PIPE_DEPTH, BLK_M, BLK_K), a_type,
    ↪layout=A_layout)
        Bsmem = pool.alloc((PIPE_DEPTH, BLK_N, BLK_K), b_type,
    ↪layout=B_layout)
```

(continues on next page)

(continued from previous page)

```

Dsmem = pool.alloc((BLK_M, 128), d_type, layout=D_layout)

# --- 屏障初始化 ---
tma2mma.init(1)
mma2tma.init(1)
mma2ld.init(1)
ld2mma.init(128 * CTA_GROUP) # 两个 CTA 的回写线程
pool.commit()

# --- TMEM 分配 (协作式) ---
if wg_id == 0:
    if warp_id == 0:
        T.ptx.tcgen05.alloc(T.address_of(tmем_addr), n_
→cols=512, cta_group=CTA_GROUP)
        T.ptx.fence.proxy_async("shared::cta")
        T.ptx.fence.mbarrier_init()
        T.cuda.cta_sync()

    tmem = T.decl_buffer(
        (128, 512), acc_type, scope="tmem", allocated_addr=tmem_
→addr[0],
        layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)]))

# --- 分块调度器 (集群分块) ---
tile_scheduler = ClusterPersistentScheduler2D(
    "ts", num_m_tiles=M // 256, num_n_tiles=N // 256,
    l2_group_size=8, num_clusters=SM_COUNT // CTA_GROUP)
tile_scheduler.init(bx // CTA_GROUP)
m_idx = T.meta_var(tile_scheduler.m_idx)
n_idx = T.meta_var(tile_scheduler.n_idx)
m_st = T.meta_var((m_idx * CTA_GROUP + cbx) * BLK_M)
n_st = T.meta_var((n_idx * CTA_GROUP + cbx) * BLK_N)

# --- 跨 CTA 屏障视图 ---
tma2mma_cta0 = tma2mma.remote_view(0)

# =====
# Warpgroup 1:TMA 生产者 (warp 3)+ MMA 消费者 (warp 0)
# =====
if wg_id == 1:
    if warp_id == 3:
        tma_ps = PipelineState(PIPE_DEPTH, phase=1)

        @T.inline
        def tma_load(k_offset):
            Tx.copy_async(Asmem[tma_ps.stage, :, :],
                          A[m_st:m_st+BLK_M, k_offset:k_
→offset+BLK_K],
                          dispatch="tma", cta_group=CTA_

```

(continues on next page)

(continued from previous page)

```

→GROUP,
                                mbar=tma2mma_cta0.ptr_to([tma_ps.
→stage]))
                                Tx.copy_async(Bsmem[tma_ps.stage, :, :],
                                B[n_st:n_st+BLK_N, k_offset:k_
→offset+BLK_K],
                                dispatch="tma", cta_group=CTA_
→GROUP,
                                mbar=tma2mma_cta0.ptr_to([tma_ps.
→stage]))

    if T.filter(lane_id, T.ptx.select_sync()):
        while tile_scheduler.valid():
            for k in range(K_TILES):
                mma2tma.wait(tma_ps.stage, tma_ps.phase)
                tma_load(k * BLK_K)
                if cbx == 0:
                    tma2mma_cta0.arrive(tma_ps.stage,
                    CTA_GROUP * (BLK_M * BLK_K +
→BLK_N * BLK_K) * F16_SIZE)
                    tma_ps.advance()
                    tile_scheduler.next_tile()

            elif warp_id == 0:
                mma_ps = PipelineState(PIPE_DEPTH, phase=0)
                ld_ps = PipelineState(1, phase=1)

                if cbx == 0:
                    if T.filter(lane_id, T.ptx.select_sync()):
                        while tile_scheduler.valid():
                            ld2mma.wait(ld_ps.stage, ld_ps.phase)
                            ld_ps.advance()

                        for k in range(K_TILES):
                            tma2mma.wait(mma_ps.stage, mma_ps.
→phase)

                            Tx.gemm_async(
                                tmem[:, :MMA_N],
                                Asmem[mma_ps.stage, :, :],
                                Bsmem[mma_ps.stage, :, :],
                                accum=(k != 0), dispatch=
→"tcgen05", cta_group=CTA_GROUP)
                            mma2tma.arrive(mma_ps.stage, cta_
→group=CTA_GROUP, cta_mask=3)
                            mma_ps.advance()

                            mma2ld.arrive(0, cta_group=CTA_GROUP,
→cta_mask=3)
                            tile_scheduler.next_tile()

```

(continues on next page)



(continued from previous page)

```

# =====
# Warpgroup 0: 回写 (256 列分成 2 个 128 列小块)
# =====
elif wg_id == 0:
    wb_ps = PipelineState(1, phase=0)
    reg_f16 = T.alloc_local((128,), d_type)

    while tile_scheduler.valid():
        mma2ld.wait(wb_ps.stage, wb_ps.phase)
        wb_ps.advance()
        T.ptx.tcgen05.fence.after_thread_sync()

        for no in T.unroll(2): # 2 个 128 列小块 = 共 256 列
            reg = T.alloc_local((128,), acc_type)
            reg_wg = reg.view(128, 128,
                              layout=TileLayout(S[(128, 128) : (1@tid_in_
↪wg, 1)]))
            Tx.wg.copy_async(reg_wg[:, :], tmem[:, no *
↪128:(no + 1) * 128])
            T.ptx.tcgen05.wait.ld()
            Tx.cast(reg_f16[:, :], reg[:, :])
            Tx.copy(Dsmem[warp_id * 32 + lane_id, :], reg_
↪f16[:, :])
            T.ptx.fence.proxy_async("shared::cta")
            T.cuda.warpgroup_sync(10)
            if warp_id == 0:
                if lane_id == 0:
                    n_st_epi = T.meta_var(n_idx * 256 + no_
↪* 128)
                    Tx.copy_async(D[m_st:m_st+BLK_M, n_st_
↪epi:n_st_epi+128],
                                Dsmem[:, :], dispatch="tma
↪")
                    T.ptx.cp_async.bulk.commit_group()
                    T.ptx.cp_async.bulk.wait_group(0)
                    T.cuda.warpgroup_sync(10)

                    ld2mma.arrive(0, cta_id=0, pred=True)
                    tile_scheduler.next_tile()

        # --- 清理 ---
        T.cuda.cluster_sync()
        if warp_id == 0:
            T.ptx.tcgen05.relinquish_alloc_permit(cta_group=CTA_
↪GROUP)
            T.ptx.tcgen05.dealloc(tmem_addr[0], n_cols=512, cta_
↪group=CTA_GROUP)

```

(continues on next page)

(continued from previous page)

`return kernel`

## 2 个 CTA 带来的变化。

- $\text{CTA\_GROUP} = 2, \text{MMA\_N} = \text{BLK\_N} * \text{CTA\_GROUP} = 256$
- `ld2mma.init(128 * CTA_GROUP)` —两个 CTA 的回写 WG 都到达
- TMA arrive 的字节计数包含两个 CTA:  $\text{CTA\_GROUP} * (\text{BLK\_M} * \text{BLK\_K} + \text{BLK\_N} * \text{BLK\_K}) * \text{F16\_SIZE}$
- `tcgen05.alloc` 和 `tcgen05.dealloc` 必须用 `cta_group=2`
- 回写把 256 个输出列拆成两个 128 列小块—一次读完所有 256 个 TMEM 列会超出寄存器容量。第 9 步把小块进一步缩小到 `EPI_N=64`
- 结尾用 `cluster_sync()` 取代 `cta_sync()` (确保 TMEM 释放前所有 CTA 都已完成)

所有这些额外的算术强度直接反映在墙上时间上: 第 8 步在 4096<sup>3</sup> 下达到 **0.104 ms**, 约为同样规模下 70 ms 第 1 步算法的 676×(见端到端表)。内核现在倾向于计算受限, 而这恰好为第 9 步打下基础——我们在那里加入第二个 MMA 消费者, 让更多 Tensor Core 工作并发起来。

如果第 8 步比第 7 步更慢, 罪魁祸首几乎总是某条新的集群契约被略微搞错。值得优先检查三件事: TMA arrive 的字节计数是否为  $\text{CTA\_GROUP} * (\text{BLK\_M} * \text{BLK\_K} + \text{BLK\_N} * \text{BLK\_K}) * \text{F16\_SIZE}$ ; 针对 256×256 集群分块, 调度器维度是否为 `num_m_tiles=M//256`, `num_n_tiles=N//256`; 以及回写是否发起了两次 TMA 存储, 每个 128 列小块一次, 且每次都在 Dsmem 复用前排空。

集群提升了跨 CTA 的复用。最后一步转向内部, 通过给生产者加上第二个 MMA 消费者来喂养, 在每个 CTA 内部提升计算密度。

### 1.14.3 第 9 步: 多消费者的线程束特化

到第 8 步, MMA 确实已经很忙, 但单个消费者 warp 消化一份已加载 B 分块的速度也就那么快, 而那份 B 分块在这段时间里就一直待在 SMEM 里, 任何想读它的人都可读取。最后的优化正是利用这一点: 它加入第二个 MMA 消费者, 把一块不同的 A 块乘到同一份 B 分块上。每个 CTA 的计算密度翻倍, 集群输出从 256×256 增长到 512×256。一如既往,  $M=N=K=4096$ 。

#### 本步改变了什么: 作用域 + 布局

- 作用域: 一个 MMA 消费者变成两个, 由 `warp_id` 选出。
- 布局: 一份已加载的 B 分块被两个消费者复用; A 增加一个消费者轴。
- 派发: 不变。

#### 主题。

- 多个 MMA warp(消费者) 以获得更高吞吐量
- 多个回写 `warpgroup`, 各自有独立的屏障槽位
- 本教程中最优化的 GEMM 变体所采用的结构

## 多消费者结构

加入第二个消费者意味着内核现在有更多不同角色要安排: 两个 MMA warp 而不是一个, 外加一个与之配套的第二个回写 warpgroup 来排空额外的累加器。在 NUM\_CONSUMER=2 和 WG\_NUMBER=3 下, 内核现在横跨三个 warpgroup(角色表中简称为 WG):

| Warpgroup   | Warp   | 角色                                                |
|-------------|--------|---------------------------------------------------|
| <b>WG 2</b> | warp 0 | MMA 消费者 0: Asmem[ ..., 0] x B -> TMEM 列 [0:256]   |
| <b>WG 2</b> | warp 1 | MMA 消费者 1: Asmem[ ..., 1] x B -> TMEM 列 [256:512] |
| <b>WG 2</b> | warp 3 | TMA 生产者: 每个流水级加载 2 份 A 块 + 1 份 B 块                |
| <b>WG 0</b> | 全部     | 消费者 0 的回写: 读 TMEM [0:256]                         |
| <b>WG 1</b> | 全部     | 消费者 1 的回写: 读 TMEM [256:512]                       |

整套安排的关键在于一处不对称。每个消费者把各自的 A 块乘到同一份已加载的 B 分块上, 所以一次 B 的加载现在驱动 2× 的 MMA 工作, B 的加载成本相对于有效 FLOP 实际上减半。我们共享 B 而不是 A 的原因在于, 两个消费者覆盖不同的 M 行带: 它们的 A 块是真正不同的数据, 而 B 对两者相同。练习 3 让你确信这是唯一可行的共享方式。

## 相对第 8 步的改动

具体来说, 支持第二个消费者会在几处触及内核, 而每一处改动都可追溯到同一个事实: 现在每个流水级要喂入并排空两个 A 块和两段 TMEM 范围, 而 B 保持共享。下面的编辑暂存额外一个 A 块, 给每个消费者自己的屏障槽位, 并为更高的 512×256 集群分块调整分块寻址。

- `Asmem = pool.alloc((PIPE_DEPTH, NUM_CONSUMER, BLK_M, BLK_K), ...)`  
—每个流水级 2 个 A 块, 每个消费者一个
- TMA 同时加载 `Asmem[stage, 0]` 和 `Asmem[stage, 1]`, TMA arrive 字节数现在是 `CTA_GROUP * (NUM_CONSUMER * BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE`(多出的 A 块)
- MMA warp 的 `warp_id` 选出使用哪个 A 块和 TMEM 范围
- `mma2tma.init(NUM_CONSUMER)` —两个消费者每个流水级都通知 TMA
- `mma2ld` 和 `ld2mma` 的 `depth=NUM_CONSUMER` —每个消费者用自己的屏障槽位 (MMA 侧用 `warp_id`, 回写侧用 `wg_id`)
- 分块地址: `m_st = (m_idx * NUM_CONSUMER * CTA_GROUP + cbx) * BLK_M` —M 方向上多出一个 `NUM_CONSUMER` 因子, 因为每个集群分块现在在 M 上横跨 `NUM_CONSUMER` 个消费者。分块调度器使用 `num_m_tiles = M // 256 // NUM_CONSUMER`(集群分块为 512x256)
- 回写采用分块的 `EPI_N`, 使每次迭代在寄存器中存活的 TMEM 回读值更少

实现。

```
def hgemv_v9(M, N, K):
    a_type = tvm.DataType("float16")
    b_type = tvm.DataType("float16")
    d_type = tvm.DataType("float16")
    acc_type = tvm.DataType("float32")
```

(continues on next page)

(continued from previous page)

```

CTA_GROUP = 2
NUM_CONSUMER = 2
BLK_M, BLK_N, BLK_K = 128, 128, 64
MMA_N = BLK_N * CTA_GROUP # 256
K_TILES = K // BLK_K
PIPE_DEPTH = 4
EPI_N = 64
WG_NUMBER = 3
F16_SIZE = 2 # fp16

A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_
→ATOM,
                                (PIPE_DEPTH, NUM_CONSUMER, BLK_M,
→BLK_K))
B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_
→ATOM,
                                (PIPE_DEPTH, BLK_N, BLK_K))
D_layout = tma_shared_layout(d_type, SwizzleMode.SWIZZLE_128B_
→ATOM,
                                (NUM_CONSUMER, BLK_M, EPI_N))

@T.prim_func
def kernel(
    A: T.Buffer((M, K), a_type),
    B: T.Buffer((N, K), b_type),
    D: T.Buffer((M, N), d_type),
):
    T.device_entry()
    bx = T.cta_id([SM_COUNT])
    cbx, cby = T.cta_id_in_cluster([CTA_GROUP, 1])
    wg_id = T.warpgroup_id([WG_NUMBER])
    warp_id = T.warp_id_in_wg([4])
    lane_id = T.lane_id([32])

    # --- 分配 ---
    pool = T.SMEMPool()
    tmem_addr = pool.alloc((1,), "uint32")
    tma2mma = TMABar(pool, PIPE_DEPTH)
    mma2tma = TCGen05Bar(pool, PIPE_DEPTH)
    mma2ld = TCGen05Bar(pool, NUM_CONSUMER) # depth=2, 每个消费
者一个槽位
    ld2mma = MBarrier(pool, NUM_CONSUMER) # depth=2, 每个消费
者一个槽位
    pool.move_base_to(1024)
    Asmem = pool.alloc((PIPE_DEPTH, NUM_CONSUMER, BLK_M, BLK_K),
→ a_type, layout=A_layout)
    Bsmem = pool.alloc((PIPE_DEPTH, BLK_N, BLK_K), b_type,
→layout=B_layout)
    Dsmem = pool.alloc((NUM_CONSUMER, BLK_M, EPI_N), d_type,

```

(continues on next page)

(continued from previous page)

```

→ layout=D_layout)

    # --- 屏障初始化 ---
    tma2mma.init(1)
    mma2tma.init(NUM_CONSUMER) # 每个流水级期望 2 次到达
    mma2ld.init(1)             # 每个槽位获得 1 次到达
    ld2mma.init(128 * CTA_GROUP) # 两个 CTA 的回写线程
    pool.commit()

    # --- TMem 分配 (协作式) ---
    if wg_id == 0:
        if warp_id == 0:
            T.ptx.tcgen05.alloc(T.address_of(tmем_addr), n_
→ cols=512, cta_group=CTA_GROUP)
            T.ptx.fence.proxy_async("shared::cta")
            T.ptx.fence.mbarrier_init()
            T.cuda.cta_sync()

            tmem = T.decl_buffer(
                (128, 512), acc_type, scope="tmem", allocated_addr=tmem_
→ addr[0],
                layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)]))

            # --- 分块调度器 (512x256 集群分块) ---
            tile_scheduler = ClusterPersistentScheduler2D(
                "ts", num_m_tiles=M // 256 // NUM_CONSUMER, num_n_
→ tiles=N // 256,
                l2_group_size=8, num_clusters=SM_COUNT // CTA_GROUP)
            tile_scheduler.init(bx // CTA_GROUP)
            m_idx = T.meta_var(tile_scheduler.m_idx)
            n_idx = T.meta_var(tile_scheduler.n_idx)
            m_st = T.meta_var((m_idx * NUM_CONSUMER * CTA_GROUP + cbx)
→ * BLK_M)
            n_st = T.meta_var((n_idx * CTA_GROUP + cbx) * BLK_N)

            tma2mma_cta0 = tma2mma.remote_view(0)

            # =====
            # Warpgroup 2:TMA 生产者 (warp 3)+ 2 个 MMA 消费者 (warp 0, 1)
            # =====
            if wg_id == 2:
                if warp_id == 3:
                    # === TMA 生产者: 每个流水级加载 2 份 A 块 + 1 份 B 块
→ ===

                    tma_ps = PipelineState(PIPE_DEPTH, phase=1)

                    @T.inline
                    def tma_load(k_offset):
                        m_st_c1 = T.meta_var(m_st + CTA_GROUP * BLK_M)

```

(continues on next page)

(continued from previous page)

```

        Tx.copy_async(Asmem[tma_ps.stage, 0, :, :],
                      A[m_st:m_st+BLK_M, k_offset:k_
→offset+BLK_K],
                      dispatch="tma", cta_group=CTA_
→GROUP,
                      mbar=tma2mma_cta0.ptr_to([tma_ps.
→stage]))
        Tx.copy_async(Asmem[tma_ps.stage, 1, :, :],
                      A[m_st_c1:m_st_c1+BLK_M, k_
→offset:k_offset+BLK_K],
                      dispatch="tma", cta_group=CTA_
→GROUP,
                      mbar=tma2mma_cta0.ptr_to([tma_ps.
→stage]))
        Tx.copy_async(Bsmem[tma_ps.stage, :, :],
                      B[n_st:n_st+BLK_N, k_offset:k_
→offset+BLK_K],
                      dispatch="tma", cta_group=CTA_
→GROUP,
                      mbar=tma2mma_cta0.ptr_to([tma_ps.
→stage]))

    if T.filter(lane_id, T.ptx.select_sync()):
        while tile_scheduler.valid():
            for k in range(K_TILES):
                mma2tma.wait(tma_ps.stage, tma_ps.phase)
                tma_load(k * BLK_K)
                if cbx == 0:
                    tma2mma_cta0.arrive(tma_ps.stage,
                                         CTA_GROUP * (NUM_CONSUMER * BLK_
→M * BLK_K + BLK_N * BLK_K) * F16_SIZE)
                    tma_ps.advance()
                    tile_scheduler.next_tile()

    elif warp_id < NUM_CONSUMER:
        # === MMA 消费者:warp_id 选出 A 块和 TMEM 范围 ===
        mma_ps = PipelineState(PIPE_DEPTH, phase=0)
        ld_ps = PipelineState(1, phase=1)

        if cbx == 0:
            if T.filter(lane_id, T.ptx.select_sync()):
                while tile_scheduler.valid():
                    ld2mma.wait(warp_id, ld_ps.phase)
                    ld_ps.advance()

                    for k in range(K_TILES):
                        tma2mma.wait(mma_ps.stage, mma_ps.
→phase)

                        Tx.gemm_async(

```

(continues on next page)



(continued from previous page)

```

tmem[:, warp_id * MMA_N:warp_id_
→ * MMA_N + MMA_N],
Asmem[mma_ps.stage, warp_id, :,
→:],
Bsmem[mma_ps.stage, :, :],
accum=(k != 0), dispatch=
→"tcgen05", cta_group=CTA_GROUP)
mma2tma.arrive(mma_ps.stage, cta_
→group=CTA_GROUP, cta_mask=3)
mma_ps.advance()

mma2ld.arrive(warp_id, cta_group=CTA_
→GROUP, cta_mask=3)
tile_scheduler.next_tile()

# =====
# Warpgroup 0/1: 回写 (各自读其消费者的 TMEM 范围)
# =====
elif wg_id < NUM_CONSUMER:
    wb_ps = PipelineState(1, phase=0)
    reg_f16 = T.alloc_local((EPI_N,), d_type)

    while tile_scheduler.valid():
        mma2ld.wait(wg_id, wb_ps.phase) # 等待本消费者
        wb_ps.advance()
        T.ptx.tcgen05.fence.after_thread_sync()

        # 按 EPI_N=64 列的小块读 TMEM(256 列需 4 次迭代)
        for i in T.unroll(MMA_N // EPI_N):
            reg = T.alloc_local((EPI_N,), acc_type)
            reg_wg = reg.view(128, EPI_N,
                layout=TileLayout(S[(128, EPI_N) : (1@tid_
→in_wg, 1)]))

            col_st = T.meta_var(wg_id * MMA_N + i * EPI_N)
            col_end = T.meta_var(wg_id * MMA_N + i * EPI_N_
→+ EPI_N)

            Tx.wg.copy_async(reg_wg[:,], tmem[:, col_st:col_
→end])

            T.ptx.tcgen05.wait.ld()
            Tx.cast(reg_f16[:,], reg[:,])
            Tx.copy(Dsmem[wg_id, warp_id * 32 + lane_id, :],
→ reg_f16[:,])

            T.ptx.fence.proxy_async("shared::cta")
            T.cuda.warpgroup_sync(wg_id + 10)
            if warp_id == 0:
                if lane_id == 0:
                    m_st_epi = T.meta_var(
                        (m_idx * NUM_CONSUMER * CTA_GROUP +
→wg_id * CTA_GROUP + cbx) * BLK_M)

```

(continues on next page)

(continued from previous page)

```

n_st_epi = T.meta_var(n_idx * MMA_N + i_u
→ * EPI_N)
Tx.copy_async(
    D[m_st_epi:m_st_epi+BLK_M, n_st_
→ epi:n_st_epi+EPI_N],
    Dsmem[wg_id, :, :], dispatch="tma")
T.ptx.cp_async.bulk.commit_group()
T.ptx.cp_async.bulk.wait_group(0)
T.cuda.warpgroup_sync(wg_id + 10)

ld2mma.arrive(wg_id, cta_id=0, pred=True)
tile_scheduler.next_tile()

# --- 清理 ---
T.cuda.cluster_sync()
if warp_id == 0:
    T.ptx.tcgen05.relinquish_alloc_permit(cta_group=CTA_
→ GROUP)
    T.ptx.tcgen05.dealloc(tmem_addr[0], n_cols=512, cta_
→ group=CTA_GROUP)

return kernel

```

实现说明。

- 在第 9 步的这种设计中,mma2ld 和 ld2mma 各自是单个共享对象且 depth=NUM\_CONSUMER, 而不是每个消费者各自独立。槽位 0 把 MMA warp 0 连到 Warpgroup 0, 槽位 1 把 MMA warp 1 连到 Warpgroup 1; MMA 侧用 warp\_id 索引, 回写侧用 wg\_id 索引。

#### 1.14.4 端到端结果

下表给出从朴素基线到线程束特化集群内核的实测里程碑, 并附 cuBLAS 参照。NVIDIA B200 上的参照数据, M=N=K=4096, fp16, 锁定频率, 1000 次迭代计时基准:

| 步骤 | 技术         | 耗时       | 加速比         |
|----|------------|----------|-------------|
| 1  | 同步加载 + MMA | 70 ms    | 1×          |
| 2  | K 循环累加     | —        | 处理 K 大于一个分块 |
| 3  | 空间分块       | 53.6 ms  | ~1.3×       |
| 4  | TMA 异步加载   | 0.49 ms  | ~142×       |
| 5  | 软件流水线      | —        | 重叠加载 + 计算   |
| 6  | 持久化内核      | —        | L2 缓存局部性    |
| 7  | 线程束特化      | 0.23 ms  | ~309×       |
| 8  | 2-CTA 集群   | 0.104 ms | ~676×       |
| 9  | 多消费者       | 0.094 ms | ~744×       |
| —  | cuBLAS(参照) | 0.094 ms | ~744×       |

这张表里的每个耗时, 包括 70 ms 的第 1 步基线, 都是在同一个 M=N=K=4096 规模下测得的, 正是这一点让整条加速链可以端到端比较。值得精确说明这 70 ms 究竟是什么, 因为它很容易

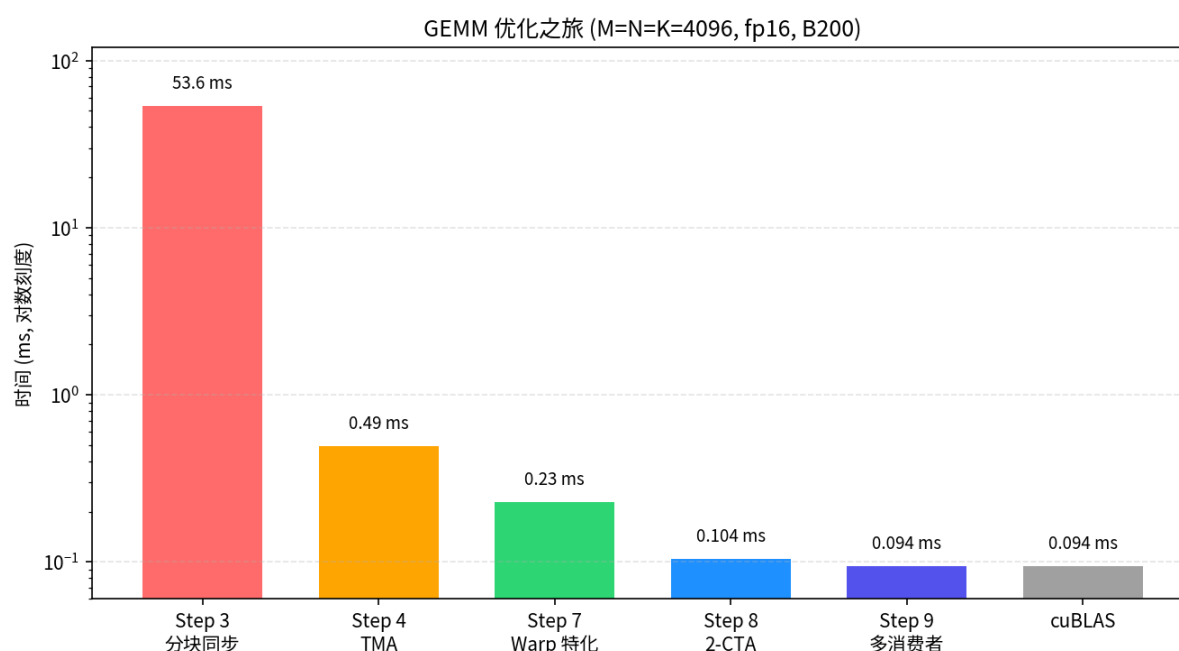
被误读。它不是构建分块 *GEMM* 中那个单分块第 1 步内核在  $4096^3$  下跑出来的结果; 那个内核只会算一个  $128 \times 128$  分块, 只在较小规模下运行。这 70 ms 是一个朴素的全规模基线, 采用同样的串行单分块方法, 把它放大到完整的  $4096^3$  问题。第 1-3 步在构建分块 *GEMM* 中以小规模 ( $128 \times 128$  和  $256^3$ ) 引入, 以让最初几趟走读保持简单; 此处的第 1、3 步两行是它们在全规模下的基准对应。余下的横线 (第 2、5、6 步) 标记那些为展示结构而呈现、但并未单独计时的步骤。

请把这些数字看作受控条件下 B200 的一次参照运行, 而不是榜单条目。各步骤中内嵌的 `{.python .input}` 基准测试单元是冒烟基准: 它们适合发现趋势, 不适合声称峰值性能。

四项技术几乎贡献了全部增益:

1. **TMA 异步数据搬运:** 硬件拷贝引擎取代软件拷贝 (第 1 步 → 第 4 步约  $142\times$ )。正确解读这个  $142\times$  很重要: 它反映的是从一个  $128 \times 128$  单分块内核 (grid  $1 \times 1$ ) 一路走到一个带 K 循环、空间分块、多 CTA 的完整分块并行内核, 再加上 TMA; 它并不是 TMA 单独的贡献。要单独剥离 TMA, 得比较两个仅在拷贝机制上不同的全规模内核。
2. **软件流水线 + 线程束特化:** 通过让加载和计算各有专职角色来重叠二者 (第 4 步 → 第 7 步约  $2.2\times$ )。
3. **CTA 集群:** 一次 2-SM 协作式 MMA 改善了跨 CTA 的 B 分块复用 (本基准中第 7 步 → 第 8 步约  $2.2\times$ )。
4. **多消费者:** 两个 MMA warp 提高计算密度 (第 8 步 → 第 9 步约  $10\%$ )。

按实测里程碑绘制, 这四项贡献正好描绘出从同步分块内核向 cuBLAS 参照下探的轨迹。下图展示所选的实测点:



注意越往下增益越小, 这背后有结构性原因, 而不是努力减弱。前几步针对访存瓶颈 (TMA 取代软件拷贝, 集群提升算术强度), 而 70 ms 大部分确实花在那里, 所以这些步骤回报最大。到第 8 步, 内核已经距 cuBLAS 不到  $10\%$  ( $0.104$  vs  $0.094$  ms), 接近计算受限, 意味着已几乎没有可隐藏的访存停顿; 第 9 步的多消费者重叠回收了所剩无几的大部分。约  $10\%$  的最终增益正是逼近计算上限时应有的预期: 它是一个近乎已解问题的边际递减, 而非一次孱弱的优化。

本章构建的所有内容 (TMA 加载、tcgen05 MMA、TMEM 回读、线程束特化屏障) 都直接延续到下一章。FlashAttention 复用了全部这些, 随后通过在两个 MMA 阶段之间楔入一个在线 softmax 步骤, 而非简单重复单一阶段, 把难度又提了一档。

### 1.14.5 练习

1. 在第 7 步中,如果把 TMA 和 MMA 的 PipelineState 初始 phase 都设为 0 会怎样? 画出死锁场景。
2. 在第 8 步的 cta\_group=2 下,TMA arrive 的字节计数是  $CTA\_GROUP * (BLK\_M * BLK\_K + BLK\_N * BLK\_K) * F16\_SIZE$ 。每个 CTA 各自加载自己的数据,为什么还要乘以 CTA\_GROUP?
3. 在第 9 步中,每个消费者处理不同的 M 行但同一份 B 分块。为什么共享 B(而非 A)是正确的选择?

交给你的 **agent** 试试: 粘贴第 7 步内核, 让它追踪一个 K 分块穿过四个屏障 (tma2mma、mma2tma、mma2ld、ld2mma) 的全过程。针对每个屏障, 问它谁在等待、谁在到达、哪个分块变得可安全读取、之后哪个缓冲变得可复用。

## 1.15 Flash Attention 4

### Overview

- 注意力由两个 MMA 组成,softmax 夹在它们中间,因此不能像 GEMM 那样只重复一个 MMA。
- 这个内核把 Part I 的硬件原语 (TMA、tcgen05、TMEM、barrier) 与 Part III 的 GEMM 技术,和 warp 角色、在线 softmax(online softmax) 重缩放、因果掩码 (causal masking) 以及 GQA 组合在一起。

注意力是决定一个 transformer 能否运行的内核,也是我们到目前为止构建的一切最终必须协同工作的地方。我们为 GEMM 组装的每一个部件在这里都用得上:TMA 分块搬运、tcgen05 MMA、TMEM、warpgroup 寄存器分块,以及显式 barrier。

难点在于,注意力并不是一个 MMA 的简单重复。它是两个 MMA,中间夹着真正的工作: 在线 softmax、因果掩码,以及让更早和更晚的分块保持同一尺度的重缩放。

那个中间阶段才是新难点的所在。一个普通的矩阵乘只往它的累加器里累加;而注意力必须在新 key 和 value 不断流入时,重新审视并重缩放它已经计算过的结果.softmax 工作本身也运行在两个 Tensor Core MMA 之间的 CUDA core 上,因此指数运算和按行规约直接落在关键路径上。

正因如此,注意力优化的很大一部分其实是 softmax 优化: 改写 exp, 并把 softmax 与 MMA 重叠起来,而不是卡在它上面等。

本章的目标不是从头重新推导 Flash Attention。我们只保留刚好够用的算法部分,让内核保持可读,然后把注意力放到真正新的部分: 这个算法如何变成 TIRx。

最清晰的切入点是跟随单个分块,看它如何流过内核。Q、K、V 作为输入分块,从 GMEM 加载到 SMEM。score MMA 把 Q 和 K 相乘,在 TMEM 中得到 score 分块 S。softmax 把 S 转成分子分块 P,value MMA 把 P 和 V 组合起来更新输出累加器 O。

到目前为止,这看起来像是两个矩阵乘粘在一起,但有一个 GEMM 从不需要处理的转折: 每当运行中的 softmax 最大值改变时,已经累加好的 O 突然尺度就不对了。在下一个 value MMA 能够安全地往里加之前,它必须被重缩放。下面的章节先追溯这条路径,然后才展示 TIRx 如何把每个阶段交给一个 warpgroup,并把各阶段连接起来。

### 1.15.1 算法形状

在我们把分块放进内存之前,需要先有这些分块所服务的算法。对于一个 query 块,Flash Attention 计算:

$$O = \text{softmax}(QK^T/\sqrt{d})V$$

字面上读,这个公式说要先形成完整的 score 矩阵  $S = QK^T$ ,对它做 softmax,再乘以  $V$ 。这恰恰是我们不能用的一种做法,因为完整的  $S$  极其庞大。在 seq=4096 时,每个 head 大约有 16M 个元素,fp32 下约 64 MB,比 SMEM 或单个 128×512 TMEM 区域都要大好几个数量级。片上根本没有地方放得下它。Flash Attention 的答案是:根本不去物化  $S$ 。它以分块的形式流式处理  $K/V$ ,并维护三个逐行的运行状态来概括目前看到的一切:

- row\_max: 目前见过的最大 score。
- row\_sum: softmax 的运行中分母。
- 0: 运行中的输出累加器。

流式更新就是在新分块到来时保持这些状态正确。微妙之处在于,每处理一个分块,运行中的最大值都可能上升,而一旦上升,我们在旧最大值下计算的一切现在尺度就错了。所以在加入新的贡献之前,我们先把旧状态拉回到新尺度:

```
S = Q_block @ K_block.T
m_new = max(row_max, rowmax(S))
scale = exp((row_max - m_new) / sqrt(d))
P = exp((S - m_new) / sqrt(d))
row_sum = row_sum * scale + rowsum(P)
0 = 0 * scale + P @ V_block
row_max = m_new
```

这里单个 scale 因子身兼二职:它同时重缩放运行中分母和运行中输出,使得来自更早和更晚分块的贡献最终都落在同一尺度下被衡量。

上面的伪代码用自然的 exp 和显式的 /sqrt(d) 写成,因为那样最容易读,但内核走了一条更省的路线。它把 1/sqrt(d) 和 log2(e) 一起折叠进一个常量 scale\_log2 = log2(e)/sqrt(d),并用恒等式  $\exp(x/\sqrt{d}) = \exp_2(x \cdot \text{scale\_log2})$  在原始 score 上用硬件 exp2 求每一个指数。动机很简单:在这块硬件上 exp2 比自然的 exp 快。

在继续之前有一点值得敲定:这里的  $P$  并不是最终归一化的注意力矩阵。它只是当前  $K/V$  分块的 softmax 分子。归一化被刻意推迟,只有在最后一个分块之后,内核才写入  $0 / \text{row\_sum}$ 。

对 TIRx 来说,知道算法计算什么只是一半。另一半是在内核运行时每个分块住在哪里,因为那正是决定布局与 barrier 代码的东西。 $S$ 、 $P$ 、 $0$  都是分块值,而每个都有自己的家:

- $S$  是 score 分块。score MMA 把它写到 TMEM。
- $P$  是 softmax 分子分块。softmax 把  $S$  从 TMEM 读进寄存器,计算  $P = \exp((S - m\_new) / \sqrt{d})$ ,再把  $P$  写回 TMEM。
- $0$  是输出累加器分块。value MMA 从 TMEM 读  $P$ 、从 SMEM 读  $V$ ,然后累加进 TMEM 中的  $0$ 。

我们前面标记的重缩放也是一个分块操作,而不是一段标量的簿记:当 row\_max 改变时,旧的  $0$  从 TMEM 读出,在寄存器中相乘,再写回 TMEM,然后下一个 value MMA 才往里累加。后面每一节都遵循同样的结构:一个分块放置,一条硬件路径,以及一道证明下一个消费者可以运行的 barrier。



### 1.15.2 分块原语图

有了运行状态和它们的归属, 我们就能把算法铺开成一条具体的分块搬运序列。对于一个 K/V 块, 内核自上而下走这条分块路径:

```

Q, K, V in GMEM
-> Q, K, V in SMEM      by TMA load
-> S in TMEM            by score MMA: QK^T
-> P in TMEM            by softmax numerator: TMEM -> RF -> TMEM
-> 0 in TMEM            by value MMA: P V
-> 0 in GMEM            by normalization, SMEM staging, and TMA
↪store

```

与 GEMM 的区别归结为一行。GEMM 是一条 MMA 链的重复;FA4 有两个 MMA 阶段,softmax 坐在链的中间。接下来几乎所有的不同, 都是这多出来的一个阶段的后果。

如果我们把这条短路径展开成显式的生产者-消费者边, 就得到完整的图:

| 阶段        | 分块搬运或计算                                     | TIRx 原语                                                  | 硬件路径                                                   |
|-----------|---------------------------------------------|----------------------------------------------------------|--------------------------------------------------------|
| 加载 Q/K/V  | GMEM 分块 -> SMEM 分块                          | <code>Tx.copy_async(..., dispatch="tma")</code>          | TMA load                                               |
| Score MMA | SMEM 中的 Q 和 SMEM 中的 K -> TMEM 中的 score 分块 S | <code>Tx.warp.gemm_async(..., dispatch="tcgen05")</code> | <code>tcgen05.mma</code>                               |
| softmax 读 | TMEM 中的 S -> warp-group 寄存器分块               | <code>Tx.wg.copy_async(reg, tmem)</code>                 | <code>tcgen05.ld</code>                                |
| softmax 写 | 寄存器中的分子分块 P -> fp16 TMEM 视图                 | <code>Tx.copy_async(tmem_as_fp16, reg)</code>            | TMEM store, followed by <code>tcgen05.wait.st()</code> |
| Value MMA | TMEM 中的 P 和 SMEM 中的 V -> TMEM 中的输出累加器 0     | <code>Tx.warp.gemm_async(..., dispatch="tcgen05")</code> | <code>tcgen05.mma</code> with a TMEM operand           |
| 修正        | TMEM 中的 0 -> 寄存器 -> TMEM 中的 0               | TMEM 回读、寄存器相乘、TMEM 存储                                    | <code>tcgen05.ld</code> / TMEM store                   |
| 收尾        | TMEM 中的最终 0 -> 寄存器 -> SMEM -> GMEM          | TMEM 回读、 <code>Tx.copy</code> 、TMA store                 | <code>tcgen05.ld</code> + TMA store                    |

新加的行是 softmax 和修正。两者都增加了 TMEM -> 寄存器 -> TMEM 的流量, 也都在 score MMA 和 value MMA 之间制造了额外的交接。

**与你的 agent 一起试试:** 让它只追溯上面这条短路径。对每个箭头, 说出生产者阶段、消费者阶段、源分块、目的分块和硬件路径。然后问哪些箭头在 GEMM 章节里并不存在。

### 1.15.3 Warp 角色与作用域

数据路径定下来之后, 自然要问的是每个阶段到底由谁来跑。这里每个 CTA 有 4 个 `warpgroup`, 共 512 个线程, 它们的划分不是按各自碰到的数据, 而是按一个 `warpgroup` 做哪类工作:

- WG3 驱动硬件引擎:TMA load、MMA、TMA store。



- WG0、WG1、WG2 做那些夹在引擎调用之间的、寄存器密集型数学:softmax、修正、收尾。

精确的角色表是:

| 拥有者         | 角色               | 做什么                           |
|-------------|------------------|-------------------------------|
| WG3, warp 1 | TMA load         | 把 Q、K、V 分块从 GMEM 加载到 SMEM     |
| WG3, warp 0 | MMA              | 既发 score MMA 又发 value MMA     |
| WG3, warp 2 | TMA store        | 把最终 O 分块从 SMEM 存到 GMEM        |
| WG0         | Q 阶段 0 的 softmax | 从 TMEM 读 S, 计算 P, 把 P 写到 TMEM |
| WG1         | Q 阶段 1 的 softmax | 为第二个 Q 流水级做同样的工作              |
| WG2         | 修正与收尾            | 重缩放 TMEM 中的 O, 归一化, 暂存输出      |

很容易把”两个 Q 阶段”误读成两个注意力 head, 但并非如此。它们只是 Q 流水线里的两个槽,WG0 拥有一个,WG1 拥有另一个, 这样两个 Q 分块就能同时天上飞。这正是 softmax 工作出现两次的原因, 一次在 WG0, 一次在 WG1。

代码用符号坐标把这些角色挑出来:

```
wg_id = T.warpgroup_id([4])
warp_id = T.warp_id_in_wg([4])
```

读这个内核时, 先找到角色分支。它会告诉你哪个团队拥有嵌在它里面的每一个分块原语。

- WG3 warp 1 启动 TMA load 命令。一个被选中的 lane 发起拷贝,TMA 引擎搬运分块。
- WG3 warp 0 发出 tcgen05.mma 指令。
- WG0 和 WG1 在完整的 warpgroup 作用域下跑 softmax。
- WG2 在完整的 warpgroup 作用域下跑修正与收尾工作。

有一种不对称最终塑造了整个 barrier 图: 每一个 MMA, 无论是 score 还是 value, 都只由 WG3 warp 0 单独发出。WG0 和 WG1 根本不发任何 MMA。它们只消费 score 分块、跑 softmax, 再把 P 写回 TMEM。

这种分离恰恰是 softmax 需要被 barrier 围起来的原因。s\_ready 把 score 分块从 MMA warp 送到 softmax;p\_o\_rescale 携带 P 和一个对 value MMA 安全的 0 槽——要么已经重缩放, 要么因为不需要重缩放而被释放。本章剩下部分会反复回到这两个名字。

#### 1.15.4 阅读这些片段

本章的片段摘自 `flash_attention4.py`, 因此不可避免地会引用我们在内核里没有复现的部分中所定义的名字。那些自描述的名字 (`wg_id`、`warp_id`、`BLK_M/BLK_N`、`HEAD_DIM`、`kv_stage`、各种 `SMEM_PIPE_DEPTH`\*/`TMEM_PIPE_DEPTH` 深度、`should_accumulate`, 以及 `CTA_GROUP`(这里为 1)) 我们会在它们首次重要的地方介绍。其余的在下面这张表里给出一句注释, 这样当一个片段在你面前摆出一个陌生的名字时, 你有地方可以查:

| 名字                        | 含义                                                                                                                                                                               |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| q_stage, i_q              | Q 流水级, 0 或 1, 即哪个 Q 分块槽 (SMEM_PIPE_DEPTH_Q = 2)。在 WG0/WG1 的 softmax 内部, 该 warpgroup 自己的 wg_id(0 或 1) 就是这个同样的阶段索引, 所以 S_region[q_stage]、P_region[wg_id]、O_region[i_q] 都选中同一个 Q 阶段 |
| MMA_N                     | TMEM 列数上的 score/输出分块宽度 (128)                                                                                                                                                     |
| MMA_K                     | P/V 列上的 MMA 内层 K 步长 (16); K_SPLIT = 6 * MMA_K = 96                                                                                                                               |
| K_SPLIT                   | value-MMA 调度的切分点 (见两个 MMA 阶段); 第一个 value MMA 覆盖列 0:K_SPLIT(6 * MMA_K = 96)                                                                                                       |
| should_rescale            | WG2 逐行标志: 旧的 0 是否需要在下一个 value MMA 之前重缩放 (用 any_sync 在 warpgroup 内规约)                                                                                                             |
| rescale_thr               | 针对微小 row-max 变化的跳过阈值; 当前内核用 8.0, 被跳过的重缩放会把 acc_scale 置为恰好 1.0                                                                                                                    |
| scale_log2                | 以 log2 为单位的 softmax 尺度, log2(e)/√d, 即 $P = \exp2((S - m) \cdot \text{scale\_log2})$                                                                                              |
| acc_scale                 | 逐行重缩放因子, softmax 通过 SMEM mailbox 传给 WG2                                                                                                                                          |
| chunk_start p_start/p_end | 正在被读 / 写的 32 宽 softmax chunk 的列范围                                                                                                                                                |

### 1.15.5 两个 MMA 阶段

对于每个流入的 K/V 分块, Flash Attention 跑两个 MMA 阶段, softmax 把它们桥接起来:

```
Q, K -> score MMA -> S
S      -> softmax    -> P
P, V -> value MMA -> O
```

把它想成一条三个生产者排成一行组成的流水线。第一个 MMA 产出注意力分数 S, softmax 把 S 转成分子 P, 第二个 MMA 消费 P 来更新输出累加器 O。除以 row\_sum 的归一化被推迟到收尾, 在每个 K/V 分块都发过言之后才做。

下面每个分块操作都拿到和我们在 GEMM 步骤里用过的同一张 **作用域 / 布局 / 派发卡**, 只是多加了一行 **交接**, 点出把分块传给下一个角色的 barrier。

计算代码从不说原始 TMEM 列号。内核把自己单一的 TMEM 分配切成按阶段的视图 (S\_region、P\_region、O\_region), 并按流水级索引 (S\_region[q\_stage]、O\_region[i\_q]、P\_region[i\_q, 0:K\_SPLIT])。这些视图在 [TMEM 布局与复用](#) 一节里用 T.TMEMStages 定义; 就现在而言, 把每个区域当作同一块物理 TMEM 的一个具名切片就足够了。

### Score MMA

两个阶段里的第一个是 score MMA, 即每个 K/V 迭代开头那个矩阵乘。它计算:

$$S = Q_{\text{block}} K_{\text{block}}^{\top}$$

并把 128 x 128 的 score 分块写到 TMEM:

```
Tx.warp.gemm_async(
    S_region[q_stage],
```

(continues on next page)

(continued from previous page)

```

    Q_smem[q_stage, 0:BLK_M, 0:HEAD_DIM],
    K_smem[kv_stage, 0:BLK_N, 0:HEAD_DIM],
    dispatch="tcgen05",
    cta_group=CTA_GROUP,
)
if T.ptx.select_sync():
    s_ready.arrive(q_stage)

```

我们可以问 GEMM 章节问过每个分块操作的同样四个问题: 谁来跑它、分块住在哪里、如何派发、如何交接:

#### 分块原语读出:Score MMA

- 作用域:WG3 warp 0 发出它; 一个被选中的 lane 到达 s\_ready。
- 布局:SMEM 中的 Q、K → TMEM 中的 S(S\_region[q\_stage])。
- 派发:tcgen05。
- 交接:s\_ready(→ softmax)。

单个被选中的线程在 s\_ready 上到达, 就是全部的交接。它宣布这块 score 分块已经完成,softmax warpgroup 现在可以自由地读它了。

#### 两个 MMA 之间的 softmax

两个 MMA 之间坐着 softmax, 这个阶段把 score 分块 S 转成分子分块 P。它的读出卡是:

#### 分块原语读出:softmax

- 作用域:WG0(Q 阶段 0)/ WG1(Q 阶段 1), 完整 warpgroup。
- 布局:TMEM 中的 S → 寄存器 → fp16 TMEM 中的 P(P\_region[wg\_id])。
- 派发: 用 tcgen05.ld 读, 用 TMEM store 写; 两者之间是在寄存器里做按行的数学。
- 交接: 等 s\_ready; 到达 p\_o\_rescale(前 96 列) 和 p\_ready\_2(最后 32 列)。

这个阶段是完全没有 GEMM 对应的那个阶段。WG0/WG1 等 score 分块在 s\_ready 上到达, 然后一次按寄存器大小的 chunk 从 TMEM 把它读出来:

```

Tx.copy_async(
    s_chunk[:, chunk_start : chunk_end],
    S_region[wg_id, chunk_start : chunk_end],
)

```

这是一次在 warpgroup 作用域下的 TMEM-到-寄存器分块读。既然分数已经坐在寄存器里,softmax warpgroup 依次做三件事:

1. 算出行最大值和行求和,
2. 算出 softmax 分子分块 P,
3. 把 P 以 fp16 写回 TMEM。

最后一步长这样:

```
Tx.copy_async(
    P_region[wg_id, p_start : p_end],
    p_chunk[:, p_start : p_end],
)
```

为什么刚在寄存器里算完 P 还要把它写回 TMEM? 因为 value MMA 需要 P 作为分块操作数, 而 MMA 没法把散落在每个线程的标量寄存器当成矩阵来读。在本内核里 P 的 MMA 可读形式是 P\_region, 即 fp16 TMEM 别名 tmem\_as\_f16 上的一个视图。所以这次回写不是多余的动作; 它正是把 P 放进下一个 MMA 真正能消费的唯一形状。

## Value MMA

第二个阶段, 也是结束每个 K/V 迭代的那个, 是 value MMA。它计算:

$$O = O + P_{\text{block}} V_{\text{block}}$$

当这个 MMA 跑起来时, O 已经被放进了对当前 K/V 块正确的状态——在第一个块上初始化、在后续块上重缩放——所以 MMA 要做的只是累加。它与 GEMM 的区别在于操作数住在哪里: A 操作数是 TMEM 中的 P, B 操作数是 SMEM 中的 V, 累加器 O 也在 TMEM 中:

```
# 第一个子 MMA: 列 0:K_SPLIT(P 的前 96 列 / V 的前 96 行)。
Tx.warp.gemm_async(
    O_region[i_q],
    P_region[i_q, 0:K_SPLIT],
    V_smem[kv_stage, 0:K_SPLIT, 0:HEAD_DIM],
    transB=True,
    accum=should_accumulate,
    dispatch="tcgen05",
    cta_group=CTA_GROUP,
)
# 第二个子 MMA(形式相同, accum=True, 由 p_ready_2 门控) 覆盖剩下的列 K_
→SPLIT:BLK_N。
```

### 分块原语读出: Value MMA

- 作用域: WG3 warp 0。
- 布局: TMEM 中的 P + SMEM 中的 V → TMEM 中的 O(O\_region[i\_q])。
- 派发: 带 TMEM 操作数的 tcgen05。
- 交接: 等 p\_o\_rescale、p\_ready\_2、kv\_load.full; 到达 o\_ready(→ 收尾)。

这种操作数摆放正是两个 MMA 之间的硬件差异:

- Score MMA 从 SMEM 读两个操作数: Q 和 K。
- Value MMA 从 TMEM 读一个操作数 P。
- Value MMA 从 SMEM 读另一个操作数 V。
- 结果累加进 TMEM 中的 O。

accum=should\_accumulate 这个标志实现了算法里”初始化或累加”的选择: 在一个 query 块的第一个 K/V 分块上为假, 其后每个分块上为真。

你可能也注意到,value MMA 并不是一次性跑完,而是拆成了 96 + 32 的调度:

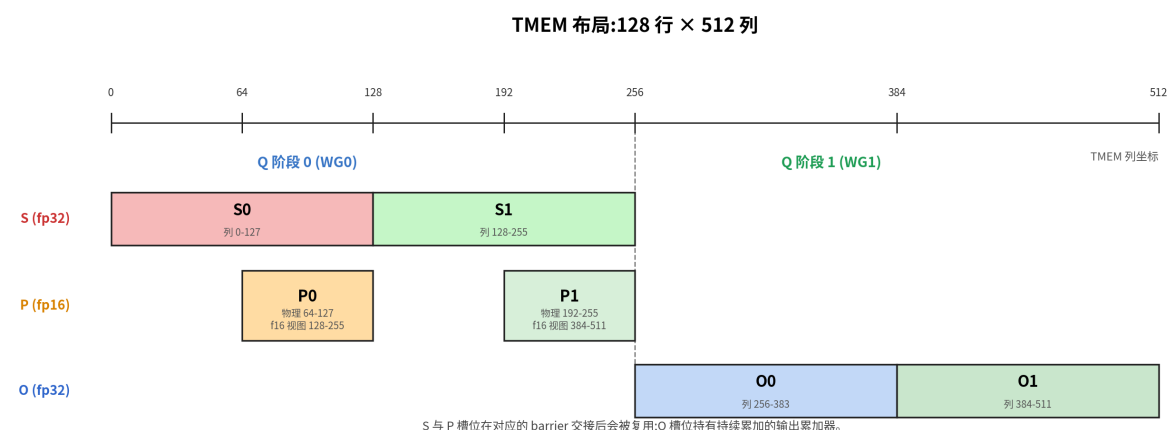
1. softmax 以四个 32 列的 chunk 写出 P。
2. 前三个 chunk 一旦就绪,value MMA 就开始在 P 的前 96 列以及 V 对应行上跑。
3. 最后 32 列等 p\_ready\_2。
4. 第二个 MMA 消费最后那个 chunk 并完成整块分块。

之所以拆分,是为了让 Tensor Core 保持忙碌。把 value MMA 当成单条指令跑,整个阶段会一直停滞,直到全部四个 32 列 P chunk 都做完指数并写好。靠在前三个 chunk 上立即开火,内核把最后一个 chunk 的 exp 和 TMEM 写与一个已经在飞的 96 宽 MMA 重叠起来,把本来是空闲的时间变成了有用的工作。

### 1.15.6 TMEM 布局与复用

S、P、O 全部要共享同一块 128 × 512 的 TMEM 分配,而它们被塞进去的方式,恰恰解释了为什么在这个内核里 barrier 与布局是不可分割的:

下面的图直接展示了那种打包方式:score 槽、分子槽、输出槽全部共享同一块 TMEM 分配,因此 barrier 协议正是让这种复用合法的东西。



这张图可以读成一组分块槽:

- score 槽放  $S = QK^T$ 。
- 分子槽放 softmax 指数步骤之后的 P 分块。
- 输出槽放 fp32 的 O 累加器。

这些并不是独立的缓冲区。它们是同一块分配的不同区域,这种共享不是风格上的选择,而是被迫的。在 Q 流水深度为 2 时,两个 S 槽 ( $2 \times \text{MMA\_N} = 256$  列)和两个 O 槽 ( $2 \times \text{MMA\_N} = 256$  列)已经占满全部 512 个 fp32 列。再没有剩下给 P 的空间,所以 P 别无选择,只能通过更窄的 fp16 视图去别名同一些字节。这之所以安全,唯一的原因是每个区域都严格在它之前的消费者用完之后才被复用,而那个时序正是 barrier 保证的。所以在 FA4 里,barrier 不仅仅是调度;它们才让布局从一开始就合法。

别名这个技巧是通过一个 `T.TMEMPpool` 搭起来的。内核先取一个 fp32 视图 (tmem) 给 score 与输出累加器用,然后把 pool 的基地址回退到 0,在同一批物理字节上取第二个 fp16 视图 (tmem\_as\_f16):

```

tmem_pool = T.TMEMPool(pool, total_cols=N_COLS_TMEM, cta_group=CTA_
    ↳GROUP, tmem_addr=tmem_addr)
tmem = tmem_pool.alloc((128, N_COLS_TMEM), "float32")
tmem_pool.move_base_to(0)
tmem_as_f16 = tmem_pool.alloc((128, N_COLS_TMEM * 2), "float16")
tmem_pool.commit()

```

因为 fp16 元素宽度只有一半,fp16 视图在同样这些字节上暴露出两倍多的可索引列,而这正是 P 住进的空间,是 fp32 布局里腾不出来的空间。两个视图在手之后,内核用 T.TMEMStages 把 S、P、O 槽切成按阶段划分的区域,让计算代码可以按流水级索引,而不是按原始列号:

```

S_region = T.TMEMStages(tmem,          col_start=0,
    ↳      width=MMA_N, stages=SMEM_PIPE_DEPTH_Q, stride=MMA_N)
O_region = T.TMEMStages(tmem,          col_start=MMA_N * SMEM_PIPE_
    ↳DEPTH_Q, width=MMA_N, stages=SMEM_PIPE_DEPTH_Q, stride=MMA_N)
P_region = T.TMEMStages(tmem_as_f16, col_start=MMA_N,
    ↳      width=BLK_N, stages=SMEM_PIPE_DEPTH_Q, stride=MMA_N * 2)

```

P\_region 步幅里那个 \* 2,是别名明显泄漏进代码的唯一一处。S\_region 和 O\_region 以 fp32 tmem 列来度量,而 P\_region 以 fp16 tmem\_as\_f16 列来度量,后者宽度只有一半,所以阶段到阶段的移动需要双倍的步幅才能落在同一批物理字节上。不过,一旦区域定义好,计算代码就保持干净:它写 S\_region[q\_stage]、读 S\_region[wg\_id, ...]、写 P\_region[wg\_id, ...]、累加进 O\_region[i\_q],从来不碰任何原始列索引。

**与你的 agent 一起试试:** 让它解释这个 FA4 内核里的 fp32(tmem) 和 fp16(tmem\_as\_f16) 视图。哪些物理 TMEM 区域放 S、P、O,而 P\_region 的步幅为什么用 MMA\_N \* 2? 把复用的问题留到下一节:在 barrier 表之后,检查每个区域被复用之前必须有哪些消费者用完。

### 1.15.7 barrier 如何把角色连起来

这是内核最难的部分,所以循序渐进地切入有好处。先从那少数几个沿主计算路径搬运数据的 barrier 入手,其余的都当作可以之后再查的簿记。那些“数据就绪”的交接是:

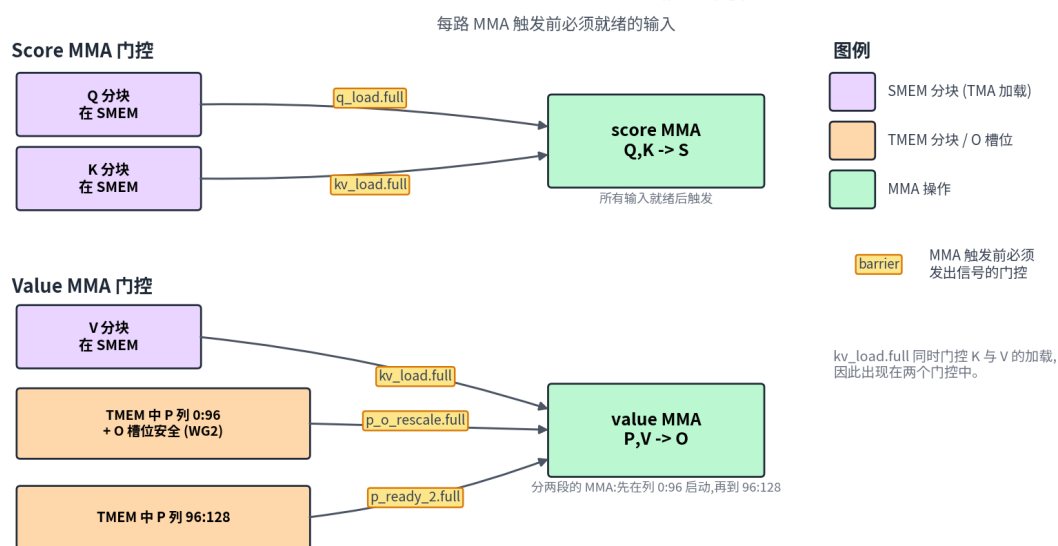
| 交接                          | 含义                         |
|-----------------------------|----------------------------|
| TMA load -> score/value MMA | Q、K 或 V 已经到达 SMEM,可以喂给 MMA |
| score MMA -> softmax        | TMEM 里的 S 已就绪              |
| softmax/修正 -> value MMA     | TMEM 里的 P 已就绪,O 可以安全累加     |
| value MMA -> 收尾             | TMEM 里的最终 O 已就绪            |
| 收尾 -> TMA store             | O_smem 已就绪可以存储             |

不在这张表里的全是流水线簿记:那些释放某个 SMEM、TMEM 或暂存缓冲区、好让另一个角色复用的 barrier。有用的一点是,每一道 barrier,无论它携带的是数据还是仅是簿记,读起来都一样,即一次分块交接。你问谁生产了数据、谁消费它、它们都做完之后哪个缓冲区变空。

下一张图把这些交接坍塌成两个 MMA 阶段的精确就绪门:score MMA 等什么,value MMA 在它能够累加之前必须等什么。



## Flash Attention 4:MMA 输入门控

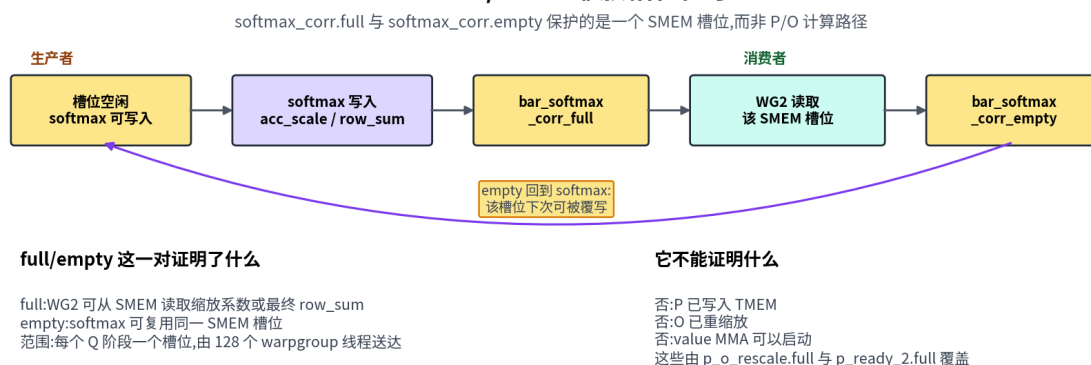


把这张图当作一组正确性门来读, 而不是一份调度。它回答”在这道 MMA 可以开火之前什么必须成立”, 而对时序只字不提。score MMA 等 SMEM 里的 Q 和 K, 然后产出 S。value MMA 同时等三样东西: SMEM 里的 V、来自 softmax 的 P 分块, 以及一个 WG2 已经释放或重缩放过的 O 槽。softmax 到 value 的那道门是拆开的, 原因我们已经见过: value MMA 在 P 的前 96 列就位后就可以开始, p\_ready\_2 再放出最后 32 列。

有一处交接不符合”分块就绪”的模式: softmax 到修正的那条边。它不传分块, 而是通过一个单槽 SMEM mailbox 把单个标量 (K/V 循环里传 acc\_scale, 收尾时传最终的 row\_sum) 传给 WG2。由于这个槽每次迭代都要复用, 必须有一对 full/empty barrier 来守护它:

下面的图放大了那次 mailbox 握手, 这也正是为什么这对 barrier 应该被读成一条标量生产者-消费者通道, 而不是一道分块就绪门。

## Softmax / WG2 缩放槽位握手



把 softmax\_corr.full 和 softmax\_corr.empty 当作一对生产者-消费者:

- softmax 在复用 scale/sum 槽之前先等 softmax\_corr.empty。
- softmax 把 acc\_scale 或最终的 row\_sum 写进那个槽。
- softmax 在 softmax\_corr.full 上到达。
- WG2 等 softmax\_corr.full, 然后读那个槽。

5. WG2 在 softmax\_corr.empty 上到达。

6. softmax warpgroup 可以在下一阶段复用这个槽。

softmax\_corr.empty 表示什么、不表示什么, 值得小心。它只表示 WG2 已经消费了 scale/sum 槽。它对 P 是否就绪只字未提, 而且绝对不是让 value MMA 开始的那道门。那道门是 p\_o\_rescale, 它在 P 的前 96 列写好、0 槽可以安全累加时开火。把两者搞混是经典的结果错误 bug 来源。

主路径在手之后, 完整的 barrier 列表可以作为参考:

| Barrier            | 生产者 -> 消费者                       | 什么变得安全                                    |
|--------------------|----------------------------------|-------------------------------------------|
| q_load.full        | TMA load -> score MMA            | Q SMEM 分块可以喂给 MMA                         |
| q_load.empty       | 这个 Q 阶段的所有 score MMA -> TMA load | Q SMEM 阶段可以复用给下一个任务                       |
| kv_load.full       | TMA load -> score/value MMA      | K 或 V SMEM 分块可以喂给 MMA                     |
| kv_load.empty      | score/value MMA -> TMA load      | K/V SMEM 阶段可以复用                           |
| s_ready            | score MMA -> softmax             | S TMEM 分块可以读                              |
| p_o_rescale        | softmax + WG2 -> value MMA       | P 的前 96 列在 TMEM 里, O 槽对 value MMA 安全      |
| p_ready_2          | softmax -> value MMA             | P 的最后四分之一在 TMEM 里                         |
| o_ready            | value MMA -> 收尾                  | 最终 O 累加器就绪                                |
| softmax_corr.full  | softmax -> WG2                   | acc_scale 或 最终 row_sum 在 SMEM mailbox 里就绪 |
| softmax_corr.empty | WG2 -> softmax                   | 同一个 SMEM mailbox 槽在 WG2 读完之后可以复用          |
| corr_epi.full      | 收尾 -> TMA store                  | O_smem 可以存储                               |
| corr_epi.empty     | TMA store -> 收尾                  | O_smem 阶段可以复用                             |

就和 GEMM 里一样, 你可以从谁产生信号来预测 barrier 的类型:

- TMA load 用 TMABar, 因为 TMA 引擎自己按字节数计数自己的完成。
- MMA 完成用 TCGen05Bar, 因为 tcgen05.commit 给完成组发信号。
- 纯线程到线程的交接用 MBarrier, 参与的线程显式到达。

拆开的 softmax 到 value 交接值得仔细看。它用了两道门:

- p\_o\_rescale 让 value MMA 在 P 的前 96 列写好、0 分块可以安全累加时就开始。
- p\_ready\_2 放出 P 的最后 32 列, 对应上一节那个 96 + 32 的 value-MMA 调度。

第一个 K/V 块是简单情形。WG2 预先在 p\_o\_rescale 上到达, 因为还没有旧 0 分块要重缩放。

后面的块得小心些。WG2 要么跳过一次不必要的重缩放、要么把旧 0 重缩放完之后, 才在 p\_o\_rescale 上到达。跳过测试是刻意保守的: softmax 计算以  $\log_2$  缩放的差值  $(m_{old} - m_{new}) * scale_{\log_2}$ ; 如果它仍高于 -rescale\_threshold, 说明新最大值移动得还不

够,不足以证明重缩放划算,于是内核保留旧最大值并把 `acc_scale` 置为恰好 1.0。只有更大的最大值跳跃才走 `exp2` 路径,并请求 WG2 重缩放 0。

然后 WG2 用 `any_sync` 在 warpgroup 内规约 `should_rescale`。如果没有一行需要这次更新,它就放过 0。这次跳过很重要,因为重缩放 0 是一次覆盖整个累加器的完整 TMEM  $\rightarrow$  RF  $\rightarrow$  TMEM 读-改-写,当阈值逻辑已经把 `acc_scale` 保持在 1.0 时,纯粹是浪费。

注意所有新的 barrier 都聚集在一处。`s_ready`、`p_o_rescale`、`p_ready_2`,以及 softmax/修正那一对,全是 softmax 周围的 barrier。它们存在出于一个原因:score MMA 和 value MMA 不再相邻。寄存器数学、TMEM 重写、输出重缩放现在夹在它们之间,而其中每一步都需要自己的交接。

与你的 **agent** 一起试试: 让它把一个 K/V 块追过 `s_ready`、`p_o_rescale`、`p_ready_2`、`o_ready`。对每道 barrier,问谁在等、谁到达、哪个分块变得可以读、之后哪个存储可以复用。

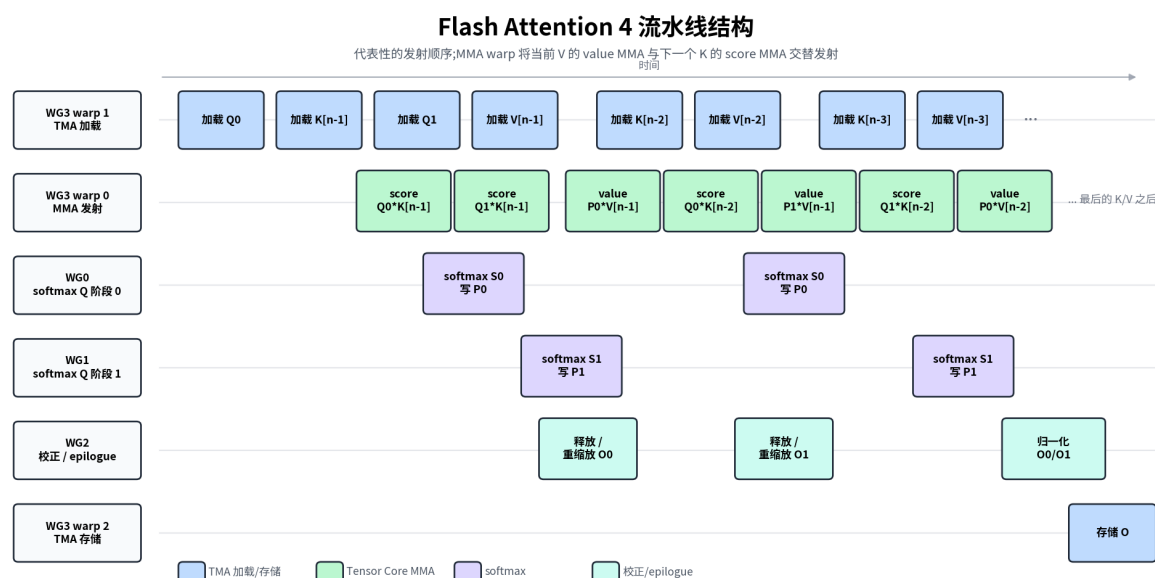
### 1.15.8 流水化结构

barrier 告诉我们在一个角色消费某个分块之前什么必须就绪。它们没告诉我们的是到底什么在并发地跑,而那正是我们现在转向的问题。这两者确实不同:一道正确性门可以在生产者碰巧运行之前很久、或之后很久被满足。

这里没有单一的流水深度,因为不同的分块流以不同的速率移动。于是内核为每条流各维护一个独立的环:

- Q 流水深度 2: 一个 CTA 同时处理两个 Q 阶段。WG0 处理一个阶段,WG1 处理另一个。
- KV 流水深度 3: K 和 V 块在内层循环里流动,同时同一批 Q 阶段被复用。
- TMEM 流水深度 2: 每个 Q 阶段有自己的 S/P/O TMEM 槽,这些槽在对应的 barrier 开火后被复用。

下面的图从正确性门切换到时间线视角,展示在这些独立环都飞起来之后,哪些角色大约能同时活动。



把这张图当作时间线,而不是 barrier 图。它展示哪些角色大约在同一时刻活动,而更早那张 barrier 流图才是你去查精确生产者-消费者等待的地方。这两张图合起来,回答了我们在本节开头提出的两个不同问题。

每一行对应代码中的一个角色分支:

- WG3 warp 1 发出 TMA load。
- WG3 warp 0 既发 score MMA 又发 value MMA。
- WG0 和 WG1 为两个 Q 阶段跑 softmax。
- WG2 释放或重缩放 0, 之后再把最终输出归一化。
- WG3 warp 2 发出 TMA store。

从左到右顺着这张图, 就追过一道有代表性的流水波。load warp 以 Q0、K[n-1]、Q1、V[n-1] 开始, 然后持续流入更低索引的 K/V 块。MMA warp 发出头几个 score MMA 来产出 S0 和 S1, WG0/WG1 把它们转成 P0 和 P1。

重要的是, MMA warp 并不是先跑完所有 score MMA 再跑所有 value MMA。两个 Q 阶段都就绪之后, 它把两类交替起来: 为当前 V 块发一个 value MMA, 再为下一个 K 块发一个 score MMA, 以此类推:

```
score Q0*K[n-1]
score Q1*K[n-1]
value P0*V[n-1]
score Q0*K[n-2]
value P1*V[n-1]
score Q1*K[n-2]
value P0*V[n-2]
...
```

正是这种交替, 使得图里 score、softmax、修正、value 这几行是重叠的, 而不是齐整地依次运行。

WG2 那一行标记为 `release / rescale`, 两半对应我们见过的两种情形。在第一个 K/V 块上还没有旧 0, 所以 WG2 只参与那道让 value MMA 继续的交接; 在后续块上它可能在 value MMA 累加进旧 0 之前先重缩放它。归一化和 TMA store 恰好各做一次, 在注意力任务的最后一个 K/V 块之后。

没有哪条单一的 GEMM 式流水能描述 FA4, 因为 Q、K/V 和 TMEM 槽都按各自的进度推进。TIRx 把这些进度显式保留, 作为独立的分块缓冲区、PipelineState 游标和 barrier 相位, 而不是把内核藏在一个大一统的原语背后。代价是更多活动部件, 但好处是复杂度保持可见、可检视。

### 1.15.9 重缩放与回写

重缩放是强制的, 不是我们可以丢掉的优化。在线 softmax 会随着每个新 score 分块抬高逐行最大值, 而每当它抬高时, 从更早分块累加来的 0 是用旧最大值标度的。这让每个更早的项都偏大了一个  $\exp(m_{\text{new}} - m_{\text{old}})$  的因子。跳过修正, 那些块就会被过度加权, 最终输出就干脆是错的。修复是一次 TMEM  $\rightarrow$  寄存器  $\rightarrow$  TMEM 的分块操作:

$$O_{\text{old}} \leftarrow O_{\text{old}} \cdot e^{(m_{\text{old}} - m_{\text{new}})/\sqrt{d}}$$

工作被拆给两个角色。softmax 计算逐行尺度并把它丢进 SMEM mailbox; WG2 等 softmax\_corr\_full, 从 TMEM 把当前 0 读出来, 乘上那个尺度, 再把 0 写回:

```
RESCALE_TILE = T.meta_var(16)
o_row = T.wg_reg_tile(RESCALE_TILE)
Tx.copy_async(o_row, 0_region[i_q, d_start : d_start + RESCALE_
```

(continues on next page)

(continued from previous page)

```

    ↪ TILE])
Tx.mul(o_row, o_row, acc_scale)
Tx.copy_async(0_region[i_q, d_start : d_start + RESCALE_TILE], o_
    ↪ row)
T.ptx.tcgen05.wait.st()

```

值得强调的是, 这是一次覆盖整个 0 累加器的完整 TMEM → 寄存器 → TMEM 分块操作, 而不是一点标量簿记, 它带着和每个其他阶段一样的读出卡:

#### 分块原语读出: 修正 (重缩放)

- 作用域: WG2, 完整 warpgroup。
- 布局: TMEM 中的 0 → 寄存器 → TMEM 中的 0(0\_region[i\_q])。
- 派发: 用 tcgen05.ld 读, 用 TMEM store 写; 两者之间是寄存器相乘。
- 交接: 等 softmax\_corr.full; 到达 p\_o\_rescale(→ value MMA) 和 softmax\_corr.empty(→ softmax)。

从端到端追一遍同步:

1. softmax 把尺度值写到 SMEM。
2. WG2 等 softmax\_corr.full。
3. WG2 重缩放 TMEM 里的 0。
4. WG2 在 p\_o\_rescale 上到达。
5. WG3 的 value MMA 现在可以消费 P 并累加进重缩放过的 0 分块。

当 WG2 读完后 softmax\_corr.empty 释放那个 SMEM 槽, 循环就合上, 这让 softmax 可以在下一次迭代复用 mailbox。

K/V 循环一结束, WG2 就从修正切换到收尾。它等最终的 row\_sum 和 o\_ready, 从 TMEM 读出最终 0, 乘上  $1 / \text{row\_sum}$  (也就是我们在一开始推迟的归一化), 转成 fp16, 写出 0\_smem。WG3 的 TMA store warp 再把 0\_smem 搬回 GMEM。

对任何打算扩展这个内核的人来说, 有一个局限值得标出。它只算前向输出, 而一次训练前向通常还会存反向所需的 log-sum-exp(LSE)。加上它要留意一个尺度细节: 这个内核把 row\_max 保持为原始、未缩放  $QK^T$  分数的最大值, 而 row\_sum 累加的是  $\exp((S - \text{row\_max}) / \sqrt{d})$ 。所以形成自然对数 LSE 时,  $1/\sqrt{d}$  因子必须重新施加到 row\_max 上:

$$\text{LSE}_i = \log(\text{row\_sum}_i) + \text{row\_max}_i / \sqrt{d}$$

这个实现只产出前向输出, 不写 LSE。

#### 1.15.10 因果掩码

因果注意力加了一个约束 (一个 query 只能注意到自己位置或之前的 key), 内核用两种互补的方式来满足它, 一种便宜, 一种精确。

便宜的方式是干脆跳过工作。很多 K/V 块完全坐在对角线之上, 对一个给定 Q 块毫无贡献, 所以 get\_n\_block\_max(...) 计算出这个块可能需要的最后一个块, 循环干脆不再加载或计算剩下的那些。

精确的方式处理那些跨在对角线上的块, 即有些列有效、有些列无效。那些块照样跑 score MMA, 但 softmax 在做指数之前把无效列掩掉。对每一行, 它从该行的 query 位置和块偏移推

导出一个列上限, 保留下限及以下各列, 把超过上限的每一列在寄存器里置为 `-inf`, 使这些列既不对行最大值、也不对 `exp2` 分子有任何贡献。

实现并不逐元素分支, 而是用 `mask_r2p(...)` 应用这个上限, 它把上限变成覆盖整个 32 宽 `score chunk` 的位掩码, 一次性把 `chunk` 掩掉。完全在对角线之下的块保留所有列, 完全不需要掩码。

从分块原语的视角看, 因果模式根本不改写数据路径。它只是修剪 K/V 的循环计数, 并在 `score MMA` 和 `P` 回写之间, 往驻留在寄存器里的 `softmax` 插入一步掩码。

### 1.15.11 GQA 支持

分组查询注意力 (Grouped-Query Attention, GQA) 让若干个 `query head` 共享单个 K/V `head`。这省显存带宽, 却带来一个打包问题: 我们如何只保留一个 K/V 分块, 同时仍把许多 `query head` 喂过它? 内核的答案是: 一次处理一整个 `query head` 组, 针对一个被调度的 `kv_head_idx`:

```
GQA_RATIO = num_qo_heads // num_kv_heads
SEQ_Q_PER_TILE = BLK_M // GQA_RATIO
```

诀窍是重新解读那 128 行 Q 分块。对 `GQA_RATIO=4`, 它们不再表示 128 个序列位置; 它们表示 32 个序列位置乘以 4 个 `query head`, 打包在一起, 使全部四个 `head` 搭同一个 K/V 分块。行的解码是:

```
seq_pos = row // GQA_RATIO
q_head = row % GQA_RATIO
```

Q load 用一个 3D 视图表达这种打包。源端是自然的 `Q[batch, seq, qo_head, dim]` 布局, 目的端是同一个 `score MMA` 之后会当作平坦 `128 x HEAD_DIM` 操作数来读的 `SMEM` 分块。这个视图正是调和两者的东西, 而且它不涉及任何拷贝:

```
Q_smem_3d = Q_smem.view(SMEM_PIPE_DEPTH_Q, SEQ_Q_PER_TILE, GQA_RATIO, HEAD_DIM)
Tx.copy_async(
    Q_smem_3d[i_q, :, :, :],
    Q[batch_idx,
        m_start : m_start + SEQ_Q_PER_TILE,
        kv_head_idx * GQA_RATIO : (kv_head_idx + 1) * GQA_RATIO,
        :],
    **tma_copy_q,
)
```

K 和 V 从不在内存中展开, 这正是 GQA 的全部意义: `kv_head_idx` 的那单个 K/V 分块, 被塞进 Q 行里的全部 `GQA_RATIO` 个 `query head` 复用。输出端与输入端对称, 收尾之后用一个匹配的 3D 视图把打包好的行存回 `O[batch, seq, qo_head, dim]`。

后果是 GQA 完全活在 Q-load 和 O-store 的边界上。计算路径内部, `score MMA` 看到的仍是普通的 `128 x HEAD_DIM` Q 分块, 分块原语图的其余部分原封不动。

### 1.15.12 分块调度

调度器的工作是把每个 CTA 映射到一个 `(batch, kv_head, m_block)` 注意力任务, 而正确策略取决于掩码是否让这些任务代价相等:



- 非因果模式用 `FlashAttentionLinearScheduler`。每个任务做同样多的工作, 所以以一个以 `num_ctas` 推进的固定 CTA 池就足以把它们均匀摊开。
- 因果模式用 `FlashAttentionLPTSchedular`, 因为因果掩码让工作量极不均匀: 靠近开头的 Q 块大约只注意一个 K/V 块, 靠近末尾的却注意全部。朴素的切分会让某些 CTA 远远晚于其他 CTA 完成, 所以最长处理时间优先 (longest-processing-time) 的调度器把重块前移以抹平完成时间, 同时仍把相邻的 batch/head 任务放在一起以保 L2 局部性。

尽管差异巨大, 两个调度器暴露出完全一致的循环接口:

```
while scheduler.valid():
    m_block_idx = scheduler.m_block_idx
    batch_idx = scheduler.batch_idx
    kv_head_idx = scheduler.head_idx
    # 处理一个 Q 块, 对应它的 K/V 块范围
    scheduler.next_tile()
```

唯一的行为差异在于 `next_tile()` 做什么: 非因果模式下它把 CTA 推进到另一个任务, 因果模式下它在当前任务之后结束循环。无论哪种, 这都纯粹是调度决策: 它选择 CTA 拥有哪个注意力分块, 从不论那个分块如何计算。循环内部跑的是同样的本地原语, 无论哪种模式: TMA load、score MMA、softmax、value MMA、修正、TMA store。

### 1.15.13 编译与验证

上面这些都是摘录, 所以要把它们拼起来真正跑内核, 我们从 `tirx-kernels` 导入真家伙, 编译它, 再对照 `torch` 参考实现检查。完整内核, 把本章走过每一块都拼到一个文件里的, 是 `tirx-kernels` 仓库里的 `flash_attention4.py`。与 GEMM 验证 cell 有两处不同: Flash Attention 有更丰富的入口 (`get_flash_attention4_kernel`), 并且它多接一个 `profiler_buf` 参数供其内置 profiler 使用。整章就跑这一个 cell:

```
import torch
import torch.nn.functional as F
import tvm
from tirx_kernels.attention.flash_attention4 import (
    get_flash_attention4_kernel, PROFILER_BUFFER_SIZE)

B, S, Hq, Hkv, D = 1, 1024, 32, 8, 128 # GQA: 32 query heads,
    ↪ share 8 KV heads
                                     # GQA: 32 个 query head 共
    享 8 个 KV head
Q = torch.randn(B, S, Hq, D, dtype=torch.float16, device="cuda")
K = torch.randn(B, S, Hkv, D, dtype=torch.float16, device="cuda")
V = torch.randn(B, S, Hkv, D, dtype=torch.float16, device="cuda")
O = torch.empty(B, S, Hq, D, dtype=torch.float16, device="cuda")
prof = torch.zeros(PROFILER_BUFFER_SIZE, dtype=torch.uint64, device=
    ↪ "cuda")

kernel = get_flash_attention4_kernel(B, S, S, Hq, Hkv, D, is_
    ↪ causal=False)
target = tvm.target.Target("cuda")
with target:
    ex = tvm.compile(tvm.IRModule({"main": kernel}), target=target, ↪
```

(continues on next page)

(continued from previous page)

```

→tir_pipeline="tirx")
ex.mod(Q, K, V, O, prof) # ex.mod takes torch tensors directly,
→like every other chapter # ex.mod 直接接收 torch 张量, 和每一章一样
torch.cuda.synchronize()

# torch 参考实现;enable_gqa 让 32 个 query head 共享 8 个 KV head
qt, kt, vt = (x.transpose(1, 2).float() for x in (Q, K, V))
ref = F.scaled_dot_product_attention(qt, kt, vt, enable_gqa=True).
→transpose(1, 2).half()
torch.testing.assert_close(0, ref, rtol=1e-2, atol=1e-2)
print(f"FA4: B={B} S={S} Hq={Hq} Hkv={Hkv} D={D}, non-causal -> PASS
→")

```

期望输出:... -> PASS。内核以 fp32 累加在线 softmax, 然而它的结果与高精度参考之间仍隔着几层不同的近似。包括: 输入与操作数的 fp16 存储与舍入; 基于 exp2 的 softmax 改写 (即  $\text{scale\_log2} = \log_2(e)/\sqrt{d}$  对每个指数的重新框架); 在线 softmax 的重排与逐行重缩放, 它是在运行尺度下逐步求和而不是一次性求和; 最后是写回时 0 的 fp16 转换。这里选用的  $\text{rtol/atol}$ , 与源内核自家测试用的容差一致, 是按“同时覆盖以上全部”相对 torch 参考来设定的, 而不是只覆盖 fp16 舍入。所以如果你哪天真在这里看到失败——不是那种擦边的险过——就把它当作指向 softmax 路径的路标: 一个被漏掉的 `s_ready / p_o_rescale / p_ready_2` 等待, 或者一次 `row_max / row_sum` 更新而重缩放步骤没施加。那恰恰是本章把 barrier 花上面的那些交接。

### 1.15.14 与 GEMM 的差异

下表沿那些发生变化的轴, 把 FA4 与 GEMM 做对比:

| 维度      | GEMM               | Flash Attention 4                 |
|---------|--------------------|-----------------------------------|
| MMA 阶段  | 一个 MMA 的重复         | score MMA 和 value MMA             |
| MMA 之间的 | 除了流水交接之外没有         | 在线 softmax、掩码, 以及 O 重缩放           |
| 运行状态    | 仅累加器               | 行最大值、行求和、O 累加器                    |
| 主要中间量   | 累加器 TMEM 分块        | S、P、O 的 TMEM 分块区域                 |
| warp 角色 | TMA 生产者、MMA 消费者、回写 | TMA load、MMA、softmax、修正、TMA store |
| barrier | 主要是 load/计算/回写交接   | 额外的 score/softmax/value/修正交接      |
| 调度单位    | 输出矩阵分块             | 注意力任务:(batch, kv_head, m_block)   |

这些差异的每一条都能追溯到我们开篇时说的那个结构性变化: 第二个 MMA, softmax 楔在两者之间。而底层的 TIRx 契约则完全没有变:

- 分块原语说哪个分块被搬运或计算,
- 外围作用域说哪些线程协作,
- 布局说分块住在哪里,
- barrier 说下一个角色何时可以消费它。

所以 FA4 比 GEMM 难, 不是因为它依赖不同的硬件, 而是因为它分块值更多、它们之间的交接更多。

### 1.15.15 练习

1. 与 GEMM 相比, FA4 的两个 MMA 阶段之间出现了什么新的分块交接? 说出生产者、TMEM 分块和消费者。
2. 为什么 softmax 把分子分块 P 写回 TMEM, 而不是为 value MMA 把它只留在寄存器里?
3. 选 p\_o\_rescale 或 p\_ready\_2。这道 barrier 究竟证明了什么? 如果 value MMA 跳过那次等待, 会出什么问题?

与你的 **agent** 一起试试: 挑一个没有注释的分块原语, 比如收尾里的一个 `Tx.copy_async`、`fp32 -> fp16` 的 `Tx.cast`, 或第二个 `gemm_pv` 子 MMA。让它给出作用域 / 布局 / 派发 / 交接卡, 然后用源码里的守卫、分配和等待来核对答案。

## 1.16 参考

主线内容贯穿第 I 至 IV 部分。参考部分收录的是你在阅读过程中需要查阅的材料:

| 需求                            | 位置                        |
|-------------------------------|---------------------------|
| 查阅 TIRx 语言特性                  | <a href="#">TIRx 语言参考</a> |
| 编译器内部机制 (降级流水线)               | <a href="#">编译器内部实现</a>   |
| 调试异步 GEMM/FA 的挂起、崩溃、错误结果或性能退化 | <a href="#">调试线程束特化内核</a> |

完整的 `tvm.tirx` Python API, 请参阅 [上游 TVM 文档](#)。

TIRx 原生层级 ([TIRx 简介](#)) 和张量布局模型 ([TIRx 布局 API](#)) 在第 II 部分介绍。

## 1.17 调试线程束特化内核

用线程束特化与集群扩展 [GEMM](#) 中的 GEMM 步骤 7-9 让 TMA 加载、`tcgen05` MMA 以及 TMEM/SMEM 写回相互重叠。同样的调试方法也适用于 Flash Attention 的交接: 先识别角色, 再识别每个角色拥有的存储, 然后用该模型去核对生成的 CUDA。

不要一上来就重写内核。首先确认这次运行本身是有效的, 然后再去检查生成的 CUDA。在排除了环境与编译期问题之后, 这些内核的运行时失败通常可以归结为一次失败的交接: 一个未初始化的屏障、错误的到达计数、被藏在角色守卫里的集合操作、陈旧的屏障相位, 或者存储在生产者让其写入可见之前就被复用了。

### 1.17.1 调试内核之前

先排除运行时环境问题:

```
python -c "import tvm, tvm.tirx; print(tvm.__file__, tvm.__version__)"
python -c "import torch; print(torch.cuda.get_device_name(), torch.cuda.get_device_capability())"
```

这些内核面向 Blackwell(sm\_100a)。如果 Python 导入的是一份陈旧的 TVM 检出版本, 或者 GPU 不是 Blackwell 系列, 请在改动内核之前先解决这些问题。然后运行该内核最小的正确性检查 (例如 `run_correctness()`), 再去看性能。

1.17.2 调试 workflow

- 1. 在仍然失败的最小形状上复现这次失败。如果失败是一次非法内存访问, 在下次运行前请重启 Python。
- 2. 如果编译失败, 在阅读运行时同步代码之前, 先检查已安装的 API、target、dispatch= 以及 buffer 作用域。
- 3. 保存 `inspect_source("cuda")` 的输出。在重新阅读 Python 之前, 先在其中搜索角色守卫、`mbarrier_init`、`tcgen05`、`cp.async.bulk.tensor` 和 `cta_sync()`。
- 4. 为失败的那条内核路径写出角色 / 存储 / 交接 / 生命周期表。
- 5. 用该表核对生成的 CUDA: 屏障初始化是否在角色分支之前、预期的 TMA 生产者、MMA 派发者、写回组, 以及是否在仅 `warpgroup` 的分支内出现了 CTA 级别的集合操作。
- 6. 把这次运行归类为死锁、崩溃、错误结果或正确但缓慢, 然后使用下方对应的章节。
- 7. 每次只改一处交接: 初始化计数、arrive/wait 相位、角色守卫、fence、TMA 存储排空、TMEM 分配/释放, 或分块调度器推进。
- 8. 在测量性能之前先重跑正确性检查。

1.17.3 要记录的内容

对任何异步内核, 在改动代码之前都先做一张小的工作表:

| 条目   | 要写下的内容                                      |
|------|---------------------------------------------|
| 角色   | 派发每个异步操作的确切线程、warp、warpgroup 或 CTA。         |
| 存储   | 每个分块在每一步的存活位置:GMEM、SMEM、TMEM 或寄存器。          |
| 交接   | 生产者、消费者、信号对象、到达计数、相位, 以及让数据可见所用的 fence 或排空。 |
| 生命周期 | 每个存储槽最早可以复用、读回或释放的时刻。                       |

然后用这张工作表去核对生成的 CUDA:

- 角色守卫与角色表一致。
- 屏障初始化出现在带守卫的角色分支之前。
- 集合操作没有意外地被 lane、warp 或 warpgroup 守卫收窄。
- arrive/wait 相位与交接表一致。
- TMA 存储排空、TMEM 释放以及 SMEM 复用只发生在生命周期表允许之后。

对 TMA->MMA-> 写回的 GEMM 流水线, 以及 Flash Attention 中 score/softmax/value/correction 的交接, 都用同一张工作表。

### 1.17.4 如果编译失败

在调试运行时同步之前, 先解决编译期失败:

| 症状                                        | 可能的方面                                          | 首先检查                                                                                                                                                           |
|-------------------------------------------|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 未知的 TIRx API 或属性错误                        | 已安装的 wheel 与教程代码不匹配                            | 打印 <code>tvm.__file__</code> 和 <code>tvm.__version__</code> ; 将 API 名与 <a href="#">TIRx 语言参考</a> 对照。                                                           |
| 不支持的 <code>dispatch=</code> Buffer 作用域不匹配 | 选定的 target 或原语不支持该路径<br>某个 buffer 被通过错误的硬件路径使用 | 检查 <code>dispatch</code> 参数与 target 能力; 本教程中的 <code>tcgen05</code> 路径需要 Blackwell。<br>检查工作表的存储行:TMEM 必须通过 <code>tcgen05</code> 访问,TMA 操作数必须使用兼容的 GMEM/SMEM 布局。 |
| 编译通过但生成的 CUDA 缺少预期路径                      | 派发没有按预期降级                                      | 在改动算法之前, 先在生成的 CUDA 中检查 <code>tcgen05</code> 和 <code>cp.async.bulk.tensor</code> 。                                                                             |

### 1.17.5 检查生成的代码

对任何已编译的内核, 都把 CUDA 保存下来, 以便搜索和 diff:

```
from pathlib import Path

cuda_source = ex.mod.imports[0].inspect_source("cuda")
Path("artifacts").mkdir(exist_ok=True)
Path("artifacts/my_kernel.cu").write_text(cuda_source, encoding="utf-8")
print(cuda_source)
```

生成的代码按如下方式把 TIRx 构造映射到 CUDA:

| TIRx                      | 生成的 CUDA                                            |
|---------------------------|-----------------------------------------------------|
| <code>wg_id == 0</code>   | <code>(warp_id_in_cta &gt;&gt; 2) == 0</code>       |
| <code>wg_id == 1</code>   | <code>(warp_id_in_cta &gt;&gt; 2) == 1</code>       |
| <code>warp_id == 0</code> | <code>(warp_id_in_cta &amp; 3) == 0</code>          |
| <code>warp_id == 3</code> | <code>(warp_id_in_cta &amp; 3) == 3</code>          |
| <code>lane_id == 0</code> | <code>((int)threadIdx.x) % 32 == 0</code>           |
| <code>.init()</code> 内部守卫 | <code>((int)threadIdx.x) &lt; 1</code> (仅 CTA 线程 0) |
| <code>elect_sync()</code> | <code>tvm_builtin_elect_one_sync_op()</code>        |

在通读整个内核之前, 先搜索这些字符串:

| 生成的 CUDA                             | 检查                                      |
|--------------------------------------|-----------------------------------------|
| <code>if (threadIdx.x &lt; 1)</code> | 单 CTA 线程守卫, 通常是屏障初始化                    |
| <code>mbarrier_init</code>           | 屏障初始化存在, 且出现在角色分支之前                     |
| <code>tcgen05</code>                 | 生成了 Tensor Core 路径                      |
| <code>cp.async.bulk.tensor</code>    | 拷贝降级到了 TMA                              |
| <code>cta_sync();</code>             | CTA 级屏障; 它绝不能出现在 <code>wg_id</code> 分支内 |

### 1.17.6 步骤 7 参考骨架

一个正确编译的步骤 7 内核具有如下的顶层形状。下方的守卫为了可读性写成角色名; 在生成的 CUDA 中, 请搜索上表中对应的表达式。

```
// (1) 屏障初始化: 顶层, 仅 CTA 线程 0
if (threadIdx.x < 1) {
    mbarrier_init(tma2mma[0..1], 1);
    mbarrier_init(mma2tma[0..1], 1);
    mbarrier_init(mma2ld, 1);
    mbarrier_init(ld2mma, 128);    // 由 WG0 的全部 128 个线程 arrive
}

// (2) TMEM 分配: WG0 的 warp 0, 派发 warp 的全部 lane
if (wg_id == 0 && warp_id == 0) tcgen05_alloc(..., 512);

// (3) fence + cta_sync, 然后相位初始化: 生产者 =1, 消费者 =0

// (4) 线程束特化循环
if (wg_id == 1 && warp_id == 3 && elect_sync) { /* TMA */
    while(valid){ ... next_tile(); } }
if (wg_id == 1 && warp_id == 0 && elect_sync) { /* MMA */
    while(valid){ ... next_tile(); } }
if (wg_id == 0) { /* WB */
    while(valid){ ... next_tile(); } }

// (5) 清理: 派发 warp, 无 lane 守卫
cta_sync();
if (warp_id == 0) { tcgen05_relinquish_alloc_permit(); tcgen05_
    dealloc(..., 512); }
```

在改动算法之前, 先检查这些项:

- 屏障初始化位于顶层, 不在 wg\_id 守卫内。
- tcgen05\_alloc 和 tcgen05\_dealloc 有 warp 守卫但没有 lane 守卫; 派发 warp 的全部 lane 都参与。
- TMA 和 MMA 循环都迭代 K\_TILES 次。
- 相位初始化为生产者 =1, 消费者 =0。

### 1.17.7 症状对照表

从症状出发, 但把它当成线索而不是最终诊断:



| 线索                            | 可能的方面               | 首先检查                                                                     |
|-------------------------------|---------------------|--------------------------------------------------------------------------|
| 内核挂起, 随后运行时报未指定的启动失败          | 死锁                  | 屏障初始化位置、到达计数、 <code>cta_sync()</code> 位置以及 <code>next_tile()</code> 参与情况 |
| 非法内存访问、XID, 或后续无关的 CUDA 调用也失败 | 崩溃 / 上下文被污染         | 重启 Python, 然后检查指针范围、存储生命周期和集合操作的参与情况                                     |
| 在 128 行或分块大小的条纹中出现错误的行        | 同步竞争或分块索引不匹配        | 生产者/消费者相位、调度器推进, 以及每个行条纹属于哪个 <code>warpgroup</code>                      |
| 出现 NaN 或明显非法的值                | 描述符、操作数设置, 或未初始化的累加 | SMEM/TMEM 描述符设置、 <code>swizzle</code> /布局, 以及累加器初始化                      |
| 有限但呈规律性错误的输出                  | 陈旧或仅部分可见的数据         | 缺失 <code>fence</code> 、缺失 TMA 存储排空, 或存储在生命周期表允许之前就被复用                    |
| 输出正确但没有预期的加速                  | 派发或资源问题             | 生成的 CUDA 路径、流水线深度、占用率以及寄存器溢出                                             |

### 1.17.8 何时重启 Python

一次 CUDA 错误并不总会自己清理干净。在非法内存访问、XID 或「CUDA 上下文被污染」错误之后, 后续无关的调用 (例如 `torch.randn`) 可能持续失败。在测试下一个修复之前请重启 Python 进程, 否则你可能是在调试上一次崩溃, 而不是当前的代码。

### 1.17.9 死锁

按以下顺序检查:

- 到达计数与初始化计数不匹配。常见情况: `MBarrier.init(128)`, 但 `arrive` 被 `if warp_id == 0: if lane_id == 0: 守卫`, 于是只有 1 个线程到达, `wait` 永不返回。

| 屏障                             | init(count) | 谁到达                                            | 到达数 |
|--------------------------------|-------------|------------------------------------------------|-----|
| TMABar (tma->mma)              | 1           | TMA 引擎, 通过 <code>arrive(stage, 1 bytes)</code> | 1   |
| TCGen05Bar (mma->tma, mma->ld) | 1           | MMA warp, 通过 <code>tcgen05.commit</code>       | 1   |
| MBarrier (ld->mma)             | 128         | WG0 的全部线程, 通过 <code>arrive</code>              | 128 |

- 屏障初始化嵌套在 `wg_id` 守卫内。 `.init()` 降级为 `if threadIdx.x < 1:`, 即 CTA 线程 0。CTA 线程 0 属于 WG0, 所以 `if wg_id == 1:` 会让所有线程都跑不到 `init`。初始化必须在顶层; 在 `inspect_source()` 中 `grep mbarrier_init` 来核实。
- `cta_sync()` 出现在 `warpgroup` 分支内。 `cta_sync` 即 `__syncthreads()`, 它要求全部 CTA 线程参与。在 `if wg_id == 0:` 内部, WG1 永远到不了它。对单个 `warpgroup` 的屏障, 请用 `T.cuda.warpgroup_sync(10)`。
- `tile_scheduler.next_tile()` 被某些消费端 `warpgroup` 的线程跳过。调度器维护每线程状态; 跳过它的线程可能永远循环下去。
- TMA 与 MMA 在 K 分块计数上不一致。如果 MMA 执行的是 `K_TILES - 1` 而不是 `K_TILES`, 屏障相位就会漂移, 第二个外层分块就会死锁。

- **PipelineState 初始相位错误。**生产者从 `phase=1` 起步, 这样第一次 `wait` 就能通过; 消费者从 `phase=0` 起步, 这样第一次 `wait` 会阻塞。如果两者从同一相位起步, 第一次交接可能立刻死锁。

### 1.17.10 崩溃与上下文污染

常见原因:

- 在 `pool.commit()` 之后调用 `pool.alloc()`。屏障封装在内部调用了 `alloc`。正确顺序: `tmem_addr -> 屏障封装 -> move_base_to(1024) -> Asmem / Bsmem / Dsmem -> commit()`。
- `tcgen05.alloc` 或 `tcgen05.dealloc` 带 `lane` 守卫。派发 warp 必须以全部 lane 参与。如果 `lane_id == 0`: 只让一个线程运行, 这是未定义行为。
- 在 `tcgen05.dealloc` 之前缺失 `cta_sync()`。在写回仍在读取时 TMEM 就被释放了。
- **GMEM 或 SMEM 越界访问。**缩小到一个分块, 检查调度器的 `m_idx / n_idx`, 并确认当前形状是内核分块或集群分块的整数倍。

### 1.17.11 错误结果

在猜测之前先按规律对错误输出分类。整行的条纹往往指向生产者/消费者相位、分块索引或角色归属不匹配。NaN 输出往往指向描述符设置、操作数设置或未初始化的累加。有限但呈规律性错误的输出往往意味着消费者读到的是旧分块、只写了一部分的分块, 或存储尚未排空的数据。

- `tcgen05.commit` 不在 `elect_sync` 内。全部 32 个线程都创建了 `commit` 组; 那 31 个空组会立刻向 `mbarrier` 发信号。于是 TMA 可能在 MMA 读取 SMEM 之前就覆盖了它。
- 在 TMA 存储之前缺失 `fence.proxy_async("shared::cta")`。TMA 引擎可能看不到来自各线程的 SMEM 写入。
- 在 TMA 存储之后缺失 `cp_async.bulk.commit_group()` 加 `wait_group(0)`。下一个分块可能在存储排空之前就复用了 Dsmem。
- 持久化内核在小尺寸 (例如 **1024x1024**) 下间歇性失败。更大的尺寸可能用更长的 K 循环掩盖了竞争。重新检查分块之间的相位复位, 以及 TMA 存储的 `commit/wait`。
- `fence.after_thread_sync()` 通常不是解药。MMA 完成的 `mbarrier` 已经带有 `release-acquire` 语义。步骤 8 和 9 出于保守在写回边缘、即 `mma2ld.wait` 之后和第一个 `tcgen05.ld` 之前添加了它; 不要在 TMA 到 MMA 的边缘常规性地添加它。

### 1.17.12 正确但缓慢

如果输出正确但性能远低于预期, 使用同样的检查循环:

| 线索                                            | 可能的方面                              | 首先检查                                                               |
|-----------------------------------------------|------------------------------------|--------------------------------------------------------------------|
| 生成的 CUDA 没有 <code>cp_async.bulk.tensor</code> | 拷贝没有降级到 TMA                        | 检查 <code>dispatch="tma"</code> 、 <code>target</code> 能力以及操作数布局     |
| 生成的 CUDA 没有 <code>tcgen05</code> 路径           | MMA 没有降级到 Blackwell Tensor Core 指令 | 检查 <code>dispatch="tcgen05"</code> 、 <code>target</code> 能力以及操作数布局 |
| TMA 与 MMA 没有重叠                                | 流水线太浅, 或相位让生产者/消费者串行化              | 检查生成的 CUDA 中 <code>wait/arrive/advance</code> 的顺序                  |
| 小形状正确性良好但大形状速度差                               | 寄存器溢出、占用率或暂存缓冲压力                   | 检查编译器资源报告; 减小分块大小、分块写回或降低流水线深度                                     |

### 1.17.13 如何提交一个好的 issue

如果这次失败在上述检查之后依然存在, 在向 [Apache TVM GitHub 仓库](#) 提交 issue 之前先把它缩小。请附上:

- `tvm.__file__ / tvm.__version__` 的输出以及 GPU 能力;
- 复现失败的最小形状;
- 这次失败是编译期、死锁、崩溃、错误结果还是正确但缓慢;
- 最小的内核或 notebook 单元, 以及它的正确性检查;
- 保存的 `inspect_source("cuda")` 输出, 或能体现可疑守卫、屏障或派发路径的最小摘录。

## 1.18 编译器内部实现

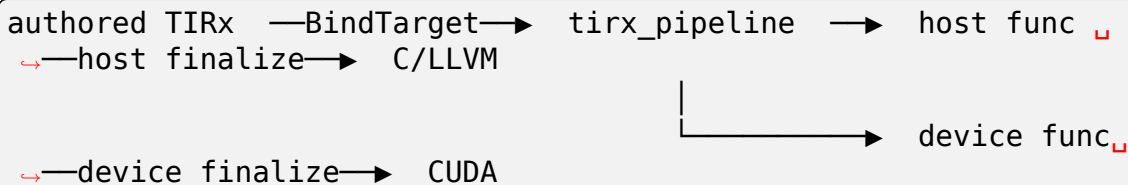
面向贡献者的 TIRx 编译器内部实现介绍。

### 1.18.1 TIRx 降级流水线

`tvm.compile(mod, target, tir_pipeline="tirx")` 会将一个手写的 TIRx 模块送入 **tirx pipeline**——一条有序的 TIR pass 序列, 它把你编写的高层构造 (tile 原语、TileLayout 类型的 `buffer`、执行作用域 id) 转换成拆分后的 **host + device** 函数, 再由 CUDA 后端渲染成源代码。该流水线定义于 `python/tvm/tirx/compilation_pipeline.py` (`tirx_pipeline`); 本页按顺序逐个介绍这些 pass。

#### 它的位置

`tvm.compile` 先绑定 `target`, 运行 **tirx pipeline** (下面的模块级 pass), 然后分别对 `host` 和 `device` 函数应用 **finalization** (收尾) pass, 最后把每个 `device` 函数交给 CUDA 代码生成器:



#### 各 pass

`tirx_pipeline` 模块 pass 按如下确切顺序执行 (少数几个受 `PassContext` 配置控制):

| #  | Pass                | 作用                                                                              |
|----|---------------------|---------------------------------------------------------------------------------|
| 1  | LowerTIRx           | 核心降级——见下文 Inside LowerTIRx                                                      |
| 2  | UnifyThreadBinding  | 合并等价的线程轴绑定, 使得每个 <code>threadIdx/block-Idx</code> 轴只声明一次                        |
| 3  | StmtSimplify        | 语句级算术化简 (arith analyzer)                                                        |
| 4  | LowerTIRxOpaque     | 把剩余的不透明 TIRx 构造降级为普通 TIR                                                        |
| 5  | FlattenBuffer       | 把多维 <code>BufferLoad / BufferStore</code> 拍平为一维                                 |
| 6  | BF16ComputeLegalize | 把 <code>bfloat16</code> 计算改写为合法形式 (上转为 <code>f32</code> )                       |
| 7  | NarrowDataType(32)  | 在可证明安全的前提下, 把 <code>index/loop PrimExpr</code> 的 <code>dtype</code> 收窄为 32 位    |
| 8  | VectorizeLoop       | 把 <code>T.vectorized</code> 循环转成向量操作 (若 <code>tir.disable_vectorize</code> 则跳过) |
| 9  | UnrollLoop          | 展开标记为 <code>T.unroll</code> 的循环 (以及小的常量循环)                                      |
| 10 | StmtSimplify        | 再次化简, 因为 <code>vectorize/unroll</code> 暴露出了常量                                   |
| 11 | CommonSubexprElim   | 把重复出现的子表达式提取为临时变量 (若 <code>tir.disable_cse_tir</code> 则跳过)                      |
| 12 | FP8ComputeLegalize  | 把 <code>float8</code> 计算改写为合法形式                                                 |
| 13 | VerifyMemory        | 检查 <code>host</code> 侧代码不会直接解引用 <code>device</code> 内存 (一道安全闸门)                 |
| 14 | AnnotateEntryFunc   | 把唯一的 <code>PrimFunc</code> 标记为模块入口                                              |
| 15 | SplitHostDevice     | 在 <code>launch_thread</code> 边界处把每个内核拆成一个 <b>host</b> 函数和一个 <b>device</b> 函数    |
| 16 | MakePackedAPI       | 把 <code>host</code> 函数改写为 <code>packed-func</code> ABI (TVM 调用的启动器)             |
| 17 | FP8StorageLegalize  | 合法化 <code>float8</code> 存储 (打包进受支持的容器类型)                                        |
| 18 | BF16StorageLegalize | 合法化 <code>bfloat16</code> 存储                                                    |

随后 **Finalization** (收尾) 按函数类型分别运行:

- **host**: `LowerTVMBuiltin` (降级 `tvm_*` builtins)、`LowerIntrin` (target 相关的 intrinsics)
- **device**: `LowerWarpMemory` (warp 作用域 `buffer` → `shuffles`)、`StmtSimplify`、`LowerIntrin`

## LowerTIRx 内部

LowerTIRx 本身是一小段序列 (`src/tirx/transform/lower_tirx.cc`):

```
LowerTIRx = Sequential([ TilePrimitiveDispatch, LowerTIRxCleanup ])
```

- **TilePrimitiveDispatch** 把每个 `TilePrimitiveCall` (`copy`、`gemm`、`reduction`、……) 替换成由其选中的后端派发所生成的函数体——即它的变体选择与代码生成。
- **LowerTIRxCleanup** 运行 `LayoutApplier`: 它把每个 `TileLayout` 类型的 `buffer` 访问解析为具体的物理地址算术 (`addr = data + elem_offset + layout.apply(coord)`), 拍平 `buffer`, 并降级执行作用域 `id` (`T.cta_id / T.thread_id / ...` 经由 `launch_thread` 变成 `blockIdx / threadIdx`)。

因此在 LowerTIRx 之后, 模块就是普通 TIR: 没有 `tile` 原语, 没有 `TileLayout` 间接寻址, 作用域 `id` 已解析为线程轴。

## 一个完整示例

取一个一行的 scale 内核:

```
@T.prim_func
def scale(A_ptr: T.handle, B_ptr: T.handle):
    A = T.match_buffer(A_ptr, (256,), "float32")
    B = T.match_buffer(B_ptr, (256,), "float32")
    T.device_entry(); bx = T.cta_id([1]); tx = T.thread_id([256])
    B[tx] = A[tx] * T.float32(2.0)
```

``LowerTIRx`` 之后, 作用域 id 已是真实的线程轴, layout 也已应用 (A\_1/B\_1 是拍平后的一维视图):

```
with T.launch_thread("blockIdx.x", 1) as blockIdx_x:
    threadIdx_x = T.launch_thread("threadIdx.x", 256)
    bx: T.let = blockIdx_x
    tx: T.let = threadIdx_x
    B_1[threadIdx_x] = A_1[threadIdx_x] * T.float32(2.0)
```

``SplitHostDevice`` + ``MakePackedAPI`` 之后, 原来这一个函数变成了两个——一个 host 启动器和一个 device 内核:

```
@I.ir_module
class Module:
    def main(...): # host: packed-API 启动器 (计算 grid/block_
        ...
    def scale_kernel(...): # device: __global__ 函数体, 在 GPU 上运行
```

随后 CUDA 后端把 scale\_kernel 渲染成 \_\_global\_\_ 函数 (B\_ptr[threadIdx.x] = A\_ptr[threadIdx.x] \* 2.0f)。

## 自己复现一遍

你可以手工运行流水线的任意前缀来检视某个阶段——本文档各处的 IR 片段就是这样得到的:

```
from tvm.tirx import transform as TT

target = tvm.target.Target("cuda")
mod = TT.BindTarget(target.with_host("llvm"))(tvm.IRModule({"main":
    scale}))
mod = TT.LowerTIRx()(mod) # tile 原语已派发, layout 已应用
print(mod.script()) # 检视降级后的 TIRx IR
```

或者编译整个模块, 再读出生成的 CUDA:

```
exe = tvm.compile(tvm.IRModule({"main": scale}), target=target, tir_
    pipeline="tirx")
print(exe.mod.imports[0].inspect_source())
```

## 1.19 TIRx 语言参考

编写 TIRx 设备内核所需的完整语言特性集合, 从 [TIRx 简介](#) 演练中抽离而来: 解析器工具、数据类型与表达式、buffer 与内存、控制流, 以及线程同步。当你需要某个特性的精确拼写或语义时, 请查阅这些页面。

### 1.19.1 解析器工具

少数几个辅助工具作用于 **\*\* 解析期 \*\***(即 TVMScript 被转换为 TIRx 的时刻), 让你可以内联 Python 计算出的值、抽离可复用片段, 并把解析器侧的状态打包起来。

#### T.meta\_var ——内联一个 Python 值

T.meta\_var(x) 告诉解析器把 x——一个在 **Python** 中算出的值——当作一个编译期 *meta* 值, 直接内联进 IR, 而不是把它当作脚本变量来解析。它避免了那种用完即弃的临时局部变量, 并驱动元编程: 一个普通的 Python for 遍历某个 meta 值时会在解析器中展开。

```
n = T.meta_var(4)           # n 是一个 Python int, 被内联
for j in range(n):         # 解析期展开
    acc[0] = acc[0] + A[tx, j]
```

#### @T.inline ——内联函数

@T.inline 定义一个函数, 其函数体在解析期间 **\*\* 在每个调用点被内联 \*\***——生成的代码中不会出现调用。它遵循 Python 的词法 (LEGB) 作用域与延迟绑定, 因此参数会遮蔽外层变量:

```
@T.inline
def add_into(acc, x):
    acc[0] = acc[0] + x

add_into(acc, A[tx, j])    # 内联 -> acc[0] = acc[0] + A[tx, j]
```

#### @T.meta\_class ——解析器侧状态对象

@T.meta\_class 标记一个普通 Python 类, 其 **\*\* 实例就是解析器的 meta 值 \*\***: 它们的字段可以持有 buffer 和标量, 所以你可以把相关的分配与状态打包进一个对象, 并在内核体中使用它。

```
@T.meta_class
class State:
    def __init__(self, smem):
        self.acc = T.alloc_local([1], "float32")
        self.buf = T.decl_buffer([64], "float16", smem, scope=
            ↪ "shared.dyn")

s = State(smem.data)
s.acc[0] = T.float32(0.0)    # 像普通 buffer 一样使用它的字段
# ... s.buf[i] ...
```

这很适合用来把一个内核的流水线状态 (barrier、累加器、临时视图) 打包成一组, 而不是把许多零散的局部变量穿过函数体。



## T.constexpr

T.constexpr 标记一个编译期内核参数, 由 @T.jit 的 .specialize(...) 烘焙进来。细节见 *TIRx* 简介。

### 1.19.2 数据类型与表达式

每个 TIRx 表达式都携带一个底层 **dtype** 和一个高层 **type**。

#### 表达式 dtype

一个 PrimExpr 的 .dtype 是它的标量(或向量)元素类型——float32、float16、bfloat16、int32、uint8、bool、低精度的 float8\_e4m3fn/float4\_e2m1fn……、handle``(指针), 以及诸如 ``float32x4 的向量形式。每种都打印为对应的 CUDA 类型。跨若干 dtype 分配 local 和 shared buffer, 外加一次向量化的 float32x4 加载/存储:

```
@T.prim_func
def dtypes(A_ptr: T.handle, O_ptr: T.handle):
    A = T.match_buffer(A_ptr, (256,), "float32")
    O = T.match_buffer(O_ptr, (256,), "float32")
    T.device_entry(); bx = T.cta_id([1]); tx = T.thread_id([64])
    f16 = T.alloc_local((1,), "float16")           # 寄存器标量 .....
    bf16 = T.alloc_local((1,), "bfloat16")
    i32 = T.alloc_local((1,), "int32")
    u8 = T.alloc_local((1,), "uint8")
    b1 = T.alloc_local((1,), "bool")
    sm = T.alloc_shared((64,), "float16")          # ..... 以及一个
    ↪ shared 分块
    v = T.alloc_local((1,), "float32x4")           # 一个向量 dtype 寄存
器 (float4)
    v[0] = A.vload([tx * 4], dtype="float32x4")    # 向量化加载
    O.vstore([tx * 4], v[0])                       # 向量化存储
    # ... (使用 f16/bf16/i32/u8/b1/sm) ...
```

降级为 (生成的 CUDA, 已省略):

```
half      f16_ptr[1];           // float16
nv_bfloat16 bf16_ptr[1];        // bfloat16
int      i32_ptr[1];           // int32
uchar     u8_ptr[1];            // uint8
signed char b1_ptr[1];         // bool
__shared__ alignas(64) half sm_ptr[64]; // shared float16
float4     v_ptr[1];            // float32x4 (向量)
v_ptr[0]   = *(float4*)(A_ptr + tx * 4); // 向量化加
载
*(float4*)(O_ptr + tx * 4) = v_ptr[0];    // 向量化存储
```

一个 buffer 的 dtype 本身就可以是 \*\*向量类型\*\*: T.alloc\_local((1,), "float32x4") 直接声明一个 float4 寄存器 (你以 v[0] 索引它), 而一次 float32x4 的 vload/vstore 随后把它作为一次 16 字节访问来移动。向量 dtype 并不与 vload 绑定——任何 buffer 或标量都可以携带它。

所以 dtype → CUDA 的映射是:

| dtype → CUDA       | dtype → CUDA     | dtype → CUDA           |
|--------------------|------------------|------------------------|
| float32 → float    | float16 → half   | bfloat16 → nv_bfloat16 |
| int32 → int        | uint8 → uchar    | bool → signed char     |
| float32x4 → float4 | handle → T* (指针) | (向量 dtype → CUDA 向量类型) |

## dtype vs type

dtype 是 \* 底层 \* 的——它说的是「什么比特」。另外, 一个值还有一个高层 **type**: 标量是 `PrimType(dtype)`, 指针是 `PointerType(PrimType(dtype), scope)`。大多数表达式都是标量 (`PrimType`); 类型系统主要对 **\*\* 指针 \*\*** 有意义。

## 指针 (handle)

一个 `buffer` 的 `data`——即它的指针——是一个指针类型的 `Var`, 且它是 **\*\* 不可变 \*\*** 的 (指针不会被重新赋值)。这决定了你如何获得一个指针:

- `T.alloc_buffer(...)` 分配存储 **\*\* 并 \*\*** 定义它的 `data` 指针。
- `T.decl_buffer(..., data=ptr)` 在一个已有指针 `Var ptr` 之上声明一个 `buffer`。
- 若要用一个指针 **\*\* 表达式 \*\*** 来支撑一个 `buffer`——例如 `T.ptx.map_shared_rank``(PTX ``map)` 给出另一个集群 CTA 的共享地址——你必须先把该表达式绑定到一个指针 `Var``(``data 必须是一个 Var, 不能是表达式), 使用一个 PointerType 的 T.let:`

```
from tvm.ir.type import PointerType, PrimType

ptr: T.let[T.Var(name="ptr", dtype=PointerType(PrimType("uint64"
→ ")))] = \
    T.reinterpret("handle", T.ptx.map_shared_rank(mbar.ptr_
→ to([0]), 0))
remote_mbar = T.decl_buffer([1], "uint64", data=ptr, scope=
→ "shared")
```

## 1.19.3 Buffer 与内存

参数 `buffer` 用 `T.match_buffer` 绑定; 临时 `buffer` 用下面两种声明 API 之一在函数体内创建。用 `A[i, j]` 索引 `buffer`; 用 `A[m0:m0+BM, 0:BK]` 对其切片 (得到一个 `BufferRegion`), 用 `A.ptr_to([i, j])` 取指针, 或用原始数据指针 `A.data`。

## 声明 buffer

两种基本 API 用于创建 `buffer`:

- `T.alloc_buffer(shape, dtype, scope=..., ...)` ——分配新存储 (生成一个 `AllocBuffer` 节点) 并返回该 `Buffer`。 `T.alloc_shared` / `T.alloc_local` 只是分别带 `scope="shared"` / `scope="local"` 的 `alloc_buffer`。

- `T.decl_buffer(shape, dtype, data=..., ...)` —— 在一个已有指针 `data` 上 **\*\* 声明一个视图 \*\*** (不分配存储); 用于给存储起别名或重新解释——可以是池的一个子区域, 或一个张量内存 (tensor memory) 地址。当 `data=None` 时它会像 `alloc_buffer` 一样分配存储。

`buffer` 的 `data` 指针是一个不可变 `Var` (`alloc_buffer` 定义它; `decl_buffer` 接收一个)。若要用一个指针 `* 表达式 *` 来支撑一个 `buffer`, 需先绑定它——见[数据类型与表达式](#)。

二者共享同一个描述符; 最关键的参数如下:

| Parameter                                 | 含义                                                                                                                                                        |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>dtype</code>                        | 元素类型——"float32"、"float16"、"float4_e2m1fn"、……                                                                                                              |
| <code>shape</code>                        | 逻辑形状 (由各维 <code>extent</code> 组成的元组)                                                                                                                      |
| <code>layout</code>                       | 物理映射 ( <a href="#">TileLayout</a> ); "" default"" = 稠密行主序                                                                                                 |
| <code>elem_offset / allocated_addr</code> | <code>elem_offset</code> (或 <code>byte_offset</code> ) 把一个 <code>* 视图 *</code> 放在 <code>data</code> 内的某个偏移处; <code>allocated_addr</code> 携带一个预分配地址 (张量内存) |
| <code>align</code>                        | 数据指针对齐, 以字节为单位                                                                                                                                            |

`scope` 参数选择内存空间:

| Scope        | 简写                          | 内存                                 |
|--------------|-----------------------------|------------------------------------|
| "global"     | (默认)                        | 设备全局内存                             |
| "shared"     | <code>T.alloc_shared</code> | 静态共享内存 ( <code>__shared__</code> ) |
| "shared.dyn" | (池)                         | 动态共享内存 (池化——见下文)                   |
| "local"      | <code>T.alloc_local</code>  | 每线程寄存器                             |
| "tmem"       | (TMEM 池)                    | Blackwell 张量内存 (见下文)               |

```
A = T.match_buffer(A_ptr, (M, K), "float16", align=16)  # 参数 ↵
↪buffer
As = T.alloc_shared((BM, BK), "float16")                # 新的共享分
块
acc = T.alloc_local((4,), "float32")                    # 寄存器累加
器
view = T.decl_buffer((BM, BK), "float16", data=As.data) # As 上的一个
视图
```

基于指针的 **buffer** 只是指针之上的元数据。对任何非 `tmem` 的 `buffer`, 其声明就是「一个指针 + 一个布局」, 而索引解析为一个地址:

```
addr(buffer[coord]) = buffer.data + elem_offset + layout.
↪apply(coord, shape=shape)["m"]
```

(`layout.apply` 返回逐轴映射; 其中的 `"m"` 分量是元素偏移。) 因此, \* 同一个 \* 逻辑访问会纯粹依据 `buffer` 的元数据被编译成不同的地址算术。在一个 `4×8` 区域上写 `B[i, j] = A[i, j] + 1`, 把 `B` 以四种方式声明:

```
from tvm.tirx.layout import TileLayout, S
```

(continues on next page)

(continued from previous page)

```

B = T.match_buffer(p, (4, 8), "float32")
    ↪      # 行主序
B = T.match_buffer(p, (4, 8), "float32", layout=TileLayout(S[(4, ↪
    ↪8):(1, 4)])) # 列主序
B = T.match_buffer(p, (4, 8), "float32", elem_offset=64)
    ↪      # 平移视图
B = T.match_buffer(p, (4, 8), "float32", layout=TileLayout(S[(4, ↪
    ↪8):(16, 1)])) # 行步长 16

```

每一种都让  $B[i, j]$  在生成的 CUDA 中降级 (lowering) 为不同的索引 ( $A[i, j]$  的加载保持  $i*8 + j$  不变——只有  $B$  的元数据变了):

```

B_ptr[((i * 8) + j)] = ...; // 行主序:       $i*8 + j$ 
B_ptr[((j * 4) + i)] = ...; // 列主序:       $j*4 + i$ 
B_ptr[((i * 8) + j) + 64] = ...; // elem_offset=64:  $i*8 + j + 64$ 
B_ptr[((i * 16) + j)] = ...; // 行步长 16:     $i*16 + j$ 

```

## 共享内存

共享内存有两种形式——**静态** (编译期固定) 与 **动态** (启动时确定大小)——外加一个管理动态情形的池辅助工具。

### 静态

最简单的共享 buffer 是 **静态** 的——`T.alloc_shared` (即 `scope="shared"`), 在编译期确定大小。把数据搬进来, `__cta_sync__` 让整个 block 都看到这些写入, 然后再读回:

```

@T.prim_func
def smem_demo(A_ptr: T.handle, B_ptr: T.handle):
    A = T.match_buffer(A_ptr, (128,), "float32")
    B = T.match_buffer(B_ptr, (128,), "float32")
    T.device_entry()
    bx = T.cta_id([1])
    tx = T.thread_id([128])
    sm = T.alloc_shared((128,), "float32") # 静态共享内存
    sm[tx] = A[tx]
    T.cuda.cta_sync()
    B[tx] = sm[tx] * T.float32(2.0)

```

它降级为一个普通的 `__shared__` 数组 (生成的 CUDA, 样板已省略):

```

extern "C" __global__ void __launch_bounds__(128)
smem_demo_kernel(float* __restrict__ A_ptr, float* __restrict__ B_
    ↪ptr) {
    int tx = ((int)threadIdx.x);
    __shared__ alignas(64) float sm_ptr[128]; // T.alloc_shared
    sm_ptr[tx] = A_ptr[tx];
    __syncthreads(); // T.cuda.cta_
    ↪sync()

```

(continues on next page)

(continued from previous page)

```

    B_ptr[tx] = sm_ptr[tx] * 2.0f;
}

```

## 动态

动态共享内存 (scope="shared.dyn") 按启动确定大小 (即 sharedMemBytes 启动参数), 而非编译期。一个内核 \*\* 只能有唯一一个 \*\* 动态共享分配——即那个 *arena*。所以你只需分配它一次, 然后把每个 buffer 作为一个视图 decl 进去: 用 T.decl\_buffer, "data=" 指向 arena 指针, 并带上一个 elem\_offset:

```

arena = T.alloc_buffer((128,), "float32", scope="shared.dyn") # 唯一的那个 arena
As = T.decl_buffer((64,), "float32", data=arena.data, scope="shared.dyn") # 偏移 0
Bs = T.decl_buffer((64,), "float32", data=arena.data, elem_offset=64, scope="shared.dyn") # 偏移 64
As[tx] = A[tx]
Bs[tx] = B[tx]
T.cuda.cta_sync()
C[tx] = As[tx] + Bs[tx]

```

两个视图共享同一个 extern \_\_shared\_\_ arena (生成的 CUDA, 样板已省略; 为清晰起见把 arena 命名为 smem):

```

extern __shared__ __align__(64) float smem[]; // 唯一的那个动态共享 arena
smem[tx] = A_ptr[tx]; // As — 偏移 0 处的视图
smem[tx + 64] = B_ptr[tx]; // Bs — 偏移 64 处的视图
__syncthreads();
C_ptr[tx] = smem[tx] + smem[tx + 64];

```

(两次单独的 alloc\_buffer(scope="shared.dyn") 调用是错误的——只允许一次动态共享内存分配。) 所以静态共享内存存在编译期确定大小 (\_\_shared\_\_ T x[N]); 动态共享内存则是这个唯一的、按启动确定大小的 arena, 视图在它内部的各偏移处被声明。

### Note

**TVM 如何标注动态共享大小。** arena 的大小在编译期已知 (此处是 128 个 float = 512 字节)。在降级过程中, TVM 会给设备内核的 tidx.kernel\_launch\_params 追加一个 "tidx.use\_dyn\_shared\_memory" 标签, 而 host 端启动器会算出总字节数, 把它作为最后一个启动参数传入:

```

# 设备内核属性:
"tidx.kernel_launch_params": ["blockIdx.x", "threadIdx.x", "tidx.use_dyn_shared_memory"]

# host 端启动调用 (... , gridDim.x, blockDim.x, dyn_shared_bytes):
T.call_packed("dyn_kernel", A.data, B.data, C.data, 1, 64, 512)

```

运行时, 那个 512 会成为 cuLaunchKernelEx 调用中的 config.sharedMemBytes。你永远不需要手动设置它——它是从 shared.dyn 分配的大小推导出来的。

## 池语法糖

T.SMEMPool 自动完成这套 arena 簿记——它会 bump 分配各偏移, 这样你就不必手动 decl 视图。除了 alloc/commit 之外, 它还提供逐 buffer 的 align=、一个能为你构造与 MMA 兼容的 swizzle 布局的 alloc\_mma 辅助工具, 以及一个把游标倒回以复用空间的 move\_base\_to:

```
pool = T.SMEMPool()                # shared.dyn 上的 bump
    ↪ 分配器
As = pool.alloc((BM, BK), "float16", align=128)  # 切出一个分块
Bs = pool.alloc((BK, BN), "float16", align=128)
Cs = pool.alloc_mma((BM, BN), "float16")        # 与 MMA 兼容, swizzle 自动推断
pool.commit()                                  # 定稿池的大小
# pool.move_base_to(offset) 把游标倒回以复用空间
```

下文的 T.MEM 池 (tensor memory) 就建立在 SMEMPool 之上。

## 寄存器

每线程的临时数据存放在寄存器里。用 T.alloc\_local(shape, dtype) (即 scope="local") 分配: 它对每个线程私有, 降级为一个保存在寄存器中的本地数组。

```
r = T.alloc_local((4,), "float32")  # 每线程寄存器数组
for k in T.unroll(4):
    r[k] = A[tx, k]
# ... 在 r[0..3] 上计算 ...
```

```
alignas(64) float r_ptr[4];          // 每线程, 驻留寄存器
r_ptr[0] = A_ptr[tx * 4 + 0];
r_ptr[1] = A_ptr[tx * 4 + 1];
// ...
```

### Note

这个 alignas(64) 是 \* 默认 \* 的 buffer 对齐——一个 buffer 的 data\_alignment 默认为 runtime::kAllocAlignment (64 字节), CUDA codegen 会把它盖到每个分配上, 包括那些对齐毫无意义的逐线程 local 数组。对这类驻留寄存器的数组, 它 \*\* 没有任何性能影响 \*\*: 一个索引可在编译期解析的线程本地数组, 会被 nvcc/ptxas 提升为寄存器 (聚合体的标量替换, SROA), 因此它从不落入可寻址的本地内存, 这个对齐就是个空操作。(一个被动态索引、溢出到本地内存的数组, 才会真正吃到这个过度对齐, 但那是少见情形。) 寄存器本地变量的这种过度对齐是一个已知的粗糙之处, 我们计划修复 (对 local 作用域改用 dtype 的自然对齐)。



## 标量

标量就是 **\*\* 单元素 \*\*** 的寄存器数组——严格说,你并不需要一个单独的概念。你可以分配一个大小为 1 的 local buffer 并索引 [0]:

```
phase = T.alloc_local((1,), "int32")    # 单元素寄存器数组
phase[0] = 0
while phase[0] < 4:
    acc = acc + A[tx, phase[0]]
    phase[0] += 1
```

但到处写 phase[0] 很笨拙,所以 **\*\* 标量 \*\*** 正是为此而生的语法糖——一个单元素寄存器 buffer,你 **\*\* 按名字 \*\*** 读写它:

```
phase: T.int32 = 0                      # 可变标量 (上面写法的语法糖)
while phase < 4:
    acc = acc + A[tx, phase]
    phase += 1

s = T.local_scalar("int32")            # 显式形式; 按名字赋值 (s = ..., 而非 s[0])
acc: T.float32 = 0.0                   # 一个带类型标注的赋值也会生成一个标量
```

二者不只是相似——它们被解析为 **\*\* 结构上完全相同的 TIRx \*\***。语法糖完全在解析器中消解: phase: T.int32 就是 \* 那个单元素 `local` buffer, 而 `phase` / `phase += 1` 就是 phase[0] / phase[0] += 1。对两个内核跑 `tvm.ir.assert_structural_equal` 会通过,而且 `printer` 甚至会把显式的 `alloc_local + [0]` 形式 **\*\* 反过来 \*\*** 渲染成标量形式——所以一旦解析完成,二者毫无差别。因此二者都降级为同一个 `alignas(64) int phase_ptr[1];`; 标量只是让你省掉 [0]。(T.local\_scalar / T.shared\_scalar / T.alloc\_scalar 显式选择作用域。)

### Note

**为什么不是 Var?** TIRx 的 Var 是 \* 不可变 \* 的——一个静态绑定 (它正是下文 T.let 所产生的)。而标量需要是 \* 可变 \* 的——你会在循环和累加器中重新赋值给它——所以它必须由一个可反复写入的单元素 buffer 支撑,而不是一个 Var。

## let

一个 T.let 绑定是 **\*\* 不可变 \*\*** 的——单个 LetStmt (一个具名值,而非 buffer)。用它来表达派生常量:

```
n: T.let = M * K                        # 不可变绑定 (LetStmt)
half: T.let[T.int32] = N // 2           # .....带一个显式类型
```

它降级为一个 **\*\* 普通的标量 C 变量 \*\***——不是 buffer (没有数组,没有 [0])。对 half: T.let = m \* 2 (其中 m 是运行时值):

```
int half = m * 2;                      // 这个 'let' -> 一个类 const 的局部变量
```

因为该值不可变,简化器可以自由地传播它并对其做 CSE,所以在使用点你常常直接看到 `m * 2` 被替换进去 (或通过一个公共子表达式临时量共享),而不是对 half 的引用。

**Note**

**为什么要有不可变绑定?** 因为该值不会改变, 算术分析器在简化一个 `LetStmt` 时会把它绑定到该 `var(analyzer.Bind(var, value))`, 于是关于该值所证明的事实——常量边界、模集合 (整除性 / 对齐)、范围——**会传播到每一处使用**。这会喂给索引简化、边界检查消除, 以及对齐 / 向量化决策。而 \* 可变 \* 标量是一次内存加载 (`buf[0]`): 分析器不能假设它保持恒定, 所以那些性质都无法传递。`let` 也是一个纯值——不分配存储, 可自由内联 / 替换 / CSE——而标量是一个带加载 / 存储语义的单元元素 `buffer`。

**张量内存**

Blackwell 的 \* 张量内存 \* 不是一个普通的临时作用域: 它必须用 `warp` 一致的 `T.ptx.tcgen05.alloc / tcgen05.dealloc` 内显式预约和释放, 每个张量都是其中用 `T.decl_buffer(..., scope="tmem", allocated_addr=<column>, layout=<tmem layout>)` 声明的一个视图。`allocated_addr` (一个列偏移) 是必填的——`tensor-core` 派发会断言它——所以 `T.alloc_buffer(scope="tmem")` (它 \*\* 不 \*\* 设置该地址) 行不通。与共享内存不同, 张量内存不可直接寻址: 它只能通过 `tcgen05` 的 `mma / ld / st / cp` 来读写。

手动做法是: 一个 `warp` 把分配发到一个共享槽位, 你把每个张量在某列偏移处 `decl` 为视图, 最后一个 `warp` 在结尾释放它:

```
addr = T.alloc_shared((1,), "uint32")           # 存放已分配基址的槽位
if warp_id == alloc_warp:                       # tcgen05.alloc 是
    ↪warp 一致的
    T.ptx.tcgen05.alloc(T.address_of(addr), n_cols=512, cta_
    ↪group=cta_group)
acc = T.decl_buffer((CTA_M, 512), "float32", scope="tmem",
                    allocated_addr=0, layout=tmem_layout) # 列 0 处
的视图
# ... 把 acc 用作 gemm_async / copy_async 的操作数 ...
if warp_id == alloc_warp:
    T.ptx.tcgen05.relinquish_alloc_permit(cta_group=cta_group)
    T.ptx.tcgen05.dealloc(addr, n_cols=512, cta_group=cta_group)
```

列偏移和 `tmem_layout` (一个数据通路 D/F 布局) 由你自己管理。这正是下面那个池所生成的序列。

**池**

`T.TMEMPpool` 把以上全部封装起来——`warp` 一致的 `alloc/dealloc`、列的 `bump` 分配, 以及数据通路布局:

```
tmem_addr = pool.alloc((1,), "uint32")          # pool = 内核的 smem
    ↪池
tmem_pool = T.TMEMPpool(pool, total_cols=512, cta_group=cta_group,
                        tmem_addr=tmem_addr)
acc = tmem_pool.alloc((CTA_M, 512), "float32") # 为你设好 allocated_
    ↪addr
tmem_pool.commit()                             # 发出 tcgen05.
    ↪alloc(由一个 warp)
```

(continues on next page)

(continued from previous page)

```
# ... 使用 acc ...
tmem_pool.dealloc() # 发出 tcgen05.
↪dealloc(由一个 warp)
```

完整示例见第三部分的 GEMM 内核。

## Buffer API

一个 Buffer 是指针之上的元数据 (见上文 \* 声明 `buffer*`), 所以它的大多数方法都是 \* 编译器 \* 的 `reshape` / 重解释, 改的是索引算术或给你一个指针——它们本身不发出任何运行时操作。常用方法:

| Method                                                                       | 是什么                                                                        |
|------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <code>B.data</code>                                                          | 原始数据指针 (一个 Var); 打印为 <code>B_ptr</code>                                    |
| <code>B.ptr_to([i, j])</code>                                                | 指向某个元素的有类型指针 ( <code>address_of</code> ); 打印为 <code>&amp;B_ptr[...]</code> |
| <code>B.vload([i], dtype="float32x4")</code> / <code>B.vstore([i], v)</code> | 向量化加载 / 存储; 打印为 <code>*(float4*)(B_ptr + ...)</code>                       |
| <code>B.view(*shape, layout=...)</code>                                      | 在新形状 / 布局下重新解释同一份存储 (不拷贝)                                                  |
| <code>B.local(*shape, layout=...)</code>                                     | 调用线程对 local buffer 拥有的私有寄存器切片                                              |
| <code>B.permute(*dims)</code>                                                | 轴被置换后的视图 (一个转置后的布局)                                                        |
| <code>B.access_ptr(mask, ...)</code>                                         | 一个带掩码的访问指针 ( <code>tvm_access_ptr</code> 内建), 用于把一个区域传给某个 intrinsic        |

指针——`ptr_to` / `data`。 `ptr_to` 是你把一个元素地址交给某个 intrinsic 或内联函数的方式; `data` 是基地址指针:

```
B[tx] = T.cuda.func_call("ld", A.ptr_to([tx]), source_code=SRC,
↪return_type="float32")
```

```
B_ptr[tx] = ld(&A_ptr[tx]); // ptr_to([tx]) -> &A_ptr[tx];
↪ A.data -> A_ptr
```

向量化访问——`vload` / `vstore`。把多个元素作为一次宽传输移动 (另见数据类型与表达式):

```
B.vstore([tx * 4], A.vload([tx * 4], dtype="float32x4"))
```

```
*(float4*)(B_ptr + tx * 4) = *(float4*)(A_ptr + tx * 4);
```

**reshape** / 重解释——`view` / `permute`。二者都是纯元数据; 数据指针不变, 只是索引算术不同。 `A.view(64, 4)` 把这个 256 元素的 buffer 看作 `64×4`; `A.permute(1, 0)` 转置各轴:

```
A2 = A.view(64, 4); y = A2[tx, 0] + A2[tx, 3] # A2[tx, j] ->
↪A_ptr[tx*4 + j]
```

(continues on next page)

(continued from previous page)

```
At = A.permute(1, 0);    z = At[i, j]           # At[i, j] -> _
->A_ptr[j*4 + i]
```

```
A2_ptr[tx * 4] /* +3 */           // view: 行主序 64x4 索引
At_ptr[(j * 4) + i]              // permute: 步长被交换
```

寄存器——“**local**”。把一个线程轴的 `local` 布局分解为调用线程的平坦寄存器束 (被分块原语广泛使用):

```
R = T.alloc_buffer((32, 8), "float32", scope="local", _
->layout=TileLayout(S[(32, 8) : (1 @ laneid, 1)]))
Rl = R.local(8)           # 当前 lane 的 8 个寄存器
```

```
alignas(64) float Rl_ptr[8];           // 该 lane 的私有寄存器
```

### 1.19.4 控制流

控制流就是 `if`、循环族, 以及 `while`——每一种都映射到显而易见的 CUDA。

#### `if`

Python 的 `if / else` 变成 CUDA 的 `if / else`。用一个线程 / lane 比较来守卫工作, 或者用 `T.ptx.select_sync()` 选出一个单独的发令线程:

```
if tx < 128:
    A[tx] = A[tx] * T.float32(2.0)
else:
    A[tx] = A[tx] + T.float32(1.0)

if T.ptx.select_sync():
    ...                               # 单个被选中的 lane (例如用于发出 _
->TMA/MMA)
```

```
if (((int)threadIdx.x) < 128) {
    A_ptr[tx] = A_ptr[tx] * 2.0f;
} else {
    A_ptr[tx] = A_ptr[tx] + 1.0f;
}
```

若要做表达式级的选择 (无分支), 用 `T.if_then_else(cond, a, b)`。它降级为一个三元表达式, 因此不引入任何控制流发散:

```
O_ptr[tx] = (A_ptr[tx] > 0.0f) ? A_ptr[tx] : 0.0f;
```

## 一致 vs 发散的控制流

像 `if tx < 128` 这种逐线程守卫, 对普通工作没问题, 但 `** 集体 **` 操作必须被它所同步的每一个线程 \* 一致地 \* 到达。

例如, `T.cuda.cta_sync()` 映射为 `__syncthreads()`, 后者要求 thread block 中的所有线程到达。它绝不能位于一个线程或 warpgroup 发散的分支内: 若放在 `if wg_id == 0:` 之内, 其他 warpgroup 永远不会到达, 内核将死锁。当只需要一个 warpgroup 同步时, 使用 warpgroup 作用域的 `T.cuda.warpgroup_sync(id)` (见[用线程束特化与集群扩展 GEMM](#)与[CUDA C++/PTX intrinsics](#))。

同样的谨慎也适用于 barrier 的初始化。一个 mbarrier 的 `.init()` 降级为单线程守卫 (`if (threadIdx.x < 1)`)。把它嵌套在另一个发散分支内, 可能让 barrier 处于未初始化状态, 导致未指明的启动失败。

## loop

循环有四种形式; 普通的 Python range 变成 `T.serial`:

- `T.serial(n)` —— 顺序循环 (ptxas 仍可能展开它)。
- `T.unroll(n)` —— 完全展开 (扩展为直线语句)。
- `T.vectorized(n)` —— 向量化循环。
- `T.grid(*extents)` —— 嵌套循环族。

`break / continue` 在循环内可用。

```
for i, j in T.grid(8, 8):
    B[i, j] = T.max(A[i, j], T.float32(0.0))
```

```
for (int i = 0; i < 8; ++i)
    for (int j = 0; j < 8; ++j)
        B_ptr[i * 8 + j] = max(A_ptr[i * 8 + j], 0.0f);
```

`T.unroll(4)` 则展开为四条没有循环的直线语句。

## while

`while` 循环一直运行到其条件为假为止。使用一个可变标量计数器 (见[Buffer 与内存](#)):

```
i: T.int32 = 0
while i < 64:
    A[i] = A[i] + T.float32(1.0)
    i += 1
```

它降级为带提前退出 `break` 的 `while (1)` (计数器是一个单元素寄存器 buffer):

```
int i_ptr[1];
i_ptr[0] = 0;
while (1) {
    if (!(i_ptr[0] < 64)) { break; }
    A_ptr[i_ptr[0]] = A_ptr[i_ptr[0]] + 1.0f;
```

(continues on next page)

(continued from previous page)

```
i_ptr[0] = i_ptr[0] + 1;
}
```

### 1.19.5 CUDA C++/PTX intrinsics

当没有分块原语能覆盖你的需求时, 有两条逃生路径可直接触达硬件: 调用后端 **intrinsic**\*\* (来自 ```tvm.backend.cuda``` 的 ```T.cuda.*``` / ```T.ptx.*``` 命名空间), 或 \*\* 内联原始 CUDA 源码。

#### 调用后端 intrinsic

`T.cuda.*` 和 `T.ptx.*` 直接暴露 CUDA 后端的设备 intrinsic——同步、mbarrier、归约, 以及 PTX 的数据搬移 / MMA 家族:

```
T.cuda.cta_sync()           # block 屏障 (__syncthreads)
T.cuda.warp_sync()          # __syncwarp
T.cuda.warpgroup_sync(8)    # warpgroup 屏障
T.cuda.cta_sum(val, num_warps, scratch.ptr_to([0])) # block 级归约

bar = T.alloc_shared((1,), "uint64")
T.ptx.mbarrier.init(bar.data, 1) # 用于异步完成的 mbarrier
T.ptx.mbarrier.try_wait(bar.data, phase)
```

一个完整、可运行的示例——通过 `T.tvm_warp_shuffle_xor` 做一次 warp all-reduce:

```
@T.prim_func
def warp_reduce(A_ptr: T.handle):
    A = T.match_buffer(A_ptr, (32,), "float32", align=16)
    T.device_entry()
    cta_id = T.cta_id([1]); warp_id = T.warp_id([1]); lane_id = T.
    ↪ lane_id([32])
    v = T.alloc_local((1,), "float32"); i = T.alloc_local((1,),
    ↪ "int32")
    v[0] = T.float32(31 - lane_id)
    i[0] = 16
    while i[0] >= 1:
        v[0] += T.tvm_warp_shuffle_xor(0xFFFFFFFF, v[0], i[0], 32,
    ↪ 32)
        i[0] = i[0] // 2
    A[lane_id] = v[0]
```

该 shuffle 直接降级为 `__shfl_xor_sync`:

```
v_ptr[0] = v_ptr[0] + __shfl_xor_sync(0xFFFFFFFF, v_ptr[0], i_
    ↪ ptr[0], 32);
```

`T.ptx.*` / `T.cuda.*` 下的其他家族: `cp_async` (LDGSTS)、`cp_async.bulk`、`tensor` (TMA)、`ldmatrix` / `stmatrix`、`tcgen05` (Blackwell MMA)、`atomic_add`、`fence` ……完整的 `tvm.backend.cuda` 参考见后端 API 文档。



## 同步语义

在 GEMM 和 Flash Attention 内核中, 有四种同步机制会不断出现。因为它们控制的是异步引擎和并行线程组, 误用其中任何一个通常都会导致静默的数据损坏或死锁。

**Mbarrier 的相位。** mbarrier 用单个内部相位位来跟踪到达情况。T.ptx.mbarrier.try\_wait(bar, phase) intrinsic 会阻塞, 直到 barrier 的内部相位与调用者传入的 phase 参数 \* 不同 \*。因此, 在循环迭代间复用 a barrier 时, 调用者必须在每次等待后翻转自己的本地相位跟踪量 (phase ^= 1)。否则会让随后的等待立即返回, 使引擎读到写了一半的内存。构建分块 GEMM 给出了完整的相位跟踪表。

**选举。** T.ptx.select\_sync() 选出的是 \*warp 内单个活跃的 lane\*, 不是 lane 0, 也不是每个 CTA 一个线程。要把发令者收窄到恰好一个线程, 你必须把它与一个 warp 级守卫配对使用。if warp\_id == 0: 后接 if T.ptx.select\_sync(): 这个模式, 在构建分块 GEMM 中用于发出 Tx.gemm\_async 和 tcgen05.commit。

**具名 warpgroup 屏障。** T.cuda.cta\_sync() 映射为 \_\_syncthreads(), 要求 \* 每个 \* CTA 线程都到达。一旦各 warpgroup 特化到不同代码路径上, 把 cta\_sync() 放在一个 warpgroup 分支内就会让内核死锁, 因为其他 warpgroup 永远到不了那里。硬件提供 16 个具名屏障 (ID 0 到 15); “T.cuda.warpgroup\_sync(10)” 只同步某一个 warpgroup 的线程。不同的 warpgroup 取不同的 ID (例如 warpgroup\_sync(wg\_id + 10)), 这样它们永远不会在同一个硬件屏障上相撞。见用线程束特化与集群扩展 GEMM。

**栅栏 (fence)。** fence 用来排序: 让生产者的写在消费者 (常常是某个异步引擎) 读取之前先完成:

| Fence                                       | 排序的内容                                                        |
|---------------------------------------------|--------------------------------------------------------------|
| T.ptx.fence.<br>proxy_async("shared::cta")  | 线程写入的共享内存, 先于某个异步代理 (TMA store / MMA) 读取它                    |
| T.ptx.fence.mbarrier_init()                 | mbarrier 的初始化, 先于后续的到达或等待使用该 barrier                         |
| T.ptx.tcgen05.fence.<br>after_thread_sync() | tcgen05 写回边上的一道保守排序 fence (第 8、9 步加上了它; 在 TMA 到 MMA 的路径上不需要) |

## 内联原始 CUDA

对那些根本没有 intrinsic 的需求, 用 T.cuda.func\_call(name, \*args, source\_code=..., return\_type=...) 从一个源码字符串注入一个 \_\_device\_\_ 函数:

```
SRC = r"""
__device__ __forceinline__ float my_relu(float x) { return x > 0.f ?
↪ x : 0.f; }
"""

@T.prim_func
def k(A_ptr: T.handle, B_ptr: T.handle):
    A = T.match_buffer(A_ptr, (256,), "float32")
    B = T.match_buffer(B_ptr, (256,), "float32")
    T.device_entry(); bx = T.cta_id([1]); tx = T.thread_id([256])
```

(continues on next page)

(continued from previous page)

```
B[tx] = T.cuda.func_call("my_relu", A[tx], source_code=SRC, ↵  
↵return_type="float32")
```

源码被原样输出, 调用被接上:

```
__device__ __forceinline__ float my_relu(float x) { return x > 0.f ?  
↵ x : 0.f; }  
// ...  
B_ptr[tx] = my_relu(A_ptr[tx]);
```